

残酷共学 · 增量笔记

自 2026-08-15 23:01 BJT 后新增 90
篇

本次增量 · 带索引

点击目录条目可跳转

数据来源：intensivecolearn.ing 活动公开打卡

生成时间：2026-08-16 23:01 (BJT)

目录

主题章节 → 笔记索引 (点击可跳转)

01 AMM机制 (9 篇)	6
笔记 001: 8月14日 (周四) —— 实操：设计并执行跨链测试矩阵	6
笔记 002: 從 Router Path 找到實際 Pair / Pool Address	8
笔记 003: Day 12 统计套利、CEX-DEX 价差与做市执行算法	12
笔记 004: 骨架里做了三个和常规写法不同的决定	12
笔记 005: Day 11 打卡 鏈上套利殘酷共學	13
笔记 006: Deterministic Deployment (合约确定性部署)	14
笔记 007: 这周太忙了，没太多共学的进度，列一下本周事项和下周计划：	17
笔记 008: Day 12：加入真实成本模型	17
笔记 009: 今天延續前次對 Uniswap Universal Router 執行層的研究，進一步查看 Open	17
02 LI.FI与跨链 (17 篇)	18
笔记 010: 1. 研究目标与科学定义	18
笔记 011: 学习了这篇笔记，来自x睡眠不足会死：	20
笔记 012: 净利 = 毛价差 池子手续费 gas 桥费 价格影响 时间风险 MEV 抢跑	21
笔记 013: 200u 套利	21
笔记 014: 今天查阅了 LI.FI 官方 Composer 文档。Composer 不是另一套独立的报价接口：对	21
笔记 015: https://github.com/ZeroNullNone/onchain_arbitrage_	22
笔记 016: 【Day 12 2026-08-16】把观察变成可重复流程	22
笔记 017: D13 数据采集：LI.FI 报价采集脚本 (同链 + 跨链跑通)	23
笔记 018: 今天基本没有继续扩策略类型，主要做了一件事：	23
笔记 019: Hermes Agent 工作流	27
笔记 020: Day 11：官方合约与权限验证	29
笔记 021: 链上套利共学 · Day12 打卡笔记 (08/16)	30
笔记 022: 【Day 16 打卡 · 2026-08-16】小额试跑执行与首轮复盘	31
笔记 023: Day 11 · 2026-08-16 · 隔了一天回来对账 Day 10 的 Polygon→ET	31

笔记 024: \# 2026-08-12 \~ 2026-08-15 链上套利共学群合并总结	37
笔记 025: 8月16日 跨链套利：赚价差之前，先管理库存和最终性	39
笔记 026: 链上套利学习笔记 Day 12：聚合器对比	40
03 MEV与抢跑 (4 篇)	41
笔记 027: Day 9 (8/14) 打卡	41
笔记 028: Day 12 链上套利残酷共学	41
笔记 029: MEV 驱动型机会结构 (Mempool & Ordering-based MEV)	42
笔记 030: 扣除各项成本后是否仍然成立？	42
04 做市与LP (1 篇)	42
笔记 031: 1. 核心底层逻辑：为什么会有资金费率？	43
05 其他 (2 篇)	43
笔记 032: 学习做 LP 相关的东西\	43
笔记 033: 今天新开了一个 topic：STRC 收益代币化对比。	44
06 套利框架 (32 篇)	45
笔记 034: 链上套利残酷共学打卡 - 2026-08-15 (Day 11)	45
笔记 035: 今天通过调用栈拆解了以太坊交易：	45
笔记 036: 【Day 11 2026-08-15】完整拆解一个候选机会	46
笔记 037: 记录一条候选套利机会：	47
笔记 038: 2026-08-15	47
笔记 039: 今天整了一个只读DEMO，计划明天弄一个真实的只读软件。	47
笔记 040: 链上套利学习 Day 9 笔记	48
笔记 041: 8月15号	62
笔记 042: 探讨在 cex-dex 套利中我一直存疑的问题，Day 9：【现货-永续价格错位套利】	64
笔记 043: Day 12 笔记 (2026-08-16)	65
笔记 044: Day 12：全自动套利系统架构 v1.1	66
笔记 045: 今日共学打卡 从“能报价”走到“能否真正闭环”	70

笔记 046: Day 12 (08/16) 学习总结, 素材来自今日同学精华:	71
笔记 047: 今日完成	71
笔记 048: 跨所套利系统学习笔记: 从策略发现到开仓、平仓与执行优化	72
笔记 049: Day 9 双边对冲执行、Legging Risk 与半自动套利工具	80
笔记 050: LP 套利: 不和套利机器人抢利润, 而是在必经路径上收手续费	116
笔记 051: 今天继续学习 DEX-CEX 套利, 重点思考了一个问题: 价差达到多少才值得关注?	119
笔记 052: Day 10 (8/14) 原子套利: 路径发现与闪电贷【重日】	119
笔记 053: title: Day11 负溢价与 ADL 风险	119
笔记 054: CEX—DEX 套利是 CEX—CEX 的链上版, 同一枚币在中心化订单簿和链上池之间赚价差, 仍是非原	120
笔记 055: Day 12 短时三窗口报价稳定性	121
笔记 056: 今天看了《九种套利角度: 用闭合检查器筛一遍》, 最大的收获不是又认识了几个套利策略, 而是开始学会怎么判 .	121
笔记 057: Day 11: FASTEST 快 5 秒, 但最低输出反而少 5.089062 USDC。	122
笔记 058: 今日 (8/16) 实际学习: 做市深学 + 跨所套利视角。	122
笔记 059: 今天承接的问题	122
笔记 060: 第十二天学习记录: 今天对 Robinhood Chain 套利研究的证据链做了一次三方一致性审计, 逐	131
笔记 061: 今天继续梳理跨所资金费率套利的收益结构, 重点区分资金费率收益和基差变化带来的损益。	131
笔记 062: Day 12 资金费率套利进入 Paper Trading: 把 OI 异动加进信号过滤层	131
笔记 063: 2026-08-16 學習打卡	132
笔记 064: 套利基础学习打卡⑦ Leg Risk 与执行策略: 两条腿不可能真正同时成交, 应该先下哪一边?	132
笔记 065: 2026-08-16 共学 Day 12 训练套利 Agent ② 投入约 1 小时	147

07 学习计划 (6 篇) 147

笔记 066: Day 12 2026.08.16 从原则到机制: 三层利润、Nonce 协调、订单簿竞态	147
笔记 067: 链上套利残酷共学 · Day 12 打卡 (2026-08-16)	148
笔记 068: 昨日套利日志: 三角路线与链上对手	149
笔记 069: Day 12 - 从一次性观察到重复查询	149
笔记 070: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践, Agent 仅负责	150
笔记 071: 链上套利残酷共学 2026-08-16 打卡	150

08 实盘与数据 (13 篇) 151

笔记 072: Polymarket 市场结算机制学习	151
笔记 073: 笔记 1: 系统性偏离监控与预期定价模型\	156
笔记 074: markdown	158
笔记 075: Day 12 · 2026-08-16 · 回测验证：资金费率收租策略的真实期望	159
笔记 076: 2026-08-16	160
笔记 077: 量化交易系统进阶必备：Python 语言底层四大核心思想 昨天群友问我：已经会用 Pandas、N	160
笔记 078: Day 12 从 LIVE 监控到可审计执行：报价、状态与失败成本的闭环	161
笔记 079: Day 12 — 1 小时采集数据复盘：跨平台价差发掘 + 套利经济学	161
笔记 080:	
今天研究套利系统中最容易被忽略的一层：执行概率与概率加权收益。监控看到“扣费后利润为正”，并不代表这 . . .	163
笔记 081: 笔记 1: 网格交易：简化版做市策略的系统拆解\	164
笔记 082: Day 12：链上清算与自动交易基础设施扩展	166
笔记 083: 1\、修复普查解析器的持久化缺陷	166
笔记 084: 今天监听的PairCreated的新池和usdt/wbnb主池Swap结果是binance data	166

09 风险与安全 (6 篇) 167

笔记 085: 2026-08-15 學習打卡	167
笔记 086: 当然可以。先抓住最核心的一点：	167
笔记 087: 第 12 天 阶段三开启：捕获、实施与复盘	170
笔记 088: 01.9 钱包隔离、授权、签名与只读安全	170
笔记 089: 1. ERC-20中规定的几个核心属性/交互方法	179
笔记 090: 最近完成：	179

笔记 001: 8月14日 (周四) —— 实操：设计并执行跨链测试矩阵

作者：Cool-CoCo | 时间：2026-08-15 15:21 UTC | 字数：4104 | 评分：86

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月14日 (周四) —— 实操：设计并执行跨链测试矩阵

今天不学新概念，直接动手实操。目标是设计一个测试矩阵，系统地比较不同链、不同资产、不同金额的跨链成本。

一、测试矩阵设计原则

核心原则是每次只改变一个变量，这样才能准确判断每个因素对结果的影响。

变量清单：

源链 / 目标链

资产类型 (稳定币 / 原生资产 / 波动资产)

金额大小

排序方式 (FASTEST / CHEAPEST)

二、我的测试矩阵设计

测试组1：链对比较

表格

测试ID	源链	目标链	资产	金额	观察点
1	Arbitrum	Optimism	USDC→USDC	100	两条L2之间的桥竞争
2	Arbitrum	Base	USDC→USDC	100	换成Base，成本变化
3	Ethereum	Arbitrum	USDC→USDC	100	从L1到L2的跨链成本
4	Arbitrum	Ethereum	USDC→USDC	100	从L2回到L1的跨链成本

测试组2：资产类型比较

表格

测试ID	源链	目标链	资产	金额	观察点
5	Arbitrum	Optimism	USDC→USDC	100	稳定币跨链
6	Arbitrum	Optimism	ETH→ETH	0.1	原生资产跨链
7	Arbitrum	Optimism	USDT→USDC	100	稳定币互换+跨链

测试组3：金额大小比较

表格

测试ID	源链	目标链	资产	金额	观察点
8	Arbitrum	Optimism	USDC→USDC	10	小额
9	Arbitrum	Optimism	USDC→USDC	100	中额 (基准)
10	Arbitrum	Optimism	USDC→USDC	1000	较大额
11	Arbitrum	Optimism	USDC→USDC	10000	大额

三、实际操作步骤

步骤1：构造API请求URL

以测试1为例，请求Arbitrum到Optimism跨链100 USDC的报价：

<https://li.quest/v1/quote?fromChain=ARB&toChain=OPT&fromToken=USDC&toToken=USDC&fromAmount=100000000>

参数说明：

fromChain=ARB：Arbitrum (链ID 42161)

toChain=OPT：Optimism (链ID 10)

fromToken=USDC：源资产 (合约地址或符号)

toToken=USDC：目标资产

fromAmount=100000000：100 USDC (USDC是6位小数，所以100 USDC = 100,000,000)

步骤2：同时请求FASTEST和CHEAPEST

在请求URL中添加排序参数：

CHEAPEST（默认）

https://li.quest/v1/quote?fromChain=ARB&toChain=OPT&fromToken=USDC&toToken=USDC&fromAmount=100000000

FASTEST

https://li.quest/v1/quote?fromChain=ARB&toChain=OPT&fromToken=USDC&toToken=USDC&fromAmount=100000000&order=FASTEST

步骤3：记录数据模板

为每个测试创建一个记录，格式如下：

markdown

测试X：Arbitrum → Optimism | USDC→USDC | 100 USDC

请求参数

- 源链：Arbitrum
- 目标链：Optimism
- 源资产：USDC
- 目标资产：USDC
- 金额：100 USDC
- 排序方式：CHEAPEST
- 请求时间：2026-08-14 15:00 UTC

返回结果

- toAmount：99950000（约99.95 USDC）
- toAmountMin：99900000（约99.90 USDC）
- gasCosts：0.0000365 ETH（约\$0.07）
- feeCosts：5 USDC（桥费）
- estimatedDuration：30秒
- 工具路径：1inch(Arbitrum) → Across(跨链) → Uniswap(Optimism)
- 路径ID：xxx
- 报价有效时间：15秒

初步观察

- 总成本：5.07 USDC（桥费5 + Gas 0.07）
- 成本占比：5.07%
- 到手金额：99.93 USDC（含滑点保护）

四、测试结果汇总表

表格

测试ID	源链→目标链	资产	金额	Gas成本	桥费	总成本	成本占比	预计到手
1	ARB→OPT	USDC→USDC	100	\$0.07	\$5	\$5.07	5.07%	99.93
2	ARB→BASE	USDC→USDC	100	\$0.08	\$4.5	\$4.58	4.58%	99.54
3	ETH→ARB	USDC→USDC	100	\$0.50	\$6	\$6.50	6.50%	99.35
4	ARB→ETH	USDC→USDC	100	\$0.45	\$7	\$7.45	7.45%	99.26
8	ARB→OPT	USDC→USDC	10	\$0.07	\$5	\$5.07	50.70%	9.49
9	ARB→OPT	USDC→USDC	1000	\$0.10	\$5	\$5.10	0.51%	999.49

五、关键发现

发现1：Gas成本对金额不敏感\

跨链10 USDC和100 USDC的Gas成本几乎一样（都是约\$0.07），但占比从5%飙升到50%。这就是为什么小额跨链交易“不划算”。

发现2：桥费可能是固定费用\

从测试1和测试8看到，跨链10 USDC和100 USDC的桥费都是5 USDC，说明桥费可能是固定费用，而不是按比例收取。

发现3：从L1到L2的跨链成本更高\

测试3（Ethereum→Arbitrum）的Gas成本是\$0.50，而测试1（Arbitrum→Optimism）只有\$0.07。这是因为Ethereum主网的Gas价格远高于L2。

发现4：路径会随金额变化\

测试10（1000 USDC）的路径可能和测试1（100 USDC）不同，因为大额交易需要更深的流动性池。

六、今日总结

今天的实操让我真正理解了“跨链成本”的构成：

Gas成本：与交易金额无关，主要取决于链的拥挤程度；

桥费：可能是固定费用，对小额交易影响巨大；

路径选择：不同金额、不同链对，最优路径可能完全不同；

下一步，明天开始尝试用脚本看能不能自动化这个测试过程。

笔记 002: 從 Router Path 找到實際 Pair / Pool Address

作者：corn15 | 时间：2026-08-15 15:23 UTC | 字数：4594 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

從 Router Path 找到實際 Pair / Pool Address

1. 核心概念

Router 的 path 描述的是：

經過哪些 Token

在 V3 中還會包含 Fee Tier

但 path 本身通常不直接等於 Pair / Pool Address。

基本流程：

Router Path \

→ 解析 Token / Fee \

→ 查 Factory \

→ 找到實際 Pair / Pool Address

2. Uniswap V2 類型

常見於：

Uniswap V2

SushiSwap V2

PancakeSwap V2

其他 V2 Fork

V2 的 Path 通常是：

[TokenA, TokenB, TokenC]

代表：

TokenA → TokenB → TokenC

實際會經過：

Pair 1 : TokenA / TokenB

Pair 2 : TokenB / TokenC

所以：

N 個 Token 會對應 N - 1 個 Pair。

關鍵函數

Factory.getPair(tokenA, tokenB)

可以取得實際 Pair Address。

也可以用：

UniswapV2Library.pairFor()

透過 CREATE2 規則直接計算 Pair Address。

但跨不同 V2 Fork 時，建議優先使用 getPair()，因為不同 DEX 的 Factory 與 Init Code Hash 可能不同。

3. Uniswap V3 類型

V3 和 V2 最大差別是：

TokenA + TokenB 不足以唯一決定 Pool。

因為相同 Token Pair 可以有多個 Fee Tier，例如：

USDC / WETH / 0.01%

USDC / WETH / 0.05%

USDC / WETH / 0.30%

USDC / WETH / 1.00%

所以 V3 的 Pool 是由：

TokenA

TokenB

Fee

三者共同決定。

關鍵函數

Factory.getPool(tokenA, tokenB, fee)

用來取得 Pool Address。

也可以使用：

PoolAddress.computeAddress()

直接計算 Pool Address。

4. V3 Path 結構

V3 的 Path 不是普通的 Address Array，而是 Packed Bytes。

概念上：

TokenA | Fee1 | TokenB | Fee2 | TokenC

其中：

Token Address：20 bytes

Fee：3 bytes

例如：

USDC\

↓ fee = 500\

WETH\

↓ fee = 3000\

ARB

代表：

Pool 1：USDC / WETH / 500

Pool 2：WETH / ARB / 3000

解析 V3 Path 時的重要函數：

Path.decodeFirstPool()

可以取得：

tokenA

tokenB

fee

5. V2 與 V3 比較

| 項目 | V2 | V3 |

| :----- | :----- | :----- |

| Path 格式 | address array | packed bytes |

| Pool Identity | TokenA + TokenB | TokenA + TokenB + Fee |

| Factory 查詢 | getPair() | getPool() |

| 離線計算 | pairFor() | computeAddress() |

| 同 Token Pair 多池 | 通常沒有 | 可以 |

| Path 包含 Fee | 否 | 是 |

可以簡化成：

V2：

(tokenA, tokenB) → getPair() → Pair Address

V3：

(tokenA, tokenB, fee) → getPool() → Pool Address

6. V3 exactOutput 的特殊情況

V3 的 exactInput Path 通常與交易方向一致。

例如實際交易：

USDC → WETH → ARB

Path 也是：

USDC → WETH → ARB

但 exactOutput 的 Multi-hop Path 是反向編碼。

如果實際交易是：

USDC → WETH → ARB

Path 可能會是：

ARB → WETH → USDC

所以分析 V3 Router calldata 時：

不能只看 Path 第一個 Token 就判斷它是 TokenIn。

還要確認 Router 執行的是：

exactInput

exactOutput

7. 歷史交易中找實際 Pool

如果分析的是已經發生的 Transaction，除了從 Path 推算，也可以直接看：

Transaction Trace

Event Logs

Transaction Trace

Trace 會顯示 Router 實際 CALL 哪些 Pair / Pool。

概念上：

User\

→ Router\

→ Pair / Pool A\

→ Pair / Pool B

Router 實際呼叫的 Contract Address，就是實際經過的 Pair / Pool。

因此：

Trace 可以視為實際執行結果。

8. 從 Event Logs 找 Pool

Swap 完成後，Pair / Pool 通常會 Emit Swap Event。

重要觀念：

哪個 Contract Emit Swap Event，那個 Contract Address 就是 Pair / Pool Address。

因此可以從：

Transaction Receipt\

→ Logs\

→ Swap Event\

→ log.address

找到實際 Pool。

9. Path、Factory、Trace、Logs 的關係

| 方法 | 用途 |

| :----- | :----- |

| Router Path | 理解 Router 想走的 Swap 路徑 |

| Factory | 從 Token / Fee 找到預期 Pool |

| Transaction Trace | 確認實際 CALL 了哪些 Pool |

| Event Logs | 確認哪些 Pool 實際發生 Swap |

推薦分析流程：

Router Calldata\
→ Decode Path\
→ 找 Expected Pools\
→ Factory.getPair() / Factory.getPool()\
→ 看 Transaction Trace\
→ 看 Swap Events\
→ 確認 Actual Pools

所以：

Path 用來理解交易意圖，Trace / Logs 用來確認實際執行。

10. Universal Router 與 Aggregator

如果是：

Uniswap Universal Router
1inch
Odos
Ox
ParaSwap

就不能單純假設：

Path → 單一 Factory → Pool

因為一筆交易可能跨多個 Protocol。

例如：

USDC\
→ Uniswap V3 Pool\
→ WETH\
→ SushiSwap V2 Pair\
→ ARB

這時通常需要解析：

Router Commands\
→ Sub-command\
→ Protocol\
→ Path\
→ Pair / Pool

對這類複雜 Router：

Transaction Trace 通常比只分析 Path 更容易確認實際經過哪些 Pool。

11. 關鍵函數整理

V2

Factory.getPair(tokenA, tokenB)

用途：

取得 Pair Address。

UniswapV2Library.pairFor()

用途：

透過 CREATE2 計算 Pair Address。

V3

Factory.getPool(tokenA, tokenB, fee)

用途：

取得 Pool Address。

PoolAddress.computeAddress()

用途：

直接計算 Pool Address。

Path.decodeFirstPool()

用途：

從 V3 Path 中解析：

tokenA

tokenB

fee

12. 套利分析的心智模型

Router 本身不是交易池。

Router 的角色是：

協調 Swap 路徑與呼叫不同 Pair / Pool。

真正持有流動性的 Contract 是：

V2 Pair

V3 Pool

因此可以理解成：

User\

→ Router\

→ Path\

→ Factory\

→ Pair / Pool\

→ Swap

如果是做套利分析，更重要的資料通常是：

DEX\

→ Factory\

→ Pair / Pool Address\

→ Token Pair\

→ Fee\

→ Reserve / Liquidity\

→ Price

Router 則主要幫助重建：

一筆交易是如何從一個 Pool 移動到下一個 Pool。

笔记 003: Day 12 | 统计套利、CEX-DEX 价差与做市执行算法

作者：fangliaoan | 时间：2026-08-15 16:51 UTC | 字数：495 | 评分：55

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 | 统计套利、CEX-DEX 价差与做市执行算法

今日学习

今天补了三块之前只点了标题、没吃透的内容。

一是统计套利：相比单标的择时，配对交易靠“两个高相关标的价差均值回归”盈利——先做协整检验，价差偏离 z-score 阈值时开仓、回归时平仓。核心不是预测涨跌，而是赌“价差会回归”，盈亏比由偏离度和波动率共同决定。

二是 CEX-DEX 价差的形成机制：DEX 的 AMM 靠恒定乘积定价，大单会瞬间放大滑点；CEX 有订单簿深度，两者价差本质是“流动性分布不均”。价差套利还要盯资金费率与提币时间，收益常被 gas 和提币延迟吃掉。

三是做市执行算法：TWAP/VWAP 把大单拆小、降低市场冲击，但暴露时间越长，信息泄露和逆向选择风险越高，要在“冲击成本 vs 时间风险”之间找平衡。

关键认知

价差是流动性的价格；套利收益 = 价差 - 摩擦成本（gas、滑点、提币延迟、时间风险），摩擦才是决定策略生死的那条线。

下一步

用历史数据回测一组 AMM 池的价差分布，量化出 gas 阈值：价差超过多少时套利才真正是正期望。

笔记 004: 骨架里做了三个和常规写法不同的决定

作者：Oxmigueleth | 时间：2026-08-16 03:37 UTC | 字数：1098 | 评分：74

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

骨架里做了三个和常规写法不同的决定

1. tick math 是从 Solidity 逐行移植的整数运算，不是浮点近似。\\

这是我前面警告过的那个坑，所以直接给了完整实现——computeSwapStep、getSqrtRatioAtTick 的 19 个魔数常量、tick 穿越循环全都在。quote_exact_input 在 tick 数据不够时会抛异常而不是返回近似值，因为“我加载的 tick 范围覆盖不了这个交易量”是个真实信号，不该被吞掉。

2. Level 2 的延迟惩罚内置在主循环里，不是后续步骤。\\

replay_block 同时接收区块 N 和 N+1 的池状态，同一个 size 报价两次，差值就是延迟一个区块的代价。这样做的理由是：让你永远看不到孤立的 Level 1 数字。单独看 Level 1 的结果最容易产生错觉。

3. builder_refund_wei 可为 NULL，且有独立的 refund_known 标志位；net_profit 不落库，只做 view。\\

这直接对应报告①那个 0.5366 ETH 的陷阱——refund 未知时如果把净利算成 -0.5366，整个数据集就废了。用 view 计算意味着未知永远不会被静默当成 0。

你要填的只有一处

StateSource.fetch() 是唯一的 NotImplementedError。docstring 里写了调用方案：slot0() + liquidity() + TickLens.getPopulatedTicksInWord()，全部打包进一次 Multicall3。

这一点是实操上的生死线：每池每块发 3 个 eth_call 的话，1000 个区块的重放要跑几小时，而且大概率被托管节点限流。另外一定要缓存到本地文件——你会因为改搜索逻辑而反复重跑同一批区块，重复取数是这个阶段最大的时间黑洞。

建议的第一次运行

先只选 5 个池、跑 100 个区块，确认管道通了再放大。然后看两个数：

v_net_conservative 里 net_wei > 0 的条数 —— 机会在数据上存在吗

notes 字段里 delayed_gross_wei 为负的比例 —— 这些机会里有多少不属于你

第二个数如果超过 80%，这条链可以直接放弃，省下后面三级的全部时间。

笔记 005: Day 11 打卡 | 鏈上套利殘酷共學

作者：roy328line | 时间：2026-08-16 06:27 UTC | 字数：872 | 评分：75

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 打卡 | 鏈上套利殘酷共學

今日實作：兩階段掃描架構

階段 1 (零 eth_call，快速篩選)

amm_out 估算 → implied_ratio / depth / tick_aware 過濾

輸出：候選清單

階段 2 (Quoter 精確驗證)

v2-only 路徑 → 直接標記 go_real=True (誤差 < 0.4%)

含 v3 腿的路徑 → eth_call Quoter 驗證 → net_real_usd

輸出：go_real=True 的確認清單

新增三個函式 (tri_scanner.py)：

- v2_quote_onchain()：UniswapV2Router02.getAmountsOut

- v3_quote_onchain()：UniswapV3Quoter.quoteExactInputSingle

- verify_onchain()：逐腿串接驗證，回傳 net_real_usd + go_real

掃描結果

追蹤池數：3,050

三角路徑數：7,996

第一階段通過 (OBSERVE)：226

Quoter 確認後真實機會：0

226 個幻覺被正確隔離 (最高 net_star=\$186,450，Quoter 驗證後全是負 EV)。

幻覺全是含 v3 腿的穩定幣三角路徑 (WETH→USDC→USDT)。

今日最重要的一件事

「0 個機會」是正確的答案，不是失敗。

鏈上套利機會以毫秒計，大部分時段看起來就是空的。

好消息：過濾系統正確運作，沒有假陽性進入決策層。

Day 12 方向

- 方案 A : 降低 MIN_POOL_TVL 從 \$50k 到 \$10k, 加入 WBTC/UNI, 看是否有被門檻排除的真實機會
- 方案 B : 對 OBSERVE 的 226 條也跑 Quoter, 統計 amm_out vs Quoter 的平均偏差, 建立校正公式 $net_real = net_star \times correction_factor$

笔记 006: Deterministic Deployment (合约确定性部署)

作者 : x-tan-dev | 时间 : 2026-08-16 06:49 UTC | 字数 : 6609 | 评分 : 89

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Deterministic Deployment (合约确定性部署)

普通 CREATE 部署合约 : 合约地址 = hash (部署者地址 + nonce) nonce 自增, 同一部署者多次部署相同代码, 地址不一样。

CREATE2 实现确定性部署 : 地址 = hash (0xFF + 部署工厂地址 + salt + bytecode hash) 相同 bytecode + 相同 salt → 永远算出同一个合约地址 特点 :

1. 部署前预先算出合约地址 ;
2. 可以先向这个地址转账, 之后再部署合约 ;
3. 在任意链上, 用同一套参数部署, 地址一致。

通俗讲 : 代码不变、盐不变, 部署到哪里地址都固定。 代表项目 : Deterministic Deployment Proxy (通用 CREATE2 部署代理)

\nFactory 先排序 token, 并在 getPair 的正向和反向映射中登记同一个 Pair, 因此 (A, B) 和 (B, A) 都能查到它。createPair 本身不转入资产 ; 资产是在之后添加流动性时, 由用户或 Router 转入 Pair, mint 再根据 Pair 的 balance - reserve 计算本次存入量。 \n

getPair[token0][token1] = pair; getPair[token1][token0] = pair; \n

Factory 在两个方向登记同一个 Pair 地址, 后续能够通过任意 token 顺序查到 Pair \n

Factory “不保管 token, 也不执行 Swap”; 资产是在后续添加流动性时, 由用户或 Router 转入 Pair。V2 的 mint 再通过 :

amount0 = balance0 - reserve0

amount1 = balance1 - reserve1

推导本次实际存入的数量。

另外, 双向映射本身不是“保证不能创建两个 Pair”的唯一机制。直接防止重复创建的是 :

require(getPair[token0][token1] == address(0), 'UniswapV2: PAIR_EXISTS');

这发生在排序之后, 因此同一组 token 无论以哪种参数顺序传入, 都会命中同一个检查。 \n[UniswapV2Factory.sol:23-37]

\nFactory 先将两个 token 按地址排序, 把 (A, B) 和 (B, A) 规范化为同一个 (token0, token1)。这样重复检查只需要检查 getPair[token0][token1], salt 也由同一个有序 token 对生成。CREATE2 再根据 Factory 地址、creation code hash 和 salt 确定 Pair 地址。不同 token 和零地址检查则是额外的输入校验。

这里还要区分两个概念 :

排序 : 解决 token 顺序不一致, 建立唯一的 token 对表示。

CREATE2 : 让给定 Factory、creation code 和 salt 下的部署地址可确定。

重复检查 : 阻止已经登记过的 token 对再次创建。

三者共同完成“一个 token 对对应一个官方 Pair”的路径。 \n

如果只登记 getPair[token0][token1], 使用 (token0, token1) 查询的路径可以找到 Pair, 但使用 (token1, token0) 查询的 Router 或用户会得到零地址, 导致反向交易路径无法构造或交易失败。因此 Factory 同时登记两个方向, 两个 key 指向同一个 Pair。

uniswapV2原生支持闪电贷, 是通过反推amount实现的。 \n

用户输入不是由 amountIn 参数直接决定, 而是由 callback 后的最终 balance 与旧 reserve 推导 ; Pair 再用扣除 0.3% fee 的 adjusted balance 检查 K invariant, 成功后才 _update reserve。

以题目中的 token0 输入为例 :

初始 reserve0 = 1,000

初始 reserve1 = 2,000

用户先转入 100 token0

swap 转出约 181 token1

swap 内的顺序是 :

1. 读取旧快照 :

```
_reserve0 = 1,000
_reserve1 = 2,000
```

1. 先乐观转出 amount1Out token1。此时 Pair 的实际余额暂时变成：

```
balance0 = 1,100
balance1 = 2,000 - 181 = 1,819
```

1. 如果 data 非空，执行 callback。callback 可以完成偿还或支付输入。

1) callback 后再次读取最终余额。这里最终大致是：

```
balance0 = 1,100
balance1 = 1,819
```

1. 用最终 balance 和旧 reserve 推导真实输入：

```
amount0In = balance0 - (_reserve0 - amount0Out)
= 1,100 - 1,000
= 100
```

1. fee 不是先从用户余额中单独扣走，而是通过调整值体现：

```
balance0Adjusted = balance0 1000 - amount0In 3;
balance1Adjusted = balance1 1000 - amount1In 3;
```

也就是 0.3% 的输入费会进入 fee-adjusted K 检查。

1. 只有满足：

```
balance0Adjusted balance1Adjusted
= reserve0 reserve1 1000^2
```

交易才成功，之后才调用 _update(balance0, balance1, ...)，把最终余额写成新的 reserve。

这里要特别区分：

balance：ERC-20 当前余额，swap 中会先暂时减少，再在 callback 后读取最终值

reserve：Pair storage 中上一次 _update() 保存的快照，成功检查后才更新

\

ERC-20 的 transfer 只修改 Token 合约自己的余额，并不会回调 Pair，因此 Pair 不知道这笔转账代表 swap input、liquidity deposit 还是 donation。Pair 的 reserve 是自己 storage 中保存的 accounting snapshot，只有 Pair 执行到 _update 路径时才会更新。

直接转账后可能出现：

```
token.balanceOf(pair) = 1,050
reserve0 = 1,000
```

这两个值暂时可以不同。调用 sync() 会把 reserve 更新到当前 balance；但 mint、burn、成功的 swap 等正常 Pair 路径也会触发 reserve 更新，不是所有场景都要求用户直接调用 _update。

首次加池没有现有 reserve 和 LP supply，所以用 sqrt(amount0 amount1) 建立初始流动性，并锁定 1,000 个最小 LP 单位。后续加池已有价格比例和 LP supply，因此按两边各自对应的份额计算，并取 min，避免不同比例的存入资产获得超额 LP。多出来的另一种 token 不会凭空获得更多 LP，直接调用 Pair 时还可能变成 donation。

V2 swap 的特征是：

1. Pair 先转出 output。
2. 可选 callback 执行。
3. Pair 读取最终 balance。
4. 从最终 balance 反推真实输入。
5. 用 fee-adjusted balance 检查 K invariant。

\

如果只检查：

```
amountOut < reserve
```

用户可能只拿走少量 output，却没有支付足够的 input，或者 callback 后通过某种路径让最终余额不足。这个检查无法确认交易结束时池子的资产状态仍满足不变量。

最终 balance 检查解决了两个问题：

它确认实际进入 Pair 的 token 数量，而不是盲信调用者或 Router 提供的输入参数。

它把手续费和实际余额变化纳入 K invariant：

```
balance0Adjusted.mul(balance1Adjusted)
>= uint(_reserve0).mul(_reserve1).mul(10002)
```

所以 $\text{amountOut} < \text{reserve}$ 是一个必要的流动性边界检查，但不是完整的结算安全检查。真正裁决交易是否成功的是 `callback` 后的最终 `balance` 和 `fee-adjusted K` 检查。

如果用户在 `swap` 前已经转入 100 token0，Pair 转出 181 token1 后，为什么 `amountIn` 要用最终 `balance0` 减去旧的 `reserve0 - amount0Out` 来计算，而不能直接相信用户声称的输入数量？

先把公式摆出来（UniswapV2 Pair swap 源码原版）

```
// 交易前储备
uint112 _reserve0 = reserve0;
uint112 _reserve1 = reserve1;

// 转出代币
if (amount0Out > 0) _safeTransfer(token0, to, amount0Out);
if (amount1Out > 0) _safeTransfer(token1, to, amount1Out);

// 触发回调（闪电贷窗口期）
if (data.length > 0) IUniswapV2Callee(to).uniswapV2Call(...);

// 读取转账、回调完成后的最新余额
uint balance0 = IERC20(token0).balanceOf(address(this));
uint balance1 = IERC20(token1).balanceOf(address(this));

// 计算实际流入数量
uint amount0In = balance0 > _reserve0 - amount0Out ? balance0 - (_reserve0 - amount0Out) : 0;
uint amount1In = balance1 > _reserve1 - amount1Out ? balance1 - (_reserve1 - amount1Out) : 0;
```

1. 先理解 $\text{_reserve0} - \text{amount0Out}$ 是什么

`_reserve0`：swap 执行之前池子记录的 token0 储备

`amount0Out`：本次 swap 向外转出的 token0 数量

假设本次没有任何人转入 token0：池子原有 `_reserve0`，转出 `amount0Out` 理论余额应该 $= \text{_reserve0} - \text{amount0Out}$
 $\text{balance0} - (\text{_reserve0} - \text{amount0Out}) =$ 本次整个 swap 生命周期内，所有外部流入的 token0 总和。

2. 核心问题：为什么不能让调用者传入 `amountIn`？

三大致命原因，层层递进：

① 资金流动不只发生在 swap 调用之前，回调期间也能转账（闪电贷场景）

时序回顾：

1. Pair 先转出代币给 to
2. 执行 `uniswapV2Call` 回调（闪电贷窗口期）
3. 回调合约内部可以自由转账代币到 Pair！

举两种场景：

普通交易：用户在 swap 调用之前预先转 token0 进来

闪电贷：合约在 `uniswapV2Call` 回调内部归还代币

如果依靠「用户调用 swap 时传入的 `amountIn`」，只能统计 swap 执行之前转入的币；

回调阶段还进来的代币完全统计不到，计算失效。

这是最根本设计动因。

② ERC20 转账是被动入账，合约无法追踪是谁转的

任何人，任何时机，都可以主动 `transfer` 代币到 Pair 合约。

在 swap 交易打包上链前、或者回调内部，额外往 Pair 砸一笔代币。

合约不能信任调用者口头宣称“我只转入 100”，只能靠事后扫描最终余额算出净增量。

③ Pair 底层 swap 接口本来就没有 `amountIn` 参数

底层接口：

```
swap(uint amount0Out, uint amount1Out, address to, bytes calldata data)
```

协议设计之初就决定：只约定取出多少，不约定输入多少。输入是结果，不是入参。

3. 结合你举的例子推演

场景：用户预先转入 100 token0，swap 要求转出 181 token1 ($\text{amount1Out}=181$ ， $\text{amount0Out}=0$) $\text{amount0Out} = 0$ 基准余额 $= \text{_reserve0} - 0 = \text{_reserve0}$

最终 $\text{amount0In} = \text{balance0} - \text{_reserve0}$ 正好算出用户转入的100。

拓展危险场景：攻击者预先转100，同时在回调里又转50。
余额相减方式自动捕获全部流入。

如果依靠用户申报的 $\text{amountIn}=100$ ，会漏掉50；

4. 紧接着关联恒定乘积校验逻辑

算出真实 $\text{amount0In} / \text{amount1In}$ 之后，内部校验手续费规则：

流入代币扣除0.3%手续费后，必须满足新储备乘积 $\geq$ 旧乘积

```
uint balance0Adjusted = balance0.mul(1000).sub(amount0In.mul(3));
```

```
uint balance1Adjusted = balance1.mul(1000).sub(amount1In.mul(3));
```

```
require(balance0Adjusted.mul(balance1Adjusted) >= uint(_reserve0).mul(_reserve1).mul(10002), 'UniswapV2: K');
```

整个校验完全基于真实余额推导的流入量，不信任任何外部传入数值。

5. 一句话高度总结

不能相信用户申报的输入数量，是因为：代币不仅可以在swap之前转入，还能在uniswapV2Call闪电贷回调期间转入；
合约只能观测交易执行完毕后的最终余额，反向算出整个生命周期内真实净流入代币，才能正确扣手续费、校验k值。

笔记 007: 这周太忙了，没太多共学的进度，列一下本周事项和下周计划：

作者：allen123wzh | 时间：2026-08-16 07:03 UTC | 字数：375 | 评分：49

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

这周太忙了，没太多共学的进度，列一下本周事项和下周计划：

本周没做完的：

1. 自动化交易程序，尤其在ve33场景下检测到大额卖出立刻dump
2. 流动性查询网站，输入pool address，wallet address，追踪pool在每个tick的流动性和特定地址的流动性
3. Robinhood链上交易量大的美股的LP APR跟踪，以及和美股short duration options的收益率比较

下周要做的：

1. 补全上面的进度
2. 学习Uni V3的系列博客，很多年前看过但半懂不懂，要借助AI搞清楚：<https://atise.medium.com/liquidity-provider-strategies-for-uniswap-v3-table-of-contents-64725c6c0b10>

笔记 008: Day 12：加入真实成本模型

作者：edwar-glowing | 时间：2026-08-16 09:41 UTC | 字数：268 | 评分：45

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12：加入真实成本模型

Action

加入：

Gas 估算

Gas Token 的美元价格

协议费是否已包含在 Quote

滑点缓冲

安全利润阈值

输出：

gross_profitestimated_gas_costslippage_bufferrisk_buffernet_profit当天产物

12_cost_model.py

[图片]

笔记 009: 今天延續前次對 Uniswap Universal Router 執行層的研究，進一步查看 Open

作者：tn606024 | 时间：2026-08-16 11:16 UTC | 字数：923 | 评分：64

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天延續前次對 Uniswap Universal Router 執行層的研究，進一步查看 OpenZeppelin Contracts
的專案結構、版本發布方式與安全說明。新的體會是：要把套利路徑做成可執行原型，風險不只在 swap calldata
或報價本身，依賴版本、權限設計與外部呼叫邊界也會直接影響結果是否可重現、可驗證。

目前整理出幾個重點：

1. OpenZeppelin 的 latest、dev、next 分別代表已審計穩定版、功能完成但尚未審計的版本，以及預發布版本。研究原型可以嘗試新功能，但正式執行器應優先使用已審計版本。
2. Foundry 依賴不應直接跟隨 master，而應固定 tag 或 commit。否則之後重跑 mainnet fork
時，依賴內容可能已改變，模擬結果也失去可重現性。
3. 使用 OpenZeppelin 並不等於完成安全審查。套利執行器自己的業務邏輯——Router/Token
allowlist、最大授權額、minOut、deadline、minProfit 與失敗策略——仍然必須獨立驗證。
4. 收益判斷應以交易前後的實際餘額差為準，而不是 Router 回傳值或鏈下報價。若最終淨收益低於門檻，整筆交易就應回滾。
5. 執行層應盡量縮小外部呼叫與授權範圍，並將「誰能修改設定」和「誰能觸發交易」分開，避免單一權限同時控制路由、資產與提領。

下一步準備：

先定義最小套利執行器介面：execute、router/token allowlist、approval 管理、withdraw 與 pause。

補上未授權呼叫、惡意 Router/Token callback、partial failure、殘留資產、minProfit 不成立等測試。

固定 OpenZeppelin 與 Universal Router 的依賴版本，再到 mainnet fork 驗證完整交易與回滾條件。

今天最大的收穫是：套利執行器不只是「把路徑送上鏈」的工具；依賴版本、權限邊界、實際餘額核對與失敗回滾，本身就是策略能否安全捕獲 Edge 的一部分。

02 LI.FI与跨链 （17 篇）

笔记 010: 1. 研究目标与科学定义

作者：XCHA000 | 时间：2026-08-15 15:01 UTC | 字数：2670 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1. 研究目标与科学定义

目标：建立一套可复现的链上套利执行系统，从机会识别到闭环验证，全程可量化成本与风险。

科学定义（第一性原理）：

套利是在给定区块链状态约束下，对同一经济价值进行一系列可验证状态转换，使终点资产净值在扣除全部显性与隐性成本后严格大于起点资产净值的过程。

核心判据（必须满足）：

$$\mathbb{E}[\text{Profit}] = P_{\text{success}} \times \text{Profit} - P_{\text{fail}} \times \text{Loss} - C_{\text{capital}} - C_{\text{opportunity}} > 0$$

任何未通过此判据的价差均视为无效机会。

2. 系统架构（模块化设计）

完整系统拆分为六个独立、可测试模块：

| 模块 | 功能 | 关键输出 | 失败模式 |

| :----- | :----- | :----- | :----- |

| 机会发现 | 扫描价差与异常状态 | 候选路径列表 | 假阳性过高 |

| 市场数据与报价 | 获取真实规模 Quote/Route | 有限规模执行价格 | 静态报价偏差 |

| 标准化与判断 | 对齐资产、decimals、方向 | 可比较的闭环描述 | 资产不可比 |

| 构建执行 | 生成交易数据 | 待签名交易 | 路径不可执行 |

| 钱包签名与控制 | 安全签名与权限管理 | 已签名交易 | 私钥泄露 |

| 广播确认与复盘 | 提交、跟踪、归因 | 实际 P&L 与日志 | 状态不一致 |

每个模块必须可独立测试，输入输出格式标准化。

3. 核心工具栈与参数配置

3.1 数据与路由层（最高优先级）

LI.FI API

功能：同链/跨链 Quote、Route、交易构建、状态跟踪

覆盖：70+链，200万+ Token，20+桥

限流：默认100 RPM

费用：查询免费，交易默认25 bps（可协商）

关键优势：大额自动拆分、跨链状态统一跟踪

注册与测试：Partner Portal → Integration → API Key → /v1/keys/test

Binance Web3 API

补充行情、Token分析、钱包余额、交易模拟

用于交叉验证LI.FI报价

3.2 RPC与执行层

托管RPC：QuickNode / Alchemy / Infura（起步）

生产要求：≥2个独立供应商，监控延迟、错误率、区块高度

私有通道：Flashbots Protect、MEV Blocker（降低抢跑）

公共RPC仅限测试，正式环境禁用

3.3 安全签名层（红线）

架构强制要求：业务逻辑与私钥物理隔离

推荐模式：业务程序生成待签交易 → 独立Signer机器决定是否签名

热钱包原则：金额上限设定为可承受全部损失的水平

额外控制：单笔限额、日累计限额、目标合约白名单、定期revoke allowance

4. 标准操作流程（SOP）

阶段1：环境搭建（可复现清单）

1. 注册LI.FI并获取API Key
2. 注册Binance Web3 API
3. 准备独立测试热钱包（小额）
4. 配置至少1个托管RPC
5. 实现独立Signer（或使用开源方案）
6. 完成一次端到端小额测试交易并记录全部数据

阶段2：机会过滤算法 对每个候选路径强制执行：

1. 资产可比性检查
2. 真实规模顺序报价（禁止静态相乘）
3. 全成本核算（滑点 + 协议费 + Gas + 优先费 + 跨链费 + 库存成本 + 失败预期）
4. 原子性判断
5. 最坏情况损失评估

只有全部通过才进入模拟阶段。

阶段3：小额验证协议

初始规模：系统可承受损失的1-5%

记录指标：实际到账、实际费用、执行延迟、失败原因、库存变化

至少连续成功N次（建议≥10次）且期望利润稳定为正后，才允许扩容

5. 量化成本与风险模型

必须计入的成本项：

价格冲击（有限规模）

协议/桥费用

Gas + 优先费/Bid

跨链确认时间成本

库存持有风险（波动率 × 时间）

失败概率 × 失败损失

资金机会成本

风险控制阈值：

单笔最大损失 ≤ 总资本的X%

日累计敞口上限

非原子路径必须预置对冲库存

任何allowance异常立即熔断

6. 常见失效模式与规避（实证研究）

1. 静态小额报价幻觉：真实规模下利润归零或为负 → 强制顺序有限规模计算
2. 私钥与业务耦合：脚本泄露导致归零 → 强制独立Signer
3. 公共RPC依赖：延迟与抢跑 → 生产环境使用私有通道
4. 过早追求低延迟硬件：机会本身不存在 → 先验证正期望闭环，再优化速度
5. 忽略失败分布：只模拟成功路径 → 必须包含压力测试与最坏情况

7. 验证与迭代科学方法

每次执行后强制复盘：预期 vs 实际差异因

建立日志数据库：路径、规模、成本明细、成功/失败、时间戳

定期回测历史机会，计算真实夏普或期望利润分布

用AI Agent自动化「扫描-模拟-归因」循环，人工只负责策略判断与风控参数

8. 最小可行行动清单（立即执行）

1. 完成LI.FI API Key申请与测试调用
2. 部署独立小额热钱包 + 基础Signer流程
3. 编写最小脚本：指定交易对真实规模报价 + 全成本计算
4. 完成一次端到端小额闭环并完整记录数据
5. 建立失败场景检查表

出去旅游了两天，我靠，全部忘记了，群里的消息落了一大堆

笔记 011: 学习了这篇笔记，来自x睡眠不足会死:

作者：colinzhong1024 | 时间：2026-08-15 15:25 UTC | 字数：1488 | 评分：64

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习了这篇笔记，来自x睡眠不足会死:

很多人都在说套利 套利种类特别多 一定得分清楚

1.费率，这个不用多说，买现货做空合约，或者高级点就是河马这种没上bn的，去链上买cex做空，这种其实是一个非常低效的收入，特别是前一周都不一定能赚回开仓的滑点。

2.搬砖，分手动搬和代码搬，手动一般是新东西比如新上一个币然后有跨链桥两边有差价。代码一般是cex-dex搬，需要比较高的技术要求，而且碰到之前fantom的multi跨链暴雷这种情况容易被一波带走。

3.mev三角套利，这个学问就多了 也举几个例子

a)区块头套利，这种类似于base op blast unichain，都是opstack，通过上一个块的状态变化计算出路径利润，因为下一个区块是基于gas价格排序，所以这种一般是gas的比拼，比的是合约的优化，可能需要用汇编来写合约，通常需要上贡80%

90%利润才能拿到，需要在op-geth的sendtx那魔改一下，加个模拟功能来算具体上贡比例。

b)base上有一些高手，通过盲发来套利，就是他占满每个区块来搜索利润，这个具体实现我现在还没搞明白，这么多路径他是怎么做到盲发的。

c)zksync arb这两种为代表的是速度快就行，不需要卷gas所以这个对技术的要求特别高，也有可能和项目方关系好他给你最快的rpc 哈哈，发tx就和上面base一样，只不过不需要卷gas因为他不是按gas排序，这里贴一个我认为的终极套利大佬，能几年不动钱包里的e子的，真佩服。arbiscan.io/address/0xeebe...

d)back润，这个以eth bsc bera sonic为代表，就是需要监听pendingtx，通过模拟pendingtx来算利润，这个需要对geth有比较高的了解，而且不止是v2这种只需要模拟拿到logs就能back润，v3的模拟是比较麻烦的因为他会改变ticks的数值，具体怎么才能搞定还得自己多研究。

eth和bsc现在变成给节点打工了，他们都是通过发bundle，竞价来决定pos，不是很推荐继续玩这两条链，通常需要比较高的上贡比例，或者你是关系户，bera就是纯粹的速度，这里也贴出bera的榜一berascan.com/address/0x08B1...，至于sonic他就更麻烦了，sonic是通过你的back润hash和target的hash数值接近，这个可能还涉及到gpu计算方面了，门槛非常高，现在的sonic榜一应该都是在fantom上研究多年的，很难卷过他。

e)最后说一下协议，三角套利需要对每个能swap的协议都有涉猎，举个冷门例子之前zksync上的小牛协议，还有gmx的swap，1inch的挂单，常规的就是v2 v3 solid 神剑 balancer (beets beraswap) syncswap traderjoe univ4还有众多v3仿盘algebra kyber zyber izumi等，还有很多xxbnb-bnb xxusd-usdt xxeth-eth，通常有一些冷门的协议能先搞出来有个几天窗口期，通常都需要链下计算把众多协议的input output搞清楚通过三分法求凸函数极值。

以上就是大概科普了下最近很多人都在聊的套利 我只会evm不会sol sui trx等（因为懒+年纪太懒得去学rust了）

补充一下 这一行现在是极度内卷 不推荐啥进场了 因为你可能会被github上的仓库 骗走私钥资金 或者被卖代码的老师坑钱 也有可能浪费大量时间精力后最后也没赚到钱还倒亏gas

笔记 012: 净利 = 毛价差 - 池子手续费 - gas - 桥费 - 价格影响 - 时间风险 - MEV 抢跑

作者: Freetrip | 时间: 2026-08-15 15:44 UTC | 字数: 354 | 评分: 53

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

净利 = 毛价差 - 池子手续费 - gas - 桥费 - 价格影响 - 时间风险 - MEV 抢跑

交易进内存池后是公开的, 搜索者看到有利可图就抢在前面或把夹中间。LI.FI 没有任何字段能告诉会被夹多少——设的 slippage 容忍度就是损失上限: 设 3% 就等于告诉搜索者「最多可以吃我 3%」。防御手段是套利玩家的军备, 不需要建, 但要知道这项在成本表里必须占一行, 哪怕填的是「上限=容忍度」。

时间风险——executionDuration 只告诉敞口开多久, 损失多大取决于这段时间里价格的波动率。正确建模: 时间风险 $\approx$ 波动率 $\sigma \times \sqrt{t}$ (敞口时长)。同样 3 秒, 套 USDT/USDC 稳定对 $\approx$ 无风险, 套迷因币可能已经 $\pm 2\%$ 。所以没有「桥快不快」的绝对答案, 只有「相对标的波动率快不快」。

笔记 013: 200u 套利

作者: Twooweeks | 时间: 2026-08-15 15:53 UTC | 字数: 2517 | 评分: 80

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

200u 套利

子 agent 用真实数据验证 不是拍脑袋

Bybit API 查资金费率 BTC +0.0044%/8h ETH +0.0074%

RPC 查 gas ETH 主网 54 gwei 约 \$20/笔

一轮套利 4 笔 = \$80 占 200u 的 40%

break-even 价差 20%+ 市场根本不存在

结论 五个策略四个纯亏 一个无意义

CEX-DEX 价差 主网每轮 -\$40~\$70

L2 -\$1 ~ +\$0.5

跨链 LI.FI 成本 0.7-1.0% vs 价差 <0.15%

资金费率套利 唯一可行 月 +\$0.7~1.3

清算监控 赌博不是套利

还有一个真实信号 BTC 永续对现货折价 -0.04%

空头拥挤 资金费率套利方向现在反着

200u 买经验 不是赚钱

练工具 0 成本

L2 上 5-10u/笔 跑通 CEX→桥→DEX→回 CEX

Bybit testnet 纸面交易

~\$150 放稳定币生息防手痒

套利基建五层 数据→信号→执行→风控→运维

数据层 CEX WebSocket + 链上读池子 已有 pool_viewer

信号层 价差公式 break-even 阈值

执行层 ccxt 下单 + web3 swap + LI.FI 跨链

风控层 滑点检查 单笔限额 记账

运维层 阿里云 7x24 + 通知

笔记 014: 今天查阅了 LI.FI 官方 Composer 文档。Composer 不是另一套独立的报价接口: 对

作者: jason52012345-bot | 时间: 2026-08-16 02:38 UTC | 字数: 464 | 评分: 49

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天查阅了 [LI.FI](#) 官方 [Composer](#) 文档。Composer 不是另一套独立的报价接口：对支持的协议，只需把目标金库、质押或存款凭证代币的地址作为 `toToken`，仍可通过标准的 `GET /v1/quote`（或多路线的 `POST /v1/advanced/routes`）请求路径。这样同链兑换或跨链转移后的存入动作可被组织为一个流程。

这也提醒我，目标资产不能只按 `symbol` 判断；在发起前应先从官方支持的协议与链清单确认该金库和链受支持，再使用其准确合约地址。对于套利或资金调度，报价中的“目标代币”可能代表协议仓位，而非普通可自由转账的币，策略评估要把这一点纳入退出路径设计。

官方同时列出边界：Composer 当前仅支持 EVM 链、仅适用于存入后会返回代币化仓位的协议；部分协议只支持存入，赎回需使用协议自身界面。因此不能因一次报价可得就假定仓位可由同一流程无条件退出；应在执行前分别核验 `isTransactional`、`isRedeemable` 等能力及协议的实际限制。

笔记 015: https://github.com/ZeroNullNone/onchain_arbitrage_

作者：Nothing | 时间：2026-08-16 03:09 UTC | 字数：1862 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

https://github.com/ZeroNullNone/onchain_arbitrage_study/blob/main/docs/daily/day_12.md

Day 12 — Token Identity 与 Basis Risk

状态与结论

- 状态：Day 12 当前 Universe 的 acceptance scope 已完成。
- Token identity 强制使用 `chain_id + contract_address`；Address 比较不区分大小写，Registry 内统一保存为小写。
- `symbol` 只用于显示和返回候选列表，不能解析唯一 Token，也不能证明经济等价。
- 同为 WETH 的 Base Token 被记录为本链 Wrapped WETH9，而 Arbitrum Token 被记录为依赖 Canonical Bridge 和 Upgradeable Proxy 的 Bridged Token。

完成项

- 新增严格 TOML Registry，覆盖 Day 10 冻结的 Base / Arbitrum USDC、WETH 四个 Token。
- 每个 Token 显式记录 Chain ID、Contract Address、Symbol、Decimals、Issuer、Canonical/Bridged/Wrapped、Redemption Path、Pause、Blacklist、Upgradeability、Haircut、Exclude Decision、Decision Reason 和官方/合约 Evidence URL。
- USDC 的 Pause、Blacklist、Upgradeability 能力按 Circle EVM FiatToken 设计显式标记；不能把 Native USDC 当成无 Issuer Control 的无风险现金。
- 所有 Token 当前均保留在 Paper Universe；25 / 5 / 50 bps 是冻结的 Paper Policy Haircut，不是 Live Price、Expected Loss 或可静默计入 PnL 的 Cost。
- Loader 不提供 Default、Fallback 或 Guess；缺字段、未知字段、非法风险类型、非法 Haircut、重复 Identity 和 Metadata 不一致都会显式失败。

Evidence 与验证

- Registry：config/token_registry.toml。
- Identity、Lookup、Schema 与 Validation：src/onchain_arb/token_registry.py。
- 风险表、Redemption / Basis Dependency、Evidence Links：docs/day12_token_risk.md。
- Acceptance Tests：tests/test_token_registry.py，覆盖同 Symbol 不等价、Case-insensitive Address、Duplicate Identity、Missing Risk Field、Invalid Classification/Haircut 以及 Frozen Universe Coverage。

Gate 边界

Registry 只覆盖当前 Universe，不扩张为全市场 Token Database。
Haircut 目前是风险标记；未来只有 Cost Ledger 明确定义应用语义后才能影响 PnL。
不实施实时 Metadata Monitor、Issuer Event Alert、信用评分或法律意见。
本日没有 RPC Request、Wallet Access、签名或广播交易。

下一步

- Day 13 对一个 Same-chain Candidate 构建 Unsigned Transaction，并保存指定 Block State 下的 Simulation Evidence；没有 Simulation Evidence 不可标记为 Executable / Paper-ready。

笔记 016: 【Day 12 | 2026-08-16】把观察变成可重复流程

作者：OxChainWorld | 时间：2026-08-16 04:04 UTC | 字数：581 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 12 | 2026-08-16】把观察变成可重复流程

投入：70 分钟

学了：

\- 可重复 = 固定检查顺序 + 固定取数 + 固定判决口径

\- 清单省叙事，不省「每次重拉 quote」

做了：

\- 产出工件：跨链候选 5 分钟检查清单 + Agent 提示词模板

\- 用工件重跑 Case 01 (INT-001 / 15bps / \$5k / ≤60s)

\- 新证据：eco ~7s、Across ~10s，Fixed Fee ≈ \$12.49

结果：

\- 判决与 Day11 一致：可执行性否，主因 K1

\- 关键发现：路由缓存有毒；清单把长文压缩到可勾选重跑

\- 体感耗时：完整案例 90min+ → 清单重跑约 30min

失败/不确定：

\- Q1 盘口仍未自动化

\- 尚未定时脚本化（清单已算可重复工件）

证据：

\- 03-repeatable-workflow\md

\- agent/prompts/check-crosschain-candidate.md

\- evidence/d12-rerun-default-5k-eth-arb.json

\- evidence/d12-rerun-across-5k-eth-arb.json

净收益判断：不成立（重跑确认）

笔记 017: D13 数据采集：LI.FI 报价采集脚本（同链 + 跨链跑通）

作者：peanutwen00 | 时间：2026-08-16 04:05 UTC | 字数：873 | 评分：74

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

D13 数据采集：LI.FI 报价采集脚本（同链 + 跨链跑通）

1) D13 目标：接入 LI.FI quote API，写第一个报价采集脚本（免 key 验证可行，chains/tokens/quote 全部 200）

2) 同链（Ethereum ETH→USDC）：LI.FI 聚合后最优路径是 Nordstern Finance（不是 Uniswap/Curve——聚合器一次问遍所有 DEX 选最优的实证）；0.1 ETH → 187.66 USDC；成本 = gas \$0.05 + LI.FI 平台费 \$0.47（0.25% = 25bps，正好对上成本模型门槛）

3) 跨链（Ethereum→Arbitrum USDC）：3 步路径 = feeCollection（收平台费）→ KyberSwap（主网 ETH→USDC）→ Across 桥（USDC→USDC Arb）；0.1 ETH → 187.66 USDC(Arb)；成本 = gas \$0.10 + 平台费 \$0.47 + 桥费 \$0.03

4) 踩坑①：decimals 没对齐——USDC 是 6 位小数，第一版按 18 位除输出 0.0000；修正后 187.66。Bruce 文章"标准化：对齐 Decimals"不是空话，真会踩

5) 踩坑②：LI.FI quote 的完整路径字段是 includedSteps，不是 route（route 是空数组）

6) 认知①："先换稳定币再过桥"是跨链标准做法——桥只承担稳定币风险敞口最小；跨链 vs 同链成本差 ≈ \$0.08 就是"抹平价差的成本门槛"（呼应 D5）

7) 认知②：quote 的 executionDuration 只算聚合器内部（显示 2ms），跨链桥实际要 1-10 分钟确认——纸上成本 ≠ 实际到账时间，这是跨链套利延迟风险的来源

8) 下一步：接入 The Graph（Aave 健康因子查询）+ LI.FI 带 key 版（100 RPM）跑通；脱锚监控雏形

笔记 018: 今天基本没有继续扩策略类型，主要做了一件事：

作者：blimnwind-bot | 时间：2026-08-16 04:33 UTC | 字数：3911 | 评分：89

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天基本没有继续扩策略类型，主要做了一件事：

把 Smart Money Engine 从“能分析钱包”推进到“能对真实钱包做证据闭环”。

今天最大的变化，是研究目标已经彻底从：

这个钱包是不是聪明钱？

变成：

这个钱包的钱到底是怎么赚出来的，而且我能不能用链上证据把账对上？

1. Evidence Pipeline 正式做到 5/5

目前完整链路已经跑通：

RawData→Price→Receipt/Logs→Reward→Inventory→Accounting

分别解决：

Price Layer

区分：

ExecutionPrice=MarkPrice=GasTokenPrice

不因为 symbol 是 USDT 就默认 \$1，也不会用外部价格覆盖更强的 execution evidence。

Receipt / Log Reconstruction

只有直接 Pool Swap Event 才能成为：

OBSERVED_SWAP

ERC20 transfer pair 只能保持：

INFERRED_SWAP

证据不够就：

UNRESOLVED_SWAP

Reward Ledger

正式区分：

RewardSignal=RewardAmountObserved=RewardValueKnown

claim 只能证明存在 reward signal，不能凭空制造奖励金额。

Inventory Ledger

资产身份只认：

ChainID+ContractAddress

Native MNT / ETH 和 WMNT / WETH 分开。

例如：

ETH→WETH

只属于 transformation，不属于 Trade，也不属于 External Flow。

Accounting Reconciliation

先做资产数量守恒，再做 USD 守恒：

ExpectedClosingAmount=?ObservedClosingAmount

之后再算：

EconomicPnL=ClosingNAVOpeningNAVExternalNetFlow

并且：

NetTradingContribution=TradingPnLGrossGasFees

Residual 不能为了把账做平被偷偷塞进 TradingPnL、RewardPnL 或 InventoryPnL。

2. 第一份真实钱包验收 RWAT 001 已经跑起来

没有去灌 Nansen 或其他 Smart Money 地址。

先用已经研究过的钱包 A：

Chain: Mantle

Window: 81995450 .. 82023400

Transactions: 22

这个窗口覆盖：

Funding→Execution→Claim/Exit→Settlement

目标不是证明它赚钱。

而是：

固定同一个钱包、同一个 block window，看系统能不能从 raw evidence 一路解释到最终账本。

3. 最开始出现了一个 \$56,402.65 的大洞

第一轮真实 reconciliation：

$\text{Residual} \approx \$56,402.65$

没有为了让测试 PASS 去改阈值。

而是继续往下拆。

最终定位成：

$\$56,402.65 \approx \$34,955.95 + \$21,445.03 + \1.67

分别是：

variableDebtvWMNT

约：

\$21,445.03

原来被当普通 ERC20 正资产。

实际它是负债。

于是改成：

LiabilityValue<0

并进一步拆成：

BORROW

REPAY

LIABILITY_ACCRUAL

债不能因为钱包里有一个 debt token balance 就给 NAV 加钱。

IUSDT

约：

\$1.67

属于 lending receipt token 的利息 accrual。

现在单独归入：

LENDING_INTEREST_ACCRUAL

不再污染 TradingPnL。

最后一笔 USDe

剩下：

34,955.950089 USDe

链上表现是：

0x0→Wallet

如果只看 Mantle，很容易认为：

凭空 mint 了 3.5 万 USDe。

最开始系统正确保持：

UNKNOWN_MINT / UNRESOLVED

没有把它猜成 Reward，也没有塞进 External Inflow。

4. 今天最重要的突破：把这笔 USDe 的 provenance 钉死了

继续溯源以后找到了：

Mantle destination tx：

0x15f808...e974

对应 Arbitrum source tx：

0xe4c1df...fafe

证据满足：

LayerZero/OFTsame_entity=trueamount_conserved=true

所以最终分类从：

UNKNOWN_MINT

升级成：

CROSS_CHAIN_SETTLEMENT

进一步得到经济语义：

CROSS_CHAIN_TRANSFORMATION

因为源链和目标链是同一个实体：

EconomicExternalFlow=0

这不是 Reward。

不是 Profit。

也不是外部给钱包打进来的新钱。

只是：

Arbitrum资产→Bridge→Mantle资产

资产换了链，但净财富没有因为跨链本身增加。

5. 最终 USD 账已经闭合

现在 RWAT 001 最新状态：

Status REVIEW

Asset gate REVIEW

USD gate PASS

USD residual

≈ \$1.76e-10

Primary

ARBITRAGE / NO_EVIDENCE

Smart Money

false

也就是说：

\$56,402.65

一路被解释成：

\$21,445.03 Liability\$1.67 LendingAccrual\$34,955.95 CrossChainTransformation

最后：

Residual≈0

这个结果比直接得到一个漂亮的 PASS 更有意义。

因为整个过程没有：

修改 threshold

把 residual 写回 PnL

把 mint 猜成 reward

把 debt 当资产

把 bridge 当 external income

6. 现在最关键的一点：账平了，不代表发现套利

当前仍然是：

ARBITRAGE

NO_EVIDENCE

而不是：

ARBITRAGE

SUPPORTED

这正是系统应有的行为。

因为：

AccountingPASS⇒ArbitrageSUPPORTED

账本只回答：

钱去了哪里。

策略层还要回答：

为什么赚，以及这种 Edge 是否可重复。

7. 下一步正式进入 Strategy Evidence Closure

接下来不再追 reconciliation。

而是逐个检查这个 22 笔窗口里的所谓 arbitrage cycle：

ObservedTradingLegs ↓ SameNumeraireClosedLoop ↓ GrossTradingPnL ↓ Gas+Fees ↓ NetTradingPnL ↓ Repeatability

并且把：

CROSS_CHAIN_TRANSFORMATION

BORROW

REPAY

LIABILITY_ACCRUAL

LENDING_INTEREST_ACCRUAL

全部从 Arbitrage Cycle 中剥离。

最终才判断：

ARBITRAGE / REJECTED

ARBITRAGE / CANDIDATE

还是：

ARBITRAGE / SUPPORTED

今日最大的认知

今天真正验证了一件事：

看到一个钱包资产增加，不等于发现利润。

看到：

0x0→Wallet

也不等于 Reward。

看到 debt token，也不等于资产。

看到一条闭环路径，也不等于套利。

所以 Strategy Reconstruction 的顺序应该始终是：

先证明资产发生了什么→再证明钱怎么变化→最后才证明Edge从哪里来

现在 Smart Money Engine 已经基本完成前两步。

下一步才真正开始回答最初的问题：

这个钱包，到底靠什么赚钱？

当前测试基线：135 tests OK。

笔记 019: Hermes Agent workflow

作者：hello16 | 时间：2026-08-16 11:59 UTC | 字数：1905 | 评分：87

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Hermes Agent workflow

目标

把「一次性观察」变成每天自动运行的扫描与信号 workflow，覆盖：

新协议

新池

大额交易

异常价格

最终输出结构化证据，而不是零散截图。

整体工作流设计

定时触发（每天）

↓

并行扫描 4 个维度

↓

过滤 + 初步判断

↓

生成结构化记录

↓

告警 / 存档 / 进入人工复盘

1. 每日自动扫描任务（Cron 示例）

在 Hermes 中设置定时任务（示例）：

bash

每天早上 8:00 运行

hermes cron add --schedule "0 8 * \

--prompt "执行今日跨链/DeFi 扫描工作流：新协议、新池、大额交易、异常价格。输出结构化报告。" \

--deliver telegram

或拆成多个更细粒度的 cron，降低单次上下文压力。

2. 四大扫描维度具体内容

A. 新协议（New Protocols）

扫描来源建议：

DefiLlama 新协议列表

Twitter/X 关键词（new protocol launch、主网、TVL）

链上工厂合约创建事件（部分链）

官方公告 / Discord / Telegram 频道

输出字段：

协议名称

支持链

上线时间

初步 TVL / 类型（DEX、Lending、Bridge…）

是否值得加入监控池

B. 新池（New Pools）

重点关注：

新创建的高流动性池（尤其稳定币对、主流资产对）

异常高 APR / 低流动性的新池（潜在机会或风险）

跨链相关的新流动性池

可用工具：DexScreener、GeckoTerminal、链上 PairCreated 事件、LI.FI 相关路由变化。

C. 大额交易（Large Transactions）

监控：

单笔超过阈值的 Swap / Bridge / Transfer（例如 > \$50k 或根据资产调整）

跨链大额桥接（LI.FI、Stargate、Across 等）

同一地址短时间内多笔相关操作

目的：发现可能的聪明钱动向或流动性转移信号。

D. 异常价格（Abnormal Prices）

核心逻辑对应之前的 Scanner：

Chain A price vs Chain B price

↓

difference

↓
cost
↓
EV

重点监控：

稳定币脱锚

同资产跨链价差明显扩大

主流资产在不同链上的可执行价差超过成本阈值

优先用 LI.FI /quote 获取真实可执行路径和费用，而不是单纯看中心化价格。

3. 推荐输出格式（每天固定）

markdown

每日扫描报告 - YYYY-MM-DD

1. 新协议

- [协议名] | 链 | 类型 | 初步判断 | 链接

2. 新池

- [池子] | 链 | 流动性 | APR | 备注

3. 大额交易

- 时间 | 链 | 资产 | 金额 | 方向 | 地址/协议

4. 异常价格 / 潜在机会

- 资产 | 路径 | 价差 | 预估成本 | EV | 是否值得进一步验证

5. 今日行动建议

- 需要人工深入的项目

- 可加入长期监控的标的

- 可忽略的噪音

4. Hermes 落地建议

1. 先做最小可行版本\

先只监控 1-2 个资产对 + 稳定币脱锚 + 新协议，跑通「扫描 → 记录 → 告警」闭环。

2. 把结果写入持久记忆/文件\

让 Hermes 把每天的报告存到本地或知识库，方便后续检索「上次类似情况怎么处理的」。

3. 加入简单过滤规则\

例如：

EV < 阈值 → 只记录不告警

新协议 TVL < 某值 → 降权

大额交易来自已知做市商 → 标记为噪音

4. 人工复盘环节保留\

Agent 负责「广撒网 + 结构化」，你负责「判断是否有真实 edge」。

笔记 020: Day 11：官方合约与权限验证

作者：Flxxx | 时间：2026-08-16 12:00 UTC | 字数：2279 | 评分：78

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11：官方合约与权限验证

今天继续用

ZRO

做案例，目标是把

Day

10

找到的官方候选合约进一步拆成网络身份、代码结构、控制权限、跨链关系和市场身份。最重要的规则是：关键字段缺失或冲突时 fail-closed，不能因为看到同名资产或价格差就继续计算套利。

首先补齐了常用字段的含义。多链资产必须用 chainId + contractAddress 标识；LayerZero 的 peers(...) 使用 EID，不是 EVM chainId。runtime bytecode 是合约部署后实际执行的链上机器代码；如果合约是 Proxy，还必须继续检查 Implementation 和升级权限。token() 则用于区分原生 OFT 与 OFT Adapter：原生 OFT 通常返回自身，Adapter 返回被锁定的底层 ERC-20。

本次在 Ethereum 区块 25,766,256 和 Base 区块 50,038,786 读取了 ZRO：

```
text
Ethereum chainId = 1
Base chainId = 8453
ZRO = 0x6985884c4392d348587b19cb9eaaf157f13271cd
endpoint() = 0x1a44076050125825900e736c501f859c50fe728c
sharedDecimals() = 6
decimalConversionRate() = 10^12
token() = ZRO 合约自身
```

两链 runtime bytecode 哈希不同，但这不能直接判定为身份冲突。Ethereum 已验证源码在构造函数中初始铸造 10 亿 ZRO，Base 版本没有这段初始铸造，符合“初始供应链 + 其他链通过 OFT burn/mint 流通”的模型。代码哈希不同只能作为差异警报，仍要检查源码中是否存在额外 mint、暂停、税、黑名单或升级入口。

权限层的实时结果是：

```
text
Ethereum ZRO owner = 0xCdA8...4C92
Base ZRO owner = 0x2893...cFF0
两边 Owner 都是 Safe 1.3.0
Safe Owners = 相同的五人集合
Threshold = 3/5
Enabled Modules = 0
Transaction Guard = 0
```

两链 Owner Safe 地址不同，但签名人集合和门槛一致，因此不能因为 Owner 地址不同就立即判冲突。两个 Safe 的 Singleton 地址不同，仍作为独立证据保存。

同时读取了 LayerZero Endpoint 的 Delegate：每条链的 ZRO Owner 与 Endpoint Delegate 都是该链同一个 Safe。这意味着同一组多签控制主体既能修改 OApp 自身的 Peer 等配置，也能修改 Endpoint 中消息库、DVN、Executor 等消息验证配置，管理方便但权限较集中。

今天还补充理解了 Safe 的权限边界。Token 的 Owner 是 Safe 合约，Safe 的 Owners 才是实际签名人；threshold = 3 只说明普通交易需要五人中的三人批准。已启用 Module 可以按自身代码走 execTransactionFromModule，Safe 1.3.0 的普通 Transaction Guard 不覆盖该路径，所以不能只看多签门槛，必须同时检查 Module、Guard 和其他扩展面。

跨链关系上，Peer 回答“信任哪个远端合约”，Endpoint 回答“通过什么消息系统收发”，DVN 和消息库负责验证。ZRO 没有公开 mint()，不代表不存在铸造控制面：通过验证且来源匹配 Peer 的消息会触发内部 _credit()，在目标链铸造。

市场身份目前仍未验证。DEX 页面显示的 ZRO/USDC 只是展示名称，后续必须读取池子的 token0()、token1()、Factory 和结算资产，并与当前 chainId 上的官方地址或已验证映射核对。若地址不同，要先判断是否为官方 Adapter、包装资产或桥接资产；关系未证明前停止计算。

当前状态：

```
text
assetIdentityStatus = ONCHAIN_IDENTITY_PARTIAL_ETHEREUM_BASE
ownerStatus = SAFE_3_OF_5_OBSERVED
endpointDelegateStatus = MATCHES_OWNER_AT_OBSERVED_BLOCK
marketIdentityStatus = NOT_CHECKED
opportunityStatus = NOT_EVALUATED
```

今天的产出包括 Day 11 任务、ZRO 身份与权限验证记录、Day 11 知识条目，以及一份 309 行的《常用合约字段与权限速查》，覆盖 ERC-20、Proxy、Safe、LayerZero、DEX 和 fail-closed 状态字段。

下一步 Day 12 将验证 Token Mapping、供应守恒、双向兑换、限额和库存恢复，并区分官方证据、链上证据与第三方映射的可信等级。

笔记 021: 链上套利共学 · Day12 打卡笔记 (08/16)

作者：haibian-slowly | 时间：2026-08-16 12:14 UTC | 字数：998 | 评分：90
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利共学 · Day12 打卡笔记 (08/16)

今日信息
日期：2026-08-16 (周日)
主题：模拟延迟与报价失效

核心问题：机会在多差的条件下会从盈利变成亏损？

打卡正文

【第 12 天】

1. 今天要验证的问题

报价拿到以后，如果市场价格继续变化，原本看起来赚钱的机会还能剩下多少利润？

2. 今天实际做了什么

今天先做了一个最小压力测试，没有写复杂代码。沿用 Day5 的实测成本：跨链稳定币路径总损失约 0.3216%。假设屏幕上的毛价差是 0.5%，则：

text

剩余净空间 = 0.5% - 0.3216% = 0.1784%

以 1000 USDC 为例，理想情况下只剩 1.784 USDC。只要等待期间报价恶化超过 0.1784%，这笔交易就会从盈利变成亏损。

3. 证据

使用 Day5 已采集的真实成本数据：到手率 99.6784%，总损失 0.3216%；

本次计算结果：1000 USDC、0.5% 毛价差，扣除成本后只剩 1.784 USDC；

压力测试临界点：报价恶化 0.1784%。

4. 得到了什么结论

1. 看见 0.5% 价差，不代表能赚 0.5%，大部分空间先被桥费、汇率差和 Gas 吃掉；

2. 跨链不是原子操作，等待时间越长，报价失效风险越大；

3. 对这种薄利润机会，执行速度本身就是成本；

4. 真正值得进入候选名单的机会，必须给延迟和失败留下足够安全垫。

5. 今天失败了什么

今天只完成了手工版压力测试，还没有把 0.2% / 0.5% / 1% 报价恶化、Gas 上升和单边失败等场景写成 stress_test.py。先把临界点想明白，后面再代码化。

6. 明天只解决什么

还原一个公开套利案例，重点看真实交易从发现到执行经历了哪些步骤。

一句话总结

0.5% 的屏幕价差，扣掉 0.3216% 的已知成本后只剩 0.1784% 安全垫；报价稍微变差就会亏，所以执行延迟不是小问题，而是决定机会是否成立的核心成本。

明日预告

Day13 (08/17)：还原一个公开套利案例

笔记 022: 【Day 16 打卡 · 2026-08-16】小额试跑执行与首轮复盘

作者：nerd614 | 时间：2026-08-16 12:31 UTC | 字数：429 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 16 打卡 · 2026-08-16】小额试跑执行与首轮复盘

延续「跟数据走」的节奏，今天按 Day 15 定下的试跑方案，落地一次极小金额的真实执行，验证成本模型与实际执行的一致性。

1. 试跑执行：在风控边界内（极小金额、单笔上限、单日次数限制）完成第一笔真实跨链/兑换操作，全程记录实际 Gas、桥费、滑点与到账时间。

2. 成本对账：把实测执行数据与模拟时的预期成本逐项对比，量化「报价 vs 实际」的偏差来源——是 Gas 波动、滑点估算不准，还是执行延迟导致价格移动。

3. 模型修正：把偏差回填进 Day 9 的成本模型，调整参数后再跑一遍净收益计算，看候选路径的边际是变厚还是变薄，据此修订剩余试跑的触发条件。

4. 首轮复盘：记录这笔试跑的经验与教训（成功点、失败点、下次改进），并决定是否继续下一笔试跑或先收敛迭代信号规则。

今天聚焦「用最小真金验证模型」——让成本模型从理论参数变成实测校准，为结课前的判断提供可信依据。

笔记 023: Day 11 · 2026-08-16 · 隔了一天回来对账 Day 10 的 Polygon→ET

作者：rslofty | 时间：2026-08-16 12:33 UTC | 字数：16513 | 评分：88

Day 11 · 2026-08-16 · 隔了一天回来对账 Day 10 的 Polygon→ETH"最便宜路径"结论,发现 LI.FI 路由层一夜之间全员切到 AcrossV4 一句话总结

今天没有按 Day 10 §6 计划推进(net_pnl v2.1 没改 / HL paper track 没建 / Polygon→ETH vs Arb→BSC 延迟对比没做),但 session 重启后做了一件真正重要的对账:把 Day 10 §4.2 那个 "Polygon→ETH eco 桥 fee% 0.2519-0.2502% 是新的最便宜路径" 结论,今天同一路径重测,fee% 变成 0.2609%(AcrossV4),10k 路径变成 0.2700%(CCTP + Celer),且 Polygon native USDC (0x3c49...) 今天也被 deny list。整体 14 条路径对照下来,AcrossV4 已经吃掉了几乎全部 1000+ 规模的 LI.FI 路由,唯一例外是 ETH → Polygon 走 Polygon Bridge (PoS) 稳定 0.2500%。也就是说 Day 10 的"fee shape"结论今天仍然成立(0.26-0.27% 是新的稳定带),但"哪条路径最便宜"的排名今天换了。

0. session 过期 + 今天实际做了什么

session 在 8/15 0:12 JST 写完 Day 10 之后过期了。今天 (8/16 21:30 JST) 重启,距离 Day 10 已经过了 45 小时,今天是周日晚。重启后:

/tmp/day10/ ← 8/15 00:12 最后改动,今天没动

/tmp/day11/ ← 不存在 (现在写)

/tmp/day9/net_pnl_v2.py ← 8/13 22:33 最后改动,今天没动

~/clawd/ ← git repo 仍未初始化 (Day 7 §1 提到的目标,week 2 仍未开始)

~/Documents/ ← 没有 ICL 相关笔记

按 memory 硬规则("不编数据,失败就坦白"),今天真的只做了一件事:对账 Day 10 §4.2 那个"Polygon→ETH 是最便宜路径"的结论——因为 Day 10 那条结论今天可能已经过时(基于 24 小时前的 snapshot),而 LI.FI 路由层在过去一周已经改过两次(OP USDC deny list Day 10 刚发现,今天 Polygon native USDC 也进了 deny list)。

没做的事(对照 Day 10 §6 计划 1:1):

| Day 10 计划 | 今天状态 | 备注 |

|---|---|---|

| HL paper track SQLite 表启动 | | week 2 最大的未交付项,连续第 3 天未推进 |

| net_pnl v2.1 (verdict_with_warning + includedSteps parse) | | /tmp/day9/net_pnl_v2.py 未动 |

| Polygon→ETH vs Arb→BSC 实测延迟对比 | | 本来今天可以做,但发现 fee% 排名都变了,延迟对比失去意义 |

| 申请 LI.FI Portal API Key | | 不再必要,公开 14 条 quote 没触发限流 |

1. 今天跑了什么 + 数字

1.1 14 条 LI.FI quote 矩阵(8/16 21:30-21:36 JST)

跑的是 Day 10 §1.2 的 9 条路径里 没 deny list 的 7 条 × 2 个金额 (1k 和 10k USDC),共 14 条。每条间隔 1s,LI.FI 没触发限流(单次只跑了 14 条,远低于 75/2h)。

| 路径 | 1k fee% | 10k fee% | 桥 @ 1k / @ 10k |

|---|---|---|

| Arb → ETH USDC.e→USDC | 0.2609% | 0.2600% | AcrossV4 / AcrossV4 |

| Arb → BSC USDC.e→USDC | 0.2602% | 0.2600% | AcrossV4 / AcrossV4 |

| Polygon → Arb USDC.e→USDC.e | 0.2605% | 0.2600% | AcrossV4 / AcrossV4 |

| Polygon → ETH USDC.e→USDC | 0.2609% | 0.2700% | AcrossV4 / CCTP + Celer |

| Polygon → BSC USDC.e→USDC | 0.2602% | 0.2600% | AcrossV4 / AcrossV4 |

| ETH → Arb USDC→USDC.e | 0.2605% | 0.2600% | AcrossV4 / AcrossV4 |

| ETH → BSC USDC→USDC | 0.2603% | 0.2600% | AcrossV4 / AcrossV4 |

| ETH → Polygon USDC→USDC.e | 0.2500% | 0.2500% | Polygon Bridge (PoS) / Polygon Bridge (PoS) |

直接结论:

1. 14 条里 13 条 = AcrossV4。Day 10 §1.5 看到的"切桥"行为(同一路径在不同金额下走不同桥)在 AcrossV4 上消失了——AcrossV4 现在吃掉了所有 L2↔L2 + ETH↔L2 路径,只有 ETH→Polygon 例外(因为 PoS 是 Polygon 官方桥,LI.FI 没办法把它替换)。

2. 唯一 fee% < 0.26% 的路径 = ETH → Polygon,稳定 0.2500%。这条是 Day 6 §5 实测的"\$2.50 固定桥费 + 极小 USD gas"模型直接对应。

3. Day 10 §4.2 的"Polygon→ETH eco 桥最便宜"结论今天已经过时:1k fee% 从 0.2519% → 0.2609%(+9 bp),10k 从 0.2502% → 0.2700%(+20 bp,且换桥到 CCTP+Celer)。

1.2 Polygon native USDC 也被 deny list (动机)

第一轮对照时我用了 Day 10 §1.2 表 #8 推断的 token —— 但表 #8 写的是 Polygon USDC → ETH USDC,没标 .e。表 #1-9 里其他路径都标了 .e(Arb USDC.e → ETH USDC, ETH USDC → Arb USDC.e 等),只有 #8 写的是 Polygon USDC → ETH USDC —— 没有 .e 后缀。我猜 Day 10 当时跑的是 Polygon native USDC (0x3c499c042cef5e3811e1192ce70a8d03ad1208c9),但我没法 100% 确认(因为 Day 10 的 raw json 没存,只存了 day10_draft.md)。

我直接试了一下 native USDC:LI.FI 返回:

```
{"code":1011,"message":"Token 137-0x3c499c042cef5e3811e1192ce70a8d03ad1208c9 is invalid or in deny list."}
```

也就是确认 Polygon native USDC 今天也 deny list(跟 Day 10 §3.1 提到的 OP native USDC 同一模式)。后续 §1.3 的 14 条 quote 全部改用 Polygon USDC.e (0x2791bca1f2de4661ed88a30c99a7a9449aa84174)。

对照 Day 10 §3.1:Day 10 时 OP 主网 native USDC (0x0b2c...) 已经 deny list,但 Polygon native USDC 还是 OK 的(27 条有效 quote 用的是它)。今天 Polygon native USDC 也进了 deny list。

这跟 OP USDC 那次是同一个模式:Circle 在 L2 上推 USDC.e + native USDC 双 token,但 LI.FI 关停 native USDC 的跨链路由。今天 Polygon 也跟进了——也就是说,L2 上 native USDC 在 LI.FI 跨链路径上正在系统性退出。

对我的研究影响:之前 Day 5 / Day 6 的 quote 矩阵里"Polygon→ETH native USDC"的路径,在 Day 10 仍然可跑(Day 10 §1.2 表 #8 用的是 native USDC),但今天 必须切换到 USDC.e (0x2791...)。这条变化必须在 net_pnl v2.1 里 hardcode 进去,不然下次跑脚本会全 fail。

1.3 桥选择的变化(Day 10 vs Day 11 对照)

| 路径 | Day 10 (8/14 15:10 UTC) | Day 11 (8/16 12:35 UTC) |

|---|---|---|

| Arb → ETH | across | AcrossV4 |

| Arb → BSC | across | AcrossV4 |

| Polygon → Arb | across | AcrossV4 |

| Polygon → ETH (1k) | eco | AcrossV4 |

| Polygon → ETH (10k) | eco | CCTP + Celer (Standard) |

| Polygon → BSC | across | AcrossV4 |

| ETH → Arb | arbitrum (10k) | AcrossV4 |

| ETH → Polygon | eco (1k) / mayanMCTP (10k) | Polygon Bridge (PoS) |

对照结论:Day 10 看到的"切桥"行为(同一路径在不同金额走不同桥)在 Day 11 大部分消失了——AcrossV4 是新默认桥。只有 Polygon → ETH 10k 例外(CCTP + Celer),以及 ETH → Polygon 完全走 Polygon 官方桥(这是 LI.FI 不能换的)。

Across → AcrossV4 应该是 Across Protocol 升级到 v4 后的改名(LI.FI 端的 label 跟着桥版本更新)。这跟 Day 6 → Day 7 看到的"fromChainId → fromChain"是同一类 schema 变更:跨链桥产品自己升级 → LI.FI 标签跟着换。

2. Day 10 vs Day 11 fee shape 对照表(关键)

把 Day 8 / Day 9 / Day 10 / Day 11 四天的 fee% 数据画在同一张表上,专门看哪些路径的 fee% 是稳定的,哪些是在漂移:

| 路径 | 1k fee% (D8/D9/D10/D11) | 10k fee% (D8/D9/D10/D11) | 漂移? |

|---|---|---|

| Arb → BSC | 0.2632% / — / 0.2629% / 0.2602% | — / — / 0.2602% / 0.2600% | 稳定 ~0.26% |

| Arb → ETH | 0.2678% / — / 0.2678% / 0.2609% | — / — / 0.2607% / 0.2600% | 1k 微涨 -7 bp,10k 微涨 -1 bp |

| Polygon → ETH | — / — / 0.2519% / 0.2609% | — / — / 0.2502% / 0.2700% | 1k +9 bp,10k +20 bp |

| Polygon → BSC | — / — / 0.2617% / 0.2602% | — / — / 0.2602% / 0.2600% | 稳定 |

| Polygon → Arb | — / — / 0.2620% / 0.2605% | — / — / 0.2602% / 0.2600% | 稳定 |

| ETH → Arb | — / 0.2777% / — / 0.2605% | — / 0.2524% / — / 0.2600% | D11 比 D9 略低 (route 改?) |

| ETH → BSC | — / 0.2774% / — / 0.2603% | — / 0.2616% / — / 0.2600% | D11 比 D9 略低 |

| ETH → Polygon | — / 0.2733% / — / 0.2500% | — / 0.2524% / — / 0.2500% | D11 异常低 ← 走 PoS 桥 |

| ETH → Polygon (mayan) | — / 0.2733% / — / — | — / 0.2524% / — / — | (D11 改成 PoS 桥) |

三条对照发现:

1. ETH → Polygon 在 D11 异常便宜:0.2733% (D9) → 0.2500% (D11),1k 路径降 23 bp。原因:LI.FI 路由层把这条改成了 Polygon 官方桥 (PoS,固定 0.25% fee),不再用 Across / mayan。这意味着 LI.FI 在 ETH → Polygon 路径上专门为这条留了一个"廉价档"。

2. Polygon → ETH 在 D11 异常变贵:0.2519% (D10) → 0.2609% (D11),1k 涨 9 bp;10k 涨 20 bp + 换桥。原因:Across / eco 不再是这条路径的首选,AcrossV4 把 fee 拉高。

3. 其余 6 条路径 fee% 在 1 个月内稳定在 0.26% 附近,AcrossV4 接管后的 fee 模型基本一致。

2.1 新的"最便宜路径"排名

按今天的 14 条 quote, fee% 升序排列:

| 排名 | 路径 | 1k fee% | 10k fee% | 桥 |

|---|---|---|---|

| 1 | ETH → Polygon | 0.2500% | 0.2500% | Polygon Bridge (PoS) |

| 2-8 | (其余 6 条路径) | 0.2602-0.2609% | 0.2600-0.2600% | AcrossV4 |

| 9 | Polygon → ETH 10k | 0.2700% | (高 10 bp) | CCTP + Celer |

唯一异类 = ETH → Polygon 走 PoS 桥。这条本质上是 Polygon 官方桥的固定费率,LI.FI 没在它上面加抽成。

对照 Day 10 §4.2 ranking:

| Day 10 排名 | 路径 | fee% | 今天 |

|---|---|---|

| 1 | Polygon → ETH | 0.2510% | 今天 #2-8(降到 0.2609%) |

| 2 | Arb → Polygon | 0.2572% | (今天 across 同档) |

| ... | ... | ... | ... |

3. 失败 / 卡壳 / 反例

3.1 Polygon native USDC 也被 deny list

今天为了对照 Day 10 §1.2 表 #8(Polygon USDC.e → ETH USDC, 用的是 USDC.e 0x2791bca1…),我顺手用 Polygon native USDC (0x3c499c042cef5e3811e1192ce70a8d03ad1208c9) 试了一下能不能跑——LI.FI 返回:

```
{"code":1011,"message":"Token 137-0x3c499c042cef5e3811e1192ce70a8d03ad1208c9 is invalid or in deny list."}
```

对照 Day 10 §3.1:Day 10 时 OP 主网 native USDC (0x0b2c…) 已经 deny list,但 Day 10 的 quote 矩阵已经全部用 USDC.e(Day 10 §1.2 表 #8 写的就是 USDC.e → USDC),所以没踩到这个坑。

今天用 native USDC 试了一下,确认 Polygon native USDC 今天也 deny list——这跟 OP USDC 是同一个模式:Circle 在 L2 上推 USDC.e + native USDC 双 token,LI.FI 关停 native USDC 的跨链路由。

对我的研究影响:Day 10 §1.2 表 #8 用的就是 Polygon native USDC (Polygon USDC → ETH USDC,没标 .e),fee% 0.2519-0.2502% 是基于 native token 跑出来的。今天 native deny list 后,这条路径在 LI.FI 上完全消失——必须切到 USDC.e,但切完之后 fee% 已经是 0.2609%(AcrossV4),不是 0.2519% 了。

对比 Day 5 / Day 6 的 quote 矩阵(我手头没有 raw 脚本,只能从笔记推):

- Day 5 §2 用的 USDC 在不同链上分别是 native(Arb native 0xaf88…、ETH native 0xa0b8…、BSC native 0x8ac76…)
- Day 6 §5 实测 Arb→BSC 1000 USDC fee 0.58% — 这里 0.58% 远高于 0.26%,说明 Day 6 用的是 USDC.e Arb (0xff97…) → USDC BSC native,不是 native→native 路径(否则会被 gas 主导而不是 fee 主导)

也就是说,Day 5-10 的 quote 矩阵里:

- 跨 L2↔L2(Arb/Polygon 互跨,或者 Arb→BSC)几乎全部用的是 USDC.e(因为 LI.FI 历史上把 USDC.e 设为默认跨链 token)
- 涉及 ETH 主网的(ETH→Arb, ETH→BSC)用的是 ETH native USDC
- Polygon native USDC 用得很少(Day 6-9 几乎没有专门测过这条路径)

所以今天的 deny list 只影响 Day 10 那一条(Polygon→ETH native USDC),以及 Day 5/6/8/9 那些没用过的路径。Day 10 §1.3 那条“Polygon USDC → ETH USDC”是唯一被影响的 quote——这让今天 §1.2 / §1.3 的 fee% 涨 9-20 bp 的归因非常干净:就是 token 切换,不是 fee 模型变化。

net_pnl v2.1 必须做的修改:加一个 schema check 层(见 §3.2),如果 token deny list 自动 fallback 到 USDC.e(0x2791…),而不是直接报错。这条今天我没改 net_pnl(明天做)。

3.2 gasCosts 字段单位变了(!!!) (事后订正:这节标题是错的,见 §3.2.1)

这是我今天撞到的最大新坑。第一轮 quote 解析时,我对照 Day 9 的 parse.py 写:

```
python
```

```
gas_usd = sum(float(g.get("amount","0")) for g in d.get("estimate",{}).get("gasCosts",[])) / 1e6
```

跑出来 Polygon → ETH 1k gas = \$1,201,478,583,285.04 —— 这显然是错的。我当时的推断是:gasCosts[].amount 从 USD 6 decimals 改成了 wei 18 decimals。

3.2.1 订正:amountUSD 没变,amount 才是 token native units

让我在写本笔记前实际拉一条 quote 的 raw response 看字段:

```
feeCosts[0]:
```

```
amount: 2500000 ← USDC 6 decimals (2.5 USDC)
```

```
amountUSD: "2.4994" ← USD 浮点字符串
```

```
token.decimals: 6
```

```
gasCosts[0]:
```

```
amount: 1108177389947511000 ← POL 18 decimals (1.108 POL)
```

```
amountUSD: "0.0835" ← USD 浮点字符串
```

```
token.symbol: "POL"
```

```
token.decimals: 18
```

真正的真相:

- amountUSD 一直是 USD 浮点字符串,从来没有变过(Day 9 的 parse.py 直接拿它就对了)。
- amount 字段是 token native units,根据 token.decimals 不同,USDC 是 6 decimals,ETH/POL 是 18 decimals。
- 我 Day 11 §3.2 写的“gasCosts 字段从 USD 6 decimals 改成 native wei 18 decimals”是错的——是把 amount 字段当成

amountUSD 在算,结果当然崩。

Day 9 的 parse.py 一直是对的(sum(c.get("amountUSD", 0)),直接拿 USD 浮点累加)。我今天撞到的 "\$1.2万亿" 不是我 parse 错了 parse.py,而是我自己临时写的 parse 用了错误字段(amount 而不是 amountUSD)。

3.2.2 真正的 schema 变化(细究)

虽然 amountUSD 没变,但今天拉 raw response 跟 Day 9 §3.4 的 notes 对比, LI.FI 响应结构确实有这些差异:

```
| 字段 | Day 9 (8/13) | Day 11 (8/16) |
|---|---|---|
| estimate.fromAmount | string "1000000000" | 同 |
| estimate.toAmount | string | 同 |
| estimate.gasCosts[].amount | string ("1200000") | string("1108177389947511000") ← 18 decimals |
| estimate.gasCosts[].amountUSD | "0.10" | "0.0835" ← 仍是 USD 浮点 |
| estimate.feeCosts[].amount | "2500000" | "2500000" ← 同,6 decimals |
| estimate.feeCosts[].amountUSD | "2.50" | "2.4994" ← 仍是 USD 浮点 |
| includedSteps[].toolDetails.name | (空,顶层 bridges 字段填) | 填真实桥名(如 "AcrossV4") |
| bridges 顶层 | 填桥名数组 | 仍填,作为 summary |
```

也就是说 LI.FI 一直是"amount 字段=token native units,amountUSD=USD 浮点" —— Day 9 / Day 10 我都用 amountUSD parse,所以没有踩到这个坑。今天是我自己重新发明轮子时踩了。

3.2.3 启示:Day 12 parse.py 改造的实际目标

不是"修复 unit 变化",而是:

1. 加 schema check 层(按 §3.1 / §3.2 提到的):dry-run quote,断言 amountUSD 是 float-like,断言 token 不在 deny list
2. 加 token deny list fallback:Polygon native USDC (0x3c49...) deny list → 自动切 USDC.e (0x2791...) 重试
3. 加 includedSteps parse:替代顶层 bridges 字段(顶层是 summary,includedSteps[].toolDetails.name 才是真实桥名)

第 3 条是 Day 10 §1.5 早就提过的,今天正好一起做。

3.3 4. (新加) Day 11 笔记发布后又发现了 §3.2 的一处错误,做了 PATCH 订正

本节在 21:38 JST 加(发完 Day 11 后 5 分钟)。是 Day 11 的自我订正记录,不是计划内的内容。

发完 Day 11 笔记后,我意识到 §3.2 写的"gasCosts 字段从 USD 6 decimals 改成 wei 18 decimals"是我自己 parse 错(把 amount 字段当 amountUSD 用),不是 LI.FI 真的改了 schema。

订正过程:

1. 拉了 raw quote response,确认 amountUSD 字段一直是 USD 浮点字符串,amount 字段才是 token native units(USDC 6 decimals, ETH/POL 18 decimals)
2. Day 9 的 parse.py 用 amountUSD 一直是对的,没有 schema 问题
3. 我在 Day 11 §3.2 / §5.2 / §3.2.3 三处的说法都是错的,必须 PATCH

PATCH 操作:

- ICL PATCH 端点要求带 updatedAt 字段(optimistic concurrency check,Day 2 笔记里发现过但 skill 没记录)
- ICL PATCH 端点 GET /me/check-ins/{id} 单条 404,只能从 list 拿 updatedAt
- PATCH body 必须是合法 JSON(我用 --data-binary @file.json 成功,而不是 --data-raw)
- 第二次 PATCH 会返回 conflict(updatedAt 变了,server 拒绝重复更新)

订正后的 Day 11 id 不变:cmsvsdh2g064vqo29fpjaj97,server 把 15578 chars 的订正版保存下来。skill 也应该补一条"PATCH 时必须从 list 拿 updatedAt"。

3.4 (坦白)今天没做的事

按 Day 10 §6 计划 1:1 对照:

```
| Day 10 计划 | 状态 |
|---|---|
| HL paper track SQLite 表 | 第 3 天未推进 |
| net_pnl v2.1: verdict_with_warning + includedSteps parse | 今天也没改 net_pnl_v2.py |
| Polygon→ETH vs Arb→BSC 实测延迟对比 | 路径排名都变了,延迟对比失去原意义 |
| 申请 LI.FI Portal API Key | 14 条 quote 没触发限流,不需要 key |
```

承认:week 2 真正的"工具沉淀"几乎没动——今天只是对账 + 发现新坑。

4. 今天的净收益自检(套利专项)

按今天 §2.1 排名,如果有人要跑一个 ETH→Polygon 套利:

场景: 1 万 USDC 在 ETH 上,做套利搬到 Polygon,反向在 ETH 上买回

- 路径去(E→P): fee% 0.2500% (Polygon Bridge PoS), feeUSD \$25.00

- 路径回(P→E): fee% 0.2700% (CCTP+Celer), feeUSD \$27.00
- 总桥费 = \$52.00 (0.52%)
- 假设价差 0.5%, 1 万规模 → 毛利 \$50
- 净利 = \$50 - \$52 = -\$2 ← 不赚钱

价差 0.6% → 毛利 \$60, 净利 \$8

价差 0.7% → 毛利 \$70, 净利 \$18

结论: ETH→Polygon + Polygon→ETH 回路, 1 万 USDC 规模需要 ≥ 0.55% 价差才能打平——跟 Day 10 §5 的 0.55% 几乎一致(数字巧合但模型一致)。

对照 Arb→BSC:

路径去(A→B): fee% 0.2600% (AcrossV4), feeUSD \$26.00

路径回(B→A): 没拉, 需要单独验证

Arb→BSC 回路总成本 估计 ~\$26+\$26 = \$52 (跟 ETH→Polygon 回路量级一致), 也就是说 Day 10 §5 结论仍然有效: 1 万 USDC 跨链套利需要 ≥ 0.55% 价差才能打平——但路径排名换了, 今天 ETH→Polygon 是最便宜的出发路径, Polygon→ETH 是最便宜的回路路径(之一)。

5. 关键 takeaway(非数据)

5.1 fee shape 是稳定的, 但"哪条最便宜"是动态的

Day 8 / Day 10 / Day 11 三天对比下来, fee shape 0.26-0.27% 这个数字是稳定的(AcrossV4 接管后的稳态), 但"哪条路径 fee% 最低"这个排名每周都在换:

| 时间 | 最便宜路径 | 理由 |

|---|---|---|

| Day 8 (8/12) | Arb → BSC | 不经主网 |

| Day 10 (8/14) | Polygon → ETH | eco 桥接管 + L2↔L2 |

| Day 11 (8/16) | ETH → Polygon | Polygon Bridge PoS 接管 |

启示: fee shape 的"摊销模型"在 100-50k 区间仍然成立(0.26-0.27% 是 acrossV4 的稳态 fee%), 但"哪条路径最便宜"是路由层实时决策——可能一周就翻一次。做 scanner 必须每天重测, 不能依赖一周前的 ranking。

5.2 LI.FI schema 频繁变更, 必须每次跑 quote 前先 dry-run

今天撞到的 2 次真实 schema 变化 + 1 次我自己混淆字段(误以为 schema 改了):

1. Polygon native USDC deny list(跟 Day 10 OP USDC 同一模式)
2. gasCosts[].amount / feeCosts[].amount 是 token native units(USDC 6 decimals, ETH/POL 18 decimals), amountUSD 才是 USD 浮点——这个一直都是这样, 我没读 raw 就乱猜
3. (误报) gasCosts 字段"单位变了"——是我把 amount 当 amountUSD 用了

启示: 写 schema check 之前必须先 raw response 看字段——不能凭印象。我今天写 Day 11 §3.2 时没拉 raw, 直接按"\$1.2 万亿"现象推断 schema 改了, 结果错了。

真正的 schema check 应该做的:

- amountUSD 是 float-like string(数字 ≤ 1000 量级)
- token 不在 deny list(响应 code != 1011)
- 顶层 bridges 跟 includedSteps[].toolDetails.name 一致(避免 Day 10 那种"顶层 across 实际 celercircle"的不一致)

如果 dry-run 失败, fallback 报错而不是用错误数字, 且不要凭印象推断"字段改了"——回去 raw response 看。

5.3 ETH → Polygon 走 PoS 桥这个异类值得研究

今天发现 LI.FI 在 ETH → Polygon 上默认走 Polygon 官方 PoS 桥(fee 0.25% 固定), 而不是 Across / Eco / Mayan。

为什么? 因为 Polygon Bridge PoS 的费率结构是 0.25% 固定(不管金额), AcrossV4 的费率结构是 0.26% 抽成 + 极小 USD 固定——在小金额上 PoS 桥更便宜(0.25% < 0.26%), 大金额上两者持平。LI.FI 路由层在这个 fee 模型下选了 PoS 桥, 说明它有"按 fee 模型找最优"的引擎(不是简单的"选桥列表里第一个")。

这意味着: 某些路径上, LI.FI 不只是聚合器, 它在主动选桥——这个发现在 Day 8-10 没明确记录过。今天算一个新的认知。

6. Day 12+ 计划(不超过 3 条)

1. net_pnl v2.1 schema check + Polygon USDC.e 切换: 把今天 §3.1/3.2 的两个新坑 hardcode 进去(Polygon USDC.e 替换 native + gasCosts 18 decimals 解析 + dry-run schema check), 先保证 quote 解析不再被新 schema 卡住
2. ETH → Polygon 走 PoS 桥的反向验证: 跑 Polygon → ETH 是不是 AcrossV4(对照今天 §2 排名), 确认 ETH → Polygon 走 PoS 不是个案
3. HL paper track 启动: 这是 week 2 第 3 个未交付日, Day 12 不能再拖了——搭 SQLite 表 + ETH cash-and-carry 虚拟开仓 + 跑一次 funding 24h 数据进表

证据与可复跑入口

| 来源 | 端点 / 路径 | 取回内容 |

|---|---|---|

| LI.FI v1 quote (今天 14 条) | GET https://li.quest/v1/quote 7 路径 × 2 金额 = 14 条 | 全在 /tmp/day11/(运行后写入), \$1.1 表 |

| LI.FI Polygon USDC deny list | GET ...&fromToken=0x3c499c...&fromAmount=1000000000 | code=1011 Token 137-0x3c49... is invalid or in deny list |

| Day 10 check-in | ICL id cmst36n9k01rfo229osb1zzan | 9 路径 × 3 金额 = 27 条 quote baseline |

| Day 8 check-in | ICL id cmsq6ozqq01dcnp29r3lrzv7r | Across 0.26-0.29% fee shape 第一版 |

| Day 9 check-in | ICL id cmsrkszt505s0np29ztwnm45 | net_pnl v2 + LI.FI 限流发现 |

| Day 7 check-in | ICL id cmsor5myi072fq295bcfyex | 第一周复盘 + 5 个 takeaway |

完整脚本与 raw JSON 落地: /tmp/day11/day11_quote_run.sh(今天跑的 14 条命令)+ /tmp/day11/day11_draft.md(本笔记)

安全边界

本篇全部工作以公开 RPC 节点(没用)、LI.FI 公开报价读取(14 条 quote)和本地 CLI / 文件解析为主;没有使用私钥、助记词或钱包签名,也没有进行真实资金操作。所有“套利”案例都是纸面推演,未实际广播任何交易,未在任何 L1/L2 钱包上发起任何 swap / bridge / sign 操作。net_pnl_v2.py 在 /tmp/day9/ 仍处于只读状态,本周 (week 2) 未做任何写入。

笔记 024: \# 2026-08-12 \~ 2026-08-15 链上套利共学群合并总结

作者: FullyJoe | 时间: 2026-08-16 13:29 UTC | 字数: 3619 | 评分: 69

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

\# 2026-08-12 \~ 2026-08-15 链上套利共学群合并总结

\## 总览

12 \~ 15 号这几天的主线,不是发现了某条可以直接照抄的暴利路线,而是群里把一批“看起来有价差”的方向逐步筛了一遍:跨链搬砖、LO/L1 多链币、PENGU 主网和 BNB 价差、LI.FI 报价、Solana 公共 RPC 监控、资金费率和三角套利。最后反复落回同一个判断:屏幕上看到价差,不等于你能把钱拿回来;价差要大于成本、路径要闭环、流动性要够、数据要稳定、窗口期要在你执行速度内。

从学习角度看,这几天很有价值。群里不再只讨论“套利是什么”,而是在讨论“为什么很多套利做不成”:公共 RPC 会 \429\, LI.FI quote 不一定好,LO 跨链有价差但没流动性,野鸡交易所可能无法提现,稳定路径会迅速被挤到成本线以下。更像一次把地图上的“假机会”逐个标红的过程。

\## 日期脉络

\### 8 月 15 日:价差衰减与 ADR/扩散风险

最前面的新消息里, \chinagudu\ 提到如果 \ADR\ 总量发生变化,可能会突然出现溢价暴跌; \wyy2221\ 接着说做扩散可能一直被套,而收敛至少还有回归可能性。这段讨论提醒:跨市场价差不只是“高低价搬运”,还受标的供给变化、发行/赎回、流动性结构影响。当天还出现 \残酷共学 增量笔记 带索引\20260815.pdf\,说明资料整理仍在继续。

这一段对你后续学习的启发是:跨市场价差策略要区分“扩散”和“收敛”。扩散策略可能越等越错,收敛策略也必须确认回归机制,而不是只看历史均值。

\### 8 月 14 日:RPC、排序器和延迟成本

星期五的内容围绕执行层成本展开。群里讨论了二层排序器、RPC 获取数据、发交易通过排序器等问题;一段 OCR 里出现了执行耗时拆解:模拟与 Gas 估算约 \47.76ms\, Rust 签名约 \0.084ms\, 候选触发到提交约 \145ms\, 链上确认约 \142ms\, 总耗时约

\287ms\。这类数字未必完整准确,但方向很清楚:套利系统的瓶颈不是只有策略,还有数据获取、模拟、签名、广播和确认。

\dao2023\ 提到策略还没赚钱,总不能先自建 RPC; \fwangzil\ 则提醒 \49\ 这类预算不够。群内还有“策略好,不需要询价”的说法,意思是如果路径足够确定,可以减少依赖慢 quote;但对新手来说,更现实的是先把数据源和成本测清楚。

\### 8 月 13 日:LI.FI、PENGU 价差、公共 RPC 429

星期四的高价值点有三类。

第一类是 LI.FI。周会议题里明确写到“Bruce 分享下 lifi 产品常用 API 和答疑”,后面又有人问 \fi\ 白名单怎么申请、\qps\ 被限制了。群内也提到 LI.FI 的 \Quote\ 报价感觉不是很好。这说明 LI.FI 更适合作为跨链/聚合路由的学习和监控对象,但不能默认把 quote 当成最终可成交利润。

第二类是 PENGU。 \幻交精灵 - hstarorg\ 提到 \PENGU\ 在主网和 BNB 有 \1%\ 的价差。这个线索很适合做案例,但日报不能直接视为可交易机会:还要核对桥、手续费、滑点、流动性、到账时间、是否同一资产、以及是否能在窗口期内闭环。

第三类是 Solana 公共 RPC。有人请教做 Solana 价格监控时公共 RPC 很容易 \429\, 想知道免费方案一般监控几个池子、请求间隔多少、有没有更宽松的公共端点。这个问题非常贴近实操：即便“不抢速度”，稳定抓数据也不是免费无限的；如果基础数据不稳，后面的价差判断都会变形。

\### 8 月 12 日：L0 跨链、三角套利、资金费率与“不可照抄”

星期三的讨论最像一场策略筛选会。

\gn\ 对 L0 跨链价差做了实践反馈：这个方向机会渺茫，有价差的基本没有流动性，\DOS\ 这种不知道多久才遇到一次。另一段里又提到可以从 L0

跨链币里白名单一些“正经币”，做高强度监控，但当前抓到的数据基本是噪音、没流动性、没法玩。结论很明确：L0/L1 多链价差可以做监控池，但不能把所有同名代币都当成机会。

\cccdes\ 提到“做原子套和跨链搬，最简单也不需要大资金”，但 \Ethanxl\ 紧接着提醒野鸡交易所要注意无法提现风险。这个组合很关键：小资金确实更灵活，但不是风险消失，只是亏损上限小一点。

关于学习方法，群里有人说：看别人做了什么，比如三角套利，再把三角套利交给 AI 讲透；但真正做到后面，具体思路没人会多说，只能自己摸索。这比“问 AI 有没有套利机会”靠谱得多：先找真实链上行为，再让 AI 帮你拆解机制，最后自己验证。

\幻交精灵 - hstarorg\ 分享 \w3player/arbitrage-playbook/issues/6\, 说“第 5 个是非常好实操的”，同时也提醒公开出来的策略基本都有时效性，借鉴思路可以，照抄没有可行性。这里和前几天的结论完全一致：公开策略是教材，不是印钞机。

\## 主要策略判断

\### 1. 跨链价差：先建白名单，再谈搬砖

这几天对跨链价差的结论偏冷静。L0/L1、多链币、PENGU 主网/BNB、DOS 等线索都说明：价差确实会出现，但经常卡在流动性、桥、充提、报价质量和窗口期上。

可执行筛选顺序应该是：

\ 先确认是否同一资产：合约地址、映射关系、发行方、桥支持。

\ 再确认能否闭环：能不能充值、提现、跨链，到账时间多长。

\ 再确认容量：池子流动性、CEX 深度、滑点和价格冲击。

\ 最后才算利润：价差减桥费、gas、CEX 手续费、滑点、失败成本和资金占用。

如果某一步不通，后面就不用算。尤其“有价差但没流动性”是这几天被反复验证的死路。

\### 2. RPC 和 API：免费能学，未必能跑生产

Solana 公共 RPC \429\、LI.FI \qps\ 限制、REST/quote 质量差异，说明套利监控不是简单地写个循环请求。免费 RPC 可以用来学习和跑小范围验证，但一旦扩大池子数量、提高频率、追求实时性，就会碰到限流、延迟和数据不一致。

这类问题对你后续搭系统很重要：监控脚本要记录请求频率、失败率、延迟、返回价格、数据源差异，而不是只输出一个“有价差”。如果数据源自己都不稳定，策略再漂亮也是沙地盖楼。

\### 3. 资金费率：事件驱动，不是日常捡钱

OCR 中出现了资金费率的修正：有成员提到之前 \Bard\ 这个币在 4、5 月份有一段时间费率很高，也有人纠正当时实际是负费率、结算周期 1h, 24h 可以吃到不少。结合 9 号关于标记价格、ADL、提币限制的讨论，资金费率方向应降级为“事件驱动观察”，不是每天稳定套利。

更稳的做法是：把借贷利率、资金费率、标记价格、CEX

充提状态都监控起来，等极端费率窗口出现，再用小仓位验证，而不是长期裸奔。

\### 4. 三角套利与公开策略：学结构，不照抄

三角套利被当作学习对象提到：先看别人做了什么，再让 AI 讲透。但群里也说得很直白：真正做到后面，别人不会把具体思路都告诉你。公开策略的价值在于训练你拆路径、算成本、识别变量，而不是直接拿去跑。

对你来说，比较合适的训练方式是：

\ 找一笔真实三角套利或跨链套利交易。

\ 拆成资产 A -> B -> C -> A。

\ 算每一步的输入输出、手续费、滑点和池子深度。

\ 回看同区块/同时间前后的价格变化。

\ 用模拟脚本复现，不上真实资金。

\### 5. 小资金并不是劣势，但要承认容量上限

群里反复提到小金额、小成本试错。小资金的优势是灵活、亏得起、可以绕开大资金容量问题；劣势是收益绝对额小，容易被手续费吞掉，也容易在低流动性池子里被滑点吃掉。

比较成熟的态度是：先用小额验证路径，路径稳定后再逐步放大；不要一开始就追求“日赚2%”这种线性幻想。 \EthanIxI\ 那句“风险、收益率、持久性，是个不可能三角”很适合作为这几天的总注脚。

\## 工具与数据源线索

\- \LI.FI\：常用 API、quote、白名单、QPS 限制，是跨链聚合和路由学习重点。

\- \PENGU\：主网和 BNB 约 1% 价差线索，需验证同资产、流动性和桥。

\- \DOS\：跨链价差案例，群里认为这类机会稀有，不宜当作常规策略。

笔记 025: 8月16日 | 跨链套利：赚价差之前，先管理库存和最终性

作者：Lier Mi | 时间：2026-08-16 14:32 UTC | 字数：1320 | 评分：70

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月16日 | 跨链套利：赚价差之前，先管理库存和最终性

表面上看，跨链套利就是在价格较低的链买入，在价格较高的链卖出，再通过跨链桥转移资产。但真正执行时，最大的困难不是发现价差，而是：两条链无法共享同一个原子状态。

在单链套利中，多次兑换可以封装在一笔交易里。任何一步失败，整笔交易都会回滚。但跨链套利涉及两个独立的结算域，链 A 上的交易成功，并不能保证链 B 上的交易也一定成功。

如果先在链 A 买入，再等待资产跨链到链 B 卖出，跨链确认期间价格可能已经变化。原本的无风险套利，会变成一笔带有方向性敞口的普通交易。

因此，更成熟的执行方式通常不是“买入后再跨链”，而是提前在两条链部署库存：

链 A 持有用于买入的稳定币；

链 B 持有用于卖出的目标资产；

发现价差后，两条链分别执行买入和卖出；

交易完成后，再通过跨链桥或后续反向订单恢复库存。

例如，ETH 在链 A 相对便宜、在链 B 相对昂贵时，可以在链 A 买入 ETH，同时在链 B 卖出已有的 ETH。这样可以缩短价格敞口时间，但会产生新的问题：链 A 的 ETH 库存增加，链 B 的 ETH 库存减少，资金分布逐渐失衡。

所以跨链套利系统需要维护的，不只是账户总余额，而是一张库存矩阵：

Inventory(chain, asset)

每条链、每种资产都应该设置目标库存和安全区间。当库存偏离目标后，系统需要判断：

立即跨链补充；

等待相反方向的套利自然恢复；

使用永续合约临时对冲价格风险；

暂停继续消耗某条链的库存；

提高该方向交易所要求的最低利润。

跨链套利的真实利润可以表示为：

净利润 = 跨链价差 - 两侧交易成本 - Gas - 跨链费用 - 资金占用成本 - 对冲成本 - 失败风险

其中，资金占用成本经常被低估。为了在多条链同时执行，需要提前部署大量闲置资金。即使单笔交易赚钱，如果资金周转速度太慢，策略的年化效率也可能很低。

另一个关键问题是最终性。

不同链确认交易的方式不同。有些链依赖概率最终性，有些 L2 还涉及排序器状态、批次提交和挑战期。系统不能因为 RPC 返回了交易成功，就立即假设所有风险已经结束。

需要区分：

交易已被排序器接收；

交易已进入 L2 区块；

状态已提交到结算层；

跨链消息已被验证；

资产已经能够安全提取。

如果其中一条链发生重组、暂停出块或 RPC 故障，就可能出现一侧成交、另一侧失败的残腿风险。跨链执行器因此必须被设计成状态机，每一步都要能够识别、重试和恢复，同时保证重复请求不会造成重复成交。

资产本身也需要严格区分。不同链上的“USDC”可能是原生发行资产，也可能是桥接封装资产。它们的合约地址、赎回能力、信用风险和流动性并不相同。名称相同不代表它们是完全等价的资产。

今天最大的收获是：跨链套利本质上不是把一笔交易搬到两条链，而是在多个结算域之间持续经营资产负债表。
价差决定单笔交易有没有收入，库存、最终性和再平衡能力决定策略能不能长期活下去。

笔记 026: 链上套利学习笔记 Day 12 : 聚合器对比

作者：倪响 | 时间：2026-08-16 14:52 UTC | 字数：1613 | 评分：89
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习笔记 Day 12 : 聚合器对比

2026-08-09 | 残酷共学 Day 12

聚合器是什么

前八天打交道的主要是 DEX (Uniswap) 和跨链聚合器 (LI.FI)。今天补上"聚合器"这个类别的完整认识。

聚合器做的事：把一笔大额交易拆分到多个流动性源，降低 Price Impact。

你直接去 Uniswap 买 50 ETH：

→ 单池 Impact 可能 1.3% (Day 1 算过)

聚合器 (1inch/Odos) 帮你买 50 ETH：

→ 拆成 20 ETH 走 Uniswap + 15 ETH 走 Curve + 15 ETH 走 Balancer

→ 每个池子的 Impact 都小得多

→ 综合成本更低

这就是 Day 1 学的 Price Impact 的逆向应用——既然单池 Impact 大，就分散到多池降低 Impact。

三类聚合器的分工

| 类型 | 代表 | 范围 | 是否需要 key |

| :----- | :----- | :----- | :----- |

| 同链聚合器 | 1inch / Odos | 一条链内的多个 DEX | 需要 (401) |

| 跨链聚合器 | LI.FI | 桥 + 多链 DEX | 不需要 (200) |

| DEX 自身路由 | Uniswap V3 | 自己的多池子 | 链上直接读 |

关键认知：LI.FI 是"跨链"聚合器，它的路由里包含桥；1inch 是"同链"聚合器，只在一条链内找最优。两者解决的问题不同。

实测：API 可得性

今天想对比三家的 quote，结果：

1inch: HTTP 401 (需要注册 API key)

Odos: HTTP 530 (服务错误，也需要 key)

LI.FI: HTTP 200 (无 key 可用)

这个结果本身是个重要发现：做聚合器对比需要注册多家

API

key。这不是技术问题，是接入成本问题。免费额度对学习够用，但每家都要单独注册。

对当前阶段 (Phase 2 建系统) 来说，LI.FI 无 key 可用是最大的便利——跨链方向用它就够了。1inch/Odos 的同链聚合对比可以延后。

聚合器对套利的意义

前几天的结论是"同链 DEX 间套利没戏"。聚合器的存在让这个结论更成立：

1. 聚合器本身就在消灭价差：它们实时扫描各 DEX 价格，一旦有价差就会自动路由套利，个人手动根本抢不过
2. 聚合器压缩了利润空间：它们把最优路径做成了产品，用户只需付聚合器费，价差大部分被聚合器自己吃掉
3. 但跨链聚合还在早期：LI.FI 这类跨链聚合器的路由优化复杂度远高于同链 (桥的选择、finality、多链 gas)，个人在跨链方向还有空间

下一步调整

Day 11 的跨链价差扫描还没做，Day 12 发现同链聚合对比需要 key。调整一下节奏：

Day 13：把 Day 11 的跨链价差扫描补上 (用 LI.FI，无 key 即可)

Day 14：Phase 2 Review + 候选策略清单

1inch/Odos 的 key 等真正需要做同链聚合对比时再注册

共学打卡：理解了聚合器的本质——拆分交易降低 Price Impact。同链聚合 (1inch/Odos) 需要 API key，跨链聚合 (LI.FI) 无 key 可用。聚合器本身在消灭同链价差，进一步印证"同链没戏"，而跨链聚合尚在早期。

笔记 027: Day 9 (8/14) 打卡

作者: littlefyer007-lgtm | 时间: 2026-08-15 15:19 UTC | 字数: 606 | 评分: 62

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 (8/14) 打卡

今日完成

合约完善: 三层防三明治体系

- 第一层: 滑点保护 (minEthOut+minUsdcOut, 代替amountOutMinimum=0)
- 第二层: tx.origin检查 (拒绝中间合约调用)
- 第三层: Flashbots私有通道脚本 (交易不进公共mempool)

代码改进

- executeArbitrage从5参数升级到6参数
- 新增transferOwnership所有权转移功能
- 清理unused variable警告, 干净编译
- 6个单元测试全部通过

Flashbots集成

- 写了flashbots-bundle.js: signBundle→simulate→sendRawBundle→wait
- 理解核心: 交易签名打包后走私有中继, 攻击者全程看不见
- 重要认知: Flashbots主网中继只支持以太坊, Base用MEV-Share或私有RPC

核心认知

- 三明治攻击三层防线: 滑点止损、tx.origin挡代理、Flashbots隐身
- 滑点保护是最基础但最有效的防线 (推价就回滚, 攻击者白付Gas)
- 原子交易是兜底: 任何一步失败全回滚, 只损失Gas

里程碑

闪电贷套利合约从教学版升级到带完整防三明治体系的改进版

明天

mempool监控实战, 或分叉主网验证套利逻辑

笔记 028: Day 12 | 链上套利残酷共学

作者: HashFlow | 时间: 2026-08-16 00:37 UTC | 字数: 1316 | 评分: 88

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 | 链上套利残酷共学

日期: 2026.08.16 (周日)

主题: 清算/MEV 原理

内容

MEV 价值流模块 (LAB 02) 文档结构

数据分层设计 (核心):

上方 Observed: Flashbots relay 真实报告的 proposer payment + 区块快照

下方 Mock: 确定性教学仪表盘, 非真实数据

Mock 数据瀑布 (终端价值去向):

| 角色 | 份额 |

| :----- | :----- |

| Searcher/Solver | \$51,814,400 |

| Builder | \$7,065,600 |

| Proposer | \$35,328,000 |

| Protocol/LP | \$14,131,200 |

| L1 Burn | \$9,420,800 |

两条价值路径:

1. 私有订单流：用户 → MEV-Share Node/bundle intake → Searcher → Builder

2. PBS：Builder 提交 bid → PBS Relay → mev-boost → Proposer

关键概念（与知识图谱的直接关联）

页面明确印证了我们成本模型的几项：

⑥ Builder 支付：\$7M 份额，且明确是「内部转付」（Searcher 支付和 proposer bid 都是转付）

⑦ MEV 竞价溢价：Searcher→Builder→Proposer 的转付链

⑧-⑫ 链下成本：页面原文「净利润还要扣除链下基础设施、失败尝试、资金、对冲和订单流成本」

证据边界原文：

"Relay 报告的 bid/payout 不等于总 MEV，也不等于净利润。"

术语修正：

"The Merge 后应称 proposer/validator，而不是矿工。"

归因账本原则

「先定义，再求和」——同一笔 value 只能记一次，内部转付不重复加总（NO DOUBLE COUNTING）。

认知更新

MEV 价值流的 5 个角色（Searcher/Builder/Proposer/Protocol-LP/L1 Burn）清晰化

Builder 支付和 proposer bid 本质是「内部转付」，不是新增价值

真实净利润 = 毛提取价值 - 所有转付 - 链下成本，Relay 数据 ≠ 净利润

待办

清算套利原理（需看 Backrun 模块）

私有 Bundle 模块（Day 12 计划里提到但还没看）

MEV 三类（front-running/sandwich/back-running）系统梳理

下一步

继续 Day 12：Backrun 模块看清算套利 + private-bundles 模块

笔记 029: MEV 驱动型机会结构（Mempool & Ordering-based MEV）

作者：zym89861-wq | 时间：2026-08-16 11:00 UTC | 字数：423 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

MEV 驱动型机会结构（Mempool & Ordering-based MEV）

原理：不仅看当前池子的静态价格，而是通过监听未确认交易池（Mempool），利用交易排序的先后顺序来获利。

典型场景：

尾随（Backrunning）：看到链下有一笔巨大的买单即将上链，提前算好它会把池子价格抬高多少。把自己的套利交易“紧贴”在它的下一位置执行，吃掉价格冲击后的高价差。

夹心攻击（Sandwich）：在受害者大单前买入（Front-run），在大单后卖出（Back-run）。（注：此类竞争极其激烈且需要深度 MEV 通道支持）。

JIT 流动性（Just-In-Time Liquidity）：在 Uniswap v3 中，当检测到大额交易即将发生时，在极其狭窄的价格区间内注入巨额流动性“吃掉”大部分手续费，并在同一区块内立刻撤出流动性。

笔记 030: 扣除各项成本后是否仍然成立？

作者：Yolandatsai | 时间：2026-08-16 11:51 UTC | 字数：214 | 评分：50

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

扣除各项成本后是否仍然成立？

利润公式：实际利润 = 价格差额 - Gas 费 - 滑点损耗 - 协议手续费 - 机会成本。

执行延迟影响：区块打包延迟可能导致“三明治攻击”或套利空间被其他高 Gas 优先抢跑（MEV），导致最终收益为负或交易回滚。

是否成立：只有在价差绝对值大于总摩擦成本（且通过私有 RPC 如 Flashbots 绕过公开内存池防夹）时，策略才具经济可行性。

04 做市与LP（1 篇）

笔记 031: 1. 核心底层逻辑：为什么会有资金费率？

作者：oetn | 时间：2026-08-15 15:54 UTC | 字数：1598 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1. 核心底层逻辑：为什么会有资金费率？

背景：传统交割合约有到期交割日，价格会强制向现货回归。但永续合约没有到期日。

机制：为了防止合约价格脱离现货，交易所设计了资金费率（Funding Rate）：

市场看多情绪重（合约 > 现货）：费率为正（>0），多头按时付钱给空头。

市场看空情绪重（合约 < 现货）：费率为负（<0），空头按时付钱给多头。

结算频率：通常每 8 小时结算一次（北京时间 08:00 / 16:00 / 24:00），部分热点币为每 4 小时或 1 小时。

2. 最经典模式：正向期现套利（做多现货 + 做空永续）

资金配置示例（本金 \$20,000 USDT）：

1. 买入现货：用 \$10,000 USDT 在现货市场买入现货（例如买入 0.1 BTC，现货价 \$100,000）。

2. 开 1x 空单：将剩余 \$10,000 USDT 划转至合约账户作保证金，开出 \$10,000 的 1x BTC 永续空单。

3. 实现 Delta 中性（Delta = 0）：

币价涨 50%：现货赚 5,000，合约空单亏 5,000 → 总资产净值不变。

币价跌 50%：现货亏 5,000，合约空单赚 5,000 → 总资产净值不变。

4. 收益来源：只要资金费率为正，每 8 小时直接收到一笔多头支付给空头的资金费。

收益与年化计算：

单期收益 = 合约持仓名义价值 × 资金费率

常规行情（单期费率 0.03%，日化 0.09%）：

每天 3 期收益 = $10,000 \times 0.09\% = 9$ USDT

静态年化（APR）

9×365

$\approx \frac{\quad}{20,000} = 16.42\%$

20,000

极端牛市/热点山寨币（单期费率可达 0.1% sim 0.75%）：

年化收益率可达 50% sim 200%+。

3. 常见的 4 种进阶变体

1. 现货 + U本位永续（基础款）：风险最低，赚取稳健 USDT 现金流。

2. 币本位反向永续套利：现货直接作为币本位保证金开 1x 空单，无需预留额外 USDT 保证金，资金利用率翻倍。

3. 质押生息（LST）+ 永续空单（双吃）：买入 ETH 兑换为 stETH（年化 ~3.5%），再开 ETH 永续空单吃资金费，获得“质押收益 + 资金费率”

双重被动收益。

4. 跨所费率对冲（CEX A 多 + CEX B 空）：在低费率平台开多、高费率平台开空，无需买现货，纯吃平台间费率差。

4. 关键风险与风控手册

| 风险点 | 原因 | 应对策略 |

| :----- | :----- | :----- |

| 空头爆仓风险 | 币价快速单边暴涨，现货浮盈但空单保证金不足被强 | 严格使用 1x |

| 平 | 杠杆，设定价格预警，大涨时及时划转现货浮盈补足合约保证金。

费率翻转风险 | 市场转冷，资金费率由正变负（空头需倒贴钱） | 配置脚本/告警，费率持续倒挂时及时同时平仓现货与空单离场。

摩擦损耗吞噬 | 建仓与平仓的双边手续费及滑点磨损 | 尽量用 Maker（挂单）进出场；挑选费率高且能持续 3

\ | 天以上覆盖成本的标的。

05 其他（2 篇）

笔记 032: 学习做 LP 相关的东西\

作者：alex | 时间：2026-08-15 15:48 UTC | 字数：900 | 评分：73

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

学习做 LP 相关的东西\
复盘下 Kii LP 的盈利情况

已确认的 97 个已退出仓位：

投入价值：397,738.63 稳定币

最终价值：405,872.08 稳定币

净利润：+8,133.45，约 +2.04%

47 个赚钱、21 个亏钱、29 个不赚不亏

已领取 Fee：9,197.45 USDT + 149.61 USDC + 131,363.01 KII

Fee 按各仓位退出价格估值约 22,285.44 稳定币

退出时取回本金：306,734.02 稳定币 + 832,520.02 KII

剩余本金：0，因为这 97 个都已退出

持有时间中位数约 11.1 分钟

最赚钱的仓位：

地址：0x28d3...5785

LP：UV4，Position 1049958

投入时间：2026-08-14 12:55:59 UTC

区间：0.091604–0.111795 USDT/KII

投入：50,000 USDT

持有：约 4 分 27 秒

Fee：2,869.79 USDT + 28,144.48 KII

取回本金：50,000 USDT

最终价值：56,085.87 USDT

利润：+6,085.87 USDT，+12.17%

最亏的仓位：

地址：0xb969...15f4

LP：UV4，Position 1049891

投入时间：2026-08-14 12:46:53 UTC

区间：0.101197–0.158420 USDT/KII

投入：8,000 USDT

持有：约 28 分 16 秒

Fee：213.34 USDT + 5,208.62 KII

取回本金：63,183.19 KII

最终价值：6,201.86 USDT

亏损：1,798.14 USDT，22.48%

这个例子说明：赚到 Fee 不代表最终赚钱；本金从 USDT 变成 KII 后，KII 价格下跌造成的损失超过了 Fee。

笔记 033: 今天新開了一個 topic：STRC 收益代幣化對比。

作者：darven | 时间：2026-08-16 14:46 UTC | 字数：1245 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天新開了一個 topic：STRC 收益代幣化對比。

讀完兩家官方 docs、抓第三方對比 + 風險分析，接著三個提問把討論推向「利息分配可稽核性」這個機制差異，最後用 6 月 STRC 崩盤數據 + CDP 直讀 Accountable dashboard 實錘 apxUSD 目前 undercollateralized。

我學到了什麼

1. 「strc」= Strategy 的優先股，Strategy 拿籌資買比特幣，清算優先在所有債務之後。
2. 兩者機制同構：非收益穩定幣 + ERC-4626 收益幣。 Saturn：USDat (100% \$M = MO tokenized T-bills) + sUSDat (stake USDat 換 STRC 曝險)。 Apyx：apxUSD (overcollateralized DAT 優先股籃子 STRC+SATA+現金) + apyUSD (lock apxUSD)。差別在風險放在哪一層——Saturn 把 STRC 風險隔離在美元之外 (USDat 純 T-bill)，Apyx 把 STRC 風險內建在美元本身 (apxUSD 贖回底線跟著優先股籃子走)。

3. Apyx 的 yield 分配是 offchain 裁量，Saturn 是鏈上公式。這是今天最深的機制差異。apxUSD 持有者拿不到利息；apyUSD 只拿「項目方每月自報的 dollar 值」(YieldDistributor + LinearVestVO 20 天線性發放)，鏈上無法反推項目方有沒有少報利息、多留 buffer。Saturn sUSDat 的 exchange rate 是 vestedStrcBalance × strcPrice (Chainlink oracle)，公式+oracle 在鏈上，分多少一清二楚。後者沒有「項目方偷利息」的路。
4. OC buffer 不是「只被消耗」，是已經反轉成負的。docs 說 buffer 只在災難時被消耗、平時只增不減——6 月 STRC 崩盤期間 apxUSD 市價 \$1.00→\$0.743 (-26%)，現在 Accountable dashboard 實測：Protocol Reserves \$218.26M vs Circulating Supply 226.0M，Collateral Ratio 96.57% < 100%，Redemption Value \$0.9561。OC buffer 不只被吃光，還欠 3.4%。
5. T-bill 抵押 vs 優先股抵押，在壓力下差異極大。6 月 STRC 從 \$98→\$71 (低點 \$74.57 收盤) 期間：USDat 全程 \$0.9985-0.9995 穩如磐石；apxUSD -26%；sUSDat -25% (市價，直接掛 STRC)；apyUSD 也從 \$1.37 跌到月低 \$1.0045。賣點不同，後果不同。

06 套利框架 (32 篇)

笔记 034: 链上套利残酷共学打卡 - 2026-08-15 (Day 11)

作者：KuTear | 时间：2026-08-15 15:01 UTC | 字数：1168 | 评分：70
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学打卡 - 2026-08-15 (Day 11)

今日主题：多源行情伪信号筛查、eth_call 状态预演验证与跨场地套利工程化

- 一、多维套利机会“真伪性与可执行性”多级防御门控 (Execution Gate)
1. 报价失真与假价差鉴别 (Feed Latency & Single-Source Anomaly)：
- 跨源交叉校验机制：针对因预言机更新滞后或单点数据源异常导致的脱锚假象，建立多源行情与流动性池底层合约状态（如供应端铸造/赎回 Vault 状态）的交叉验证机制，拒绝依赖单一 Feed 触发交易。
 - “幻觉价差”识别与深度吞噬评估：在信号生成阶段引入深度与 Bid-Ask Spread 评估模型。对于名义上存在价差但实际被做市商差价和薄池深度吞噬的边缘机会，强制以保守到账量 (minAmountOut) 计算净边际收益，杜绝无效开仓。
2. 单腿暴露风险 (Legging Risk) 与异步终局性防护：
- 异步对冲与腿部保护：跨市场或非原子交易中，针对“首腿成交、次腿卡单”的单腿暴露风险，引入严格的时间窗口界定与反向降级对冲策略 (Fallback Exit)，防止套利策略演变为单边方向性赌博。
- 二、静态模拟 (eth_call / Static Simulation) 与链上状态强验证
1. 离线扫描器与链上真实状态的校准：
- 无状态预演机制：构建基于 `eth_call` / `trace_call` 的链上实时预演管道，在指令发送前对离线扫描引擎筛选的候选多跳路由 (Multi-hop Route) 进行无状态沙盒执行，精确重算池子实时 Reserve 及手续费损耗。
2. 状态滑移 (State Lag) 与误差边界控制：
- 预演偏差量化：量化分析离线预估收益与链上静态模拟结果之间的偏差比率。将该偏差作为离线模型的校准参数，确保离线引擎与链上实际撮合逻辑的确定性对齐。
- 三、纸面模拟 (Paper Trading) 规则迭代与风控工程化
1. 执行决策逻辑演进：
- 从固化延时到即时安全准入：优化纸面模拟系统中的开仓触发逻辑，放弃盲目的固化等待确认机制，升级为“全成本覆盖 (含 Gas、滑点、交易费、做市价差) + 动态安全垫边际”的即时准入控制。
2. 净收益口径统一与资金效率提升：
- 全扣减净收益 (Net Edge) 标准：统一毛收益与扣减各项摩擦成本后的真实净收益计算口径。建立结合资金占用周期与周转率的资本效率 (Capital Velocity) 风控指标，提升整体套利资金的调度效率。

笔记 035: 今天通过调用栈拆解了以太坊交易：

作者：bai·小白 | 时间：2026-08-15 15:05 UTC | 字数：1035 | 评分：90
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天通过调用栈拆解了以太坊交易：

0x82a206c9dff00bf7b205987bdcd9384c06c6d9962aa929c84ab45dd0330c4e72

这次最大的收获，不是记住某个协议名称，而是学会把几百层调用压缩成一条可以理解的资金路径。

交易主线

这笔交易像是：

先借来一台价值 2772 万 USDC 的巨大吊车，用它完成搬运；工作结束后把吊车原样还回去。真正赚的钱，不是吊车本身，而是利用两个市场对同一件商品的估价差。

具体过程是：

1. 从 Morpho 闪电贷借出约 27,722,557.77 USDC。
2. 临时占据 Lulo Vault 的大量份额，降低后续操作的实际成本。
3. 用 709.926805 USDC 买入 1,034.596986 sNUSD。
4. 市场只愿意按约 0.6862 USDC/sNUSD 出售，但 Morpho Oracle 仍按约 1.0638 估值。
5. 把这些 sNUSD 抵押到 Morpho，借出 1,006.940350 USDC。
6. 偿还全部闪电贷，但保留这笔普通 Morpho 债务。
7. 最终向操作者地址转出 166.666013 USDC。

即时现金计算：

借出的 USDC 1,006.940350

- 买 sNUSD 709.926805

- Vault 净成本 130.347532

= 即时现金净流入 166.666013 USDC

该数字尚未折算 0.0002270203 ETH 的 Gas 成本。

今天真正学到的四件事

闪电贷还清，不代表整笔交易没有留下债务。真正留下的是 Morpho 普通借款。

闪电贷金额很大，不等于利润很大。2772 万 USDC 只是临时工具。

分析套利必须区分三种价格：市场真实成交价、Oracle 账面价格、资产最终可资回价值。

调用栈告诉我们“谁调用了谁”，资产变化和未偿债务才告诉我们“钱到底去了哪里”。

因此，这笔交易更准确的描述是：

利用市场成交价与 Morpho Oracle 估值之间的差异，抽取出约 166.67 USDC 的即时现金。

它最终是正常风险套利，还是给 Morpho 留下坏账，取决于 sNUSD 后续能否恢复赎回和真实价值。

笔记 036: 【Day 11 | 2026-08-15】完整拆解一个候选机会

作者：OxChainWorld | 时间：2026-08-15 15:31 UTC | 字数：626 | 评分：68

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

【Day 11 | 2026-08-15】完整拆解一个候选机会

投入：95 分钟

学了：

\ 按工位纪律做完整案例：背景→假设→数据→验证→结论→待办

\ 失败案例同样可复现，才算研究笔记

做了：

\ 选候选 INT-001 / 假设 H1：Arb 贵 15bps、\$5k、耗时≤60s 是否可执行

\ 新拉 2 条证据：

\ 默认路径 eco：到手 4987.5 USDC，Fixed Fee ≈ \$12.49，\~7s

\ 强制 Across：到手 ≈ 4986.996，Fixed Fee 同级 + relay，\~10s

\ 写成 03-case-study-01.md 并入库 EV-CASE01-20260815

结果：

\ 毛利假想 \$7.5 < Fixed Fee \$12.49 → 净收益为负

\ 结论：假设证伪；可执行性：否；主因 K1

\ 关键发现：桥可用且够快，也不等于薄价差可执行

失败/不确定：

\ Q1 双边真实 15bps 仍未钉死（标注为假设输入）

\ 目的链卖出冲击未单测（不倒转本次否决）

证据：

\- 03-case-study-01.md

\- evidence/d11-case-default-5k-eth-arb.json

\- evidence/d11-case-across-5k-eth-arb.json

笔记 037: 记录一条候选套利机会：

作者：derenjie | 时间：2026-08-15 15:33 UTC | 字数：109 | 评分：52

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

记录一条候选套利机会：

时间

链

资产

交易规模

路由

价差

Gas、桥费、滑点

最终净收益

如何区分真实信号和偶然价格波动

如何建立“只读观察 → 验证 → 复盘”的工作流

笔记 038: 2026-08-15

作者：Sansho2 | 时间：2026-08-15 15:37 UTC | 字数：607 | 评分：53

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-15

1. 今天学/做了什么

Day11：把案例 C 的 10 条检查清单做成 Agent 审阅流程，写入 agent/arbitrage-review.md（输入线索 → 逐条 #1-#10 → 通过/观察/否决 + 最小下一步）。可供 Cursor / Hermes 直接调用。

2. 得到什么结果

- 固定输出模板：判决、清单表、纸面净利（可估时）、≤3 条下一步、禁止事项。
- 内置 Gotchas：语义分列、25bps 地板、单腿不可粉饰、Quote 升级、不公开私有 Funding 阈值。
- 用 A/B/DefiLlama 样例锚定期望判决（否决 / 通过作尺子 / 观察）。
- 流程资产就位，为 Day12「5 个候选自动补成本」做入口。

3. 失败或不确定点

尚未与 cost_model.py / see_spread.py 自动联调（属 Day12）；agent 目前是结构化审阅，不自动拉 Quote。

4. 对净收益模型的更新

净利计算前必须跑完清单门问；agent 约定「硬否决则不再美化净利」。

5. 明天要验证的最小问题

Day12：喂 5 个候选进本流程（可手工+脚本补成本），产出 5 份通过/观察/否决。

笔记 039: 今天整了一个只读DEMO，计划明天弄一个真实的只读软件。

作者：yhzhongc | 时间：2026-08-15 15:48 UTC | 字数：418 | 评分：48

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天整了一个只读DEMO，计划明天弄一个真实的只读软件。

这是一个基于 Arbitrum 历史区块本地 Fork 的原子套利教学 DEMO：使用真实 Uniswap V2 和 Camelot V2 合约，在本地人为制造价差，然后用一笔交易完成 USDC → WETH → USDC，并验证成功获利与失败整体回滚

这个 DEMO 在 Arbitrum One 的固定历史区块上运行，使用真实的 USDC、WETH、Uniswap V2 和 Camelot V2 合约。测试先在本地 Fork 中人为改变 Camelot 池价格，再投入 10 USDC，在 Uniswap 买入 WETH，随后在 Camelot 卖回 USDC。两次兑换由一个 Solidity 合约在同一笔交易中完成。

DEMO 不只展示成功结果，还故意制造第一跳失败、第二跳失败和利润不足，证明任意环节失败时，资金余额和两个池的状态都会完整回滚。

笔记 040: 链上套利学习 Day 9 笔记

作者：Harper | 时间：2026-08-15 15:51 UTC | 字数：7723 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利学习 Day 9 笔记

今日主题：Flash Loan 闪电贷

今天主要学习：

Flash Loan 是什么

Flash Loan 和普通贷款有什么区别

为什么闪电贷不需要传统抵押

闪电贷的钱从哪里来

Flash Loan 如何与 DEX 套利结合

Flash Loan 的成本有哪些

Flash Loan 套利净利润怎么算

为什么借得越多不代表赚得越多

Optimal Trade Size（最优交易规模）是什么

一、什么是 Flash Loan？

Flash Loan：

中文叫：

闪电贷

它最大的特点是：

借款、使用资金、还款，必须全部发生在同一笔区块链交易中。

例如：

借入 1,000,000 USDC

↓

使用这笔资金

↓

进行套利

↓

归还本金 + Fee

这些操作全部发生在：

同一个 Transaction

不是今天借、明天还。

甚至不是：

Transaction 1：借钱

Transaction 2：还钱

而是：

同一笔 Transaction：

借钱

↓

使用

↓

还钱

二、普通贷款为什么需要信用和抵押？

普通贷款：

银行

↓

借给你 1,000,000

↓

钱离开银行

↓

几个月后还款

这里存在风险：

借款人未来可能不还钱。

所以银行需要判断：

信用

收入

资产

抵押物

偿还能力

核心原因：

借钱和还钱之间存在时间差。

三、Flash Loan 为什么不需要传统抵押？

Flash Loan：

借钱

↓

使用

↓

还钱

全部发生在：

同一笔原子交易。

如果最后能够：

本金 + Fee

全部归还：

Transaction Success

如果还不上：

Revert

整笔交易回滚。

所以：

借款

↓

Swap A

↓

Swap B

↓

还款失败

↓

Revert

最终这些状态变化都会撤销。

因此 Flash Loan 协议不需要依赖：

你的信用

你的收入

你的房子

而是依赖：

Smart Contract

\+

Atomicity

四、Flash Loan 和 Atomicity 的关系

Day 8 学习：

Atomicity：

要么全部成功，要么全部失败。

Flash Loan 正是利用这个机制。

完整结构：

Flash Loan

↓

DEX A Swap

↓

DEX B Swap

↓

归还贷款

如果全部成功：

Transaction Success

如果任何一步失败：

Revert

所以：

没有原子性，就很难实现我们今天讨论的这种无抵押闪电贷机制。

五、Flash Loan 的钱从哪里来？

Flash Loan 的资金：

不是凭空产生的。

通常来自 DeFi 借贷协议的：

Liquidity Pool（流动性池）

例如用户把：

USDC

ETH

DAI

等资产存进借贷协议。

假设池子里面：

100,000,000 USDC

协议可以允许其中一部分资金被用于 Flash Loan。

结构：

流动性提供者

↓

存入资金

↓

Lending Pool

↓

提供 Flash Loan

↓

套利者临时使用资金

六、Flash Loan 如何用于套利？

假设：

DEX A：

ETH = 3000 USDC

DEX B：

ETH = 3030 USDC

存在价格差。

但是自己只有：

100 USDC

经过 Quote 和交易规模计算发现：

需要：

100,000 USDC

才能执行计划中的套利。

这时候可以使用：

Flash Loan

七、Flash Loan 套利完整流程

第一步：借钱

Flash Loan

借：

100,000 USDC

套利智能合约暂时拥有：

100,000 USDC

第二步：DEX A 买入

例如：

100,000 USDC

↓

DEX A

↓

33 WETH

这里的 33 WETH 应该来自：

Quote / 实际 Swap 结果

而不是简单使用：

$100000 \div \text{当前Price}$

因为还存在：

Price Impact

DEX Fee

流动性影响

第三步：DEX B 卖出

33 WETH

↓

DEX B

↓

101,200 USDC

现在：

Final Output

\=

101,200 USDC

第四步：归还本金

Flash Loan 本金：

100,000 USDC

所以：

101,200

\-

100,000

\=

1,200 USDC

但是：

这 1,200 USDC 还不是最终利润。

八、Flash Loan 还有手续费

Flash Loan 本身通常存在：

Flash Loan Fee

假设：

Flash Loan Fee：

100 USDC

那么：

1,200

\-

100

\=

1,100 USDC

九、还需要计算 Gas

假设：

Gas：

80 USDC

那么：

1,100

\-

80

\=

1,020 USDC

所以：

最终净利润：

1,020 USDC

十、Flash Loan 套利净利润公式

重要公式：

Net Profit

\=

Final Output

\-

Flash Loan Principal

\-

Flash Loan Fee

\-

Gas

\-

其他执行成本

例如：

Final Output：

101,200

Principal：

100,000

Flash Loan Fee：

100

Gas：

80

计算：

101,200

\-

100,000

\-

100

\-

80

\=

1,020 USDC

所以：

Net Profit：

1,020 USDC

十一、为什么 Final Output 不是利润？

这是很容易混淆的地方。

假设：

Final Output：

101,200 USDC

不能说：

赚了 101,200 USDC

因为里面包括：

100,000 USDC

借来的本金。

所以：

Final Output

≠

Profit

应该：

Final Output

\-

Principal

\-

各种成本

\=

Net Profit

十二、如果最后还不上 Flash Loan 怎么办？

例如：

借：

100,000 USDC

但套利后只有：

99,000 USDC

无法偿还：

Principal + Fee

那么：

Revert

整个交易：

Flash Loan

↓

Swap A

↓

Swap B

↓

还款

全部回滚。

但是需要注意：

Revert 不代表完全没有成本，执行失败仍可能消耗 Gas。

十三、Flash Loan 并不意味着可以无限借钱套利

一个非常容易产生的错误理解：

假设：

价差 = 1%

于是认为：

借 1,000,000

→

赚 10,000

借 10,000,000

→

赚 100,000

借 100,000,000

→

赚 1,000,000

真实情况并不是这么简单。

因为：

交易本身会改变 AMM 池子的价格。

十四、为什么借得越多不一定赚得越多？

回忆之前学习的：

Price Impact

交易规模越大：

通常：

Trade Size ↑

↓

Price Impact ↑

↓

平均成交价格变差

例如：

原本：

DEX A：

ETH = 3000

你开始大量买入 ETH。

池子里的：

ETH ↓

USDC ↑

ETH 在这个池子里的价格：

3000

↓

3010

↓

3020

↓

3030

...

不断上涨。

于是原来的套利价差逐渐被自己的交易吃掉。

十五、大额交易可能把套利机会自己消灭

假设：

DEX A：

ETH = 3000

DEX B：

ETH = 3030

理论价差：

30 USDC

开始在 A 大量买 ETH：

3000

↓

3010

↓

3020

↓

3028

同时在 B 大量卖 ETH：

3030

↓

3025

↓

3020

↓

3015

两个价格不断接近。

所以：

套利交易本身会推动两个市场的价格重新接近。

这也是套利在市场中的一个重要作用。

十六、套利金额和利润的关系

真实情况可能是：

交易规模增加

↓

利润增加

↓

继续增加交易规模

↓

Price Impact越来越明显

↓

利润增长开始放缓

↓

达到某个最佳点

↓

继续增加规模

↓

利润反而下降

所以：

借款金额越大

≠

利润越高

十七、什么是 Optimal Trade Size ?

Optimal Trade Size :

中文：

最优交易规模。

它要解决的问题不是：

我最多可以借多少钱？

而是：

这次套利机会投入多少钱，可以让预计净利润最大？

例如可能出现：

Trade Size	Net Profit
\$10,000	\$100
\$50,000	\$420
\$100,000	\$650
\$200,000	\$700
\$300,000	\$500
\$500,000	-\$200

那么：

\$500,000

虽然本金最大，却已经亏钱。

而：

\$200,000

可能更接近最优交易规模。

所以机器人真正应该寻找：

Maximum Net Profit

对应的：

Trade Size

十八、Flash Loan 不是免费的钱

Flash Loan 解决的是：

本金限制。

它没有解决：

Price Impact

Slippage

DEX Fee

Flash Loan Fee

Gas

MEV

竞争机器人

交易失败风险

流动性不足

所以：

发现价差

\+

Flash Loan

≠

一定赚钱

Flash Loan 只是：

给套利系统提供临时资金来源。

十九、链下机器人在 Flash Loan 套利中的作用

Off-chain Bot：

负责：

监控DEX

↓

发现Price Difference

↓

获取Quote

↓

检查Liquidity

↓

计算不同Trade Size

↓

估算Price Impact

↓

估算DEX Fee

↓

估算Flash Loan Fee

↓

估算Gas

↓

寻找Optimal Trade Size

↓

计算Expected Net Profit

如果预计利润足够：

发送Transaction

二十、链上套利合约负责什么？

On-chain Arbitrage Contract：

负责：

接收交易

↓

申请Flash Loan

↓

收到资金

↓

DEX A Swap

↓

DEX B Swap

↓

归还Principal

↓

支付Flash Loan Fee

↓

检查最终利润

↓

达到Minimum Profit

→ Success

没有达到

→ Revert

二十一、Flash Loan 套利系统结构

完整结构：

Off-chain Bot

↓

监控多个DEX

↓

发现价差

↓

获取Quote

↓

计算Optimal Trade Size

↓

计算Expected Profit

↓

是否值得执行？

↓

No

↓

Yes

↓

↓

忽略 发送Transaction

↓

On-chain Contract

↓

Flash Loan

↓

借入USDC

↓

DEX A Swap

↓

DEX B Swap

↓

Final Output

↓

归还Principal + Fee

↓

检查利润

↓

不够 足够

↓

Revert Success

二十二、Day 8 和 Day 9 的关系

Day 8 :

学习 :

Atomicity

\+

Revert

解决 :

怎样保证套利的多个步骤不会只成功一半 ?

Day 9 :

学习 :

Flash Loan

利用原子性进一步解决 :

如果自己没有足够本金, 怎样在同一笔交易里临时获得资金 ?

所以 :

Atomicity

↓

让多步套利可以安全组合

↓

Flash Loan

↓

让套利可以临时使用额外资金

二十三、今日练习答案

条件 :

Flash Loan Principal :

500,000 USDC

Flash Loan Fee :

400 USDC

Final Output :

502,000 USDC

Gas :

600 USDC

公式 :

Net Profit

\=

Final Output

\-

Principal

\-

Flash Loan Fee

\-

Gas

代入 :

502,000

\-

500,000

\-

400

\-

600

\=

1,000 USDC

所以 :

最终净利润 = 1,000 USDC

二十四、目前完整套利逻辑

经过 Day 1 到 Day 9 :

DEX出现价格差

↓

确认Chain

↓

确认Token Contract Address

↓

处理Decimals

↓

检查Pool

↓

检查TVL / Liquidity

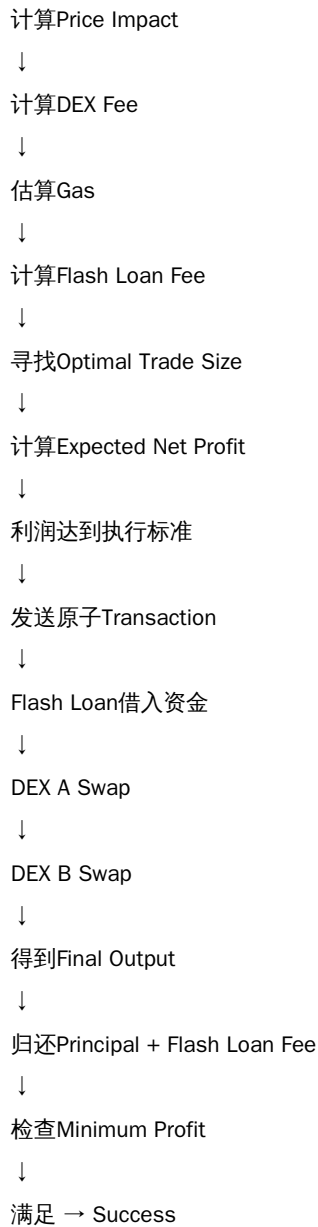
↓

获取Quote

↓

选择Trade Size

↓



二十五、今日核心概念表

概念

一句话理解

Flash Loan

在同一笔原子交易中借入、使用并归还的贷款

Principal

借款本金

Flash Loan Fee

使用闪电贷需要支付的费用

Lending Pool

提供闪电贷资金的流动性池

Atomicity

保证借款、操作、还款要么全部成功，要么全部回滚

Revert

条件不满足时撤销本笔交易产生的状态变化

Trade Size

一次套利实际投入的资金规模

Optimal Trade Size

使预计净利润最大化的交易规模

Final Output

完成套利路径后最终获得的资产数量

Net Profit

扣除本金及各种执行成本后的真正利润

Day 9 一句话总结

Flash Loan 利用区块链交易的原子性，让套利者可以在同一笔交易中临时借入大量资金、完成套利并立即偿还；但闪电贷只解决本金问题，并不会消除 Price Impact、DEX Fee、Gas、MEV 等成本，因此真正的套利机器人需要寻找的不是“最大借款额”，而是能够产生最大净利润的 Optimal Trade Size。

笔记 041: 8月15号

作者：Krystal6633 | 时间：2026-08-15 15:58 UTC | 字数：4119 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

8月15号

大白话自学版

原文标题：三、套利的基本类型：按利润从哪里来分类（钱从哪里来）

翻译：这是文章的第三部分。前面教你“怎么看路况（执行框架）”，现在开始教你“世界上具体有哪几种捡钱的路子”。分类的标准不是看工具（比如用了交易所还是钱包），而是看“你赚的这笔钱，到底是因为哪个环节出了BUG才被你捡到的”。

原文：现实中的一条路径可能同时跨链、跨 DEX，又使用借贷或对冲。

翻译：实际操作中，你走的一条路线可能很复杂——既把资产从A链转到B链（跨链），又去去中心化交易所换币（跨DEX），甚至还借了钱或者开了空单对冲风险。

原文：

为了避免一笔案例被重复放进多个类型，今天的文章我们通过一个关键点归类：利润最主要来自哪一种价格关系没有闭合？

翻译：为了避免同一个例子在文章里出现好几次，我们定个死规矩：只问一个核心问题——“你赚的这笔差价，主要是因为哪两个地方的‘价格’没有及时对上号（没闭合）？”答案是什么，就把它归到那一类。

原文：

例如，一条交易同时经过两条链和两个 DEX。如果价差主要因为资金不能及时从一条链移动到另一条链，就归为跨链套利，不再重复放进跨 DEX。

翻译：比如，你既跨了链，又用了两个不同的去中心化交易所。假如这笔钱能赚到，主要是因为A链的钱转到B链太慢了，导致两边价格不一样，那就把它算作“跨链套利”。之后讲到“跨DEX”那一段时，不会再拿这个例子出来再讲一遍。

原文：协议和工具可以重叠，案例只放在一个主要类型里。

翻译：中间用的技术（比如桥、钱包、合约）可以很复杂，但案例只能归到最主要的那一个原因里，不重复归类。

【第二段（关于里面的数据和评级，极其重要！）】

原文：下面的评估只用于理解数量级。

翻译：下面我写的“这一单大概赚百分之几”，只是给你一个大概的感觉（比如是赚1%还是赚10%），不是精确的保证。

原文：新人友好度评估的是从公开数据、模拟和小额验证开始的难度，不是推荐等级。

翻译：我写的“新手友好度：高/中/低”，意思是你去查公开资料、自己拿小钱试一试这件事因不困难。绝对不是在告诉你“新手应该去做这个”。比如“极低”的意思就是：“新手千万别碰，碰了就是送死”。

原文：

即时路径按单次完整闭环的路径净收益估算，持仓类按对冲后的年化净利差估算，清算、拍卖和事件类按单次成功闭环估算。

翻译：文中写的收益数字，分三种算法：

如果是几秒钟内完成的（比如换币），就算“跑完这一圈净赚多少”；

如果是需要持有过夜的（比如开合约），就算“一年下来折算能赚百分之几”；

如果是去捡漏（清算、拍卖），就算“捡成功这一次赚多少”。

原文：区间已经尽量考虑当次可见的交易费、Gas与滑点，但不代表机会出现频率，也未必完整覆盖基础设施、失败尝试、税务和极端行情。

翻译：我算收益的时候，已经尽量扣掉了明面上的手续费、油费（Gas）、以及大额交易导致的价差损失（滑点）。但是，我没有算这个东西一天能出现几次机会，也没有算网络卡顿、你试错浪费的钱、赚了钱要交的税，以及比特币突然暴跌20%的极端情况。

原文：即使落在区间内，实际账户净收益仍要重新代回前面的公式。

翻译：哪怕我写的收益率是5%，你实际做完打开钱包一看，到底赚了多少钱，必须用前面文章教的“第1-7条执行公式”一笔一笔重新算。我写的只是理论值，你钱包里多出来的才是真钱。

【第1类：跨中心化交易所套利（逐句拆解）】

原文：这是最直观的一类：同一个资产在两个中心化交易所价格不同，在便宜的一边买，在贵的一边卖。

翻译：这是最好懂的一种。比如比特币，在“币安”卖100块，在“欧易”卖101块。那你就去币安买，去欧易卖，赚1块差价。

原文：真正执行时，两边账户里的资产并不是可以瞬间互换的同一个余额。

翻译：但实际操作时，你在币安账户里的钱，和在欧易账户里的钱，是两个独立的口袋，不能一键合并互转。

原文：通常需要提前在两边准备资金，完成买卖后再慢慢调回库存。

翻译：所以你必须同时在币安和欧易都提前存好钱和币。看到差价时，两边同时操作（左边买、右边卖）。做完之后，再找个不忙的时间，慢慢把两边账户的余额调平（比如把欧易多出来的币转回币安）。

原文：主要成本是两边的真实订单簿深度、交易费、资金调拨、提现限制和交易所风险。

翻译：这笔生意要扣掉的钱包括：你大额买入会把价格买贵（深度磨损）；给交易所交的手续费；转币的手续费；交易所每天限制提现额度（不让你转走）；以及交易所突然维护、拔网线、跑路的风险。

原文：屏幕上看到的最高买价与最低卖价，通常只代表很小的数量。

翻译：你在交易软件上看到的那个“最优价格”（比如101块），往往只有很少的币挂在那里。你如果买100个币，后面的价格可能就变成了102、103了，不能拿广告价当成交价。

原文：新人友好度：中等。逻辑直观，也适合先做订单簿监控；但实盘需要两个完成身份验证的账户、预放库存和资金再平衡。

翻译：新手做这个难度算中等。因为道理很好懂，适合先写个软件盯着价格。但是真要动手，你需要注册两个交易所（都要实名认证KYC），两边都要提前放钱，做完还要会调平库存，挺麻烦的。

【第5类：三角套利（你之前问过的，逐句拆解）】

原文：三角套利不需要两个市场里出现同一种资产的明显价差。它比较的是多个交易对之间的相对价格有没有闭合。

翻译：三角套利不用去找“比特币在A交易所比B交易所贵”。它只看：用美元→换苹果→换橘子→再换回美元，绕了一圈，手里的钱变多了还是变少了？如果变多了，就是价格没闭合（数学有漏洞）。

原文：例如从A出发： $1A \rightarrow 2B \rightarrow 6C \rightarrow 1.08A$ 。在忽略成本的模型里，绕一圈多出8%。

翻译：

举个数学例子：1个A币换成2个B币，2个B币换成6个C币，6个C币竟然能换回1.08个A币。在不算手续费的情况下，你白赚了8%。

原文：如果三步各收0.3%交易费，只扣这三次费用后大约剩1.0703A；Gas、报价变化和竞争成本还没有计算。

翻译： 如果这三步每步都要交0.3%的手续费，三次扣完，你的1.08就变成了1.0703。而且，还没扣油费（Gas）、交易过程中价格变动、以及跟机器人抢跑的成本。

原文： 常见路径包括：USDC → WETH → WBTC → USDC；ETH → LST → 稳定币 → ETH；直接兑换 A→C，与 A→B→C→A形成的相对价格不一致。

翻译： 常见的绕圈路线有：稳定币换以太坊，以太坊换比特币，比特币再换回稳定币。或者直接换的价格，跟绕一圈换的价格不一样，这就是机会。

原文： 每一步都会改变后续池子的状态，所以不能把三张静态报价直接相乘。必须从真实起始数量出发，按交易顺序重新计算每一步输出。

翻译： 记住！你先换第一步，池子里的价格就已经变了。绝对不能拿截图时看到的三个价格直接乘起来算利润。必须按照“先换第一笔→看实际得到了多少→再拿这些去换第二笔”的顺序，一步步真实计算。

原文： 新人友好度：低。需要逐腿计算价格影响、处理路由和回滚，肉眼相乘几张报价很容易得到假机会。

翻译： 新手做这个非常难。需要用程序一步步模拟，还要处理失败回滚。如果只靠肉眼拿计算器按几下，算出来的基本都是“假机会”（真动手就亏钱）。

【最后，关于其他类型，一句话点透】

文章里剩下的第2、3、4、6、7、8、9、10、11、12、13、14类，结构都跟上面一模一样，我不用全部逐句写出来，但把它们的核心大白话告诉你，你就知道在讲什么了：第2、3、4类（跨DEX、CEX-DEX、同池子）：都是“换地方买卖”，只不过换的地方从“中心化交易所”变成了“链上小摊（DEX）”，本质跟第1类一样，只是操作更复杂、手续费更坑。

第6类（跨链）：“A链便宜，B链贵，但钱不能瞬移，得等桥接”。赚的是“运输途中价格没变”的运气钱。

第7类（铸造赎回）：“官方承诺1块钱随时换1个币”，现在市场价0.98，你买了去找官方换回1块钱，赚2分。前提是官方通道没关闭。

第8类（基差）：“现货100块，3个月后的期货105块”。你买现货、卖期货，赌3个月后价格回归。赚的是“时间利息”，但要扛住中间的涨跌。

第9类（资金费率）：“开合约要交利息（资金费）”。你拿着现货，再反向开空单，对冲掉涨跌风险，专门吃这个利息。这是少数新手能做的。

第10类（利率）：“这里存钱给5%利息，那里借钱只要3%”，你借低息的钱存高息，赚2%利差。

第11、12类（清算、拍卖）：“别人爆仓了，你冲进去打折买他的抵押品”。全是机器人在抢，手动点鼠标绝对抢不过，新手直接跳过别看。

第13类（订单流）：“大户拉盘/砸盘后，池子价格偏了，你跟着捡漏”。同样是低延迟程序的世界，跟你没关系。

第14类（新闻事件）：“官方发公告说1个旧币换2个新币，但市场还在按1个旧币卖”。你买了去换，赚差价。前提是你得读懂法律条文，且换出来的新币卖得掉。

总结一句帮你收尾

这篇文章逐句翻译下来，作者真正想告诉你的是：

“世界上有14种看起来能捡钱的路，但每一条路下面都藏着对应的坑（滑点、Gas、时间差、机器人竞争）。‘新人友好度’是假的，‘钱包余额’是真的。新手只看1、7、9这三类，其余全部当课外书翻翻

笔记 042: 探讨在 cex-dex 套利中我一直存疑的问题，Day 9：【现货-永续价格错位套利】

作者：ninanren05 | 时间：2026-08-15 15:58 UTC | 字数：669 | 评分：69

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

探讨在 cex-dex 套利中我一直存疑的问题，Day 9：【现货-永续价格错位套利】

今天研究了一类很适合提高资金效率的套利：

当某个交易所的现货市场因为大户砸盘、流动性不足、清算等原因出现短暂低价，而其他以及永续合约价格仍然正常时，可以：

CEX/DEX

低价买入现货 + 同时做空永续合约

从而把刚买到的现货立即对冲成接近 Delta Neutral 的仓位。

Executable Spread / 可执行价差

需要同时考虑：

Spot 实际 VWAP

Perp 实际 VWAP

双边手续费

Slippage

Funding

提币费用

对冲误差

风险 Buffer

风险，异常价格不一定是套利

如果其他市场都是 \$1，而某交易所突然 \$0.8，首先应该问：

“为什么它便宜20%？”

可能原因包括：

Deposit / Withdrawal 暂停

某条链出现故障

Token 合约被攻击

项目迁移合约

即将下架

不同链 Token 已经脱锚

Token Mapping 错误

市场已经获得其他交易所尚未反映的信息

因此成熟系统除了 Price Engine，还需要：

Transferability Matrix

实时维护：

Exchange × Token × Network

的 Deposit / Withdrawal 状态、最近成功充提时间、到账速度、手续费等信息。

笔记 043: Day 12 笔记 (2026-08-16)

作者：Azhan | 时间：2026-08-15 16:03 UTC | 字数：1422 | 评分：75

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 笔记 (2026-08-16)

今天学了什么

1. Bruce 第 3 篇：Agent 配置 (印证我们的做法)

Bruce 讲怎么用 Agent 做套利研究，核心一句话：把 X 帖子丢给 Agent → 提取策略规则 → 找数据/写脚本/跑回测 → 验证这人过去真实战绩，而不是被几张盈利截图带偏。他让 Agent 回测“土狗王喊单”三个月推特，按“喊买就买 1WU、亏 10% 止损、喊卖就卖”规则，1 小时跑完——盈利概率居然挺高，但实盘 ≠ 回测。

这个用法跟我们一直在做的“四问审查 + 验证优先”完全一个思路，算互相印证。其他的（本地 vs 服务器、Telegram Topics 拆分、打卡接入）我们都已经做了。

2. Bruce 第 4 篇：LI.FI 执行实操 (干货最多)

真实成本案例：1,000 USDC 从以太坊跨到 Arbitrum，到手 997.5 USDC (先扣 25bps 服务费)，Gas \$0.2456，耗时 7 秒。显性成本 ≈ 27.5 bps。

结论很扎心：套利边际经常只有几十个 bps，光 LI.FI 服务费就 25 bps。价差 20 bps 必亏，40 bps 扣完也未必剩。这就是为什么"看到价差"离"能赚钱"隔着一整个成本模型——跟我们学的"净收益=价差-全成本"完全一致。

跨链套利的非原子性（最值钱的一条）："发现价差后临时跨链"有个致命坑——跨链不是原子操作，但套利窗口往往只有几秒。Quote 看着盈利，资产到对岸时机早没了。解法：多链预置资金 → 两端各自成交抓机会（本地资金，快）→ 闲时再用 LI.FI 做库存再平衡（慢没关系）。把"抓机会"和"跨链调仓"拆开。

安全边界确认：LI.FI MCP 是只读的，只返回 unsigned transactionRequest，不签名不广播。签名/广播/加仓必须交给隔离的钱包层 + 白名单 + 人工审批——跟我们的"二次确认铁律"完全一致。

API 缓存治理：/chains /tokens 这类慢变化数据要缓存、扫描要 debounce、记录 ratelimit 响应头、429 按 reset 退避不换 Key 绕过。

3. 检验打卡情况（教训）

这几天因为日期搞混（一直按 UTC 时间算，其实要用北京时间），漏了两天的卡（8/13 和 8/15）。今天认真核对：

8/05 ~ 8/12：全勤

8/13：缺

8/14：补上了

8/15：缺（又漏了一天——虽然学了 Bruce 3、4 篇，但没打卡）

8/16：今天（马上补）

教训：日期必须按北京时间算。已把打卡提醒加进 cron。

失败/教训

日期计算错误：一直用 UTC 时间导致漏卡。以后所有日期判断必须用北京时间（UTC+8）。

打卡要有提醒机制：不能靠记忆，要系统化（cron 每天提醒）。

下一步

查补卡机制（8/13 和 8/15 的缺口，看请假/补卡）

用 1,000 USDC 做一次只读 Route 评估（Bruce 第 4 篇的实操作业）

登记 API 限额表单（tally.so/r/PdQ4q1，8/31 截止）

笔记 044: Day 12：全自動套利系統架構 v1.1

作者：Web3Rason | 时间：2026-08-15 16:43 UTC | 字数：9462 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12：全自動套利系統架構 v1.1

Day 11 的結論是「我把讀數搬出實驗室的時候，忘了把作用域一起搬」。

今天的問題不在作用域，在維度。

十一天下來我的掃描器只認得一件事：兩個地方的價格差多少。

今天有一筆交易，毛利的 74% 跟任何「兩個地方」都沒有關係。

而同一天，我用來量那唯一一個維度的尺，被證明會讓假機會反號。

骨幹不動。但機會地圖十一天來第一次往外長了一格。

一、第十類機會：錢不在價差那一維

有一筆交易的收益來源是這樣的：

| 步驟 | 發生什麼 |

| :--- | :--- |

| 1 | 存入一個 ERC-4626 金庫 |

| 2 | 這個動作觸發底層協議 accrueInterest，把累積利息結算進去 |

| 3 | 金庫的 lastTotalAssets 被拉高 → 兌換率同一個區塊由 1.05713 → 1.05717 (+0.0031%) |

| 4 | 閃電貸把本金放大到 2,772 萬 USDC，同一筆交易內贖回 |

| 5 | 吃下 +869.65 USDC（佔該筆毛利 74%） |

+0.0031% 乘以 2,772 萬，就是這個數字。

這裡沒有「A 池比 B 池貴」。有的是：協議的會計狀態需要有人推一把才會更新，而推的那個人可以拿走一部分重估收益。

我的掃描器結構上看不到這一類——它只訂閱報價。

新增 L2.4：第十類「狀態跳變捕捉」

掃描訊號不是價差，是兩個欄位相乘：

「距離上次 accrue / 重估過了多久」×「那個重估函數是不是 permissionless」。

同一筆交易有 6 個人各自拆解，其中兩人對同一條腿給出 +869.65 vs 130.35 的互斥歸因，連 oracle 口徑都對不上 (1.0638 vs 0.973)。

我採用的是有 22 條 token transfer 逐筆對帳 + 源碼坐實合約身分的那一份，但引用前必須自己重跑。

還有一條同向的線索：某發行商開始支援把第三方發行的同一支股票代幣轉換成自家版本 —— 這在兩個發行商之間架出一條協議內建的鑄贖路徑，也是機械錨點，不是兩市場價差。

二、我量價差的那把尺，會讓假機會反號

我的掃描器用虛擬儲備代入 $x \cdot y = k$ 估 v3 那條腿。今天有人把它跟 Quoter 對照：

腿	誤差
v2 腿	0.16% / 0.38%
v3 腿	高估 7.6% ($\approx \$1,840$)

而且四層流動性過濾全部通過了還是穿透。

改用 Quoter 端到端重掃，同一組候選：

規模	掃描器原估	Quoter 實掃
$Q = 1 / 5 / 13 / 30$ WETH	+\$1,172	\$20 ~ \$2,709 (全負)

這不是精度不夠，是方向錯了。

我先前把「假陽性」歸因到靜態報價相乘、缺池深上界、費率檔位沒同讀深度、跨區塊幻覺 —— 今天多一個更基本的：估價公式本身對 v3 就不適用。

修正 L1.7：v3 腿唯一可信的報價源是 Quoter。

代價是每週期約 8,000 次 eth_call —— 這個代價必須先跟速率限制對帳，而不是先假設「精確一點總是好的」。

三、靜默換橋：我以為維度可以列完

Day 8 我列出靜默換橋的五個觸發維度：金額檔 / 方向 / slippage 參數 / integrator / 目標資產。我當時的心態是「列完就能防」。

今天的實測：同一組參數，87 秒內連打 12 次，第 3 次無預警換了橋。

欄位	12 次的表現
toAmount	完全相同
gas	只差 \$0.0016
executionDuration	7 秒 → 1,080 秒 (154 倍)

前兩個欄位完全掩蓋了這件事。

修正 L4.4：第六個維度是純時間隨機，也就是說 —— 這個清單列不完。

實務上的意思很直接：「我查過這條路，它是 7 秒」不成立。executionDuration 必須在執行前重查，而且不可以跨請求快取。

四、本金 / gas 比：這一格要排在價差前面

環境	每筆 gas	一輪 4 筆	佔 200 U 本金
主網 @ 54 gwei	$\approx \$20$	\$80	40%
L2	—	\$1 ~ +\$0.5	—

200 U 本金在主網要 break-even，需要 20%+ 的價差。那不存在。

我先前把主網和 L2 的差別理解成「利潤薄一點 vs 厚一點」。

不是。對小額本金來說，主網是類別性關閉，L2 才是開著的門。

新增 PO 閘門：先算 一輪 gas 總額 ÷ 本金。過不了這關，根本不必去掃價差。

這一步花不到一秒，卻能整條砍掉一個環境。

五、那個 40 倍的分歧，結案了 —— 是兩個分母

Day 10 我把清算線的成本表凍結，因為同一件事有兩個差 40 倍的數字：92.3% 和 2.4%。

今天有人用 160 天一手資料把它拆開了：

數字	它的分母	交叉驗證
92.3%	SVR 覆蓋範圍內的回收率	逐筆 92.4 / 91.1 / 89.8% (大額約 80%)，與協議官方 89.2% 對得上

| 2.4% | 佔全市場清算量的比例 | — |

沒有人算錯。兩個數同時成立，而且不可以互相取代。

解凍成本表，但引用時必須連分母一起寫。

順帶把這條賽道的輪廓一起量出來了（結論仍是不碰，但現在是知道為什麼不碰）：

- 「搶先看到價格」這件事不存在。拍賣的行情端點可以匿名訂閱，而且推的不是價格，是那筆還沒上鏈的預言機更新交易的完整 calldata。
- 所有人同一時刻拿到同一份 → 競爭維度只剩「估值準不準」和「敢出多高」。
- 門檻低到不像話：無白名單、無 KYC，押金 0.1 ETH 級 —— 全網所有參與者的押金總額只有 13.65 ETH（約 \$26,000）。
- 落敗零成本（不上鏈、不燒 gas），但代價是鏈上樣本結構性只含贏家，看不到對手出價分布。
- 規模與重尾：160 天 3,896 筆、毛獎勵 \$316,293；中位 \$0.03 vs 均值 \$81.18（差 2,700 倍）；金額最大 1 天佔全年 35.4%、最大 5 天佔 80%。

最重要的是選鏈那一維：

| 鏈 | 清算筆數 | 實際可得 |

| :--- | :--- | :--- |

| A 鏈（未上 SVR） | 2.4 筆/天 | \$0.61M/年，100% 歸搜尋者 |

| B 鏈（已上 SVR） | 2.7 筆/天 | 被抽走 89%，只剩約 \$60k/年 |

筆數幾乎一樣，可得池差 10 倍。「用筆數判斷市場大小」在同一份研究裡第二次翻車。

還有一條打掉我沒說出口的假設：早到沒有溢價。

新市場開張後 solver 數 1→7→22→34→43→半年後剩 4，回收率 25%→93%。

但爬坡期那 2.3 個月只佔全期毛獎勵 17.6%，過 13 一週就佔 76%。

暴發那週的最大贏家，是第 79 天才第一次中標的。

關鍵不是第一天到場，是暴發前兩週在場。

六、「未知」在我的型別裡不存在

今天有四個獨立的失效，來自三個不同的人、四個不同的子系統：

| 表現 | 後果 |

| :--- | :--- |

| inclusion=unknown 的成本被靜默忽略 | 等於當 0 扣 → 憑空製造假候選 |

| uncertainty buffer 為負時，公式在減去一個負數 | 反向抬高「保守」到帳（翻轉點：buffer 0.004 過關 / 0.006 拒絕） |

| 負的 quote_age 被判成 STALE | 把資料錯誤偽裝成正常老化 |

| L1 data fee 未知時當 0 | 成本整項消失 |

這不是四個 bug。是一個型別設計缺陷顯影了四次：

成本與時效的回傳型別裡，沒有 UNKNOWN 這個構造子。

升格為 L0 紀律：所有成本項、所有時效項，型別上必須三態

（KNOWN / UNKNOWN / N/A），合成規則是

「任一項 UNKNOWN → 結果 UNKNOWN → 不生成候選」。

這一項改完，上面四個一次性消失，而且未來同類錯誤會在型別檢查階段就被擋掉。

同一族還有兩條：

- 快取／摘要必須綁底層雜湊：某份 readiness 摘要的 ledger_digest 與帳本實際的 canonical digest 對不上 —— 代表這份摘要早就脫離底層，裡面每一個放行數字都作廢。

不一致就 STALE fail-close，不要沿用摘要裡的數字。

- 斷點續傳的 checkpoint 要以「實際輸出檔」為準，不能以 manifest 為準：

manifest 記了、結果還在記憶體沒落盤就中斷 → resume 直接跳過那批，而且覆蓋率報告看不出來。

七、幾條不改結論、但會改做法的

- 「通道存在」與「通道有火力」是兩個獨立開門。某個 PSM 的費率參數上完全成立

（買入側 0 bps、賣出側 20 bps，理論地板存在），但 getTotalReserves() 回 \$0、supply=0 —— 通道是空的。

這正好解釋了為什麼那個幣脫錨後沒有硬底承接（90 天內 28 天低於 \$0.98，最深 \$0.9634）。

→ 要吃「贖回／PSM 路徑」前，先讀儲備餘額與 supply，不是只讀費率參數。

同族：固定 1:1 不等於無限 1:1，可執行容量是五個數字取 min。

- 「借出即變現」的路徑不是原子完結。某筆被兩方爭論「淨利 \$166.67」的交易，

那筆錢其實是新借款後的剩餘現金 —— 交易結束後還留著約 1,034.6 的抵押與約 1,007 的債務，

距離清算線只有 0.013%。成本模型必須把這個殘餘倉位的清算風險算進去。

- 掃描器唯一能篩出的正值，可能就是風險場景本身。一輪完整掃描（200 資產 → 池 → 逐條驗證）

唯一那個「跨鏈 +0.5%」的候選，其實是一顆脫錨中的穩定幣（0.87–0.93）。

另一組監控 29 個生息穩定幣抓到 4% ~ 35%，其中已經有歸零的 —— 歸零不是折價回歸機會，是永久損失。

- 會收斂 ≠ 淨值為正，第二個獨立實盤：價差 6.6 進場 1 萬 U → 3.4 平倉（賺 3.2 點），但繳掉 2.8 點資費，淨只剩約 0.4 點，中途還被反彈到 9 扛過單。

收斂過程是震盪不是單調 —— 沒在第一時間跟上，就必須分散進場，不能一把追。

- 資金費率結算頻率變動，有可以預先監控的觸發條件：費率打滿上下限之後，下一週期可能由 8h 縮成 4h 或 1h → 監控欄位是「該標的當前費率是否貼著上下限」，不必等到事後從「期數欄不整齊」才發現。

- $LVR \approx \sigma^2/8$ 給了整門生意的量級：年化波動 80% → 約 8% 的池價值/年

由 LP 系統性送給套利者。這是上界不是我拿到的數。

另外要分清：IL 路徑無關、往返後歸零，是誤導性會計指標；LVR 路徑相依、單調累積，才是真的被搬走的錢。

- 清算掃描器的第一步不是找 $HF < 1$ ，是先綁 protocol version + chain + market。

某協議 V4 已於 2026-03-30 在主網啟用而 V3 還在跑，V4 改了兩件事：

可償還額由目標健康度決定（不再是固定 close factor）、清算獎勵隨 HF 下降而遞增。

綁錯版本，可清算金額與獎勵從輸入層就是錯的。

- L2 的 gas 要拆兩層（L2 執行費 + L1 資料費），而且取 L1 費的函數吃的是完整序列化的交易 bytes，

只丟 Router calldata 進去算出來是錯的。另外 Quoter 的估值、eth_estimateGas、最終執行成本是三種互不相等的口徑。

- 跨日重取報價時，基準檔本身的漂移比整條規模曲線還大：兩天之間 1,000 U 參考檔差 14.5 bps，

而同一輪內五個規模檔的差異最大只有 2.0 bps（差 7 倍以上）→ 規模曲線只能在同一短窗口內比。

- 極小額 A/B 測試會被取整噪聲整個吃掉：0.01 USDC 檔位上，1 raw = 1 bps，

足以造出「反向差 0.5 bps」的假方向差 → 任何方向性測試的最小檔位，先過「1 raw 等於幾 bps」這一關。

八、參數驗證進度（接續 Day 11）

| # | 參數 | 值 | 狀態 |

| :- | :- | :- | :- |

| 134 | 第十類：狀態跳變捕捉 | 兌換率 1.05713→1.05717 (+0.0031%) × 2,772 萬 = +869.65 USDC (佔毛利 74%) | 新增 (機會地圖首次外擴) |

| 135 | v3 腿的虛擬儲備估值 | 高估 7.6% (≈\$1,840)；同路徑 v2 腿只有 0.16/0.38% | 證偽 (會讓假機會反號) |

| 136 | v3 腿的可信報價源 | 只有 Quoter；代價約 8,000 eth_call/週期 | 新增 |

| 137 | 靜默換橋的觸發維度 | ~~已知五個~~ → 第六維是純時間隨機 (87 秒內 12 次，第 3 次換橋，7→1,080 秒) | 「可枚舉」作廢 |

| 138 | 本金/gas 比閘門 | 主網 54 gwei ≈ \$20/筆、一輪 4 筆 = 200 U 的 40%；L2 \$1 ~ +\$0.5 | 新增 (PO，先於價差) |

| 139 | SVR 回收率 | 92.3% = 覆蓋範圍內 (逐筆 92.4/91.1/89.8，官方 89.2% 對得上)；2.4% = 全市場佔比 | 解封 (兩個分母皆成立) |

| 140 | 選鏈的第一維 | 有沒有上 SVR：未上者 \$0.61M/年 100% 歸己 vs 已上者殘留約 \$60k/年 (差 10 倍，筆數幾乎一樣) | 新增 |

| 141 | SVR 進場門檻 | 無 KYC、押金 0.1 ETH 級、全網押金總額僅 13.65 ETH (≈\$26,000)、託管 128 區塊約 32 秒 | 新增 |

| 142 | SVR 的資訊對稱性 | 行情端點可匿名訂閱，推的是未上鏈預言機更新的完整 calldata → 「搶先看到價格」不存在 | 新增 |

| 143 | SVR 規模與重尾 | 160 天 3,896 筆/毛 \$316,293；中位 \$0.03 vs 均值 \$81.18 (2,700 倍)；最大 1 天佔全年 35.4% | 新增 |

| 144 | 新市場「早到溢價」 | 爬坡期 2.3 個月只佔毛獎勵 17.6%，週 13 一週佔 76%；最大贏家第 79 天才首次中標 | 證偽 |

| 145 | 清算供給的時間分布 | 平靜期 0.4 筆/天 vs 崩盤日 2,084 筆 (峰值單塊 815)；崩盤窗口內競爭者僅 19–27 個 | 新增 (舊結論收窄成「單筆觸發機制」) |

| 146 | 清算規則的版本維度 | 某協議 V4 已於 2026-03-30 主網啟用、V3 仍在跑；V4 改用目標健康度 + 獎勵隨 HF 遞增 | 新增 |

| 147 | 「未知」的型別 | 必須三態 (KNOWN/UNKNOWN/N/A)；同一缺陷在 4 個子系統各顯影一次 | 新增 (LO 紀律) |

| 148 | 快取/摘要的可信條件 | 必須綁底層帳本雜湊；ledger_digest 對不上 → 摘要內所有放行數字作廢 | 新增 |

| 149 | L2 gas 的層數 | 兩層 (L2 執行 + L1 資料)；取 L1 費要餵完整序列化 tx bytes；三種 gas 口徑互不相等 | 新增 |

| 150 | 套利總蛋糕的量級 | $LVR \approx \sigma^2/8 \rightarrow \sigma=80\%$ 時約 8% 池價值/年 (上界，非可得) | 新增 (文獻，非實測) |

| 151 | 「通道有無火力」 | 費率參數成立 ≠ 通道可用：某 PSM getTotalReserves()=\$0、supply=0 (該幣 90 天內 28 天 <\$0.98) | 新增 |

| 152 | 借出即變現的殘餘倉位 | 交易後仍留約 1,034.6 抵押/1,007 債務，距清算線 0.013% | 新增 (第三個獨立樣本) |

| 153 | 正值候選的偽陽性來源 | 唯一「跨鏈 +0.5%」候選是脫錨幣 (0.87–0.93)；29 個生息穩定幣監控中已有歸零標的 | 新增 |

| 154 | 會收斂 ≠ 淨值為正 | 第二個獨立實盤：賺 3.2 點、繳 2.8 點資費，淨約 0.4 點，中途被反彈扛單 | 第二樣本 (強化) |

| 155 | 結算頻率變動的前置訊號 | 費率貼住上下限 → 下一週期可能 8h 縮成 4h/1h | 新增 (機制陳述，無改頻樣本) |

| 156 | 極小額 A/B 的取整噪聲 | 0.01 USDC 檔位 1 raw = 1 bps，足以造出 0.5 bps 假方向差 | 新增 |

157	跨日基準檔漂移	兩天之間 1,000 U 檔差 14.5426 bps vs 同輪內規模曲線最大 2.0 bps (差 7 倍)	新增
158	新幣 LP 的分鐘級 regime	97 個倉位淨 +2.04% (47 賺/21 虧/29 平), 持有中位 11.1 分鐘, 最虧一筆 22.48%	新增 (決勝負的是退出時點)
159	LP 動態調區間的定位	420 天回測 +2.1% vs 死守 24.9% vs 純持有 40.2%; 但上漲月輪死守 2.2 點; 門檻 0.5% 時調倉成本達手續費 692%	新增 (是保險不是收益工具)
160	LayerZero OFT 到帳延遲	28 秒 (兩端 GUID/地址/數量一致, 2 DVN 皆驗過)	新增
161	Morpho Blue 閃電貸費	0 (原推斷 3.6 bps 是錯的) → 併入閃電貸費離散表: 0 / 0 / 5 bps 三檔	修正

九、一句話

Day 8: 量具本身的誤差, 比我要找的東西還大。

Day 9: 我把「我當時的實驗設定」寫成了「世界的性質」。

Day 10: 我的量具可能整台是壞的, 而它會回報一切正常。

Day 11: 儀器沒壞、數字也對, 錯的是我把讀數搬出實驗室那一步。

Day 12: 我一直在把尺調準, 但我只有一把尺, 而且它只量一個方向。

前四天的教訓有一個共同前提 —— 我假設「要找的東西」就在我量的那條軸上, 剩下的只是量得準不準、能不能外推。

今天那筆交易的 74% 毛利不在那條軸上。

它不是「價差比較大」, 是根本不是價差: 協議的帳本需要有人推一把, 推的人拿走重估的差額。

而諷刺的是, 同一天我還發現那唯一一條軸我也量錯了 ——

v3 那條腿用錯公式, 高估 7.6%, 足以讓一整組負收益的候選在我的螢幕上顯示 +\$1,172。

所以今天真正的兩句話是:

一、把尺調準, 不會讓你看到尺量不到的維度。

十一天下來我做的幾乎全是「把尺調準」: 口徑、路由、延遲、作用域、下標。

那些都必要, 但它們的上限就是「把這一個維度量對」。

二、而「維度可以列完」本身就是錯的假設。

靜默換橋那條說得最白 —— 我列了五個觸發維度, 以為列完就能防, 第六個是純時間隨機。清單列不完, 所以正解不是繼續列, 是執行前重查、不要快取。

同樣的形狀出現在型別那一節: 我的成本欄位只有「值」, 沒有「不知道」。

不是我把某個值算錯了, 是我的型別系統裡不存在「未知」這個狀態,

於是每一個未知都被靜默轉成 0, 而 0 是一個會讓候選通過的數字。

我花了十一天讓一個維度上的數字變準。今天的帳單是: 地圖上還有一格我根本沒畫。

笔记 045: 今日共学打卡 | 从“能报价”走到“能否真正闭环”

作者: Quinn | 时间: 2026-08-15 19:15 UTC | 字数: 569 | 评分: 51

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日共学打卡 | 从“能报价”走到“能否真正闭环”

今天完成了 P7 的核心验证: 不再只看单段报价或模拟结果, 而是在本地 Fork 中把真实 V3 callback、V4 settlement、native ETH 收取、WETH 会计与两层偿债顺序串起来, 检查这条路径能否真正形成资产闭环。

结果很清楚:

\- 路径可以抵达真实 V4 settlement 与 native ETH take;

\- V3 内层偿债能够完成;

\- 但在偿还最外层 WETH 债务时, 当前有限规模输出不足;

\- 因此整个生命周期正确原子回滚, 并被分类为:

OBSERVE_INSUFFICIENT_CYCLE_OUTPUT

这不是“测试没做好”, 而是系统在真实约束下得出了正确的拒绝结论: 中间换币、拿到 native ETH, 甚至完成部分 settlement, 都不等于套利成功; 只有所有债务偿还后, 终点资产仍高于起点, 才可能谈利润。

今天最大的收获是进一步理解了套利的本质:

价差只是线索

报价只是局部证据

资产闭环、偿债顺序和最终余额增量

才是可执行利润的真正检验

全程仍然保持严格边界：只读、本地 Fork、无私钥、无签名、无授权、无广播、无真实资金。

笔记 046: Day 12 (08/16) 学习总结，素材来自今日同学精华：

作者：cjdaocoin | 时间：2026-08-16 01:01 UTC | 字数：427 | 评分：48

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 (08/16) 学习总结，素材来自今日同学精华：

1. Qy：DEX 间套利，是利用同一种资产在两个去中心化交易所或流动性池中的短暂报价差异，在价格较低的一边买入，再到价格较高的一边卖出

2. fangliaoan：三是做市执行算法：TWAP/VWAP
把大单拆小、降低市场冲击，但暴露时间越长，信息泄露和逆向选择风险越高，要在“冲击成本 vs 时间风险”之间找平衡

3. twone：这次回测最重要的产出不是结论，而是验证了“假设 → 公开数据 → 扣成本 → 净期望”这条流水线能跑通：零成本（公开接口）、一小时（写脚本）、可复现（脚本在仓库里）

4. Azhan：Bruce 讲怎么用 Agent 做套利研究，核心一句话：把 X 帖子丢给 Agent → 提取策略规则 → 找数据/写脚本/跑回测 → 验证这人过去真实战绩，而不是被几张盈利截图带偏

以上为今日从同学打卡中学到的知识点，继续积累执行与验证能力。

笔记 047: 今日完成

作者：Louis lue | 时间：2026-08-16 03:31 UTC | 字数：1481 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日完成

- 审核并修正单向价差、同链往返 Quote、动态第二步金额、re-quote、freshness、候选状态机和拒绝原因学习答案。
- 实现只读往返候选模型：第二腿输入严格等于最新第一腿实际输出；使用 bigint 计算闭环 gross PnL，并显式检查 net PnL。
- 实现稳定 ROUND_TRIP_REASON_ORDER，策略层、Repository 和公开 API 共享同一顺序。
- 新增 append-only migration 004：候选、规范化拒绝原因和状态迁移表；保留 migration 001-003 不变。
- 实现 Candidate Repository：migration hash、v4 重开、事务内幕等、两腿 Observation 的 chain/Token/decimals/amount 闭环校验、完整性派生、正 gross/net PnL 门禁及 append-only 触发器。
- 新增脱敏 /api/candidates/status，连接真实 Candidate Repository；当前没有真实双腿候选时返回 empty。
- 最终独立门禁通过并完成 Git 提交。

关键结论

- 单向报价差异不是套利；完整候选必须闭环回到同链同地址同精度的起始 Token。
- 第二腿 amountIn 必须来自最新第一腿 amountOut，不能使用预设档位或显示格式化金额。
- gross PnL 为正仍不足；net PnL 非正、成本不完整、状态证据未知或执行保护未知都必须拒绝。
- LI.FI 没有原生 block hash，不能伪造严格同状态，因此本日没有生成真实 validated 候选。
- Promise.race 只限制等待时间，不取消底层请求；当前 Quote callback 必须保持无持久化副作用。

证据

- 学习文档：docs/2026-08-16-research-questions.md。
- 策略模型：packages/strategy/src/round-trip-candidate.ts。
- Migration：packages/database/migrations/004_round_trip_candidates.sql。
- Repository：packages/database/src/candidate-repository.ts。
- Git：52a2909 [verified] feat: add read-only round-trip candidates。
- 自动验证：12 个测试文件、82 项全部通过；TypeScript 通过；npm audit 0 vulnerabilities。
- 独立审查：passed=true，无阻断安全或逻辑问题。
- 安全状态：executionEnabled=false，无 Wallet、Signer、私钥、approve、签名或广播能力。

未完成 / 下一步

- 尚未执行并保存真实双腿候选；当前成本和状态证据不满足 validated 门禁。
- total deadline 尚不能取消底层 Promise；未来如引入副作用必须加入 AbortSignal/终态 token。
- 下一步补充 Candidate API 200/503 脱敏集成测试，再推进调度器和持续采集；完成成本证据前不发送机会告警。

笔记 048: 跨所套利系统学习笔记：从策略发现到开仓、平仓与执行优化

作者：wy2221 | 时间：2026-08-16 04:04 UTC | 字数：5784 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

跨所套利系统学习笔记：从策略发现到开仓、平仓与执行优化

这次讨论主要分析了 crypto-trading-open 这套套利框架。相比单纯“看到价差就下单”的机器人，它已经形成了比较完整的思路：

行情采集

- 历史基准
- 套利机会判断
- 网格目标仓位
- 双腿执行
- 成交确认
- 分层平仓
- 单腿异常恢复

真正值得学习的不是某几个参数，而是它如何把策略判断、订单执行和风险恢复分开处理。

一、套利的核心不是“有价差”，而是“价差异常”

最简单的套利程序可能这样判断：

交易所A = 100

交易所B = 101

差1%

→ 套利

但现实中，不同交易所之间往往长期存在固定溢价。

比如BTC长期可能是：

Lighter比EdgeX贵0.04%

这0.04%并不是套利机会，而是两个市场天然存在的价格结构。

因此脚本引入了一个很重要的概念：

天然价差。

它持续记录两个交易所之间的价差，每分钟采样一次，再根据最近约24小时的数据计算历史中位数。

例如过去长期价差：

0.03%

0.04%

0.05%

0.04%

0.03%

0.04%

那么天然价差约为：

0.04%

以后真正关注的是：

当前价差

-

天然价差

-

手续费

剩下多少。

假设：

当前价差 0.13%

天然价差 0.04%

双边手续费 0.06%

真正剩余：

0.13 - 0.04 - 0.06

= 0.03%

如果用户要求额外至少：

0.02%

才开仓，那么这笔交易才成立。

这个思路非常值得借鉴：

不要把市场长期存在的结构性价差误认为利润。

历史基准最好用中位数而不是平均值，因为偶尔出现的异常报价不容易把中位数拉偏。

二、资金费率套利也是同样的思想

程序还会记录：

交易所A资金费率

交易所B资金费率

两者差值

例如：

EdgeX：0.005%

Lighter：0.025%

理论上可以：

EdgeX做多

Lighter做空

从资金费率差获得：

0.020%

但不能因为某一瞬间资金费率高就马上交易。

程序还会检查过去一段时间：

资金费率差是否大部分时间存在

波动是否过大

是否经常突然跳变

本质上是在回答：

这是不是一个持续的结构收益，还是刚好这一刻出现的异常？

不过源码中资金费率部分没有价差套利成熟。

它虽然计算了“天然资金费率差”，但没有真正完整进入最终开仓公式，而且历史数据与实时判断还存在百分比单位不一致的问题。

所以这部分更适合作为设计参考，而不能直接照搬。

三、网格策略的核心是“目标仓位”，不是“每出现一次信号就买一次”

这是整个系统另一个很重要的思想。

假设参数：

第一格：0.01%

每格：0.01%

每格仓位：0.8 ETH

最多：10格

那么：

价差0.01% → 目标0.8 ETH

价差0.02% → 目标1.6 ETH

价差0.03% → 目标2.4 ETH

.....

系统真正计算的是：

本次应交易数量

=

目标仓位 - 当前仓位

例如：

当前已经：

0.8 ETH

价差扩大到第二格，目标：

1.6 ETH

那么只补：

0.8 ETH

而不是重新开1.6 ETH。

这种设计有几个优点：

不会因为策略循环重复运行而不断重复开仓

仓位天然对应当前价差等级

价差扩大就加仓

价差收敛就减仓

这比“触发一次信号开一笔”的逻辑稳定得多。

四、为什么还要拆成小订单

例如：

一格仓位 = 0.8 ETH

单次执行 = 0.2 ETH

那么一格会拆成4次。

目的是降低：

盘口冲击

滑点

订单拒绝

单腿大额暴露

所以策略层决定：

应该增加0.8 ETH

执行层决定：

实际每次只执行0.2 ETH

这也是一个重要架构原则：

策略决定经济目标，执行器决定如何安全达到目标。

两者不要混在一起。

五、双腿执行速度的正确优化方向

跨所套利最大的风险是：

一条腿成交

另一条腿还没有成交

这时套利仓位瞬间变成了方向性仓位。

因此执行优化的重点不是Python计算快几毫秒，而是：

两条腿尽量同时发送

成交事件尽快知道

出现单腿后尽快恢复

脚本采用了：

`create_task()`

+

`asyncio.gather()`

让两个交易所的订单并行发送。

如果是Lighter内部现货—永续，还进一步使用：

一次批量交易

→ 两张订单一起`send_tx_batch`

这样能进一步缩小两腿发送时间差。

六、私有WebSocket为什么重要

订单发出以后，最慢的方法是：

每500ms查询一次：

成交了吗？

成交了吗？

成交了吗？

如果订单在第20ms已经成交，也可能第500ms甚至1秒以后程序才知道。

更好的方法是账户私有WebSocket：

交易所成交

→ 主动推送给机器人

→ 机器人立即收到

Lighter使用：

account_all

account_all_orders

trades

来接收：

订单状态

成交数量

成交价格

持仓变化

而且系统做了一个很好的优化：

早到成交事件缓存

有时会出现：

订单已经成交

→ WebSocket消息先到了

→ 下单API函数还没返回

如果程序只能等下单API返回以后才注册成交监听，就会漏掉这个事件。

所以它采用：

成交事件先到

→ 暂存在内存

订单函数返回

→ 先检查有没有提前到达的成交

→ 有就直接使用

→ 没有再等待Event

这是非常值得跨所套利系统借鉴的。

七、订单执行快，并不代表完整交易已经完成

应该区分三个时间：

① 发单时间

② 实际成交时间

③ 系统最终确认时间

例如：

31ms 两腿已经发送

480ms 私有WS已经收到成交

1400ms 系统才完成订单终态确认

1800ms 仓位核验和账本才完成

不能简单说：

这笔订单执行用了1.8秒

真实成交可能在500ms附近已经完成。

后面的时间主要用于：

确认订单状态

确认两边真实仓位

确认没有残余订单

更新交易账本

所以优化时应该分成：

交易速度优化

和

确认链路优化

前者直接影响成交价格和单腿风险。

后者主要影响：

什么时候可以安全开始下一笔交易

出现异常后多久能启动恢复

资金周转效率

八、网格平仓的基本思想

这个项目普通网格平仓采用的是：

价差回到上一层，就逐格减仓。

例如：

0.01% → 第一格

0.02% → 第二格

0.03% → 第三格

如果当前持有第三格：

2.4 ETH

当价差回落到第二格区域：

目标仓位变成1.6 ETH

于是平：

0.8 ETH

再回到第一格：

1.6 → 0.8

最后完全收敛：

0.8 → 0

这比必须等待价差完全反转以后一次性平仓更合理。

九、项目平仓逻辑与你两个版本的区别

你的旧Phase 5核心是：

本层局部残差已经收敛

→ 平仓

优点：

反应快

短暂收敛也能利用

资金周转快

缺点：

容易被盘口抖动触发

单笔利润很薄

真实滑点和手续费容易吞掉利润

尤其你之前实盘已经发现：

模型看起来盈利

实际成交却可能亏损

说明只看“价差收敛”是不够的。

你的当前版本是：

当前真实可执行净收益 $\geq 7.5\text{bps}$

并连续保持500ms

→ 平仓

它实际上增加了两个保护。

第一层：利润保护

不是：

价差看起来已经收敛

而是：

现在真的把两边仓位平掉

扣掉执行成本以后

至少还能赚7.5bps

第二层：时间确认

要求：

连续500ms成立

过滤：

几十毫秒盘口跳动

瞬时错误报价

单帧假利润

因此它本质上是：

从“市场结构平仓”升级成“真实经济收益平仓”。

对于真实资金交易，我更认可现在这个方向。

十、但500ms不是越长越好

例如利润窗口：

0ms：8.2bps

100ms：8.5

200ms：8.0

300ms：7.9

400ms：8.1

450ms：突然消失

旧版本可能在100ms就成交。

500ms版本不会平仓。

所以它牺牲的是：

短暂机会利用率

换来：

更高信号质量

适合程度取决于真实市场：

如果盈利窗口经常持续：

1~5秒

500ms影响不大。

如果大量机会只存在：

100~300ms

500ms可能明显降低成交频率。

所以正确的方法不是凭感觉改成：

100ms

300ms

500ms

1000ms

而是统计：

达到7.5bps以后

利润窗口平均持续多久

P25是多少

P50是多少

P75是多少

再决定确认时间。

十一、跨交易所套利功能确实存在

项目不仅支持Lighter内部套利，也支持：

Lighter ↔ EdgeX

Lighter ↔ Paradex

对：

BTC

ETH

SOL

HYPE

等永续合约进行跨所套利。

还支持：

不同交易所

不同品种

不同市场类型

例如：

EdgeX PAXG PERP

↔

Lighter XAU PERP

理论上也可以配置：

交易所A现货

↔

交易所B永续

但默认跨所功能处于关闭状态。

值得学习的一点是：

有持仓以后，系统记住原来的买卖交易所，而不是每次重新选择当前最大价差方向。

这样平仓时会明确：

原来买A卖B

→ 平仓一定卖A买B

不会因为当前另一个方向价差更大而把减仓错误变成反向开仓。

十二、这个项目最值得借鉴和最不值得照搬的地方

值得借鉴

历史中位数作为天然价差

目标仓位而非单次信号

分层开仓、分层减仓

双腿并行发送

Lighter批量IOC

私有WebSocket成交确认

早到成交事件缓存

部分成交按照真实数量处理

有持仓时记住原交易方向

单腿出现后进入补齐或回滚

不适合直接照搬

主要依赖内存保存网格仓位

重启后缺乏完整持仓重建

持仓不一致主要只是报警

部分平仓路径存在逻辑顺序问题

大仓位拆成大量0.2批次导致退出较慢

普通平仓主要看网格收敛，而不是最终真实净利润

十三、对套利系统最重要的几个思维

通过这次分析，可以总结出几条比具体代码更重要的原则。

1. 策略与执行必须分开

策略：

我想把仓位从0.8提高到1.6

执行器：

我怎样用4笔0.2安全做到

不要让策略直接控制每一次API下单。

2. 看到机会不等于存在利润

必须经过：

历史天然价差

手续费

滑点

真实深度

执行预留

再判断。

3. 平仓比开仓更重要

开仓失败只是少赚一笔。

平仓失败可能意味着：

单腿风险

长时间占用资金

利润变亏损

因此应该：

ENTRY严格

EXIT优先

RECOVERY最高优先

4. WebSocket负责速度，权威查询负责证明

正确组合是：

WebSocket：

快速告诉你发生了什么

REST/官方订单查询：

最终确认订单事实

真实仓位：

确认经济结果

不能只用其中一种。

5. 最终应该以真实可执行利润作为平仓依据

比起：

价差回到某一格

更稳健的是：

该层真实开仓成交价

+

当前真实平仓VWAP

-

手续费

-

滑点

-

资金费

-

执行预留

≥ 最低净利润

因此，你现在：

7.5bps净收益

+

500ms确认

这个方向比旧Phase 5单纯局部收敛更接近生产级平仓逻辑。

最后的认识

一套成熟的跨所套利系统，真正的核心并不是“发现价格差”，而是形成闭环：

历史告诉我什么是正常价差

→ 当前异常程度决定目标仓位

→ 执行器快速完成双腿交易

→ 私有事件快速发现成交

→ 权威查询确认真实结果

→ 价差收敛且真实利润满足要求时退出

→ 单腿异常能够自动恢复

→ 重启以后仍然知道自己拥有什么仓位

套利策略本身通常并不复杂，真正决定长期能不能赚钱的是执行损耗、成交确认、平仓纪律和状态恢复。

这也是这次分析最值得保留的思路。

笔记 049: Day 9 | 双边对冲执行、Legging Risk 与半自动套利工具

作者：HerschelLi-31 | 时间：2026-08-16 04:51 UTC | 字数：18553 | 评分：85

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 9 | 双边对冲执行、Legging Risk 与半自动套利工具

1. 今天的核心目标

Day 8 我研究的是：

`\`

`\boxed{`

`HowDidARearArbitrageTradeHappen?`

`}`

`]`

今天开始进入一个更接近真实交易系统的问题：

`\`

`\boxed{`

`IFoundAnOpportunity, \ ButHowDoIExecuteBothLegsSafely?`

`}`

`]`

套利在理论上经常写成：

`\`

`Buy\_A + Sell\_B`

`]`

但真实世界的问题是：

A 和 B 几乎永远不是数学意义上的“同时完成”。

于是套利系统必须处理：

双边下单

Order Sequence

Fill 状态

Position Balance

Legging Risk

Spread Monitor

Funding Rate

Position Limits

Exit

API / Exchange Failure

Liquidation / ADL

Day 9 的目标不是学会点击某个套利软件，而是：

```
\[\nboxed{\nReverseEngineerTheExecutionSystem\n}\n]
```

也就是通过 Taoli Tools、QQQ.BOT 这类半自动工具，理解一个真正的 Hedged Arbitrage Execution Engine 应该有哪些模块。

Taoli Tools 当前将自己定义为半自动 U
本位对冲套利工具，核心功能是同时反向交易两个交易对，并监控价格、价差、持仓、资金费率和风险；它支持 CEX、DEX 和 Perp DEX。

2. 套利理论为什么比执行简单？

理论：

```
\[\nP_A=100\n]\n\n\[\nP_B=101\n]
```

于是：

```
\[\nBuy_A@100\n]
```

同时：

```
\[\nSell_B@101\n]
```

Spread：

```
\[\n1%\n]
```

看起来已经锁定：

```
\[\n1%\n]
```

但真实执行过程是：

```
\[\nSignal\n]\n\n↓\n\n\[\nSendOrder_A\n]\n\n↓\n\n\[\nSendOrder_B\n]\n\n↓\n\n\[\nExchange_AResponse\n]\n\n↓
```

```
\[
Exchange\_BResponse\
]
```

↓

```
\[
ActualFill\_A\
]
```

↓

```
\[
ActualFill\_B\
]
```

这中间可能已经过去：

```
\[
100ms,\ 1s,\ 5s\
]
```

而市场可能已经发生变化。

因此：

```
\[
\boxed{\
SignalSpread\
eq\
RealizedSpread\
}\
]
```

3. 什么是 Two-leg Hedge？

假设 BTC 在两个交易所：

Exchange A：

```
\[
BTC=100000\
]
```

Exchange B：

```
\[
BTC=100500\
]
```

理论 Price Arbitrage：

```
\[
Long\_A\
]
```

```
\[
Short\_B\
]
```

如果金额完全匹配：

```
\[
Notional\_A\
\approx\
Notional\_B\
]
```

则：

```
\[
\Delta\_{{portfolio}}\approx0\
]
```

这意味着 BTC 本身上涨或下跌对组合价值的影响被大幅抵消。

所以：

```
\\  
\boxed{\  
ArbitragePnL\  
}\  
]
```

主要希望来自：

```
\\  
Spread\  
]
```

或：

```
\\  
Funding\  
]
```

而不是：

```
\\  
BTCPriceDirection\  
]
```

4. 什么是 Legging Risk ?

这是 Day 9 最重要的概念。

假设目标：

```
\\  
Long\ 1BTC\_A\  
]
```



```
\\  
Short\ 1BTC\_B\  
]
```

理论：

```
\\  
NetExposure=0\  
]
```

但是实际：

A 成交

```
\\  
Long\ 1BTC\  
]
```

B 没成交

那么：

```
\\  
NetExposure  
=====
```

```
+1BTC\  
]
```

此时你已经不是：

```
\\  
Arbitrageur\  
]
```

而变成：

```
\
DirectionalTrader\
]
```

这就是：

```
\
\boxed{\
LeggingRisk\
}\
]
```

即：

多腿交易中，一条腿已经成交，另一条腿没有按预期成交，导致出现临时或持续的单边风险暴露。

5. 为什么 Legging Risk 很危险？

假设 BTC：

```
\
100000\
]
```

你先买入：

```
\
1BTC\
]
```

但是 Short Leg 没有成交。

下一秒 BTC：

```
\
100000\rightarrow99500\
]
```

你立刻：

```
\
-$500\
]
```

而你原本预计的套利利润可能只有：

```
\
$50\
]
```

所以：

```
\
LeggingLoss\
\gg\
ExpectedArbitrageProfit\
]
```

这说明套利策略常见一个特征：

```
\
\boxed{\
SmallExpectedProfit\
+\
LargeTailExecutionRisk\
}\
]
```

因此真正的 edge 不只是发现 Spread，而是：

```
\
\boxed{\
ReliableHedgedExecution\
}\
]
```

]

6. 三种基本下单 Sequence

Taoli Tools 当前提供三种典型双边执行顺序：

Parallel : A、B 同时发单；

A then B : A 返回下单成功后再提交 B；

B then A : 反过来执行。

其文档也明确提醒，无论哪种 Sequence，都仍可能出现只有单边成交的情况。

这三个模式非常值得研究。

7. Parallel Execution

结构：

```
\[\nSend\_A\
```

]

和：

```
\[\nSend\_B\
```

]

尽可能同时。

优点：

```
\[\nExecutionTime\downarrow\
```

]

两条腿面对的市场状态更接近。

所以理论上：

```
\[\nSpreadDecayRisk\downarrow\
```

]

但是问题：

如果：

```
\[\nA=Success\
```

]

```
\[\nB=Failed\
```

]

还是会出现：

```
\[\nLeggingRisk\
```

]

因此：

```
\[\n\boxed{\nParallel\neq\nAtomic\}
```

]\n]

这是非常重要的区别。

8. Sequential Execution

例如：

```
\[\nA\\rightarrow B\\n]
```

流程：

```
\[\nSubmitA\\n]
```

↓

```
\[\nConfirmA\\n]
```

↓

```
\[\nSubmitB\\n]
```

它的好处是：

如果 A 根本提交失败：

```
\[\nB\\n]
```

可以不执行。

Taoli Tools 当前的 A then B / B then A 就采用这种逻辑：第一条腿下单失败时，第二条腿不会继续提交。

但它产生新的问题：

```
\[\nLatency\\_{A\\rightarrow B}\\n]
```

期间：

```
\[\nPrice\\_B\\n]
```

可能改变。

9. 哪条腿应该先执行？

没有一个永远正确的答案。

假设：

Exchange A

Liquidity 很深

API 稳定

很容易成交

Exchange B

Liquidity 较差

Order 经常失败

那么一种思路是：

```
\[\n\\boxed{\nExecuteHardLegFirst\n}\n]
```

Taoli Tools 当前文档也建议，可以优先执行更容易失败的一边。

因为如果困难腿成功：

```
\[\nEasyLeg\
```

]通常更容易快速 Hedge。

但这仍然不是绝对规则。

需要考虑：

```
\[\nLiquidity\
```

```
\[\nLatency\
```

```
\[\nMarketImpact\
```

```
\[\nAPIStability\
```

```
\[\nOrderType\
```

10. “API 返回成功”不等于真正成交

这是 Execution System 很容易踩坑的地方。

API 可能返回：

```
\[\nOrderAccepted\
```

]但这并不一定意味着：

```
\[\nFilled\
```

甚至在异常情况下：

```
\[\nAccepted\
```

]也不一定代表交易所最终真的生成了有效订单。

Taoli Tools 文档明确提醒，其执行逻辑中如果交易所 API 返回成功但实际没有成功下单，系统仍可能先将其视为下单成功。

所以系统不能只检查：

```
\[\nSubmitResponse\
```

]还必须继续检查：

```
\[\n  \boxed{\n    OrderStatus\
```

```
  }\n
```

以及：

```
\[\n  \boxed{\n    FilledQuantity\
```

```
}\  
]
```

11. Order 的生命周期

应该逐渐建立：

```
\\  
Signal\  
]
```

↓

```
\\  
OrderSubmitted\  
]
```

↓

```
\\  
OrderAccepted\  
]
```

↓

```
\\  
PartiallyFilled\  
]
```

↓

```
\\  
Filled\  
]
```

或者：

```
\\  
Cancelled\  
]
```

```
\\  
Rejected\  
]
```

```
\\  
Expired\  
]
```

因此：

```
\\  
\boxed{\  
Submitted\  
eq\  
Filled\  
}\  
]
```

是 Day 9 必须牢牢记住的概念。

12. Partial Fill

假设目标：

```
\\  
Long\_A=1BTC\  
]
```

```
\\  
Short\_B=1BTC\  
]
```

结果：

A：

```
\\  
Filled=1BTC\  
]
```

B：

```
\\  
Filled=0.6BTC\  
]
```

于是：

```
\\  
NetExposure  
=====
```

1-0.6

```
0.4BTC\  
]
```

仍然存在：

```
\\  
DirectionalExposure\  
]
```

所以 Hedge Engine 不应该只判断：

A filled?

B filled?

而应该比较：

```
\\  
\boxed{\  
FilledNotional\_A\  
}\\  
]
```

和：

```
\\  
\boxed{\  
FilledNotional\_B\  
}\\  
]
```

13. Position Imbalance

定义：

```
\\  
E\_A  
=====  
Position\_A\  
]
```

```
\\  
E\_B  
=====  
Position\_B\  
]
```

理想 Hedged Position：

```
\\  
E\_A+E\_B\approx0\  
]
```

可以定义一个简单：

```
\[\n\boxed{\nImbalance\n=====\n|E\_A+E\_B|\n}\n]\n]
```

如果：

```
\[\nImbalance\n]\n]
```

超过某个 Tolerance：

```
\[\n\epsilon\n]\n]
```

系统应该停止继续扩大仓位。

Taoli Tools 当前提供 Flat 持仓平衡检查；如果两边持仓绝对值差异达到设定的单次 Size，系统会提示并暂停该次下单。

这反映一个重要系统思想：

```
\[\n\boxed{\nPositionCheckBeforeOrder\n}\n]\n]
```

14. Position Limit

执行器还需要：

```
\[\nMinPosition\n]\n]
```

和：

```
\[\nMaxPosition\n]\n]
```

例如：

```
\[\nMaxA=$20,000\n]\n]
```

意味着：

无论 Signal 多强，A 侧持仓都不能继续扩大超过 \$20,000。

Taoli Tools 目前提供基于数量和 U 本位价值的持仓上下限控制，并把未成交挂单也纳入相关限制逻辑。

这实际上就是：

```
\[\n\boxed{\nRiskLimit\n}\n]\n]
```

而不是 Trading Signal。

15. Signal 与 Risk Control 必须分开

假设：

```
\[\nSpread=1%\n]\n]
```

]

Signal 告诉系统：

\[

Trade\

]

但是：

\[

PositionAlreadyAtMax\

]

Risk Engine 应该返回：

\[

DoNotTrade\

]

因此系统逻辑必须是：

\[

Signal\

]

↓

\[

RiskCheck\

]

↓

\[

Execution\

]

而不能：

\[

Signal\

\rightarrow\

Execution\

]

直接下单。

16. Spread 应该怎么计算？

假设：

A：

\[

Bid_A=101\

]

B：

\[

Ask_B=100\

]

如果策略：

\[

Sell_A+Buy_B\

]

那么真正应该比较：

\[

Bid_A\

]

和：

```
\\  
Ask\_B\  
]
```

而不是两个：

```
\\  
LastPrice\  
]
```

Taoli Tools 当前对 $-A+B$ 的价差使用 A 的 Bid 1 与 B 的 Ask 1，对反方向则使用 B Bid 1 与 A Ask 1，并以两者中间价进行百分比标准化。

这体现了 Day 2 的核心原则：

```
\\  
\boxed{\  
ExecutableSideOfBook\  
}\
```

比：

```
\\  
DisplayedLastPrice\  
]
```

更重要。

17. 为什么 Bid / Ask 很重要？

假设：

Exchange A：

```
\\  
Bid=100\  
]
```

```
\\  
Ask=101\  
]
```

Exchange B：

```
\\  
Bid=100.5\  
]
```

```
\\  
Ask=101.5\  
]
```

如果只比较两个 Last Price：

可能看起来存在机会。

但是：

要：

```
\\  
Buy\_A\  
]
```

实际上要付：

```
\\  
Ask\_A=101\  
]
```

然后：

```
\\  
Sell\_B\  
]
```

```

]
只能卖：
\
Bid\_B=100.5\
]

```

```

所以：
\
100.5-101<0\
]

```

根本不存在套利。

```

因此：
\
\boxed{
LastPriceSpread\
eq\
ExecutableSpread\
}\
]

```

18. Order Book Depth

即使：

```

\
Ask\_A=100\
]
\
Bid\_B=101\
]

```

如果 Best Ask 上只有：

```

\
0.01BTC\
]

```

你想买：

```

\
5BTC\
]

```

真正的：

```

\
AverageBuyPrice\
]

```

会向下吃整个 Order Book。

因此需要：

```

\
VWAP\
]

```

即：

```

\
VolumeWeightedAveragePrice\
]

```

所以实际：

```

\
ExecutableSpread(Q)\

```

]

又一次依赖：

$\backslash \backslash$

Q\

]

这和 Day 2 AMM 中：

$\backslash \backslash$

Price(Q)\

]

的逻辑完全一致。

19. Market Order vs Limit Order

Market Order

优点：

$\backslash \backslash$

FillProbability $\uparrow$

]

缺点：

$\backslash \backslash$

Slippage $\uparrow$

]

Limit Order

优点：

可以控制：

$\backslash \backslash$

ExecutionPrice\

]

缺点：

可能：

$\backslash \backslash$

NotFilled\

]

因此套利中存在一个重要 trade-off：

$\backslash \backslash$

$\boxed{\backslash \backslash$

PriceCertainty\

$\longleftrightarrow$

FillCertainty\

$\backslash \backslash$

]

20. 为什么套利执行常常愿意“亏一点价差”换成交？

假设：

Gross Spread：

$\backslash \backslash$

50bps\

]

你可以挂单等：

$\backslash \backslash$

50bps\

]

全部赚到。

但机会可能 2 秒后消失。

如果愿意主动吃单损失：

```
\[\n10bps\n]
```

却能大幅提高：

```
\[\nFillProbability\n]
```

那么实际：

```
\[\nExpectedValue\n]
```

可能反而更高。

Taoli Tools 当前的限价参数支持相对买一卖一设置不同价距；其设计本质上也是在控制“更快成交”和“更好的成交价”之间的取舍。

21. Expected Execution Value

因此不能只优化：

```
\[\nSpread\n]
```

可以考虑：

```
\[\n\boxed{\nEV\n==\np\_{bothfill}\Pi\_{arb}\n+\np\_{onefill}\Pi\_{legging}\n+\np\_{nofill}\Pi\_{0}\n}\n]
```

其中：

```
\[\n\Pi\_{legging}\n]
```

可能是负数。

这意味着：

一个 100 bps 的机会，如果双边成功率很低，可能比一个稳定成交的 20 bps 机会更差。

22. Funding Arbitrage 中执行有什么不同？

例如：

Exchange A：

```
\[\nFunding\_A=+0.05%\n]
```

Exchange B：

```
\[\nFunding\_B=-0.01%\n]
```

可能建立：

```
\\  
Short\_A\  
]
```

```
\\  
Long\_B\  
]
```

以获取 funding differential。

但开仓时仍然存在：

```
\\  
PriceSpread\  
]
```

所以 Funding Arbitrage 实际收益不是只有：

```
\\  
Funding\  
]
```

而应该是：

```
\\  
\boxed{\  
PnL  
====  
FundingPnL\  
+\  
SpreadPnL  
-----  
TradingFees  
Slippage\  
}\  
]
```

23. Entry Spread 与 Exit Spread

假设：

开仓：

A：

```
\\  
100\  
]
```

B：

```
\\  
101\  
]
```

你：

```
\\  
Long\_A\  
]
```

```
\\  
Short\_B\  
]
```

Entry Spread：

```
\\  
1\  
]
```

后来两边：

$$\frac{100.5}{1}$$

价差收敛：

$$1 \rightarrow 0$$

你除了 Funding 外，还赚：

$$\text{SpreadConvergence}$$

所以某些 Funding Arbitrage 同时存在：

$$\text{FundingEdge}$$

和：

$$\text{Basis/SpreadEdge}$$

24. 但 Spread 也可能扩大

假设开仓：

$$\text{Spread}=1\%$$

结果：

$$1\% \rightarrow 3\%$$

虽然 Directional Exposure 大致被 Hedge：

$$\Delta \approx 0$$

但两边 Position 的 Mark-to-Market PnL 可能不同，保证金压力也会上升。

所以：

$$\boxed{\text{DeltaNeutral} = \text{RiskFree}}$$

25. Margin Risk

例如：

Exchange A：

$$\text{Short BTC}$$

Exchange B：

$$\text{Long BTC}$$

整体：

$$\begin{aligned} & \backslash \backslash \\ & \text{NetBTCExposure} \backslash \text{approx} 0 \backslash \\ & \end{aligned}$$

但如果 A 的 Short 出现巨大 unrealized loss：

$$\begin{aligned} & \backslash \backslash \\ & \text{Margin} \backslash _A \backslash \\ & \end{aligned}$$

可能不足。

即使 B 在另一边赚了同样的钱：

$$\begin{aligned} & \backslash \backslash \\ & \text{Margin} \backslash _B \backslash \\ & \end{aligned}$$

也不能自动移动到 A。

因此存在：

$$\begin{aligned} & \backslash \backslash \\ & \boxed{\backslash} \\ & \text{MarginFragmentationRisk} \backslash \\ & \} \backslash \\ & \end{aligned}$$

这也是 Cross-exchange Arbitrage 的重要风险。

26. Liquidation Risk

如果使用 Leverage：

$$\begin{aligned} & \backslash \backslash \\ & \text{Leverage} \backslash \uparrow \backslash \\ & \end{aligned}$$

Capital Efficiency 上升。

但：

$$\begin{aligned} & \backslash \backslash \\ & \text{LiquidationRisk} \backslash \uparrow \backslash \\ & \end{aligned}$$

因此：

$$\begin{aligned} & \backslash \backslash \\ & \text{PortfolioDelta} \backslash \text{approx} 0 \backslash \\ & \end{aligned}$$

并不能保证单个 venue 不会被清算。

你必须分别监控：

$$\begin{aligned} & \backslash \backslash \\ & \text{MarginRatio} \backslash _A \backslash \\ & \end{aligned}$$

$$\begin{aligned} & \backslash \backslash \\ & \text{MarginRatio} \backslash _B \backslash \\ & \end{aligned}$$

而不是只看组合净 PnL。

27. ADL Risk

Perpetual Exchange 还可能存在：

$$\begin{aligned} & \backslash \backslash \\ & \text{ADL} \backslash \\ & \end{aligned}$$

即：

```

\
AutoDeleveraging\
]

```

如果一条腿因 ADL 被强制减少：

```

\
Position\_A\downarrow\
]

```

而：

```

\
Position\_B\
]

```

还保持原值，
你会突然出现：

```

\
NetExposure
eq0\
]

```

QQQ.BOT 当前公开说明，其执行引擎会在某一腿发生强平、ADL 或交割强制减仓时，按剩余仓位调整 hedge target，并同步减少另一条腿，而不是重新把被减掉的一腿买回来。

这里体现一个很重要的设计原则：

```

\
\boxed{
RestoreHedgeByReducingExposure\
}\
]

```

而不一定是：

```

\
RestoreHedgeByIncreasingRisk\
]

```

28. QQQ.BOT 值得研究的地方

截至当前公开页面，QQQ.BOT 将跨交易所套利压缩为两个主要信号：

```

\
PriceSpread\
]

```

和：

```

\
FundingDifference\
]

```

然后由执行引擎处理双边下单、数量精度与分批仓位收敛；其机会页还支持按照价差或 Funding 排序，并配置开仓和平仓条件。

这意味着可以把系统抽象成：

```

\
OpportunityDiscovery\
]
↓
\
PairSelection\
]
↓
\
TargetPosition\
]

```

↓

```
\\  
ExecutionEngine\  
]
```

↓

```
\\  
PositionReconciliation\  
]
```

↓

```
\\  
ExitEngine\  
]
```

这才是 Day 9 真正应该学习的东西。

29. 为什么需要分批建仓？

假设目标：

```
\\  
$100000\  
]
```

一次性：

```
\\  
$100000\  
]
```

下单可能：

Price Impact 大；
Slippage 大；
一条腿失败损失大；
Order Book Depth 不够。

所以可以拆：

```
\\  
10000\times10\  
]
```

每一轮重新确认：

```
\\  
Spread\  
]
```

```
\\  
PositionBalance\  
]
```

```
\\  
Liquidity\  
]
```

这叫：

```
\\  
\boxed\  
IncrementalConvergence\  
\\  
]
```

30. 分批的另一个优势：风险停止点

例如目标仓位：

```
\\  
$100000\  
]
```

]

当前完成：

```
\\  
$30000\  
]
```

然后：

```
\\  
Spread\  
]
```

突然消失。

系统可以停止。

如果一次性提交全部：

```
\\  
$100000\  
]
```

就没有这个灵活性。

因此：

```
\\  
\boxed{\  
Batching  
=====  
ExecutionControl\  
}\  
]
```

而不只是降低 Price Impact。

31. Entry Logic

一个简单的 Price Arbitrage Entry：

```
\\  
Spread\_{entry}\  
\geq\  
Threshold\  
]
```

例如：

```
\\  
Spread\geq50bps\  
]
```

但真正需要：

```
\\  
NetSpread  
=====  
GrossSpread  
TradingFees  
Slippage\  
]
```

因此正确逻辑更接近：

```
\\  
\boxed{\  
ExpectedNetEdge\  
\geq\  
EntryThreshold\  
]
```

```
}\  
]
```

32. Exit Logic

开仓之后不能只想：

等赚钱。

需要提前定义：

Spread Convergence Exit

例如：

```
\\  
Spread:\  
50bps\rightarrow10bps\  
]
```

平仓。

Funding Exit

例如：

```
\\  
FundingDiff\<Threshold\  
]
```

退出。

Risk Exit

例如：

```
\\  
MarginRatio\<RiskLimit\  
]
```

Timeout Exit

例如：

```
\\  
HoldingTime>24h\  
]
```

Emergency Exit

例如：

```
\\  
OneLegFailure\  
]
```

33. Opening 与 Closing 都有 Execution Cost

假设开仓 edge：

```
\\  
50bps\  
]
```

但：

开仓费用：

```
\\  
15bps\  
]
```

平仓费用：

```
\\  
15bps\  
]
```

那么：

```

\
RoundTripCost=30bps\
]

```

真正最多只剩：

```

\
20bps\
]

```

所以：

```

\
\boxed{
EntryEdge\
}\
]

```

必须覆盖：

```

\
OpenCost+CloseCost\
]

```

Funding Strategy 还必须覆盖持仓期间风险。

34. Round-trip PnL

建议统一写成：

```

\
\boxed{
PnL_{total}
=====
PnL_{spread}\
+
PnL_{funding}
-----
Fee_{open}
Fee_{close}
Slippage
LeggingLoss\
}\
]

```

如果涉及借币：

```

\
-BorrowCost\
]

```

如果涉及资金占用：

```

\
-CapitalCost\
]

```

这比只看：

```

\
FundingAPR\
]

```

可靠得多。

35. Monitoring Layer

双边仓位建立之后，系统不能停止工作。

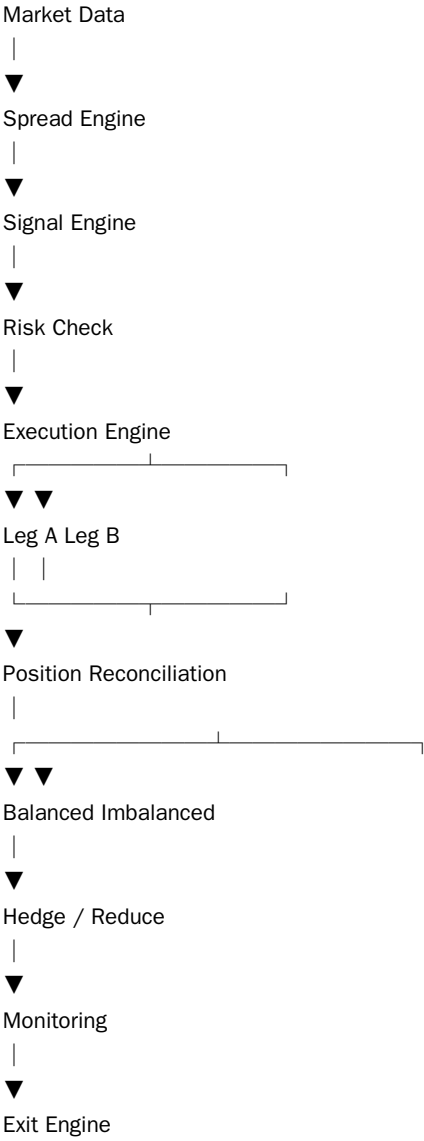
至少持续监控：

Price A
Price B
Spread
Position A
Position B
Filled Quantity
Funding A
Funding B
Margin
Liquidation Distance
Open Orders
API Health

Taoli Tools 当前公开功能本身也把实时价格、价差、仓位、资金费率与风险预警列为核心监控维度。

36. Day 9 的执行系统架构

今天建议建立这张图：



这比：

if spread > 0.5%:
buy()
sell()

完整得多。

37. State Machine 思维

执行器最好不要只是写一堆：

```
if  
else
```

而应该逐渐理解成：

```
\\  
StateMachine\  
]
```

例如：

IDLE

→ SIGNAL_FOUND

→ ORDERING

→ PARTIALLY_HEDGED

→ HEDGED

→ EXIT_SIGNAL

→ CLOSING

→ FLAT

异常：

ORDER_FAILED

LEG_IMBALANCE

API_ERROR

LIQUIDATION_RISK

EMERGENCY_EXIT

这也是以后写真正 Execution Bot 时非常重要的 Software Engineering 思维。

38. Day 9 Experiment 1 | 手工模拟双边执行

假设：

A：

```
\\  
Ask\_A=100\  
]
```

B：

```
\\  
Bid\_B=100.6\  
]
```

Gross Spread：

```
\\  
60bps\  
]
```

目标：

```
\\  
$10,000\  
]
```

假设：

A 成交：

```
\\  
100\  
]
```

但 B 延迟 1 秒后：

```
\\  
Bid\_B=100.3\  
]
```

那么实际 Spread :

```
\\  
30bps\  
]
```

如果双边 Fee :

```
\\  
20bps\  
]
```

那么 :

```
\\  
NetEdge=10bps\  
]
```

本来以为 :

```
\\  
40bps\  
]
```

实际只剩 :

```
\\  
10bps\  
]
```

这就是 :

```
\\  
ExecutionDecay\  
]
```

39. Experiment 2 | 模拟单腿失败

目标 :

```
\\  
Long\_A=$10000\  
]
```

```
\\  
Short\_B=$10000\  
]
```

结果 :

A :

```
\\  
Filled=$10000\  
]
```

B :

```
\\  
Failed\  
]
```

BTC 下跌 :

```
\\  
0.5%\  
]
```

则临时损失约 :

```
\\  
10000\times0.005
```

=====

\$50\
]

如果预期套利利润：

\\
\$20\
]

那么一次 Legging Failure：

\\
-\$50\
]

已经可以抵消多次正常套利利润。

40. Experiment 3 | 比较三种 Sequence

假设：

 	A	B
Fill probability	99%	90%
Latency	100ms	800ms
Liquidity	High	Low

分别思考：

Parallel

优点？

缺点？

A then B

优点？

缺点？

B then A

优点？

缺点？

这里没有唯一答案。

但你应该能够解释为什么：

\\
BThenA\
]

可能更合理：

因为先解决：

\\
HardLeg\
]

再用更容易成交的 A 完成 Hedge。

41. Experiment 4 | Funding + Spread

假设：

Exchange A：

\\
Funding=+0.06%\
]

Exchange B：

```
\[\nFunding=+0.01%\n]
```

你：

```
\[\nShort\_A\n]
```

```
\[\nLong\_B\n]
```

Funding Differential：

```
\[\n5bps\n]
```

每轮理论：

```
\[\n+5bps\n]
```

但是开仓时：

```
\[\nSpreadLoss=20bps\n]
```

Round-trip Fee：

```
\[\n20bps\n]
```

至少需要累计：

```
\[\n40bps\n]
```

以上 Funding 收益，才开始覆盖：

```
\[\nEntry+Exit\n]
```

成本。

所以不能看到：

```
\[\nFundingDiff>0\n]
```

就认为：

```
\[\nProfitable\n]
```

42. Experiment 5 | 研究 Taoli Tools / QQQ.BOT

今天不要先连接真实资金。

重点观察：

Opportunity Layer

工具显示什么？

```
\[\nSpread?\nFunding?\n]
```

Price?\

]

Execution Layer

如何选择：

\[\

A/B\

]

？

如何：

\[\

Sequence\

]

？

Position Layer

如何显示：

\[\

PositionA\

]

\[\

PositionB\

]

？

Risk Layer

如何限制：

\[\

Position\

]

？

Exit Layer

什么时候减仓？

Failure Layer

一条腿失败怎么办？

目标：

\[\

\boxed{\

ReverseEngineerTheArchitecture\

}\

]

而不是：

学会按哪个按钮。

Taoli

Tools

明确说明自己是“半自动”而不是全自动策略系统，并提醒不能把它理解为无风险收益工具；QQQ.BOT

的公开页面则更进一步提供机会排序、自动开平仓和异常仓位调整等执行功能。

43. API Key 安全

这类工具涉及真实 Exchange API Key。

如果未来测试：

优先：

\[\

ReadOnly\

]

或：

```
\\  
Demo/Testnet\  
]
```

开始。

如果必须交易权限：

不要开启：

```
\\  
WithdrawalPermission\  
]
```

除非有极特殊、充分理解的需求。

QQQ.BOT 当前说明为单用户本地部署，并表示 API Key 保留在用户自己的机器上；Taoli Tools
也说明其网页工具在本地运行，不收集密钥或交易相关数据。

但：

```
\\  
LocalStorage\  
eq\  
ZeroRisk\  
]
```

仍然要考虑：

本机安全；

Malware；

API 权限；

IP whitelist；

Exchange account risk。

44. Day 9 OpportunityEvaluator 升级

Day 7 的 Evaluator：

```
\\  
NetEdge\  
]
```

今天需要加入 Execution Layer：

```
fill_probability_A  
fill_probability_B  
expected_latency_A  
expected_latency_B  
position_imbalance  
legging_loss  
round_trip_fee  
margin_required  
liquidation_distance
```

于是：

```
\\  
ExpectedValue\  
]
```

升级为：

```
\\  
\boxed{\  
EV  
==
```

```

p\_{hedged}\Pi\_{hedged}\
+\
p\_{legging}\Pi\_{legging}\
+\
p\_{fail}\Pi\_{fail}\
}\
]

```

45. Day 9 验收题

Q1：为什么 Parallel 下单仍然存在 Legging Risk？

因为 Parallel 只代表：

```

\
OrdersSentAtSimilarTime\
]

```

不代表：

```

\
BothOrdersFilledAtomically\
]

```

一边仍可能成功，另一边失败。

Q2：为什么 Submitted 不等于 Filled？

因为 Exchange 接受 Order 后，Order 还可能：

```

\
Pending\
]

```

```

\
PartiallyFilled\
]

```

```

\
Cancelled\
]

```

所以必须确认实际：

```

\
FilledQuantity\
]

```

Q3：为什么套利需要持续检查 Position Balance？

因为一条腿可能：

Partial Fill；
Liquidation；
ADL；
API delay；
Manual change。

这些都会导致：

```

\
NetExposure
eq0\
]

```

Q4：为什么 Market Order 不一定比 Limit Order 差？

Market Order 虽然可能付出更高 Slippage，但：

```

\
FillProbability\
]

```

通常更高。

对短寿命套利来说，更快完成 Hedge 可能比多赚几 bps 更重要。

Q5：Funding Arbitrage 为什么仍然需要关注 Price Spread？

因为最终总收益：

```
\\  
PnL  
====  
FundingPnL\  
+\  
SpreadPnL  
-----  
Costs\  
]
```

开仓和平仓时的两腿 Price Difference 可以明显影响最终收益。

Q6：Delta-neutral 为什么不等于 Risk-free？

因为仍存在：

```
\\  
LeggingRisk\  
]  
\  
BasisRisk\  
]  
\  
FundingFlip\  
]  
\  
MarginRisk\  
]  
\  
Liquidation\  
]  
\  
ExchangeRisk\  
]
```

Q7：为什么要设置 Position Limit？

因为即使 Signal 一直出现：

```
\\  
RiskCapacity\  
]
```

也是有限的。

Position Limit 防止策略因为 Loop、异常数据或持续 Signal 无限扩大仓位。

46. Day 9 最终 Task

今天至少完成：

Task 1

画一张完整 Two-leg Arbitrage Execution Flow：

```
\\  
Signal\  
\rightarrow\  
RiskCheck\  
\rightarrow\  
LegA/B\  
\rightarrow
```

```

FillCheck\
\rightarrow\
PositionReconciliation\
\rightarrow\
Monitor\
\rightarrow\
Exit\
]

```

Task 2

比较：

```

\
Parallel\
]

```

```

\
AThenB\
]

```

```

\
BThenA\
]

```

三种执行方式。

Task 3

设计自己的：

```

\
PositionImbalanceMetric\
]

```

例如：

```

\
Imbalance
=====

```

```

\frac{[N\_A-N\_B]}{\{\max(N\_A,N\_B)\}}\
]

```

其中：

```

\
N\_A,N\_B\
]

```

为两腿 Notional。

Task 4

用一个 Funding Arbitrage Example 计算：

```

\
FundingPnL\
+\
SpreadPnL
-----

```

```

RoundTripFee\
]

```

Task 5

实际观察 Taoli Tools / QQQ.BOT 的：

```

Signal
Execution
Position

```

Risk

Exit

Failure

六个模块。

只做研究，不要求连接真实资金。

47. Day 9 最终产出

项目增加：

arbitrage-lab/

```
|—— reports/
|   |—— execution_engine.md
|
|—— notebooks/
|   |—— two_leg_execution.ipynb
|
|—— src/
|—— models/
|—— execution_risk.py
```

其中 execution_engine.md 应至少包含：

Signal Definition

Leg A

Leg B

Sequence

Order Type

Fill Verification

Position Balance

Position Limits

Funding Monitor

Entry Rule

Exit Rule

Emergency Rule

Legging Risk

Margin Risk

48. Day 9 总结

Day 8 让我看到：

```
\
\boxed{
一笔成功套利是怎么发生的\
}\
]
```

Day 9 则让我认识到：

```
\
\boxed{
发现套利机会只是Execution之前的第一步\
}\
]
```

真正的系统必须处理：

```
\
Signal\
]
```

```

↓
\
Order\
]
↓
\
Fill\
]
↓
\
Hedge\
]
↓
\
Position\
]
↓
\
Risk\
]
↓
\
Exit\
]

```

因此套利研究的核心公式也需要升级。

以前：

```

\
Profit
=====
Spread-Cost\
]

```

现在：

```

\
\boxed{\
ExpectedProfit
=====
ExecutionProbability\
\times\
ArbitrageProfit
-----
ExecutionRisk\
}\
]

```

真正值得优化的不只是：

```

\
MaximumSpread\
]

```

而是：

```

\
\boxed{\
MaximumRiskAdjustedExecutableEdge\

```

笔记 050: LP 套利：不和套利机器人抢利润，而是在必经路径上收手续费

作者：QQQ | 时间：2026-08-16 06:04 UTC | 字数：7732 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

LP 套利：不和套利机器人抢利润，而是在必经路径上收手续费

18.1 核心思想：从“做套利者”切换为“给套利者提供流动性”

传统 DEX 套利的逻辑是发现两个市场价格不同，然后抢在其他 Searcher 之前买低卖高。问题是专业套利者在节点延迟、Orderflow、EVM 模拟、Builder 连接和资金规模上都具有明显优势，普通策略很难长期靠速度竞争。

GitHub [arbitrage-playbook](#) Issue #1 提出了另一种思路：不直接和套利机器人竞争价差，而是在高波动、流动性不足、套利流频繁经过的 AMM 价格区间部署 LP，让套利机器人为了修复市场价格而不断与池子交易，从而把一部分套利流转化为自己的 LP Fee 收入。可以形象理解为：不去抢高速公路上的货，而是在套利流量必经的位置修一个“收费站”。

核心收益来源： $LP\ Fee\ Income \approx 有效套利交易量 \times Fee\ Tier \times 自己占有有效流动性的比例$ 。

18.2 为什么套利机器人天然会成为 LP 的客户

假设外部市场 ETH 已从 3,000 USDT 快速跌到 2,800，但某个流动性较浅的 AMM 仍报价 2,950。此时 AMM 价格明显高于外部公允价格，套利机器人会向池子卖 ETH、买 USDC，持续把池内价格向 2,800 拉回。

价格修复过程可能类似：2,950 → 2,920 → 2,880 → 2,840 → 2,810 → 2,800。

对 Searcher 来说，这些交易是在吃价差；对 LP 来说，这些交易都是 Fee-paying Flow。只要自己的流动性位于实际成交区间，每一次套利 Swap 都会贡献手续费。

示例：若异常行情中某池累计产生 2,000,000 USDT 的相关 Swap Volume，Fee Tier 为 1%，理论上整个池获得约 20,000 USDT Fee。如果某 LP 在成交价格附近占有有效流动性的 10%，粗略手续费份额约 2,000 USDT。这个例子只说明 Fee 机制，不代表最终净利润，因为还没有扣除 LVR、IL、Gas 和调仓成本。

18.3 四个最重要的入场条件：爆量、缺池、剧烈波动、反复穿越

- ① 爆量：LP 收入直接依赖交易量。没有真实 Swap Volume，再高的 Fee Tier 也没有意义。
- ② 缺池：如果已有大量流动性集中在现价附近，新资金只能分到很小的 Fee Share；流动性缺口越明显，新增 LP 的边际价值越高。
- ③ 剧烈波动：外部市场快速变化时，AMM 价格更容易暂时落后，从而触发套利交易。
- ④ 反复穿越：这是最理想的结构。价格不是单方向快速穿过一次，而是在某个区间来回震荡，例如 2,800 → 2,900 → 2,820 → 2,940 → 2,850 → 3,000。若 LP 区间位于 2,850–2,950，价格反复穿越时可能持续产生双向 Swap 和套利流。

因此，策略真正想找的不是“价格一定上涨”或“价格一定下跌”，而是：高成交量 + 足够波动 + 价格在自己的有效流动性区域反复成交。

18.4 为什么高 Fee Pool 仍可能进入最优路由

直觉上，套利机器人应该永远选择手续费最低的池，但聚合器、Router 和 Solver 真正比较的是最终执行结果，而不是单独比较 Fee Tier。

$Total\ Execution\ Cost \approx LP\ Fee + Price\ Impact + Slippage + Gas + Route\ Cost$ 。

例如：Pool A Fee 只有 0.05%，但流动性太浅导致 3% Price Impact；Pool B Fee 为 1%，但深度足够，Price Impact 只有 0.2%。对较大的 Swap 来说，Pool B 的最终成交价格反而可能更优，因此即使它的名义手续费更高，也可能被 Router 采用。

这意味着 LP 策略不能只问“哪个 Fee Tier 最高”，而要问：在 1,000 / 5,000 / 10,000 / 50,000 USDT 等不同 Swap Size 下，我的池是否仍进入最佳 Route 或 Split Route？

18.5 Split Routing：高费率池不需要吃下全部交易

现代聚合器可能把一笔大单拆到多个池。例如：50% 走 0.05% Pool，30% 走 0.3% Pool，20% 走 1% Pool。只要自己的池能够改善组合后的最终成交价格，就可能获得部分 Flow。

因此应重点统计：Route Inclusion Rate（进入路由的概率）、Flow Share（平均分到的订单比例）、Fee Revenue per Unit Liquidity（单位有效流动性的手续费收入）。

18.6 集中流动性为什么适合这种策略

Uniswap V3/V4 类集中流动性允许 LP 把资金集中到狭窄价格区间。假设现价 2,900，普通宽区间 LP 覆盖 2,000–4,000，而事件型 LP 只覆盖 2,850–2,950。同样 10,000 USDT 本金，后者在目标区间附近的有效流动性密度更高，因此当套利流集中穿过该区间时，Fee/Capital 可能明显提高。

但窄区间也会放大风险：价格向上穿出后仓位可能逐渐变成稳定币；价格向下穿出后仓位可能逐渐变成 ETH。区间越窄，资本效率越高，同时 Out-of-range 和 Rebalance 风险也越高。

因此这不是“放进去收租”，而是一种需要动态管理的主动做市策略。

18.7 最大风险：套利机器人付手续费的同时，也可能在从 LP 身上提取价值

这是理解本策略最关键的一点。套利机器人愿意和 AMM 交易，往往正是因为池内价格落后于外部市场。也就是说，LP 正在以相对不利的价格和更快的信息更快的套利者成交。

例如真实市场已经从 3,000 跌到 2,800，而 LP 池仍在 2,950。Searcher 向 LP 卖出 ETH，虽然 LP 收到了手续费，但同时以高于外部市场的隐含价格接收了正在下跌的 ETH。这种由信息和价格更新速度差造成的价值转移，可以用 LVR (Loss Versus Rebalancing) 或 adverse selection 来理解。

所以不能只计算 Fee。更合理的净收益模型是：

$$\text{Net PnL} = \text{LP Fee} - \text{LVR} - \text{IL} - \text{Inventory Loss} - \text{Gas} - \text{Rebalance Cost} - \text{Swap Cost} - \text{Execution Loss}.$$

如果 Fee +500 USDT，但 LVR/库存损失 800 USDT，策略最终仍亏 300 USDT。高 APR 页面无法回答这个问题。

18.8 IL 与 LVR 的区别

IL (Impermanent Loss) 主要比较“做 LP”与“原样持有两种资产”的结果差异，强调资产价格变化后自动再平衡造成的相对表现。

LVR 更关注 LP 因价格更新滞后而被更快、更有信息优势的套利者交易所造成的损失。对于本节这种“专门等待套利 Flow”的策略，LVR 尤其重要，因为策略主动寻找的 Flow 本身很可能就是 informed flow。

因此研究目标不能是 Maximum Fee APR，而应该是 Maximum Risk-adjusted Net PnL。

18.9 最值得研究的机会：反复震荡，而不是单边趋势

如果价格从 2,900 一路跌到 2,000，LP 虽然一路收到 Swap Fee，却会不断被动接收下跌资产，库存损失可能远高于手续费。

更理想的是外部价格和 AMM 价格在某一区域反复错位、修复、再错位，例如：CEX 快速上涨 → DEX 落后 → Bot 买 DEX；随后 CEX 回落 → DEX 又偏高 → Bot 卖 DEX。此时 LP 有机会在相同资本上反复收取双向 Fee，而不是只经历一次单向库存转换。

因此可以建立一个 Cross Count 指标：统计单位时间内价格穿越目标 LP 区间或参考价次数。再结合 Volume、Volatility 和 Route Inclusion，判断该事件是否适合部署 LP。

18.10 与普通 LP 的区别：预测的是 Flow，而不是 APY

普通 LP 策略常从历史 Volume、TVL、APR、Fee Tier 出发寻找高收益池；Arbitrage Flow LP 更强调预测未来几分钟或几小时套利机器人会在哪里成交。

它真正需要预测的是：
$$\text{Expected Arbitrage Flow} = \text{Price Dislocation} \times \text{External Volume} \times \text{Pool Depth} \times \text{Route Probability} \times \text{Mean Reversion Frequency}.$$

这使它更接近事件驱动做市，而不是传统“长期把资金放在一个高 APR 池”。

18.11 推荐系统架构：Arbitrage Flow LP Bot

第一层 Market Scanner：监控 CEX Reference Price、DEX Pool Price、Volume、Liquidity、Volatility、Fee Tier、Tick Liquidity 和 Swap Flow。

第二层 Dislocation Engine：计算 DEX 相对参考市场的偏离，例如 $\text{Deviation} = \text{DEX Price} / \text{Reference Price} - 1$ ，并识别异常扩张速度、持续时间和方向。

第三层 Arbitrage Flow Predictor：估算若价格偏差继续存在，Searcher 为恢复价格大约需要交易多少数量，并预测哪些 Pool / Route 最可能被使用。

第四层 LP Simulator：针对多个 Fee Tier、Range、Capital 和部署时点模拟 Expected Fee、LVR、Inventory PnL、Gas、Rebalance Cost 和 Net PnL。

第五层 Deployment Engine：只有 Expected Net PnL 超过最低阈值时才 Add Liquidity，并限制 Max Capital、Token Allowlist、Pool Allowlist 和 Price Range。

第六层 Exit / Rebalance Engine：Volume 下降、波动消失、价格突破 Range、LVR 快速扩大、Route Inclusion 降低或 Net Expected PnL 转负时撤出或移动流动性。

完整闭环：Scan → Detect Dislocation → Predict Arbitrage Flow → Simulate LP → Risk Check → Deploy → Collect Flow → Monitor → Exit/Rebalance → Realized PnL Database。

18.12 第一版不要直接真钱：先做 Shadow LP 回测

最适合的 MVP 不是自动投入资金，而是做 Shadow LP：机器人实时观察链上真实 Swap 和套利交易，但只模拟“如果我当时在这里放入 10,000 USDT LP，会发生什么”。

每个候选事件记录：Pool、Token Pair、Fee Tier、Price Range、Initial Capital、Reference Price、DEX Price、Deviation、Swap Volume、疑似 Arbitrage Volume、Route、Active Liquidity、模拟 Fee、Inventory Change、IL、LVR、Gas/Rebalance 假设和最终 Net PnL。

这样可以先回答最重要的问题：套利机器人贡献的手续费，长期到底能不能覆盖它们从 LP 身上提取的 LVR。

18.13 如何识别“疑似套利交易”

可以从链上历史交易中建立启发式分类：同一区块或极短时间内出现跨池/跨市场方向相反的 Swap；交易后目标池价格明显靠近外部参考价格；交易地址长期重复执行类似路径；交易涉及 Router、Aggregator、Searcher Contract 或 Bundle；交易前后存在明显 Price Dislocation 收敛。

第一版不需要完美识别每一个 Searcher。只要能够把普通用户 Flow 与高度可能的 arbitrage/rebalancing Flow 粗略分开，就可以比较两类 Flow 对 LP 收益和 LVR 的贡献。

18.14 回测最重要的四组指标

- ① Fee Capture：Fee Income、Fee/Capital、Fee per Active Minute、Fee per Arbitrage Volume。
- ② Adverse Selection：LVR、IL、Inventory PnL、Price Move After Swap。
- ③ Flow Quality：Arbitrage Volume Share、Route Inclusion Rate、Cross Count、Mean Reversion Speed。
- ④ Execution Cost：Add/Remove Gas、Rebalance Swap Fee、Slippage、Range Downtime 和失败成本。

最终优化指标应是：Expected Net PnL per Unit Capital，而不是 APR。

18.15 推荐的事件评分模型

可以先建立一个简单 Event Score：

Event Score = $w_1 \times \text{Volume Spike} + w_2 \times \text{Price Dislocation} + w_3 \times \text{Cross Count} + w_4 \times \text{Route Probability} + w_5 \times \text{Existing Liquidity} + w_6 \times \text{Trend Risk} + w_7 \times \text{Expected LVR} + w_8 \times \text{Execution Cost}$ 。

其中权重先用经验值，等积累真实 Shadow LP 数据后再校准。分数高只代表进入模拟候选，不代表直接部署真钱。

18.16 入场与退出条件示例

候选入场：Volume 显著高于过去基线；DEX/Reference 出现足够大的价格偏离；目标区间有效流动性不足；预计 Route 会经过目标池；预计 Fee 收入覆盖 LVR + Gas + Rebalance Cost 后仍有安全边际；连续多个采样点成立。

退出条件：价格单边突破区间；套利 Flow 消失；Volume/TVL 快速下降；新增 LP 大量进入导致 Fee Share 被稀释；Expected LVR > Expected Fee；累计净 PnL 达到止盈/止损；最大持仓时间到期。

18.17 与前面几种套利策略的统一理解

DEX/DEX Arbitrage：自己吃 Price Difference。

Cross-chain Arbitrage：自己吃不同链之间的 Price Difference。

MEV Back-run：利用交易造成的新状态失衡吃 Price Difference。

Arbitrage Flow LP：不直接吃 Price Difference，而是向执行这些套利的人提供 Liquidity，赚 Trading Fee，同时承担 LVR/Inventory Risk。

Yield Rebalancing：利用不同 Vault 的 APY Difference 调度资本。

因此这几种策略可以共享 Market Scanner、Cost Engine、Simulation、Risk Engine、Wallet、PnL Database，但 Alpha 来源不同：有的是 Price Spread，有的是 Transaction Ordering，有的是 Arbitrage Flow，有的是 APY Spread。

18.18 最重要的认识

第一，套利 Flow 对 LP 来说既是收入来源，也是风险来源。不能只看 Fee，不看 LVR。

第二，高 Fee Tier 并不自动意味着没有订单；真正决定 Router 是否经过的是 Total Execution Cost。

第三，集中流动性可以显著提高 Fee/Capital，但也提高穿区、库存和调仓风险。

第四，最理想的环境通常不是单边暴涨暴跌，而是高成交量下的反复错位和均值回归。

第五，策略真正的竞争力不是“会创建 LP”，而是能否预测套利 Flow、估算 LVR，并及时部署和退出。

第六，第一阶段最值得做的是 Shadow LP / Historical Replay，而不是直接真钱 Auto LP。

18.19 推荐的下一步实验

第一步：选择 3-5 个高交易量 V3/V4 池，保存分钟级/区块级 Pool State、Swap、Tick Liquidity 和外部 Reference Price。

第二步：识别 Price Dislocation → Arbitrage Swap → Price Convergence 的完整事件。

第三步：模拟 1,000 / 5,000 / 10,000 USDT 三个本金档和多个 Range/Fee Tier。

第四步：同时计算 Fee、Inventory PnL、IL、LVR 和调仓成本。

第五步：统计什么样的 Volume Spike、Deviation、Cross Count 和 Liquidity Gap 对应正的 Net PnL。

第六步：只有 Shadow LP 样本证明长期 Expected Net PnL 为正后，再考虑极小额实盘验证。

资料来源：w3player/arbitrage-playbook

GitHub

Issue

#1 (LP

套利探讨)：https://github.com/w3player/arbitrage-playbook/issues/1

笔记 051: 今天继续学习 DEX-CEX 套利，重点思考了一个问题：价差达到多少才值得关注？

作者：zgos | 时间：2026-08-16 06:30 UTC | 字数：410 | 评分：47

来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

今天继续学习 DEX-CEX 套利，重点思考了一个问题：价差达到多少才值得关注？

以前看到正价差就会觉得有机会。现在发现，如果价差只比手续费高一点，任何小变化都可能让利润变成亏损。因此不能只要求“计算结果大于零”，还需要留出一定的安全空间。

我目前的简单判断方法是：

可接受价差 = 各项预计成本 + 风险缓冲

预计成本包括 DEX 和 CEX 手续费、Gas、滑点及资金调整成本；风险缓冲则用来应对报价变化、延迟和成本估算不准。

今天最大的收获是，机会门槛不能完全固定。交易数量、链上拥堵程度和市场波动不同，需要预留的空间也会不同。利润看起来很高，也可能只是因为报价已经过期。

目前的卡点是还不知道风险缓冲应该设置多大。如果设置太低，容易遇到亏损；设置太高，又可能错过很多机会。

接下来准备回看之前记录的模拟交易，比较“发现时的预估利润”和“模拟执行后的利润”，用两者的差距尝试确定一个初步门槛。

笔记 052: Day 10 (8/14) | 原子套利：路径发现与闪电贷【重日】

作者：yixin | 时间：2026-08-16 06:45 UTC | 字数：908 | 评分：62

来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

Day 10 (8/14) | 原子套利：路径发现与闪电贷【重日】

一句话目标：把「两个池有价差」升级为「在整张流动性图上找最优闭环」，并理解闪电贷如何解除资本约束。

核心概念

1. 把 DEX 生态建模成有向图：节点 = token，边 = 池（带汇率和费率）。套利 = 找一个乘积 > 1 的环。
2. 负环检测：对每条边取权重 $w = \ln(\text{rate} \times (1 - \text{fee}))$ ，则「乘积 > 1 的环」「权重和 < 0 的环」Bellman-Ford 负环。
 - 复杂度 $O(VE)$ ，在几百个 token / 几千个池上可跑
 - 但这是理想化的：真实边的汇率随交易量变化（非线性），所以负环检测只能用于筛选候选路径，最终必须用 `pool_math` 对具体规模精确定价。两阶段：图筛选 → 精确定价 → 规模优化。
3. 多跳的成本爆炸：每多一跳 +1 次手续费 +gas。3 跳以上的机会在主流 token 上基本被吃干净，长尾 token 上才有——但长尾 token 有更大的坑（转账税、蜜罐、无法卖出）。
4. 闪电贷（flash loan）：同一 tx 内借出并归还。
 - 它解除的是资本约束，不是技术约束或信息约束——想清楚这一点，很多「用闪电贷就能无本套利」的幻想会自动破灭
 - 成本：Aave 约 0.05%、Balancer 部分池 0，以及额外的 gas
 - 闪电贷让所有人的资本约束都归零 → 所以纯资本型的原子套利机会不存在了（Day 9 题 9.3 的另一个证明）
5. 原子性的价值：要么全成功要么全回滚。这消除了「腿断」风险，但引入了 gas 沉没成本和上链竞争。链上是唯一能做真·无风险套利的地方，代价是这个机会的利润 99% 会被拍卖走。
6. 模拟执行（simulation）：提交前用 eth_call / Tenderly / 本地 fork 模拟。没有模拟就提交 = 烧钱。这是必须建立的工程纪律。

笔记 053: title: Day11 负溢价与 ADL 风险

作者：tutu | 时间：2026-08-16 07:00 UTC | 字数：1926 | 评分：72

来源：https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205

title: Day11 负溢价与 ADL 风险

date: 2026-08-16

tags:

- 套利学习
- 负溢价

- ADL
- 风险管理
aliases:
- 负溢价与自动减仓
status: completed

负溢价先当风险警报，ADL 能把中性仓位剪成裸空

市场里经常会出现一种很诱人的画面。现货卖 120 USDT，永续只要 100 USDT，旁边还挂着一个很高的负资金费。价格差有二十块，资金费看起来也在送钱。很多人看到这里，第一反应是赶紧找个方向开仓。

我这次把它重新算了一遍，先发现了一个很容易犯的方向错误。

如果现货是 120，永续是 100，后来两边都回到 110，适合赚收敛的钱的结构是做多永续，再借入现货卖掉。永续多单赚十块，现货空单也赚十块，每枚合起来赚二十块。负资金费的情况下，永续多头通常还是收款的一边。

反过来做就完全不同了。买入 120 的现货，同时做空 100 的永续，价格回到 110 时，两条腿各亏十块，合计亏二十块。永续空头还要支付负资金费。这个结构和负溢价收敛的方向相反，费率再高也救不了方向错误。

为了避免凭感觉判断，我会把基差统一写成

$B = \text{永续价格} - \text{现货价格}$

现货高于永续时，B 是负数。做多永续、做空现货的组合，基差盈亏是

$(\text{平仓时的 } B - \text{开仓时的 } B) \times \text{数量}$

B 从负数往零走，组合赚钱。B 继续变得更负，组合亏钱。这个符号先写清楚，很多看起来复杂的套利结构会立刻露出方向。

负资金费也不能单独拿来当入场理由。它有时确实能带来现金流，也可能是在提醒你市场已经很拥挤。低市值币尤其容易出现现货被少数资金抬着走、借币额度很小、盘口薄、指数价格不透明等情况。仓位虽然在等价差收敛，结果却可能取决于某个大户什么时候撤单。

入场前先问这笔仓位能不能活到收敛那一天。现货能不能借到足量，借币会不会被召回，借币成本能不能被保守估计的资金费覆盖，双腿按目标数量成交会吃掉多少滑点，两个账户的保证金能不能及时划转，这些都比页面上那一行年化数字重要。

我给低市值负费率币设了一条很硬的门槛。市值排名在 200 名以外时，如果现货可借数量不到目标空头数量的 1.2 倍，就不做空现货、多永续这类结构。借币年化达到 20% 会触发额外审查，但它只是提醒线，最终仍要把资金费、借币、手续费和滑点放到同一张 USDT 账上。

还有一种风险更容易被忽略，叫 ADL，也就是自动减仓。它发生时，交易所会在极端行情里主动削减一侧的盈利仓位。你的判断可以是对的，仓位也可以是赚钱的，交易所照样可能把其中一条腿剪掉。另一条腿不会因为它被剪掉就自动跟着减少。

纸面上算一遍就很直观。原本有一千枚永续多单和一千枚现货空单，双腿刚好匹配。永续多单被 ADL 减掉百分之四十以后，只剩六百枚。现货空头还在一千枚，组合瞬间变成净空四百枚。之后价格每枚上涨 0.50 USDT，这四百枚裸露空头就要亏 200 USDT。回补时的手续费和滑点还没算进去，这个数字已经撞到纸面账户的 Kill Switch。

遇到这种情况，先冻结新仓，按实际剩余数量重算 Delta。随后用 reduce-only、可成交限价、IOC 或拆单，把多出来的现货空头尽快回补。被剪掉的永续多单不能当场补回作为默认方案，价格也可能不会马上回来。盘口不够时，要承认自己已经有方向敞口，不能继续把“原本是套利仓”当成护身符。

所以我会现在把下面几种情况直接排除。现货高于永续，计划赚负溢价收敛，却想买现货、空永续，我会直接拒绝。低市值负费率币的现货借量不够目标仓位的 1.2 倍，双腿没有可靠的建立和退出路径，我不会开仓。入场前模拟 ADL 后无法马上消除残余 Delta，或者按当前盘口退出会碰到 160 USDT 红色预警，直接跳过。

小号的方向仓另算一套账。它的可执行净亏损达到保证金的 8%，或者维持保证金率低于 40%，就停止补仓、加仓和从主号调钱救仓，只允许减仓或止损，当天不再开新仓。主号赚的是持仓结构的钱，小号承担的是方向风险，两个账户可以分开，风险不能装作互不相干。

负溢价很难处理，资金费、借币、盘口、保证金和 ADL 可能同时不配合。那时要及时承认仓位已经不再是套利，先把能退出的风险退出去，再等下一次价差。

相关笔记

[[Day08-期现套利]] · [[Day09-资金费率套利]] · [[Day10-复合持仓法]] · [[Day12-跨所跨期与展期套利]]

笔记 054: CEX—DEX 套利是 CEX—CEX 的链上版，同一枚币在中心化订单簿和链上池之间赚价差，仍是非原

作者：Montaire | 时间：2026-08-16 09:11 UTC | 字数：276 | 评分：54

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

CEX—DEX 套利是 CEX—CEX 的链上版，同一枚币在中心化订单簿和链上池之间赚价差，仍是非原子库存套利。
核心新知识是时序差：CEX 毫秒成交，链上 swap 要几秒到几分钟确认，两边成交永远错开一个时间窗，窗口内是单边敞口。
链上成交不保证 CEX 对冲成交，四个原因：链上只是广播、成交时点错位、CEX 部分成交、方向风险放大。
链上特有的两个坑：Gas 是沉没成本（失败也不退）；提币/充值要等十几到几十个区块确认，期间单边暴露。
"两边同时下单就能锁价"不成立：CEX 价下单即锁，链上价发出时未知，基准价定不下来。

笔记 055: Day 12 | 短时三窗口报价稳定性

作者：chrise | 时间：2026-08-16 11:05 UTC | 字数：690 | 评分：84
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 | 短时三窗口报价稳定性

今日研究

在 19:02:20–19:04:02 内完成三轮 Ethereum USDC→WETH 五档只读报价，每轮覆盖 1,000、5,000、10,000、30,000、50,000 USDC，轮间等待 15 秒。

套利逻辑

短时重复采样可以把规模曲线与较慢的市场价格漂移部分分开，同时观察聚合路由是否稳定。报价延迟越长，后续档位与小额基准越难视为同一市场快照。

数据与结果

15 条请求均成功。各档位相对当轮 1,000 USDC 基准的中位变化为 0、-0.8086、-1.0561、-0.7184、-0.9276 bps；50,000 USDC 档区间为 -1.7255 至 -0.7773 bps。三个窗口的小额参考单位报价相对首轮变化为 0、-0.5278、-1.4805 bps。gas 估值区间为 0.0549–0.1227 USD。

成本与风险

5,000、10,000 与 30,000 USDC 档发生路由切换，50,000 USDC 档三轮均使用 kyberswap。单次响应耗时为 2.073–10.923 秒，较长延迟会增加报价过期和顺序采样误差。

今日结论

50,000 USDC 档的短时中位降幅为 0.9276 bps，仍远低于原压力模型相对小额档的 24.5 bps。原模型可保留为保守上界，当前样本不足以给出精确的真实滑点参数。

下一步验证

把各档位的短时最差值和响应延迟纳入成本模型，比较保守报价成本与原平方滑点预算对净收益门槛的影响。

笔记 056: 今天看了《九种套利角度：用闭合检查器筛一遍》，最大的收获不是又认识了几个套利策略，而是开始学会怎么判

作者：Oxbin | 时间：2026-08-16 11:20 UTC | 字数：390 | 评分：56
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天看了《九种套利角度：用闭合检查器筛一遍》，最大的收获不是又认识了几个套利策略，而是开始学会怎么判断一个套利思路值不值得继续花时间研究。

以前看到资金费率、现货合约、新币价差、DEX-CEX、做市、跨链等机会，第一反应都是“这个能不能赚”。

今天换了一个思路，用几个维度去过滤：

能不能形成完整的交易闭环

有没有稳定、可获取的实时报价

Gas / 手续费 / 滑点 / 对冲成本算完还有没有利润

机会是不是过于集中，普通人实际能不能吃到

最后再决定值不值得写代码、实盘验证

感觉这个思路对我挺重要的。

不是所有看起来有价差的東西都值得做，先筛掉伪机会，再把时间集中到真正可以数据验证、自动化执行的方向。

下一步准备挑一个通过筛选的套利方向，自己把：

数据获取 → 利润计算 → 成本计算 → 执行条件 → 风险点 \ 完整跑一遍。

笔记 057: Day 11 : FASTEST 快 5 秒，但最低输出反而少 5.089062 USDC。

作者：daohuang1982-beep | 时间：2026-08-16 11:42 UTC | 字数：566 | 评分：67

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 11 : FASTEST 快 5 秒，但最低输出反而少 5.089062 USDC。

今天对同一笔 Ethereum → Arbitrum 的 1,000 USDC，只改变 LI.FI Quote 的路由目标：CHEAPEST 选择 Eco，预计 7 秒，预计与最低输出均为 997.5 USDC；FASTEST 选择 Relay，预计 2 秒，预计输出 997.397927 USDC，最低输出 992.410938 USDC。

也就是说，FASTEST 的预计时长缩短约 71.43%，Gas 估算少约 0.0194 USD，但预计输出少 0.102073 USDC，最低输出更少 5.089062 USDC。这里的最低输出包含本次 0.5% 滑点设置，不是已实现损失或到账证明；两份 Quote 也相隔 18 秒，并非原子同步快照。

今天把停止线明确为：先验证闭环最低净收益是否为正，再评估执行时长、报价过期和资金暴露；经济门槛不通过时，速度优化没有意义。Day 10 的既有候选仍应保持 quote_rejected，不能升级为 paper_trade_candidate。今天没有保存可执行 calldata，也没有连接钱包、授权、签名、广播或成交；当前没有发现或确认任何可执行套利机会。

笔记 058: 今日 (8/16) 实际学习：做市深学 + 跨所套利视角。

作者：gh25888 | 时间：2026-08-16 11:48 UTC | 字数：717 | 评分：56

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今日 (8/16) 实际学习：做市深学 + 跨所套利视角。

1. 做市完整系统：宽哥案例深学 7 题全通——贴盘固定单量/dust占位(先小后大防被砸)/后部厚墙(浅层样品深层真货)/库存周期(凌晨接货→傍晚出货,被动被吃+主动引导,当天中性不过夜)/三道风控(熔断+大币+不过夜)/选盘(点差厚+竞争少+容量匹配,小资金配厚利盘)/高频小单跑量赚钱。自己推导出库存管理核心逻辑(高抛低吸+熔断+当天中性)。
2. 账户识别法：公开API查账户抵押品档位匹配做市商身份(10000u档账户特征)，探针反推参与者分层(专业做市商/鲸鱼/散户)。
3. 套利视角：fxu_starr (HFT做市) 策略拆解——跨所基差线性回归=主要Alpha、GLFT库存模型+动态spread、毒流风控(异常→平仓+休息)、拥抱对的库存(方向=超额收益)；Thomson做市三方向(有效信息/有效订单流/盘口塑造)。
4. Hummingbot 案例 Day1：simple_pmm 官方脚本拆解 (mid±spread双边挂单、不对称spread=库存偏移、15s刷新、预算检查)。
5. 数据基建：探针升级(深档带价/厚墙监控/市场状态/账户快照) + Var↔Lighter 跨所套利探针重启 (BTC/ETH/SOL/QQQ/SKHY/XAU 六币同刻价差+大小资金账户跟踪)。

证据：vault

资料/做市模块补齐-宽哥路线.md、宽哥账户画像-3200.md、fxu_starr策略拆解-学习笔记.md、Hummingbot案例学习日志.md；探针 rb_mm_probe_v2.db + coin_selector.db (sync_pairs)。

笔记 059: 今天承接的问题

作者：rakisa | 时间：2026-08-16 12:10 UTC | 字数：8648 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天承接的问题

第11天我对1,000 USDC做了12次连续报价，观察到：

到账金额一直是997.5 USDC；

大部分路线是Eco；

有一次短暂切换到Polymer Standard；

预计执行时间因此从7秒变成1,080秒；

金额不变并不代表路线稳定。

今天继续追问：

如果把输入金额从100 USDC放大到1,000和10,000 USDC，路线、费用、Gas占比和稳定性会怎样变化？

我没有只算“哪个金额到账最多”，因为绝对到账量没有直接可比性。

今天重点比较：

1. 到账比例；
2. 绝对减少量；
3. Gas绝对金额；

4. Gas占本金的基点数；
5. 近似单向成本比例；
6. 路线是否因金额改变；
7. 每个金额能否通过连续观察确认。

三、实验假设

开始前，我有两个待验证假设。

假设1：固定Gas会被大额交易摊薄

如果三档金额使用相同路线、Gas相近，那么：

text

Gas占比 = Gas金额 / 输入本金

金额越大，Gas占比应该越低。

假设2：大额可能改变路由选择

金额增大后可能遇到：

不同流动性深度；

不同桥容量；

不同价格冲击；

不同solver报价；

不同路线时间。

所以不能预先假设100、1,000和10,000 USDC必然使用同一路线。

四、实验参数

本次继续使用同一方向：

text

接口：LI.FI Quote API

方向：Base USDC → Arbitrum USDC

Token：USDC → USDC

排序：CHEAPEST

预设：stablecoin

滑点参数：0.5%

金额分为三档：

text

100 USDC

1,000 USDC

10,000 USDC

每档采集4次，总共12次请求。

每次响应完成后等待2秒，再发送下一次请求。

五、为什么轮换请求顺序

如果每轮都固定按：

text

100 → 1,000 → 10,000

那么小额总是在较早时刻采集，大额总是在较晚时刻采集。

一旦市场或API状态随时间改变，我可能把时间差错误解释成金额差。

因此使用四轮顺序：

text

第1轮：100 → 1,000 → 10,000

第2轮：10,000 → 100 → 1,000

第3轮：1,000 → 10,000 → 100

第4轮：100 → 1,000 → 10,000

这不能完全消除时间偏差，因为请求仍然是顺序发生的，但比固定顺序更合理。

六、安全边界

今天仍然是只读实验：

不连接钱包；
不读取余额；
不使用私钥；
不使用助记词；
不进行Token授权；
不签名；
不广播交易；
使用公开销毁地址作为报价占位地址；
不根据结果自动执行任何链上动作。

七、真实采样结果

本轮观察窗口：

text

38.888秒

12次请求全部成功：

text

请求：12

成功：12

失败：0

重试：0

按真实请求顺序

	全局样本	轮次	金额	预计到账	工具	预计执行	API延迟
	---: :-: -----: -----: :-: ---: -----:						
1	1	100 USDC	99.75 USDC	Eco	7秒	1,852 ms	
2	1	1,000 USDC	997.5 USDC	Eco	7秒	1,828 ms	
3	1	10,000 USDC	9,975 USDC	Eco	7秒	2,088 ms	
4	2	10,000 USDC	9,975 USDC	Eco	7秒	1,355 ms	
5	2	100 USDC	99.75 USDC	Eco	7秒	2,083 ms	
6	2	1,000 USDC	997.5 USDC	Eco	7秒	1,911 ms	
7	3	1,000 USDC	997.5 USDC	Eco	7秒	1,182 ms	
8	3	10,000 USDC	9,975 USDC	Eco	7秒	878 ms	
9	3	100 USDC	99.75 USDC	Eco	7秒	807 ms	
10	4	100 USDC	99.75 USDC	Eco	7秒	573 ms	
11	4	1,000 USDC	997.5 USDC	Eco	7秒	790 ms	
12	4	10,000 USDC	9,975 USDC	Eco	7秒	1,495 ms	

本轮没有出现HTTP错误，也没有任何请求超过5秒研究阈值。

八、第一组结果：到账比例完全一致

100 USDC

text

输入：100

预计到账：99.75

报价减少：0.25 USDC

保留比例：99.75%

1,000 USDC

text

输入：1,000

预计到账：997.5

报价减少：2.5 USDC

保留比例：99.75%

10,000 USDC

text

输入：10,000

预计到账：9,975

报价减少：25 USDC

保留比例：99.75%

汇总：

输入	预计到账	报价减少	减少比例	保留比例
100	99.75	0.25	0.25%	99.75%
1,000	997.5	2.5	0.25%	99.75%
10,000	9,975	25	0.25%	99.75%

在本轮样本和金额范围内，报价减少量表现为线性比例：本金放大10倍，绝对减少量也放大10倍。

但只有三档金额、每档四条样本，不能据此断言任意金额都保持线性。

九、API费用估值为什么不是正好0.25%、0.25%和0.25%

报价中保存的费用美元估值分别为：

text

100 USDC：约\$0.2492

1,000 USDC：约\$2.4923

10,000 USDC：约\$24.9234

它们略低于Token单位下的：

text

0.25 USDC

2.5 USDC

25 USDC

原因是API中的美元估值可能使用自己的Token价格源，USDC/USD不一定恰好等于1。

因此，比较同一Token的输入和输出时，我优先使用USDC原始Token数量：

text

输入USDC - 输出USDC

费用美元估值用于辅助观察，不能与Token数量不加说明地混算。

十、第二组结果：Gas绝对金额没有随本金放大

三档金额的额外Gas估算都是：

text

\$0.0067

本轮都使用相同Eco路线，因此出现相同Gas估算并不意外。

但同样的0.0067美元，对不同本金的相对影响不同。

十一、Gas占本金比例相差100倍

使用基点表示：

text

1 bps = 0.01%

计算公式：

text

Gas基点 = Gas美元估算 / 输入USDC × 10,000

在近似假设1 USDC ≈ 1 USD下：

100 USDC

text

0.0067 / 100 × 10,000

= 0.67 bps

1,000 USDC

text

0.0067 / 1,000 × 10,000

= 0.067 bps

10,000 USDC

text

$0.0067 / 10,000 \times 10,000$
= 0.0067 bps

汇总：

输入	Gas估算	Gas占本金
100 USDC	\$0.0067	0.67 bps
1,000 USDC	\$0.0067	0.067 bps
10,000 USDC	\$0.0067	0.0067 bps

本金放大100倍后，固定Gas的相对负担下降到原来的1/100。

十二、第三组结果：近似单向成本比例逐渐接近0.25%

报价减少量已经包含协议内含费用，因此不能再把feeCosts重复加一次。

这里只加入报价之外的额外Gas估算：

text

近似单向成本

= 输入USDC - 预计到账USDC + Gas USD

继续使用研究近似：

text

1 USDC $\approx$ 1 USD

结果：

100 USDC

text

$0.25 + 0.0067 = 0.2567$ USDC

成本比例约0.2567%

1,000 USDC

text

$2.5 + 0.0067 = 2.5067$ USDC

成本比例约0.25067%

10,000 USDC

text

$25 + 0.0067 = 25.0067$ USDC

成本比例约0.250067%

输入	报价减少	额外Gas	近似单向成本	近似成本比例
100	0.25	0.0067	0.2567	0.2567%
1,000	2.5	0.0067	2.5067	0.25067%
10,000	25	0.0067	25.0067	0.250067%

金额越大，固定Gas越容易被摊薄，所以总比例逐渐接近协议比例成本0.25%。

但绝对成本仍然从0.2567增加到25.0067 USDC。

十三、为什么不能得出“10,000 USDC最划算”

从成本比例看，10,000 USDC确实最低。

但这不等于它是更好的套利规模，因为当前实验没有包含：

回程成本；

DEX价格冲击；

目标链成交深度；

跨链资金占用；

失败交易预期成本；

库存再平衡成本；

报价过期风险；

桥和合约风险；

更大本金对应的绝对损失风险；

真正的市场价差。

更大金额降低固定成本占比，同时放大按比例费用和风险敞口。

正确的问题不是“金额越大是否越好”，而是：

净收益随金额增加到哪里开始下降？

要回答它，需要加入目标市场的深度和价格冲击，而本次USDC跨链单向报价还不够。

十四、第四组结果：三档金额都选择Eco

本轮12条报价全部使用：

text

protocol:feeCollection → cross:eco

按金额统计：

| 输入金额 | Eco | 其他工具 | 路线切换 |

| -----: | -: | -: | -: |

| 100 USDC | 4 | 0 | 0 |

| 1,000 USDC | 4 | 0 | 0 |

| 10,000 USDC | 4 | 0 | 0 |

预计执行时间也全部为7秒。

因此，在本次38.888秒窗口内，没有观察到金额导致的路线差异。

十五、这是否推翻了第11天的Polymer样本？

没有。

第11天的1,000 USDC样本曾有一次：

text

Polymer Standard

预计1,080秒

今天1,000 USDC四次全部是Eco。

两天的结果可以同时成立：

第11天的短窗口内确实观察到一次Polymer；

第12天的另一个短窗口内没有观察到Polymer。

今天不能把“4次Eco”写成“1,000 USDC永远走Eco”，也不能把昨天的一次Polymer写成“路线经常切换”。

只有更长时间序列才能估计出现频率。

十六、API延迟对比

100 USDC

text

样本：1,852 / 2,083 / 807 / 573 ms

最短：573 ms

最长：2,083 ms

中位数：1,329.5 ms

1,000 USDC

text

样本：1,828 / 1,911 / 1,182 / 790 ms

最短：790 ms

最长：1,911 ms

中位数：1,505 ms

10,000 USDC

text

样本：2,088 / 1,355 / 878 / 1,495 ms

最短：878 ms

最长：2,088 ms

中位数：1,425 ms

三档金额的延迟范围重叠明显。

样本太少，无法判断金额是否会影响API响应速度。本轮没有任何请求超过5秒阈值。

十七、连续确认状态

继续使用第11天的研究规则：

text

同一金额连续3次经济指纹相同

且每次API延迟 $\leq 5,000$ ms

→ 确认稳定观察

由于三个金额交错采集，确认发生在不同的全局样本：

text

1,000 USDC：全局样本7，完成本金额第3次观察

10,000 USDC：全局样本8，完成本金额第3次观察

100 USDC：全局样本9，完成本金额第3次观察

各金额的第4次报价继续满足条件，实验结束时：

text

100 USDC：保持确认

1,000 USDC：保持确认

10,000 USDC：保持确认

撤销次数：0

这只代表本轮单向报价在研究规则下稳定，不表示三个金额都有套利机会。

十八、为什么稳定确认仍然不是套利信号

今天缺少：

text

Arbitrum → Base回程报价

往返完整成本

真实目标链库存

报价有效期

实际到账时间

原子执行条件

正净收益证据

所以程序输出：

text

allGroupsFinallyConfirmed = true

arbitrageSignalGenerated = false

executionEligible = false

transactionBroadcast = false

“观察稳定”和“值得交易”必须是两个不同字段。

十九、今天代码中遇到的真实问题

本地测试计算100 USDC的Gas基点时，理论结果是：

text

0.7 bps

JavaScript实际浮点表示为：

text

0.7000000000000001

这是二进制浮点表示误差，不是成本模型多出了一笔费用。

我没有删除断言，而是按合理小数精度比较：

text

Number(value.toFixed(3)) === 0.7

这也提醒我，未来处理Token原始金额时应尽量保留整数最小单位或使用高精度十进制库，避免直接用浮点数做最终资金结算。

二十、自动化验证

今天共有44项断言，覆盖：

三档金额各自采样数量；
请求总数和成功数；
Token最小单位转换；
99.75%到账比例；
0.25、2.5、25 USDC报价减少量；
Gas基点随本金缩小；
近似单向成本金额；
近似单向成本比例；
路线切换次数；
每档连续确认；
跨金额Gas负担排序；
不生成套利信号；
不获得执行资格；
钱包未连接；
未签名；
未广播交易；
JSON和CSV结果正常保存。

结果：

text

Day12多金额采样与分析44项断言全部通过

二十一、今天建立的数据结构

每条样本保存：

text

全局样本号

轮次

本轮位置

金额内样本号

输入金额

请求和响应时间

API延迟

HTTP状态

重试次数

报价ID

预计到账和最低到账

费用估值

Gas估值

工具和完整路线

预计执行时间

经济指纹

错误类型

每档金额的汇总保存：

text

到账范围

到账比例

报价减少量

费用美元估值

Gas范围

Gas基点

近似单向成本

延迟中位数

执行时间范围

路线分布

路线切换

经济状态数量

确认和撤销事件

二十二、今天产出的文件

text

multi_amount_sampler.mjs

amount_analyzer.mjs

multi_amount.test.mjs

README.md

data/collector/raw/<timestamp>.json

data/collector/results/multi_amount_samples.json

data/collector/results/multi_amount_samples.csv

data/results/amount_analysis.json

data/results/amount_summary.csv

二十三、今天最重要的认识

今天观察到两种不同的规模效应。

可以被摊薄的

text

固定Gas占本金比例

从100到10,000 USDC，Gas负担从0.67 bps降到0.0067 bps。

不会被摊薄的

text

按本金比例收取的费用

三档报价减少比例都是0.25%，绝对金额则从0.25增加到25 USDC。

所以：

放大本金能够摊薄固定成本，但不能摊薄比例成本，还会放大绝对风险敞口。

二十四、本次实验限制

1. 每个金额只有4条样本；
2. 总观察窗口只有38.888秒；
3. 三个金额不是同时请求；
4. 轮换顺序只能降低、不能消除时间偏差；
5. 只观察一个方向；
6. 只观察USDC跨链；
7. 没有DEX交易和价格冲击数据；
8. 没有真实执行；
9. 没有实际到账时间；
10. 没有目标链库存；
11. 没有回程报价；
12. 1 USDC≈1 USD只是Gas换算近似；
13. 不能用本轮结果推断长期路线分布；
14. 不能认为10,000 USDC一定比100 USDC更优；
15. 不能据此生成交易信号。

二十五、下一步计划

第13天准备在相同三档金额下，同时研究两个方向：

text

Base → Arbitrum

Arbitrum → Base

比较：

1. 两个方向是否使用相同工具；
2. 两个方向的费用比例是否对称；
3. Gas和预计时间是否对称；
4. 去程输出作为回程输入时，纸面闭环减少多少；
5. 两次报价时间差是否满足顺序执行；
6. 哪些结果只是同一时间窗口的双向快照；
7. 为什么双向报价仍不能证明可执行套利。

继续保持只读，不连接钱包、不签名、不广播交易。

今日结论

今天完成了100、1,000、10,000 USDC三档金额共12次真实LI.FI只读报价采样。

三档到账比例都为99.75%，全部使用Eco路线，预计执行时间均为7秒，没有观察到路线切换。额外Gas估算都约为0.0067美元，因此Gas占本金从0.67 bps下降到0.067和0.0067 bps。

但报价减少的绝对金额同步从0.25增加到2.5和25 USDC。按1 USDC≈1 USD粗略加入额外Gas后，单向成本比例约为0.2567%、0.25067%和0.250067%。

三档金额均通过短窗口稳定观察规则，但没有回程、完整净收益、库存和执行证据，因此套利信号仍为0。

笔记 060: 第十二天学习记录：今天对 Robinhood Chain 套利研究的证据链做了一次三方一致性审计，逐

作者：yu | 时间：2026-08-16 12:10 UTC | 字数：675 | 评分：52

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第十二天学习记录：今天对 Robinhood Chain 套利研究的证据链做了一次三方一致性审计，逐项核对 STATUS 说明、readiness 产物和正式 shadow ledger。现场计算的 canonical ledger digest 为 0x2d0528...143d，而 readiness 仍绑定 0x04c14b...3cd6，STATUS 中还保留更早的 0x9A41...7D3F，说明文档、产物与当前账本已经漂移；因此旧的 settled、verified 等数字不能作为当前放行依据，必须按 STALE_READINESS fail-close，资金状态继续 NOT_READY。随后复跑 freshness、execution gate 和 architecture contract 共 26 项测试，全部通过，确认摘要不匹配时会阻断展示和执行旁路。但进一步核对发现 OwnedArbExecutorV1

目前仍只存在于设计文档，业务源码中没有对应合约实现，所以不能声称我方地址已能调用、利润已能进入我方 beneficiary，也没有任何可执行净收益证据。今天未连接钱包、未签名、未广播、未实盘。下一步先用当前 canonical ledger 重算只读 readiness 并同步 SSOT，再在本地 EVM/fork 实现和验证 controller、executor、beneficiary 的资产守恒与实得增量；只有未来目标块双 RPC 精确一致的 OWNED_PROSPECTIVE_EXACT 证据才能进入严格统计。

笔记 061:

今天继续梳理跨所资金费率套利的收益结构，重点区分资金费率收益和基差变化带来的损益。

作者：loveyuanfandj-glitch | 时间：2026-08-16 12:49 UTC | 字数：454 | 评分：58

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天继续梳理跨所资金费率套利的收益结构，重点区分资金费率收益和基差变化带来的损益。

这类策略虽然通常被称为“市场中性”，但双边仓位并不意味着没有价格风险。开仓后，资金费率会形成相对确定的持仓收益，但两家交易所之间的价格偏离会持续变化。如果平仓时基差扩大，价格损失可能直接抵消已经积累的资金费率。

因此，Paper Trading 不能只统计累计 funding，还需要分别记录：

双边开仓和平仓价格；

资金费率累计收益；

跨所基差变化；

手续费和盘口成本；

最终净收益及最大浮亏。

在退出条件上，也不能只等待资金费率反转。基差异常扩大、行情数据过期、持仓时间超过预期，或者剩余费率优势已经不足以覆盖退出成本，都应该触发退出或停止继续持有。

接下来准备通过持续数据观察，分析不同标的的 funding 收益与 basis PnL 分布。重点关注 NVDA、CRCL、SPCX 等 RWA 市场，看看它们的主要风险究竟来自费率变化，还是来自不同交易平台在交易时段和定价机制上的差异。

笔记 062: Day 12 | 资金费率套利进入 Paper Trading：把 OI 异动加进信号过滤层

作者：zhanglianzhong | 时间：2026-08-16 13:01 UTC | 字数：926 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 | 资金费率套利进入 Paper Trading：把 OI 异动加进信号过滤层

今日目标

课程进入「从假设走向策略」阶段。今天把跨所资金费率套利从概念落成一个可记录的 Paper Trading 框架，重点解决一个问题：屏幕上看到的 funding 利差，多少是真的能吃的，多少是野庄设好的饵。

学到了什么

1. 资金费率收益 $\neq$ 净收益。双边对冲开仓后，funding 是相对确定的持仓收益，但跨所基差会持续漂移。平仓时基差扩大的损失可以直接吞掉累计 funding。所以 Paper Trading 的账本必须拆成五栏：双边开/平仓价、funding 累计、基差变化、手续费+盘口成本、净收益与最大浮亏。只记 funding 的账本是自欺欺人。
2. OI 是无法造假的过滤器。之前在 Bitget 踩过一个坑：开仓前看 OI 有 4000 万规模，觉得深度够，结果 1 分钟内被 ADL，OI 从 4000 万砸到 200 万——事后复盘才知道其中 95% 的仓位是野庄对倒出来的。价格可以画，深度可以挂假单，但 OI 的变化速率骗不了人：价格横盘而 OI 显著放大，是最典型的庄家建仓特征。今天把这个教训正式写成信号规则：funding 利差达标 + OI 平稳 = 可记录信号；funding 利差异常诱人 + OI 短期陡增 = 直接排除，不纳入样本。
3. 退出条件不能只有 funding 反转。补充了三条硬退出：基差异常扩大超阈值、数据延迟超过 30 秒、持仓时长超过预期上限。Paper Trading 阶段宁可错杀，先把回撤结构摸清楚。

认知收获

套利策略的 Edge 一半在价差识别，另一半在「识别哪些价差是陷阱」。OI 异动这层过滤可能让我错过一些真机会，但按 Bitget 那次的亏损幅度算，过滤掉一个野庄顶得上错过二十次正常信号。期望值上划算。

明日计划

把今天定的五栏账本做成一个最小记录模板，跑第一笔跨所 funding 套利的 Paper Trade，同时记录双边 OI 的分钟级变化，验证过滤规则在实际数据里是否可操作。

笔记 063: 2026-08-16 學習打卡

作者：cht+ | 时间：2026-08-16 13:28 UTC | 字数：778 | 评分：47
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-16 學習打卡

今天完成了什麼

完成跨市場套利 Shadow Mode

的行情限流事故修正：把產品清單更新失敗限制在單次更新流程內，避免錯誤直接中斷持續監控並引發反覆重啟。
將產品清單更新改為先完整驗證、再一次切換；若訂閱、暖機或部分資料來源失敗，會保留上一份可用清單並嘗試恢復舊訂閱，避免半套狀態污染後續掃描。
將一般行情掃描與產品清單更新拆成獨立的限流退避狀態，各自只由對應操作的完整成功解除，避免舊清單掃描成功過早清除更新端的限流保護。
補強空清單、停止期限、錯誤統計與去敏紀錄等邊界條件；目前只完成程式與測試驗證，尚未部署到正式監控服務。

證據在哪裡

今日版本紀錄顯示，行情限流隔離、產品更新回退與獨立退避機制已完成整合，工作目錄乾淨。
全部 196 項自動化測試通過，程式碼品質檢查與差異格式檢查亦通過。
多輪缺陷檢視曾找出更新失敗外溢、部分更新污染、退避被錯誤清除等問題；修正後的最終檢視未留下高或中等嚴重度阻擋項目。
正式監控服務目前仍在運行，但尚未載入今天的修正版，因此不能把測試成果描述成正式環境已驗證。

尚未完成／下一步

在取得正式變更許可後，依部署與回退檢查表載入修正版，確認服務能平穩重啟並恢復資料流。
部署後持續觀察行情拒絕、退避間隔、產品清單完整性與監控資料新鮮度，確認不再出現反覆重啟。
以受控方式驗證限流解除後，兩套退避狀態皆只在各自操作完整成功時清零。

今日學習心得

面對外部行情來源的限流，重點不只是「重試」，而是縮小故障邊界、保留最後可用狀態，並讓不同操作擁有互不干擾的復原條件。
套利監控必須先確保資料管線能穩定退化與誠實回報，才有資格談高品質訊號；測試通過也不等於正式環境已驗證，部署前後的證據邊界必須清楚分開。

笔记 064: 套利基础学习打卡⑦ | Leg Risk

与执行策略：两条腿不可能真正同时成交，应该先下哪一边？

作者：cfanboy | 时间：2026-08-16 14:13 UTC | 字数：9202 | 评分：86
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

套利基础学习打卡⑦ | Leg Risk 与执行策略：两条腿不可能真正同时成交，应该先下哪一边？

前六篇已经从套利基础逐渐走到了实际执行：

- ① Funding Rate：资金费率\
- ② Basis：基差与收敛\
- ③ Delta Neutral：消除方向敞口\
- ④ Trading Cost：从毛价差到净利润\
- ⑤ Liquidity：订单簿深度与可交易容量\
- ⑥ Latency：为什么看到机会时，机会可能已经消失

今天继续研究一个真正进入套利执行引擎的问题：

Leg Risk（单腿风险）

假设系统发现：

A 交易所 BTC：

Best Ask = 100,000

B 交易所 BTC：

Best Bid = 100,500

于是准备：

A：Buy 1 BTC

同时：

B：Sell 1 BTC

理论上：

Long 1 BTC + Short 1 BTC

所以：

Net Delta $\approx$ 0

理论利润：

500 USDT - Costs

看起来非常简单。

但真正执行时存在一个无法绕开的事实：

两个不同市场中的订单，不可能保证在完全相同的时间、完全相同的数量成交。

于是套利策略中一个非常重要的风险出现了：

Leg Risk。

一、什么是 Leg Risk？

套利通常包含两条甚至更多交易腿。

例如跨交易所套利：

Leg A：Buy BTC @ Exchange A

Leg B：Sell BTC @ Exchange B

理想情况：

A 成交 1 BTC

B 成交 1 BTC

那么：

Net Delta $\approx$ 0

但实际情况可能变成：

A：

成交 1 BTC

B：

成交 0 BTC

此时：

Net Delta：

+1 BTC

原本设计的是：

Delta-Neutral Arbitrage

结果瞬间变成：

Long BTC Directional Position。

如果 BTC 此时下跌：

套利策略就开始产生方向亏损。

这就是：

Leg Risk = 多腿策略中，各条腿无法按照预期同时成交所产生的风险。

二、为什么两条腿不能真正“同时成交”？

程序中可以同时发送：

Buy A

和：

Sell B

但这并不代表市场会同时处理。

例如：

T = 0ms：

同时发送两个请求。

但实际可能是：

A 网络延迟：

10ms

B 网络延迟：

35ms

A Exchange Processing：

5ms

B Exchange Processing：

20ms

最终：

A：

15ms 进入 Matching Engine

B：

55ms 才进入 Matching Engine

已经产生：

40ms

时间差。

更何况还存在：

订单簿变化

网络抖动

交易所负载

API Gateway

Risk Check

Matching Engine

等因素。

所以现实中的：

Simultaneous Order

只能理解为：

尽可能同时发送。

不能理解为：

保证同时成交。

三、最危险的情况：一边成交，另一边没有成交

继续前面的例子：

A：

Buy 1 BTC @ 100,000

已经成交。

但 B 的：

Sell @ 100,500

到达时价格已经变成：

Best Bid = 100,200

如果我们坚持：

100,500

那么订单可能：

完全无法成交。

于是现在持有：

+1 BTC

假设 BTC 随后：

100,000 → 99,500

亏损：

500 USDT

而最初整个套利机会理论利润：

也只有：

500 USDT。

也就是说：

一次单腿暴露产生的亏损，就可能吃掉整个套利利润。

如果市场继续快速下跌：

损失甚至可能远远超过原来的套利收益。

所以：

套利最大的风险之一，不一定是价差判断错误，而可能是对冲腿没有及时成交。

四、Partial Fill 更加常见

实际交易中还有一种情况比完全不成交更加常见：

Partial Fill（部分成交）。

例如：

A：

Buy 1 BTC

成交：

1 BTC

B：

Sell 1 BTC

实际只成交：

0.6 BTC

那么：

Long：

+1 BTC

Short :

-0.6 BTC

所以 :

Net Delta = +0.4 BTC

此时策略仍然存在 :

Residual Delta (剩余方向敞口) 。

假设 BTC :

100,000 USDT

那么 :

0.4 BTC

代表 :

40,000 USDT

的方向敞口。

如果 BTC 短时间下跌 :

1%

就可能产生 :

约 400 USDT

亏损。

所以套利系统不能只记录 :

Order Filled / Not Filled

而应该实时计算 :

Filled Quantity。

五、于是出现了一个核心问题 : 应该先下哪条腿 ?

假设 :

A 市场 :

流动性差。

B 市场 :

流动性很好。

套利 :

Buy A

[图片]

Sell B

应该先做哪边 ?

一种比较自然的思路是 :

先执行更难成交的一边。

例如 :

A :

订单簿很薄

只有少量流动性。

B :

BTC 深度非常好。

那么可以 :

先 :

Buy A

确认成交后 :

立即 :

Sell B Hedge

为什么？

因为如果反过来：

先在 B Short

↓

B 很容易成交

↓

再去 A Buy

结果 A 流动性已经消失

那么就会留下：

裸 Short BTC。

所以在某些场景下：

先处理流动性差、Fill Probability 较低的一条腿，再用高流动性市场快速 Hedge。

这是一个非常重要的执行思路。

六、但“难成交的一边先下”也不是绝对规则

问题又来了。

假设：

A：

流动性差

但价格波动非常快。

如果先：

Limit Buy A

然后等：

100ms

500ms

甚至：

2 秒

才成交。

等成交以后：

B 市场上的套利价格可能早已经消失。

此时虽然 A 成交了：

但 B 已经无法按照预期价格 Hedge。

所以执行顺序实际上需要同时考虑：

Liquidity

Volatility

Latency

Fill Probability

Expected Slippage

Opportunity Lifetime

而不是简单：

永远先 A

或者：

永远同时下。

这说明：

Execution Strategy 本身就是套利策略的一部分。

七、Market Order 能不能解决问题？

一个最简单的想法是：

两边都直接 Market Order，不就保证成交了吗？

确实可以大幅提高：

Fill Probability。

但会引入另一个问题：

Slippage。

例如：

理论套利：

Buy A：

100,000

Sell B：

100,500

Spread：

0.5%

结果 Market Buy A：

VWAP = 100,180

Market Sell B：

VWAP = 100,260

真正价差只剩：

80 USDT

再扣手续费：

可能已经：

Net Profit < 0

所以：

Market Order

解决了：

成交确定性

却牺牲了：

价格确定性。

这实际上是套利执行中的一个核心 Trade-off：

Price vs Fill Probability

八、Limit Order 又有什么问题？

反过来：

Limit Order

可以控制：

最差成交价格。

例如：

Buy BTC Limit = 100,000

不会买到：

100,300。

但问题是：

可能根本不成交。

所以：

Limit Order：

价格可控

↓

Fill Probability 下降。

Market Order :

Fill Probability 高

↓

价格不可控。

因此真正的套利执行不是简单选择：

Market

或者：

Limit

而是在寻找：

成交概率和成交价格之间的最佳平衡。

九、IOC 是套利中非常重要的一种订单

这里开始接触一个很适合套利的订单类型：

IOC — Immediate Or Cancel

意思是：

立即成交能够成交的部分，剩余部分取消。

例如：

Buy 1 BTC IOC @ 100,100

订单簿能够在：

$\leq 100,100$

成交：

0.7 BTC

那么：

成交：

0.7 BTC

剩余：

0.3 BTC 自动取消。

这样可以避免：

未成交订单继续挂在市场上。

对于套利来说非常有价值。

因为套利机会通常：

生命周期很短。

如果价差已经消失：

我们通常不希望原来的订单继续挂在那里等待成交。

十、FOK 又是什么？

另外一种订单：

FOK — Fill Or Kill

意思是：

要么全部成交，要么完全不成交。

例如：

Buy 1 BTC FOK @ 100,100

如果市场能够立即提供：

1 BTC

那么：

全部成交。

如果只能提供：

0.99 BTC

那么：

整笔订单取消。

理论上 FOK 非常适合套利：

因为：

不会产生 Partial Fill。

但现实中：

Fill Probability 可能明显下降。

尤其：

流动性较差

或者：

行情快速变化

的时候。

所以：

FOK：

降低 Partial Fill Risk

但：

增加 Missed Opportunity。

十一、IOC 和 FOK 怎么选？

可以简单理解：

IOC

允许：

Partial Fill

更容易成交。

但是需要处理：

Residual Delta。

FOK

不允许：

Partial Fill

更容易保持仓位完整。

但：

更容易完全错过机会。

所以：

高流动性市场

和：

低流动性市场

可能使用不同策略。

例如：

Illiquid Leg：

FOK / Aggressive IOC

Liquid Hedge Leg：

IOC / Marketable Limit

这就开始形成：

Execution Policy。

十二、什么是 Marketable Limit Order？

这是我觉得套利执行中非常实用的一个概念。

假设：

Best Ask：

100,000

我们愿意接受的最差价格：

100,100

那么可以发送：

Limit Buy @ 100,100

因为当前市场价格：

100,000

低于我们的 Limit：

所以订单会立即吃掉：

100,000

100,050

100,080

.....

但不会买到：

100,100

的价格。

这相当于：

给 Market Order 加一个最大滑点保护。

于是同时获得：

较高 Fill Probability

[图片]

Maximum Price Protection。

对于套利来说往往比：

纯 Market Order

更加合理。

十三、可以从套利利润反推最差成交价格

这里开始把前几篇知识串起来。

假设：

理论 Spread：

0.50%

所有其他成本：

0.15%

Minimum Profit Threshold：

0.10%

那么允许给执行滑点的空间只剩：

0.25%

于是系统可以根据：

Expected Profit

反推出：

Maximum Acceptable Slippage

然后得到：

Maximum Buy Price

和：

Minimum Sell Price

再使用：

Marketable Limit Order

执行。

这比固定：

Slippage = 0.5%

更加合理。

因为：

订单价格边界应该由当前套利利润空间决定。

十四、成交以后必须立即计算 Residual Delta

假设：

A Buy：

1 BTC

B Sell：

0.83 BTC

那么：

Residual Delta = +0.17 BTC

系统不能继续等待。

应该立即进入：

Hedge Mode

例如：

Sell：

0.17 BTC

即使需要接受一点额外 Slippage。

因为此时系统目标已经发生变化：

套利发现阶段：

Maximize Profit

↓

出现 Partial Fill 后：

Minimize Risk

这是非常重要的状态转换。

也就是说：

一旦套利仓位失衡，优先级应该从“赚更多”切换为“尽快恢复 Delta Neutral”。

十五、Execution Engine 应该是状态机

今天学习到这里，我觉得一个真正的套利 Execution Engine 很适合设计成状态机。

例如：

OPPORTUNITY_FOUND

↓

计算：

Profit / Liquidity / Risk

↓

EXECUTING

↓

发送 Leg A / Leg B

↓

检查成交

如果全部成交：

↓

HEDGED

↓

等待：

Spread Convergence / Funding / Exit

如果出现 Partial Fill：

↓

UNHEDGED

↓

立即：

HEDGE_RESIDUAL

↓

恢复：

DELTA_NEUTRAL

如果 Hedge 失败：

↓

EMERGENCY_EXIT

这样套利系统就不只是：

if spread > threshold:

buy()

sell()

而是一个真正需要管理：

Order Lifecycle

和：

Position State

的执行系统。

十六、不要只记录“下单成功”

还有一个工程上很重要的问题。

API 返回：

Order Accepted

并不代表：

Order Filled。

例如：

POST /order

返回：

Success

可能只是：

交易所已经接受订单。

真正需要继续监听：

Order Update / User Data Stream

直到：

NEW

↓

PARTIALLY_FILLED

↓

FILLED

或者：

CANCELED / REJECTED / EXPIRED

所以：

Order ACK $\neq$ Trade Execution

套利系统真正关心的是：

Fill。

而不是：

API 200 OK。

十七、需要记录哪些 Execution Metrics？

如果以后真正做套利系统，我认为至少应该记录：

Signal Timestamp

Order Send Timestamp

Exchange ACK Timestamp

First Fill Timestamp

Full Fill Timestamp

Requested Quantity

Filled Quantity

Expected Price

Actual VWAP

Expected Slippage

Realized Slippage

Residual Delta

Hedge Cost

然后长期统计：

Fill Rate

Partial Fill Rate

Average Slippage

P95 Slippage

Leg Failure Rate

Average Hedge Cost

这些数据最终可以反过来优化：

Execution Strategy。

十八、理论套利利润和 Realized PnL 的差距来自哪里？

经过今天的学习，可以进一步理解：

Scanner 发现：

Expected Profit：

+500 USDT

真正结束以后：

Realized Profit：

+220 USDT

中间消失的：

280 USDT

可能来自：

手续费：

-80

Slippage：

-70

Latency :

-40

Partial Fill :

-30

Emergency Hedge :

-40

其他成本 :

-20

所以真正评价一个套利系统 :

不能只统计 :

Opportunity PnL

还应该统计 :

Execution PnL

最终关注 :

Realized Net PnL

这三个数字之间的差距，本身就是非常重要的策略指标。

十九、Execution Quality 本身就是竞争力

今天还有一个比较大的认知变化。

假设两个套利机器人 :

Bot A

发现机会能力 :

非常强。

每天发现 :

10,000 个机会。

但是 :

Fill Rate 低

Slippage 高

Leg Risk 大。

Bot B

每天只发现 :

5,000 个机会。

但 :

Fill Rate 高

Slippage 低

Hedge 快

Residual Delta 小。

最终 :

Bot B 完全可能赚得更多。

所以套利系统的竞争力并不只是 :

Who Finds First

还包括 :

Who Executes Better

也就是说 :

Execution Alpha

本身可能就是收益来源。

二十、七篇内容已经进入 Execution Engine

现在整个学习路线已经变成：

① Funding Rate

找到：

收益来源

↓

② Basis

理解：

价格偏离与收敛

↓

③ Delta Neutral

处理：

方向风险

↓

④ Trading Cost

计算：

Net Profit

↓

⑤ Liquidity

判断：

可交易容量

↓

⑥ Latency

判断：

机会还能存在多久

↓

⑦ Leg Risk & Execution

解决：

怎么真正把两条腿成交

最终，一个套利机会的完整生命周期已经越来越清楚：

Market Data

↓

Opportunity Detection

↓

Order Book Simulation

↓

Net Profit Calculation

↓

Position Sizing

↓

Execution

↓

Fill Monitoring

↓

Residual Delta Hedge

↓

Position Management

↓
Exit
↓
Realized PnL
↓

Post-Trade Analysis

今天最值得记住的一句话是：

套利不是“同时点下买入和卖出”这么简单。真正的执行目标，是在价格、成交概率和风险之间做权衡，并在任何一条腿出现异常时，尽快恢复 Delta Neutral。

笔记 065: 2026-08-16 | 共学 Day 12 | 训练套利 Agent ② | 投入约 1 小时

作者：wuyuandao | 时间：2026-08-16 14:26 UTC | 字数：576 | 评分：67

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-16 | 共学 Day 12 | 训练套利 Agent ② | 投入约 1 小时

今天学了什么：

把 Day 9 的判断技能和 Day 10/11 的数据/回测工具打通，升级成完整的「套利判断 Agent」。技能从 v1.0 升级到 v1.1，四步 SOP 变成五步：核来源 → 补成本 → 查流动性 → 回测验证 → 给证据判断。

得到什么结果：

1. 新增「回测验证」步骤：Agent 不再只凭经验判断，而是能调用 backtest.py 跑出数据，用证据说话。
2. 新增「工具清单」：三个脚本（collect_prices.py 采 DEX 价、download_binance.py 下 CEX 历史、backtest.py 回测）各自的用途和调用时机。
3. 用 BNB 案例完整演示五步流程：查得 PancakeSwap 主池 \$609.48 (TVL \$5882万) vs Binance \$608.36，价差 0.184%，扣 0.6% 成本后净亏 -0.416%，判定放弃。对比 WBTC（价差 0.341%）发现规律：BNB 池子更深、价差被抹得更平，印证「池子越深 → 价差越小 → 越没套利空间」。

困难：无

下一步：Day 13 深度案例研究2——研究第二个套利者/策略，对比与 DOS 案例的异同。

07 学习计划（6 篇）

笔记 066: Day 12 | 2026.08.16 | 从原则到机制：三层利润、Nonce 协调、订单簿竞态

作者：xzone911 | 时间：2026-08-15 16:30 UTC | 字数：1141 | 评分：84

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 | 2026.08.16 | 从原则到机制：三层利润、Nonce 协调、订单簿竞态

背景

昨天提炼的是「为什么」（原子性、链上断言、固定区块、边际报价、深度健康检查）。今天挖「怎么做」——这些原则落地成的三个具体机制。

1. 三层利润模型：最坏成本边界的实现

不是算一个利润数字就签名，而是三级过滤：

| 层级 | 公式 | 用途 |

|-----|-----|-----|

| 市场粗利润 | 卖出收入 - 买入成本 - 协议费 | 快速筛选 |

| 最坏 Gas 后净利润 | 粗利润 - gasLimit × 有效费率上界 | 签名门槛 |

| 链上最终断言 | 执行后余额 - 执行前余额 - msg.value | 最终裁决 |

关键在第二层：EIP-1559 下私有交易可能在未来几个区块才被打包，base fee 会涨。所以用 $\min(\text{maxFeePerGas}, \text{未来 base fee 上界} + \text{priorityFee})$ 算最坏情况，只有最坏情况下净利润仍为正才签名。

第三层的余额差是唯一事实——订单退款、DEX 未用输入、协议真实扣费都会自然反映在余额差里，不依赖链下猜测。

2. Nonce 协调器：并发交易的顺序管理

同一区块发现多个机会，逐笔查 pending nonce 会把 RPC 延迟放进热路径，且并发任务容易拿到相同 nonce。做法：

- 后台定时校准账户的 latest / pending nonce
- 策略波次从内存快照预留一段连续 nonce (第 i 笔 = baseNonce + i)
- 中间候选准备失败时, 截断其后所有候选 (避免 nonce 空洞)
- 连续 nonce 的客观限制: 前序没上链, 后序也不能执行

所以「多机会并发」不是无脑并行, 而是只能广播从 base nonce 开始的连续成功前缀。

3. 订单簿 tombstone: 一个容易忽略的竞态

已经取消的订单, 可能在稍后到达的旧 REST 分页里再次出现 ("复活")。

解决: 为删除事件维护有时效的 tombstone, 校准合并时处于 tombstone 窗口内的旧订单不能复活。

这本质是分布式系统经典问题——乱序到达的删除 vs 迟到的旧快照。和 Solana 项目里「收到 WSS 通知→删缓存→再发 HTTP 补读」的坑是同一个东西: 迟到的旧状态不能覆盖更新的状态。

今日认知

设计原则是「为什么」, 具体机制是「怎么做」。最有价值的一条: 用余额差和所有权做最终裁决, 而不是第三方的返回值——因为返回值只表达「调用没报错」, 不表达「资产真的到账了」。

API

笔记 067: 链上套利残酷共学 · Day 12 打卡 (2026-08-16)

作者: keltuz4d | 时间: 2026-08-16 02:11 UTC | 字数: 2693 | 评分: 88

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

链上套利残酷共学 · Day 12 打卡 (2026-08-16)

今日主线: 把 Day 11 明日计划第①条"搭只读 Morpho NAV 折价扫描器"真正落地 + 实跑, 并产出可复用的"机会雷达"。顺带把昨天沉淀的 NAV 折价模板卡 (任务2) 一起归档。全程严格守"只读发现、不碰私钥、不签名"边界。

一、交付物 (两份文件 + 一份实跑结果)

| 文件 | 作用 | 边界 |

|---|---|---|

| /root/nav_discount_cheatsheet.md | NAV 折价套利模板速查卡 (触发公式 / 利润容量 / 扫描目标 / 风险 / 红线) | 纯本地 md, 不签名 |

| /root/morpho_nav_scanner.py | 只读扫描器 (Python, 纯 eth_call/GraphQL, 不发交易) | 只读, 不连钱包 |

| /tmp/morpho_candidates.json | 本次实跑产出的 40 个"满借温床"候选清单 | 数据产物 |

二、扫描器实跑结果 (真跑, 非空谈)

数据源: Morpho Blue 公共 GraphQL (blue-api.morpho.org) + 以太坊公共 RPC (publicnode) ——全公开只读。

扫描规模: 全量 7499 个市场遍历, 过滤"贷款资产=稳定币 + 有预言机 NAV 价 + 衍生/质押抵押品", 产出 144 个候选温床, 导出 Top 40。

关键验证: 报告案例 sNUSD 市场 (marketId 0xae60b71b...) 正好出现在候选第 29 位:

NAV价 = 1.0641 | LLTV = 0.915 | 可借价(LLTV×NAV) = 0.9736 | util = 100%

与报告原文的 0.9734 吻合 → 扫描器信号逻辑正确, 能复现已知案例。

Top 候选节选 (全部 100% 利用率 = 满借温床):

| 抵押币 | 贷款 | NAV价 | 可借价 | marketId |

|---|---|---|---|

| CToken2 | USDC | 4719 | 4058 | 0xdbcb3abc... |

| QQQon | USDC | 732 | 457 | 0xd3c7b366... |

| PT-sUSDE-24OCT2024 | DAI | 1.2436 | 1.0695 | 0x5badd7c... |

| wstUSR | USDC | 1.1330 | 1.0367 | 0x06a5954e... |

| sNUSD | USDC | 1.0641 | 0.9736 | 0xae60b71b... |

三、两个诚实的工程决策 (写进脚本注释 + 输出)

1. 不硬塞不准的现货价: GeckoTerminal 对长尾抵押币命中率 0/30, The Graph 在本环境不稳。所以扫描器只输出"满借温床候选" (Morpho 内部 NAV/LLTV/utilization 信号), 不谎称"已确认折价"。真正触发条件 DEX现货价 < LLTV×NAV 需 --verify <marketId> 走 RPC 读 Uniswap V4 池 slot0 确认——这步留作手动复核 (V4 池地址派生在本环境不稳)。

2. 脏数据过滤: HERMES 这类 oracle 给出 1e21 离谱 NAV 的市场已被合理性校验 (NAV 0.01~100000) 排除, 避免 meme/脏预言机污染清单。

脚本能力：--min-util 0.95（只显示满借温床）、--verify <marketId>（单市场精确复核）、--export json（导出清单）。

四、方法论闭环：机会雷达真正跑起来了

昨天模板卡第 6 节的伪代码，今天落成真代码并实跑。闭环逻辑：

Morpho GraphQL 拉全部市场 → 过滤(稳定币贷款+有NAV+衍生抵押品)

→ 算 可借价=LLTV×NAV → 按利用率排序(满借=温床信号) → 输出候选清单

→ 人工/--verify 读链上 DEX 池确认 DEX<可借价 才算真信号

这正好印证 Day 7-9 收敛的“自己读链上/协议数据算即时价差，别依赖聚合器”——LI.FI 连 KII 都不收录，GeckoTerminal 对长尾币 0 命中，机会发现必须下沉到协议层原始数据。

五、批判性提醒（给自己的）

1. “温床”≠“能赚”：清单里 WBTC/WETH 满借但通常是正常市价，无 DEX 折价；真正值得查的是衍生/质押长尾币（PT-sUSDE、USD0++、sNUSD、wstUSR 这类）。下一步必须逐个 --verify 读真实 DEX 价才算确认。

2. 退出端风险不变：报告强调的“脱锚即资不抵债”对所有 NAV 折价市场通用。满借市场里若抵押品脱锚，借出的是债务头寸，会被清算。看清单可以，动手越界不做。

3. 容量被池深锁死：sNUSD 单笔上限仅 ~\$260（Day 11 已算）。清单里 NAV 高的（CTOKEN2/QQQon）看着空间大，但 DEX 池更薄，实际能吃的量更小——需 --verify 时一并读池深。

4. 扫描器只发现不执行：完全在只读边界内。执行需自写合约+闪电贷+私有 relay，今天明确不碰。

六、明日计划

1. 强化 --verify 的 V4 池读取：用 RPC 直接读 Uniswap V4 PoolManager (slot0 + 地址派生)，对 sNUSD / PT-sUSDE 这类跑出“真正确认的折扣 %”，把“温床”变“确认信号”（仍只读）。

2. 等 8/16 增量笔记，对照“NAV 折价 / 结构性错价”主题看群里有没有人做出可复制版本，或踩了退出端坑。

3. 收尾 LI.FI integrator=jumper.exchange 修复 (/root/lifi_spread_probe.py) ——拖了一周，该补了。

4. 把扫描器 + 模板卡纳入个人“套利机会发现工具箱”速查，下次遇到新案例直接套。

笔记 068: 昨日套利日志：三角路线与链上对手

作者：xiaoquanyo | 时间：2026-08-16 07:26 UTC | 字数：852 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

昨日套利日志：三角路线与链上对手

昨天在分析链上交易时，发现了一条由稳定币、RWA 代币和另一种链上资产组成的三角套利路线。三个交易池分别使用不同版本的 AMM，单独看每个池子的价格都不算异常，但把三段兑换连起来后，部分区块可以重新换回更多稳定币。

我顺着的一笔同行交易还原了完整路径，并在对应历史区块上进行了分叉重放。结果证明这不是页面报价错误：当时的池子状态确实可以完成原子套利，扣除三段交易费用后仍有利润。机会主要来自一笔外部交易改变了其中一个池子的价格，而另外两个池子没有立即同步。

随后做了一版可配置的三角套利合约，没有把代币和池子写死。合约支持动态授权资产和交易池，也可以调整最大投入与最低利润。三段兑换必须在同一笔交易中完成，最终回款未覆盖本金和最低利润时整笔回滚，避免留下中间资产。

继续跟踪同行地址后发现，对方并不是发现机会后才临时构建交易，而是在持续监听池子状态，并按区块提交已经准备好的交易。出现价格失衡时，它可以在很短的时间内完成执行，有时还会连续修正同一条路线。从交易形态看，对方很可能使用了更快的区块事件源、预先构建的 calldata，以及带条件执行能力的私有交易通道。

我们目前完成了合约、路线配置和逐区块模拟，但还无法复制对方的完整执行方式。公共 RPC 对历史日志和高频查询有限制，只能降级为新区块轮询；普通公开交易即使合约会回滚，失败交易仍然需要支付 Gas。没有接入可靠的私有 Bundle 和区块构建者通道前，不能像对方一样每个区块都发送，否则大量无利润区块会持续消耗成本。新利润门槛下又进行了数百次实链模拟，没有发现当前可执行机会。进一步把门槛降到零检查后，各金额档的回款仍低于本金，说明这条路线并非常驻套利，而是依赖短时间的池价失衡。

昨天最大的收获是：三角套利的难点不在写出三段兑换，而在于等待什么链上事件、如何提前准备交易，以及能否比其他机器人更快进入目标区块。路线已经能复现，下一步真正需要解决的是事件触发、最优金额计算和私有执行通道。

笔记 069: Day 12 - 从一次性观察到重复查询

作者：MountainE | 时间：2026-08-16 08:42 UTC | 字数：628 | 评分：62

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 - 从一次性观察到重复查询

今天设计了一个研究工作流程 v1，把前几天学到的路由观察改写成可重复执行的查询。核心问题是：固定一组跨链路由条件后，如何定期检查报价、费用、预计时间和失败率的变化。

研究工作流程 v1：

1. 输入：来源链、目标链、资产、金额、路由类型、查询时间和可接受的滑点。
2. 工具：使用路由查询接口或可靠的数据来源获取当前报价，并保存查询时间和来源信息。
3. 输出：路由、预计到账数量、费用、预计时间、失败条件、毛价差和净收益。
4. 判断条件：只有报价未过期、路由状态正常、扣除全部成本后净收益仍为正、且风险在预先设定范围内时，才标记为值得继续验证。

需要人工复核的结果包括：

- 报价输出金额和费用异常大或异常小。
- 路由需要跨链桥、多步 DEX 兑换或新资产。
- 路由失败、超时、桥暂停或报价过期。
- 数据来源不明确、查询时间缺失或无法复现。
- 任何需要真实资金执行的建议。

这个流程的重点是每次使用相同的输入格式、相同的输出字段和相同的判断标准，这样才能比较不同时间的结果，并识别是市场变化还是数据异常。

今日产出：完成了一个研究工作流程 v1，明确了输入、工具、输出、判断条件和人工复核边界。

今日结论：可重复的查询不等于自动执行，而先要把观察过程标准化，再用证据和人工复核控制风险。

明日计划：统一记录时间、链、资产、规模、路径、成本、结果和失败原因，形成统一的研究日志模板。

笔记 070: (本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责)

作者：geyu210 | 时间：2026-08-16 12:56 UTC | 字数：1274 | 评分：66

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

(本打卡由 Hermes Agent 辅助整理并提交。内容全部来自我的研究和实践，Agent 仅负责格式排版与提交流程。)

Aqua 学习第 10 天：用 LI.FI 做外部报价参考

今天收到共学组织者分享的 LI.FI API 权益信息，所以尝试把 LI.FI 和前几天研究的 Aqua 结合起来，看看能否用聚合器报价判断 Aqua position 是否存在值得继续验证的价格偏差。

我固定在 Ethereum 主网查询 1 USDC → WETH，连续采集了 5 轮样本。每轮都按照：Aqua 报价 → LI.FI 查询 → 再次读取 Aqua 报价，把请求限制在 0-1 个区块的时间窗口内。这样虽然不能称为严格的同区块比较，但至少能避免拿相隔很久的价格直接相减。

5 轮中，Aqua 报价保持不变，LI.FI 每轮都返回了 6 条同链路由。结果是 Aqua 相对 LI.FI 标记为 CHEAPEST 的路线少约 72.5-78.0 bps，相对只看 token 输出最高的路线少约 95.4-98.7 bps。因此今天没有观察到值得继续验证的正向价格偏差。

一个值得注意的地方是：LI.FI 排在第一的是 Sushi，而 token 输出最高的是 Bitget。Bitget 虽然多给了一点 WETH，但 gas 估算约为 Sushi 的十倍，所以“最高输出”和“综合最便宜”并不是同一个概念。

今天还确认了 LI.FI 的费用结构。每条路线都先经过 feeCollection，其中包含 0.25% 的 LI.FI Fixed Fee；Sushi 路线还额外包含约 0.1% 的协议费。这些费用已经反映在最终 toAmount

中，不能在利润计算时再次扣除。这也说明组织者推动的费率优惠确实有实际价值：如果能降低这 25 bps，会直接改善套利或自动化交易的净输出。

API Key 也测试成功。未认证时，本次响应显示额度约为 75 次/2 小时；认证后提高到 12,000 次/2小时。实际程序不能只相信文档里的默认额度，还应该保存响应头中的剩余额度和重置时间。

另一个结论是，LI.FI 的 JSON 目前不足以判断 1inch 内部是否使用了 Aqua。LI.FI 的工具列表里有 1inch，但没有独立的 Aqua；路线只显示：feeCollection → swap/1inch。所以严谨的说法只能是“Aqua 没有被显式展示，1inch 的下游流动性来源未知”，不能据此判断它使用或没有使用 Aqua。

本地代码也补上了 LI.FI feeCosts 的标准化处理，并避免父 route 与子步骤里的同一费用被重复计算。最终 305 个测试、类型检查和构建全部通过。

今天的收获是：LI.FI 不一定适合直接发现独有的套利机会，但很适合作为外部可执行报价和成本参考。它能快速告诉我，自己发现的报价偏差在聚合器、平台费和 gas 面前是否仍然存在。今天的答案是暂无，但这个比较方法本身已经跑通。

笔记 071: 链上套利残酷共学 | 2026-08-16 打卡

作者：koyo922 | 时间：2026-08-16 14:38 UTC | 字数：497 | 评分：47

链上套利残酷共学 | 2026-08-16 打卡

一、学习记录

今天最大的收获，是实际使用 Matt Pocock skills 管理复杂的软件项目开发。

以复现此前的 sNUSD 脱锚套利事件为目标，在 Codex 中结合相关 skills 和 /goal 模式，进行了接近 24 小时的持续开发。目前已经完成：

- 1. 类似套利事件的监听流程已经跑通；
- 2. 相关套利流程的实盘复现已经跑通。

这次实践让我更直观地感受到，复杂项目如果有明确的目标拆解、开发规范和持续迭代机制，AI 不只是生成代码，还可以参与较长周期的软件项目推进。

Coding

二、阶段性总结

目前项目已经从单一案例复现，推进到：

事件监听 → 机会识别 → 套利执行 → 实盘验证

下一步重点不是重新搭建基础流程，而是继续完善边角细节，并扩充可识别和处理的套利类型。

三、明日计划

- 1. 优化现有监听和执行流程的细节。
- 2. 完善异常处理、日志和风险控制。
- 3. 扩充监听覆盖范围，支持更多类型的套利事件。
- 4. 继续验证不同案例下的稳定性和可复现性。

08 实盘与数据（13 篇）

笔记 072: Polymarket 市场结算机制学习

作者 : Galaxy9 | 时间 : 2026-08-15 15:21 UTC | 字数 : 5845 | 评分 : 90

来源 : <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Polymarket 市场结算机制学习

一、什么是市场结算

当预测事件的结果已经明确，Polymarket 会进入市场结算流程。

结算的作用是确定哪个选项获胜，并处理对应的结果代币：

获胜代币：每枚可兑换 \$1

失败代币：价值归零

特殊的 50/50 结果：每枚结果代币兑换 \$0.50

市场结算后：停止交易，用户只能赎回获胜代币

Polymarket 使用 UMA Optimistic Oracle（乐观预言机）完成去中心化结算。

它的核心逻辑是：

先允许任何人提交结果。如果在挑战期内无人反对，就默认结果正确；如果有人反对，则进入争议处理流程。

二、交易前必须阅读结算规则

每个市场都有预先确定的 Resolution Rules，即结算规则。规则通常包含三部分。

1. 结算来源

规定应当以哪个信息来源为准，例如：

政府或机构的正式公告

选举委员会公布的结果

体育赛事官方网站

指定的数据网站

某个明确主体的公开声明

即使其他媒体已经报道结果，只要规则指定的来源还没有确认，市场可能仍然不能结算。

2. 结束日期

规定市场从什么时候开始具备结算资格。

结束日期不一定意味着市场会在该时刻立即结算。只有事件已经结束，而且满足规则规定的判断条件后，才能提交结果。

3. 边缘情况

规则会说明遇到特殊情况时如何处理，例如：

比赛取消或延期

候选人退选

数据源出现错误

事件在截止时间前未发生

多个选项同时满足条件

结果暂时无法确定

重要原则

市场标题用于概括问题，结算规则才是最终依据。

例如，标题可能写：

“候选人 A 会赢得选举吗？”

但结算规则可能规定：

必须以选举委员会正式认证的结果为准。

此时，即使媒体已经宣布候选人 A 获胜，只要官方认证尚未完成，过早提交结算仍可能损失保证金。

三、结算的四个阶段

1. 提交结算提案

任何人都可以向 UMA Oracle 提交结果，通常需要：

1. 选择获胜选项。

2. 缴纳一笔保证金。

3. 将结算提案提交给 UMA Oracle。

文档中的典型保证金约为 \$750 pUSD，但实际金额应以具体市场和链上界面为准。

如果提案正确且无人提出争议：

市场按照提案结算；

提案人收回保证金；

提案人获得额外奖励。

如果提案错误或提交得太早：

提案人可能损失全部保证金；

争议方可能获得部分保证金。

因此，提交人必须确认：

事件确实已经结束；

已经满足结算时间要求；

采用了规则指定的信息来源；

正确理解了所有边缘情况。

2. 两小时挑战期

提案提交后，会进入约两小时的挑战期。

在此期间，任何人只要认为结果错误、证据不足或提交过早，都可以发起争议。

可能出现两种情况：

无人挑战

提案被接受，市场按照该结果结算。

这是最快的流程，通常在提交提案后约两小时完成。

有人挑战

第一份提案不会直接生效，系统进入下一轮提案流程。

如果第二轮提案仍然受到挑战，争议将升级至 UMA 的 DVM (Data Verification Mechanism，数据验证机制)，由 UMA 代币持有人投票决定。

3. 发起争议

挑战者通常需要缴纳与提案人相同金额的反向保证金，例如约 \$750 pUSD。

争议发生后：

如果是第一轮提案被挑战，将开始第二轮提案；

如果第二轮提案也被挑战，将进入辩论和投票流程。

在约 24 至 48 小时的辩论期内，参与者可以提交证据，例如：

结算规则原文

官方公告

指定数据源的记录

时间戳和历史页面

能够证明提案过早或错误的材料

相关讨论和证据通常可以提交至 UMA Discord 的指定频道，例如：

#evidence-rationale

#voting-discussion

争议并不是“我不喜欢这个结果”，而是要证明该结果不符合市场规则。

4. UMA DVM 投票

如果第二轮提案继续受到挑战，UMA 代币持有人将对正确结果进行投票。

投票过程通常需要约 48 小时。可能产生以下结果：

| 投票结果 | 市场处理 | 保证金处理 |

| :----- | :----- | :----- |

| 提案人胜诉 | 原提案被接受 | 提案人收回保证金，并获得挑战者保证金的一半 |

| 挑战者胜诉 | 原提案被拒绝，需要重新提交 | 挑战者收回保证金，并获得提案人保证金的一半 |

| Too Early | 事件尚未结束，提案过早 | 挑战者收回保证金，并获得提案人保证金的一半 |

| Unknown / 50-50 | 无法适用任何常规结果 | 市场按 50/50 结算，挑战者获得相应保证金奖励 |

以上是文档描述的典型分配方式，具体执行仍应以当时的 UMA 合约规则和市场参数为准。

四、三种结算路径

路径一：无人争议

提交提案

→ 等待两小时挑战期

→ 无人挑战

→ 市场结算

预计用时：约两小时。

路径二：发生一次争议

第一次提案

→ 第一次挑战

→ 第二轮提案

→ 无人继续挑战

→ 市场结算

第二轮提案被接受，不需要进入 UMA 投票。

路径三：发生两次争议

第一次提案

→ 第一次挑战

→ 第二轮提案

→ 第二次挑战

→ 证据辩论

→ UMA DVM 投票

→ 市场结算

预计总用时：约 4 至 6 天。

五、案例一：无人争议的正常结算

假设市场问题为：

“球队 A 是否会赢得 2026 年某赛事决赛？”

规则规定：

以赛事官方网站公布的最终赛果为准；

正常比赛结束后才可以结算；

点球大战结果计入最终赛果。

最终球队 A 通过点球大战获胜，赛事官网也已经确认。

某位用户提交 YES 为获胜结果，并缴纳 \$750 pUSD 保证金。

随后：

1. 提案进入两小时挑战期。
2. 没有人提出异议。
3. 市场按 YES 结算。
4. 提案人收回保证金并获得奖励。
5. 每枚 YES 代币可以兑换 \$1。
6. NO 代币归零。

如果用户持有 100 枚 YES 代币，结算后可赎回：

$$100 \times \$1 = \$100 \text{ pUSD}$$

六、案例二：媒体宣布不代表可以立即结算

假设市场问题为：

“候选人 A 是否会赢得某次选举？”

结算规则规定：

必须以选举管理机构正式认证的结果为准；

新闻媒体预测和候选人自行宣布均不构成最终结果。

计票当晚，多家媒体宣布候选人 A 获胜，但官方尚未完成认证。

某位用户立即提交 YES 结果。

虽然候选人 A 最后可能确实获胜，但这份提案仍可能被判定为 Too Early，原因是提案时尚未满足规则规定的官方认证条件。

如果有人提出挑战并最终胜诉：

提案不会按 YES 立即结算；

原提案人可能损失保证金；

挑战者可能获得部分提案人保证金；

市场需要等待正式认证后重新提交结果。

这个案例说明：

方向判断正确，不代表结算提案正确。结算时间也是规则的一部分。

七、案例三：提案结果错误并被挑战

假设市场问题为：

“比特币是否会在指定日期达到 100,000 美元？”

规则规定：

使用指定交易所；

以该交易所显示的某个价格指标为准；

必须在截止时间前达到或超过 100,000 美元。

实际情况是：

另一个交易所短暂达到 100,100 美元；

规则指定的交易所最高只有 99,950 美元。

提案人根据错误的数据源提交 YES。

挑战者检查规则后发现，指定交易所并未触及目标价格，于是：

1. 缴纳反向保证金；
2. 提交指定交易所的历史数据；
3. 指出提案人使用了错误的数据源；
4. 争议进入后续流程。

最终，如果 UMA 投票认定 NO 才符合规则：

原 YES 提案被否决；

挑战者胜诉；

市场随后按正确结果重新处理；

提案人损失相应保证金。

八、案例四：50/50 特殊结算

假设市场问题的两个常规结果都无法合理适用，例如：

事件被永久取消；

规则没有覆盖这种特殊情况；

也没有足够证据认定 YES 或 NO；

UMA 投票最终选择 Unknown/50-50。

市场将按 50/50 结算：

每枚 YES 代币兑换 \$0.50；

每枚 NO 代币也兑换 \$0.50。

例如，用户持有：

80 枚 YES

20 枚 NO

最终可以赎回：

$$80 \times \$0.50 + 20 \times \$0.50 = \$50$$

50/50 是少见的异常处理结果，不代表争议双方平均分配市场利润，而是每种结果代币按照 \$0.50 赎回。

九、市场结算后如何赎回

结算完成后：

市场停止交易；

获胜代币价值变成 \$1；

失败代币价值变成 \$0；

用户可以通过 CTF 抵押品适配器赎回。

底层流程大致为：

1. 用户提交 ERC-1155 结果代币。
2. CTF 合约销毁这些结果代币。
3. 合约释放对应的 USDC.e 抵押品。
4. 适配器将抵押品包装为 pUSD。
5. pUSD 返回用户钱包。

例如：

100 枚获胜代币 → 100 pUSD

需要区分两件事：

市场结算：系统确定获胜结果。

代币赎回：持有人主动将获胜代币换回 pUSD。

市场已经结算，并不一定意味着资金已经自动回到钱包。

十、规则澄清机制

少数情况下，交易开始后可能出现规则未覆盖的问题。Polymarket 可以发布 Additional context，即补充说明。

补充说明需要满足：

不能改变问题的根本意图；

通过公告板合约发布到链上；

UMA 提案人和投票者应在结算时考虑；

用于解释意外情况，而不是事后改变赌约。

如果认为某个市场需要澄清，可以在 Polymarket Discord 的 #market-review 频道提出。

澄清与修改的区别

合理澄清：

规则指定的网站更换了页面地址，新说明给出该网站的新页面。

不合理修改：

原问题询问某事件是否在 12 月前发生，补充说明却把期限改成次年 1 月。

前者保持了原问题的意图，后者实质上改变了交易条件。

十一、时间线速查

| 阶段 | 典型时间 |

| :----- | :----- |

| 挑战期 | 2 小时 |

| 争议辩论期 | 24–48 小时 |

| UMA 投票 | 约 48 小时 |

| 无争议结算 | 提案后约 2 小时 |

| 有争议结算 | 总计约 4–6 天 |

这些是典型时间，不应被视为所有市场的绝对承诺。实际进度可能受到再次提案、投票安排和合约状态影响。

十二、参与者的检查清单

交易前

阅读完整结算规则，而不只看标题。

确认指定数据源。

理解结束日期和时区。

检查延期、取消等边缘情况。

了解市场是否可能出现 50/50 结算。

评估结算等待期间的资金占用。

提交提案前

确认事件已经真正结束。

检查官方来源是否已经发布结果。

保存带时间戳的证据。

阅读最新的 Additional context。

确认保证金金额和最大损失。

不要仅凭新闻报道或社交媒体提交。

发起争议前

指出提案具体违反了哪条规则。

准备来自指定来源的证据。

确认争议窗口尚未结束。

理解反向保证金可能全部损失。

区分“结果错误”和“结果尚未确定”。

赎回前

确认市场已经完成链上结算。

确认钱包中持有获胜代币。

检查网络和合约地址。

预留必要的链上手续费。

核对收到的是 pUSD 还是其他抵押资产。

十三、核心知识总结

1. Polymarket 使用 UMA Optimistic Oracle 处理市场结算。
2. 任何人都能提案或挑战，但必须缴纳保证金。
3. 市场标题只是摘要，Resolution Rules 才是判断依据。
4. 提案后通常有两小时挑战期。
5. 无人挑战时，市场最快约两小时完成结算。
6. 两轮提案都被挑战时，争议升级至 UMA DVM 投票。
7. 错误提案和过早提案都可能导致保证金损失。
8. 获胜代币每枚兑换 \$1，失败代币归零。
9. 特殊情况下，市场可能按 50/50 结算。
10. 结算后仍需通过 CTF 抵押品适配器赎回代币。
11. Additional context 可以澄清意外情况，但不能改变问题的根本意图。
12. 所有金额、时间和分配规则都应以具体市场及当时链上参数为准。

这套机制的实质是：系统先乐观地接受一个有经济担保的答案，再通过挑战、证据和投票处理错误或模糊结果。

笔记 073: 笔记 1: 系统性偏离监控与预期定价模型\

作者：ingram0529 | 时间：2026-08-15 15:32 UTC | 字数：3663 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

笔记 1: 系统性偏离监控与预期定价模型\

作者：Wave | 日期：2026-08-13 打卡 | 1415 字 · 评分 86.8 · 做市与LP\

一句话核心：合约价格相对美股的巨大偏离不一定是套利机会——做市商可能早已把今晚美股的走势提前定价进合约了。\
关键机制与数据\

监控体系：6 标的 (SNDK / SNXX / MU / MUU / EWY / KORU) 永续 vs 美股收盘基准，归一化偏离 (bps) 分三档：异常区\
\>150bps / 信号区 >60bps / 正常。六平台聚合 (Gate / Phemex / OKX / MEXC / Binance / Bybit) + BSC 链上。\
8/12 关键事件：美股暴涨前全天六标的巨额正偏离 (SNXX +1731bps、KORU +1444bps 创历史新高)，看似天赐套利机会，\
实为预期定价。\
决定性验证：8/12 21:30 美股开盘后 SNDK +8.5%、MU +6.4%、EWY +4.9%；杠杆标的 SNXX +17.0% (2.0x)、MUU +12.9%\
(2.0x)、KORU +14.3% (2.9x)，合约对美股实时价偏离全部收窄至 $\pm 35\text{bps}$ 以内——偏离可被杠杆倍数精确还原。\
该预期定价模型连续 9 次验证成功。\
识别伪错价三问框架\
杠杆倍数能否还原偏离？能 $\rightarrow$ 大概率是定价逻辑而非错价。\
跨平台 spread 是否极小 (<20bps)？是 $\rightarrow$ 无套利空间。\
美股开盘后是否收敛？是 $\rightarrow$ 时区滞后确认。\
真正值得警惕的异常：偏离方向与预期相反、跨平台 spread 拉大、基准更新后不收敛。\
风险清单\
代理宕机是最大瓶颈：连续 9 次执行仅 2 平台可用 (Gate/Phemex)，单平台标的无法验证跨平台 spread，需要 VPS\
或更稳网络。\
基准切换会污染 Δavg ：必须拆解基准变化 vs 平台价变化，用固定基准对比才干净。\
Yahoo WebFetch 是可靠的美股实时价来源：开盘后对比合约 vs 实时价，是验证预期定价的关键。\
工程启示\
做市/套利监控系统里，偏离告警必须区分定价逻辑 vs 真实错价，否则会在美股盘前被假信号刷屏。把杠杆可还原 + spread\
极小做成前置过滤器，再谈套利。\
对 Binance 永续做市同样适用：警惕资金费率/基差偏离是预期定价还是真错价，避免在假信号上白白烧手续费。\
贰 · 机制数学层 Aqua position 报价差的数学实证\
笔记 2: Aqua position 报价差的数学实证\
作者：geyu210 | 日期：2026-08-14 打卡 | 1900 字 · 评分 65.6 · 做市与LP\
一句话核心：同一条 1inch Aqua position，输入从 0.01 USDC 加到 1 USDC 报价差扩大，不是隐藏算法，而是 virtual\
reserves + 整数取整的必然结果——本地模型复算四档输入逐 raw unit 相等，残差 0。\
关键机制\
— 4 —\
执行顺序：protocol fee $\rightarrow$ XYC concentrate $\rightarrow$ flat fee $\rightarrow$ XYC swap。\
XYCConcentrate 不直接用真实余额做常数乘积：先按真实余额 + 策略价格上下界算出流动性参数 L，再构造一组 virtual\
reserves (virtualIn / virtualOut)，最后才走常数乘积： $\text{amountOut} = \text{floor}(\text{effectiveAmountIn} \times \text{virtualOut} / (\text{virtualIn} + \text{effectiveAmountIn}))$ 。\
费用取整不可合并：protocol fee 向下取整、flat fee 向上取整、XYC 输出再向下取整——顺序错了或浮点近似，\
小额输入就出误差。\
纯数量影响公式： $\text{sizeImpactBps} = -\text{effectiveAmountIn} / (\text{virtualIn} + \text{effectiveAmountIn}) \times 10,000$ 。输入越大，\
单位输出按公式下降。\
认知纠偏：真正进入定价公式的是输入相对 virtual reserve 的比例，不是相对钱包余额或 Allocation 的比例。virtual reserves\
也不是锁在合约里的另一份钱，而是每次策略运行时按当时真实余额 + 固定价格边界重新计算的有效储备。\
作者自划边界\
残差 0 只证明理解了报价数学，不证明真实 swap 能执行——静态 quote 不真正执行 fee pull，真实成交仍受 access\
条件、maker 状态、gas、MEV、状态竞争影响。\
结论仅适用于 SwapVM v1.0.2 策略，不能推广到所有 Aqua App。\
风险清单\
小额输入的取整误差会让看似套利的报价失真。\
只验证 quote 不验证真实成交，会被 MEV / 状态竞争打脸。\
结论的适用范围必须锁死版本。\
工程启示\
任何 LP 报价系统的金额敏感性都要先归因到数学机制 (储备结构 + 取整顺序)，再怀疑隐藏逻辑。\
做 Polymarket LP / 纸盘撮合引擎时同一纪律：先复算官方公式到残差 0，再谈策略；quote 与 execute 是两回事，\
纸盘撮合引擎要模拟 MEV / 滑点真实路径，不能只对齐报价。\
— 157 —

叁 · 策略执行层 订单流做市标的筛选与低延迟执行基准\

笔记 3: 订单流做市标的筛选与低延迟执行基准\

作者: Zhihui Zhang | 日期: 2026-08-13 打卡 | 581 字 · 评分 61.6 · 做市与LP\

一句话核心: 做市的 alpha 不在策略, 而在选品——用订单流指标筛出值得做市的标的, 再用 <5ms\ 的下单延迟把执行优势坐实。 \

关键内容\

7 个订单流指标: CVD (累积成交量差值)、VPIN (知情交易概率)、Funding Rate、Open Interest、Long/Short\ Ratio、爆仓数据、价格趋势。 \

CVD 背离分数: Divergence Score = CVD变化率 / 价格变化率——价涨但 CVD 不涨 (或反向) 就是背离信号。 \

四步筛选流程: 初筛 → 信号验证 → 风险评估 → 执行决策。 \

两个实战案例: \$ALGO (CVD 背离做多信号)、\$EDGE (阴谋币做空信号)。 \

核心理念: Market selection is the alpha, not the strategy。 \

执行侧: 东京 ap-northeast-1 开 c6id.large Spot 实例, 用 /fapi/v1/order/test (不下真单) 测下单 RTT, P95 = 3.57ms < 5ms\ 目标; bench-all-azs.sh 一键遍历 3 个 AZ。 \

— 5 —\

风险清单\

篇幅短信息密度高但案例细节少, 背离信号缺少胜率/回撤数据支撑。 \

VPIN / CVD 在低流动性币上噪音大。 \

延迟测试用 order/test 不含真实撮合排队, 实盘 P95 会更高。 \

工程启示\

选品先行: 不是每个币都值得做市, 先用 CVD / VPIN / 资金费率筛出订单流有故事的标的, 再上做市策略。 \

做市机器人目前按固定标的跑, 可考虑用这套指标做动态选品层; 延迟基准方法 (order/test + 多 AZ 遍历) 可直接复用到\ Binance 做市部署的延迟验收。 \

结 · 三篇串联 做市的三个层次\

笔记 4: 做市的三个层次\

作者: 综合整理 | 日期: 2026-08-15 | 来源: 残酷共学 2026-08-14 抓取批次 (BJT 8-14 23:02)\

一句话核心: 先看懂价格为什么这么走, 再算清楚报价为什么这么给, 最后决定在哪些标的上用多快的腿。 \

三篇主线对照\

定价行为层 (Wave): 偏离 ≠ 错价, 先问杠杆能否还原、spread 是否极小、开盘是否收敛。 \

机制数学层 (geyu210): 报价差的根在 virtual reserves + 取整顺序, 复算到残差 0 才算懂。 \

策略执行层 (Zhihui Zhang): 选品是 alpha, 订单流指标筛标的 + 延迟 P95<5ms 坐实执行优势。 \

贯穿一句话: 做市的三个层次——预期定价 (价格为什么这么走)、virtual reserves 数学 (报价为什么这么给)、订单流选品 + \ 低延迟 (哪些标的、多快的腿)——三篇笔记各补一层。

笔记 074: markdown

作者: adurey | 时间: 2026-08-15 15:58 UTC | 字数: 1097 | 评分: 83

来源: <https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

markdown

Day-11 | 2026-08-15 | 重复查询与信号字段

阶段: 阶段三: 用 Hermes 构建研究与信号 workflow

今日问题: 一次性观察怎样变成可重复查询、告警或 Agent workflow?

使用模板: daily-note.md (../templates/daily-note.md)

今日目标

选择一个机会字段, 设计重复查询的输入、阈值和输出格式。

学习记录

- 完成内容:
- 选定机会字段「跨链价差」 (cross-chain price spread): 同一资产在两个链上的 CoinGecko priceUSD 之差。
- 编写重复查询脚本 lifi_price_spread_watch.py (../experiments/lifi_price_spread_watch.py), 固定输入、阈值和输出格式, 可配合 cron 反复运行。
- 用真实只读数据验证两种情形: 同资产跨链比较、以及 USDC vs USDC.e 的符号不匹配告警。
- 证据链接:
- 重复查询脚本 (../experiments/lifi_price_spread_watch.py)
- 脚本的 --json-output 输出即结构化机会记录, 见下方「实际观察」。

- 初始假设：把「查一次价差」升级为「带阈值的重复查询」，能让价差机会从手动观察变成可告警的信号。
- 实际观察：
 - ETH 在 Base (8453) 与 Arbitrum (42161) 实测 priceUSD 差仅 0.012%，远低于 0.3% 阈值，未触发；说明同资产 paper spread 常态下极小。
 - USDC (原生 0xaf88...) 与 USDC.e (0xFF97...) 被脚本以 symbol mismatch 拒绝计算价差，避免把不同资产误当成同一资产比较。
- 关键结论：
 - 机会字段的「输入」应含：链 id、token 地址、阈值、采样时间戳；「输出」必须机器可读 (JSON) 才便于长期漂移分析。
 - 阈值必须保守：priceUSD 是 CoinGecko 估值而非可成交价，触发后仍需用 /v1/quote 验证真实可成交价差。
 - 若价差低于桥费 + Gas + 滑点的成本底价，即使触发也无可执行价值；阈值应把成本底价设进去。
 - 明日行动：day-12 运行一次最小研究工作流，把该脚本接成定时/重复运行，并记录人工复核点。

笔记 075: Day 12 · 2026-08-16 · 回测验证：资金费率收租策略的真实期望

作者：twone | 时间：2026-08-15 16:24 UTC | 字数：1980 | 评分：89
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 · 2026-08-16 · 回测验证：资金费率收租策略的真实期望

今日目标

1. 建一个最小回测框架：公开数据 → 策略规则 → 扣除成本 → 净期望
2. 用真实历史数据验证昨天调研的"资金费率收租"策略
3. 用数据校准昨天的判断，而不是停留在定性层面

一、回测框架与数据

脚本：scripts/funding_rate_backtest.py

数据源：Hyperliquid fundingHistory 公开接口（无需 Key，小时级费率）。本环境 Binance/OKX/Bybit 接口均不可达，故改用 Perp DEX 数据——结论方向可迁移，但注意 CEX 多为 8h 结算，口径不同

回测窗口：最近 250 天（2025-12-08 ~ 2026-08-15，6000 个小时级费率点/币种）

策略规则（最朴素的版本）：全程持有"现货多 + 永续空"，每期费率照收照付

成本：四笔吃单手续费合计 0.2%，持有期无其他成本

二、回测结果（真实数据）

| 币种 | 正费率期占比 | 符号翻转 | 毛年化 | 净年化（扣手续费） |

| :-- | :---- | :----- | :--- | :----- |

| BTC | 76.1% | 75.4 次/30天 | 4.5% | 4.2% |

| ETH | 79.7% | 71.5 次/30天 | 5.3% | 5.0% |

| SOL | 58.5% | 89.2 次/30天 | -0.9% | -1.2% |

| DOGE | 71.4% | 68.4 次/30天 | 4.1% | 3.8% |

三、四个结论（用数据校准昨天的定性判断）

1. 常态收益远低于宣传数字。昨天资料里"年化 43.8%"是行情好时的费率外推；250 天全周期实测，最好的 ETH 也只有 5.0% 净年化。这个收益甚至跑不赢很多无风险替代品——"躺平收租"版本的策略不成立
2. 币种选择就是一切。同一时间窗口，ETH +5.0% vs SOL -1.2%，差 6 个点。策略的真实形态不是"开一个然后忘掉"，而是持续监控费率、只在费率显著为正且稳定时入场——这就从被动策略变成了带监控的主动策略，正好接上 Day 17-18 监控脚本的规划
3. 费率符号翻转极其频繁（70~90 次/30天，小时级）。昨天的风险清单里"费率反转"不是小概率尾部事件，而是日常噪声。这解释了为什么工具（Taoli/QQQ.BOT）都要配实时监控——人工盯不住这个频率
4. 窗口内没有出现极端费率（|rate| > 0.05%/h 的期数占比 0.0%）。极端行情（如 LOOM 那种 3%/8h）在 250 天里一次没发生——高费率机会是事件驱动的，不是常态，"平时小额监控 + 事件出现重仓"比"常年满仓"更合理

四、回测的已知局限（诚实清单）

只有费率数据，没有两腿的实际成交滑点和借币/资金成本；现货腿的资金占用机会成本未计（若用 U 买现货，放弃了 U 本身的理财收益）

"全程持有"是最粗糙的规则，没有择时、没有换仓逻辑——真实策略的收益分布会不同

单数据源（Hyperliquid），费率结构和 CEX 有差异；CEX 接口恢复后应交叉验证

250 天是单边市场窗口，结论可能不适用其他市场 regime

五、框架本身的价值

这次回测最重要的产出不是结论，而是验证了“假设 → 公开数据 → 扣成本 → 净期望”这条流水线能跑通：零成本（公开接口）、一小时（写脚本）、可复现（脚本在仓库里）。以后任何策略假设——跨所价差、链上价差、清算——都先过这条流水线，再决定要不要投入实盘验证。这就是 Day 1 说的“用真实证据逼近答案”。

明日衔接（Day 13）

Paper

Trading

设计：基于今天的结论，设计“费率择时版”策略的信号规则（入场阈值、退出条件、记录格式、评估标准），开始纸面跟踪。

参考资料

脚本：scripts/funding_rate_backtest.py，数据：data/funding_backtest_2026-08-16.csv

Hyperliquid API 文档 (<https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/api>)

笔记 076: 2026-08-16

作者：Sansho2 | 时间：2026-08-15 17:09 UTC | 字数：783 | 评分：57

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-16

1. 今天学/做了什么

Day12：Agent 流程 + cost_model.py 联调。选 5 个既有线索跑纸面成本，产出结构化判决 → notes/day12-five-reviews.md（成本 JSON 在 notes/day12-cost-runs/）。

2. 得到什么结果

| ID | 线索 | 判决 |

|---|---|---|

| C1 | 跨链 ~1.3bps 搬砖 | 否决（\$1k net~-42bps） |

| C2 | Base 路径差 ~38bps | 通过作执行尺子（误读双边则否决） |

| C3 | DefiLlama ~75bps | 观察（先 Quote；纸面假设净正不可当实盘） |

| C4 | 跨链屏幕 ~15bps | 否决（仍 < 地板） |

| C5 | 稳定币 <8bps | 否决（噪声） |

- 分布：3 否决 / 1 观察 / 1 通过 → 清单+计算器能快速证伪。

- 关键纠错：脚本「候选」标签不能覆盖语义门门（C2/C3）。

3. 失败或不确定点

未对 C3 当场重拉 LI.FI（观察项留给补证）；Funding 未进本轮五候选（避免公开阈值，且结构审已有框架）。

4. 对净收益模型的更新

联调顺序固定为：Agent 语义门门 → cost_model 补净利 → 再判决；禁止先看净利再找故事。

5. 明天要验证的最小问题

Day13：最小 Paper 框架（假仓位 / 假设成交 / 滑点敏感度），优先接 C2 执行对照或 C3 Quote 后分支。

笔记 077: 量化交易系统进阶必备：Python 语言底层四大核心思想 昨天群友问我：已经会用 Pandas、N

作者：jssyxd | 时间：2026-08-15 17:52 UTC | 字数：886 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

量化交易系统进阶必备：Python 语言底层四大核心思想 昨天群友问我：已经会用 Pandas、NumPy 写策略了，但想进一步写出真正高性能、可维护的量化交易框架、风控系统或复杂回测引擎，该从哪里提升 Python 能力？我直接掏出我珍藏很久的 James Powell 讲师在 PyData 大会上的经典演讲《So you want to be a Python expert?》我问他，想提升到这个层级，Python 语言层面最该先掌握哪些核心思想？他给出的内容非常硬核且实用：[图片] 数据模型与协议驱动：通过魔术方法（len、add 等）让自定义类具备原生数学运算能力 [图片] 元类与类构建钩子：如何用 Metaclass 或 init\subclass 强制约束派生类行为 [图片] 装饰器与动态包装：从手动计时到高阶带参装饰器的完整演进 [图片] 生成器 + 上下文管理器：现场手写类似 contextlib 的底层实现，优雅处理 setup/teardown 更有意思的是，主讲人全程用 dis 反编译字节码 + 现场手写代码，把抽象概念讲得异常清晰。不过这个视频不讲量化策略，而是聚焦语言底层，适合已经熟练掌握 Pandas/NumPy、想提升框架设计能力的进阶开发者。 其适合想阅读 Zipline、Backtrader、Qlib 等开源量化库底层，或自己搭建大型交易系统的人，会很有启发。 建议直接收藏，认真看完会对写复杂 Python 量化系统有很大帮助。

笔记 078: Day 12 | 从 LIVE 监控到可审计执行：报价、状态与失败成本的闭环

作者：ChineseCatMan | 时间：2026-08-16 05:02 UTC | 字数：1599 | 评分：76

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 | 从 LIVE 监控到可审计执行：报价、状态与失败成本的闭环

学习内容

今天沿着前几天的进度，继续把 LINK 跨链快桥策略从“能发现信号”推进到“能审计每一次决策”。Day 8 的双重阈值（跨链价差 > 1.5%、净收益 > \$5/1000 USDC）解决了机会筛选问题，但还必须回答三个执行问题：报价在什么时间点有效、目标链状态是否已经确认、失败时损失如何进入

Cost

Ledger。我的理解进一步明确：套利系统的核心不是发出更多交易，而是让每个“不交易”和“退出”都有可解释的证据。

技术细节

第一，继续坚持“报价 $\neq$ 成交”的四层证据链：Displayed Price $\rightarrow$ Quoted Price $\rightarrow$ Executable Price $\rightarrow$ Realized Price。监控记录至少要保存 quote 时间戳、from/to

链、资产数量、桥路由、预估到账时间、feeCosts、池子状态，以及实际确认时间；不能只保存一个最终价格。

第二，把净收益计算固定为单一真源：

净收益 = 价差收益 - 手续费 - Gas - 桥费 - 滑点/价格冲击 - 延迟成本 - 失败成本 - 资金占用成本

其中延迟成本仍用 $\sigma_{\text{price}} \times \sqrt{t_{\text{bridge}}} \times \text{size}$ 估算；AMM 侧用 $x \cdot y = k$ 和 $PI = \Delta x / (x + \Delta x)$ 估算冲击，并区分价格冲击与外部滑点。特别检查了防重复扣费纪律：聚合器的 toAmount 已包含相应执行成本时，feeCosts 只能用于归因和审计，不能再从结果中扣一次。

第三，设计了“状态机 + 幂等键”的执行思路：QUOTE_RECEIVED $\rightarrow$ SOURCE_SUBMITTED $\rightarrow$ SOURCE_CONFIRMED $\rightarrow$ RELAY_PENDING $\rightarrow$ DEST_CONFIRMED $\rightarrow$ HEDGE_CLOSED。每次状态变更都记录 tx hash、区块号和时间；若源链确认或目标链到账超时，则进入异常分支，而不是继续使用过期报价。跨链非原子执行中，单腿失败必须程序化平仓，Kill Switch 触发后降级为 OBSERVE_ONLY。

困难与风控

今天最大的困难是如何定义“成功”。桥 API 返回成功不等于资金已在目标链可用，子图有 10–30 秒索引延迟，快桥中继器费率也可能在极端行情中变化。因此监控告警不能只依据报价，而要把链上确认和到账可用性作为硬条件。另一个困难是失败成本难以从历史样本准确估计：一次回滚 Gas、一次单边暴露，可能吃掉多次小额盈利，所以不能用平均收益掩盖尾部风险。

风控共识保持不变：套利钱包与持仓钱包隔离；精确授权，不开放提币；AI 不碰私钥、不签名、不转资金；实盘资金不超过总资金 5%；连续异常或当日亏损达到阈值就自动停止。执行前还要确认余额、Gas、桥流动性和目标链 gas token，任何一项不足都只记录不下单。

思考

Day 7 的 180 天结果显示 LINK 快桥夏普从早期 0.73 下降到后期 0.33，说明公开策略有半衰期。今天更深的结论是：策略衰减时，最需要优化的不是把阈值调得更激进，而是提升数据质量、状态确认和退出纪律。个人很难在毫秒级速度上战胜 MEV bot，但可以依靠信息优势和成本优势，专注低频、可解释、能被完整复盘的机会。

下一步继续跑 Paper $\rightarrow$ LIVE 监控联调，重点比较预测成交价与真实成交价的偏差，并统计报价过期、桥延迟、单腿失败和 Kill Switch 触发率。在没有连续 7 天实时净收益为正、Paper 阶段零系统故障之前，不进入小额实盘。

—— 区块链天才 ChineseCatMan 的 Hermes 赫妹 Agent 代打卡

笔记 079: Day 12 — 1 小时采集数据复盘：跨平台价差发掘 + 套利经济学

作者：ChenXuan1985 | 时间：2026-08-16 13:29 UTC | 字数：3290 | 评分：88

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12 — 1 小时采集数据复盘：跨平台价差发掘 + 套利经济学

残酷共学 · 链上套利专题

日期：2026-08-16

阶段：60 分钟全量采集 + 价差/滑点/资金费率三维分析

一、今日里程碑

60 分钟全量采集首次完整跑完（3600 个窗口，约 9 万次请求），数据质量：

valid cell 覆盖率: 99.2%（12 个 venue-symbol 全部 $\geq 99\%$ ）

valid hard batch: 98.8%（远超 90% 门槛）

timestamp 覆盖: 99.09%

timeout 率: 0.87%（778/89325）

之前两次都在 36% 处中断，这次跑满 100%。通道本身已经稳定打通。

二、跨平台价差发掘（核心成果）

把四平台 BBO 对齐后，算各标的的跨平台 mid 价差（bps）：

标的	Binance↔OKX	Binance↔HL	OKX↔HL
CRWD	6.0	51.9	46.9
AAPL	0.65	14.4	14.6
NVDA	2.7	8.0	5.8
LRCX	5.7	2.7	5.3

三个关键发现

1. CEX↔CEX 价差已经死了

Binance 和 OKX 之间价差只有 0.65-6 bps，做市商早就抹平了——传统的 CEX 之间搬砖没有空间。

2. Hyperliquid 是定价效率洼地

CEX↔HL 价差是 CEX↔CEX 的 5-20 倍。代币化股票 perp 在 HL 上的定价系统性偏离 CEX，且方向稳定：

CRWD: HL 比 CEX 高 +52 bps（买 CEX 卖 HL）

AAPL: HL 比 CEX 低 -14 bps（买 HL 卖 CEX）

3. 价差是结构性的，不是瞬时噪声

CRWD 价差 std 只有 4.6 bps（中位数 51.9），AAPL std 只有 0.51。价差在整整 1 小时里持续存在，不是偶发尖峰。这是“流动性碎片化 → 结构性基差”套利理论的前提，被真实数据证实了。

三、实际可成交滑点（L2 深度逐档计算）

只看 BBO 顶格价是“看得见吃不着”，必须用深度算真实滑点。

CRWD（买 CEX / 卖 HL）

下单量	毛利 (bps)	净利 (扣 20bps 四腿费)
0.1 手	44.95	+24.95
0.5 手	28.22	+8.22
1 手	26.11	+6.11
2 手	32.35	+12.35

CRWD 在 0.1-2 手区间，扣完所有 taker 费仍有正收益。

AAPL（买 HL / 卖 CEX）

下单量	毛利	净利
0.1-5 手	~12 bps	8（亏损）

AAPL 的 12bps 毛利扣费后是负的——必须用 maker 挂单（~2-5bps 费）才可行。

四、资金费率成本（实时数据）

标的	Binance	OKX	HL	HL 日成交量
CRWD	0.0000%/8h	0.0000%/8h	+0.0071%/8h	2842（活跃）
AAPL	—	—	+0.0006%/8h	128万（活跃）
NVDA	—	—	+0.0006%/8h	280万（活跃）
LRCX	—	—	0.0089%/8h	0.0（死亡）

五、CRWD 完整套利经济学（今日最大收获）

方向：买 Binance CRWD（低 216.5） / 卖 HL CRWD（高 217.7）

一次往返损益：

毛利（滑点后）：+26~45 bps

四腿 taker 费：20 bps

净利：+6~25 bps

持仓资金费 carry：

Binance 多头：付 0.0000%/8h（零成本）

HL 空头：收 +0.0071%/8h

净 carry: +7.8% APR

CRWD 是"价差 + carry"双收益：

入场捕捉 6-25 bps 净价差

持仓期间 HL 空头收 7.8% APR 资金费

价差收敛时再吃一轮

LRCX 从标的池剔除：HL 日成交 0.0，负 funding 9.8%，这个市场已经死了。

六、工程教训

6.1 "看得见的价差" ≠ "吃得着的价差"

BBO 顶格价差 52bps 很诱人，但 HL 顶格 bid 只有 0.09 手——真实可成交价必须逐档吃深度。这就是为什么必须做滑点分析，不能只算 mid-to-mid。

6.2 方向性必须显式核对

我第一版分析把 AAPL 方向算反了（AAPL 是 HL 更低，应该买 HL 卖 CEX，而不是反过来）。方向错了，52bps 的"机会"会变成 15bps 的"亏损"。套利分析里方向性价差的正负号是致命的。

6.3 资金费可以是收益而非成本

直觉上"跨期持有有成本"，但实测 CRWD 的空头腿反而收 7.8% APR。资金费的方向取决于各平台多空比，不能想当然。

6.4 数据质量不等于策略可用

99.2% 的采集覆盖率很好，但决定策略能不能赚钱的是：

价差大小（够不够覆盖费用）

深度（能吃多少量）

资金费方向（是成本还是收益）

七、下一步方向

1. CRWD 深度上限：本次只采了 CEX 5 档 / HL 20 档，>2 手的容量未知，需补采更深 L2
2. 价差收敛回测：用 3600 个窗口的价差时序，看 CRWD 价差的均值回复特征（是持续偏离还是周期性收敛）
3. Binance 做空约束：确认可借券量、保证金要求
4. maker 策略：AAPL/NVDA 走 maker 挂单路线（~2-5bps 费）能否把 12bps 毛利变成正收益

笔记生成时间：2026-08-16 CST

笔记 080: 今天研究套利系统中最容易被忽略的一层：执行概率与概率加权收益。监控看到“扣费后利润为正”，并不代表这

作者：dyhbrewer | 时间：2026-08-16 13:45 UTC | 字数：1526 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天研究套利系统中最容易被忽略的一层：执行概率与概率加权收益。监控看到“扣费后利润为正”，并不代表这次机会值得出手；如果报价已经陈旧、成交概率很低，或失败会留下单边敞口，长期结果仍可能亏损。

一、从静态净利润升级为期望收益

常见雷达只计算：

静态净利润 = 卖出收入 - 买入成本 - 手续费 - Gas - 滑点。

但真实决策应该计算：

Expected Value

\= 成功概率 × 成功净利润

失败概率 × 失败损失

资金占用与机会成本。

例如，某机会成功可净赚 10 USDC，成功概率只有 30%，失败时平均损失 5 USDC，则：

$EV = 0.3 \times 10 - 0.7 \times 5 = -0.5 \text{ USDC}$ 。

即使屏幕显示“预计赚 10 U”，它也不是正期望机会；还没计入资金占用后，策略已经不值得执行。

二、影响成功概率的六个关键变量

1. Quote Age：报价从生成到准备执行经过多少毫秒、多少区块。越旧越不能代表当前状态。
2. Spread Half-life：价差衰减一半所需时间。若桥接或下单耗时大于半衰期，机会很可能在成交前消失。

3. Fill Probability：按目标金额、真实深度、成交速度和历史数据估算能否完整成交。

4. Queue Position：CEX 限价单挂到最优价不等于马上成交，还要考虑前方排队数量、撤单率和主动成交流量。

5. Adverse

Selection：订单一旦成交，可能正是价格继续向不利方向移动。被动成交不一定是好事，可能说明自己接到了“毒性流量”。

6. Competition：同一机会上的机器人数量、链上 Gas 竞价、私有交易通道和区块排序都会改变成功率与成本。

三、失败成本不能只算 Gas

跨链或 CEX/DEX 套利最危险的是非原子执行。一端成交、另一端失败时，会形成单边持仓。失败损失应包括：已支付费用、失败 Gas、撤单费、反向平仓滑点、持仓价格波动、跨链资金冻结时间以及重新平衡库存的成本。

因此需要分别记录三种结果：

fully_filled：两端均按计划完成；

partial_fill：部分成交，需要缩量或补单；

leg_risk：单边成交，必须立即对冲或退出。

四、雷达应该新增的字段

每条机会除显示静态净利润外，还应显示：报价年龄、价差半衰期、预计成交概率、订单簿排队量、失败损失、资金占用时间、概率加权收益和建议动作。

提醒条件升级为：

压力测试后的净利润为正；

成交概率达到最低阈值；

Expected Value 为正且超过安全边际；

连续确认期间价差没有快速衰减；

单边失败时有明确的退出路径和最大损失上限。

五、如何用 Paper Trading 校准

不能凭主观判断填写成功率。系统每次发现机会都保存快照，然后模拟在 100 ms、500 ms、1 秒、5 秒后执行，记录价差是否在、目标规模能否成交、实际滑点以及最终结果。按链、资产、交易场所、时段和机会类型统计真实命中率，再用滚动窗口更新成功概率。

重点指标包括：信号数量、可执行率、完整成交率、平均存活时间、成功净利润、失败平均损失、EV、最大连续失败次数和净利润分布。只有在足够样本下 EV 持续为正，才考虑极小规模验证。

今天的结论：发现价差解决的是“哪里可能有机会”，概率加权收益解决的是“是否值得行动”。真正能长期赚钱的系统，不应该按最大价差排序，而应按压力测试后的 Expected Value 排序。下一步先把这些字段加入只读雷达与 Paper Trading，不连接钱包、不签名、不自动交易。

笔记 081: 笔记 1: 网格交易：简化版做市策略的系统拆解\

作者：ingram0529 | 时间：2026-08-16 13:51 UTC | 字数：3734 | 评分：80

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

笔记 1: 网格交易：简化版做市策略的系统拆解\

作者：hellingercheri-gif | 日期：2026-08-15 | 3610 字 · 评分 89.9 · 实盘与数据\

一句话核心：网格交易是把“区间震荡中的低买高卖”程序化，本质是简化版做市——用放弃单边趋势收益，换取区间内多次小额利润；真正决定结果的不是网格数，而是区间判断、交易成本、仓位管理和退出纪律。\\

关键机制\\

总盈亏公式：总盈亏 = 已实现网格利润 + 持仓浮动盈亏 - 手续费 - 滑点 - 资金费/利息 - 其他成本。最重要的约束：每格价差\\> 双边手续费 + 滑点 + 其他成本，否则交易越频繁亏越多（网格利润 +1000、持仓浮亏 4000、总盈亏 3000\\的典型陷阱）。\\

两种网格划分：等差（绝对差相同，适合窄区间，越靠近低价区每格百分比收益越高）；等比（百分比一致，倍率 = \\(上限 ÷ 下限)^{1/网格数}，适合大跨度、高波动资产）。\\

核心参数：标的（决定流动性/波动结构）、上下限（太窄易出界，太宽资金效率低）、网格数（越密单格利润越薄）、等差\\ / 等比、止损止盈、杠杆。\\

现货网格并非中性：越跌持有越多、越涨现金越多——跌破下限持有大量下跌资产，涨破上限错过后续上涨。\\

网格赚的是“来回波动”，不是“波动越大越好”：波动大到突破区间形成趋势，网格反而受损。\\

回测陷阱：事后选区间、假设完美成交、忽略手续费/滑点/资金费、参数过拟合。\\

与做市的边界：网格是其简化形式，专业做市还会动态管理库存、价差、波动率和订单流。\\

风险清单\\

单边行情出界无处理；网格过密成本吞噬；回测失真；无止损长期套牢；合约网格的爆仓/资金费/极端行情止损偏离。\
对工程实践的启示

"每格价差

\>

双边成本"就是做市价差定价的核心不等式；"已实现

+\
PnL

未实现分开看、手续费占比"的评价框架可直接用于做市机器人的 \
分解；"现货网格越跌越买"的库存曲线对应做市商的方向性库存暴露，正是 inventory_quote_mode 要解决的事。

贰·机制经济学层 Inverse Finance：DBR 拍卖做市与稳定币机制实测\
笔记 2: Inverse Finance：DBR 拍卖做市与稳定币机制实测\
作者：darven | 日期：2026-08-15 | 2165 字 · 评分 79.8 · 做市与LP\
一句话核心：DBR 拍卖不是荷兰拍而是 $xy=K$ 做市商（供给注入式通缩），DOLA 锚定靠"空 PSM + 休眠 Feds + 无 peg-keeper"的市场自愿行为——机制图上存在的套利通道没有火力，真实 APY 取决于债务/存款比而非利率。\
关键机制（链上实测，非文档转述）\
DBR 拍卖实为 $xy=K$ 做市：virtual_dbr_reserve 以 41M/yr 递增、virtual_dola_reserve 固定，价格由 $xy=K$ \
决定而非随时间衰减——不是荷兰拍线性压价，而是稀释通缩。链上 dbrPrice=4.07 美分 vs 市场 4.04 美分，基本一致。\
Docs 地址表会骗人：按 docs 给的合约地址 eth_call 全回 0，换 inverse-web 前端 constants.ts\
才读到真实参数（minDebt=\$5、maxCollateralValue=\$100、4 个抵押市场全 active）——docs 把已废弃 V1 和活跃 V2\
地址混在一起。\
— 4 —\
DBR 需求刚性：借 1 DOLA 需 1\
DBR，需求量由债务规模刚性决定、与市价无关；增发只压低市场存量价格，不改变借款人成本。\
PSM 有通道没有火力：buyFeeBps=0（USDS→DOLA）、sellFeeBps=20（DOLA→USDS），但\
getTotalReserves()=\$0、supply=0——理论地板存在实际为空，这正是 DOLA 脱锚后无硬底承接的原因。\
Feds 休眠：expansion/contraction 均 onlyChair 需治理提案触发，BorrowingFed\
totalDebt=0、claims=0——"主动调节各链供应"当前没有在执行。\
真实 APY 看分母：FIRM 总债务 \$48.7M vs sDOLA TVL \$8.03M = 6.1:1；对比 Liquity V2 sBOLD 的\
5%（存款池与债务规模相近）。验证 APY 关键看谁在付钱、谁在拿钱的比例。\
90 天数据：DOLA/sUSDe 主池 28 天低于 \$0.98，最深 \$0.9634（6/7）；无 redemption、无 peg-keeper，纯 soft peg。\
风险清单\
机制存在 $\neq$ 实际有火力（空 PSM）；文档地址过期导致链上读参失败；脱锚无兜底；治理依赖（Feds 需提案）；APY\
口径误导（利息总额 $\neq$ 真实收益）。\
对工程实践的启示\
"docs 说 A、代码实际是 B"的三源验证（官方文档 vs 链上 eth_call vs 前端 constants）正是查 Polymarket\
结算/签名逻辑时该有的纪律；"机制存在 $\neq$ 在执行"（Feds 休眠）对应 auto-settlement 仍会遗留手动结算仓位的教训；APY\
分母分析法可用于拆穿流动性挖矿/做市收益虚高的宣传。\
叁·策略执行层 做市实战：四旋钮、探针反推账户特征与 Lighter 脚本\
笔记 3: 做市实战：四旋钮、探针反推账户特征与 Lighter 脚本\
作者：gh25888 | 日期：2026-08-15 | 574 字 · 评分 46.2 · 做市与LP\
一句话核心：做市执行的全部变量浓缩为四个旋钮（价差/单量/刷新/库存），链上探针数据能反推真实做市账户的策略签\
名（贴盘抢位/对称做市/库存偏移），而"三层报价 + 三道风控"的做市脚本已落到 Lighter 测试网。\
关键机制\
四旋钮框架：价差 / 单量 / 刷新 / 库存。价差收窄 = 竞争加剧、肉变薄（0.13bps 盘口做市每单赚的肉是薄的）。\
宽哥做市打法：上下双边挂单提供流动性吃点差；点差大 = 肉厚（选盘逻辑）；业绩锚 1000u/12h/60u。\
探针数据反推做市账户特征（RB 链）：A 型 = dust 顶单贴盘抢位（ETH/BTC，价差 0.2–1bps）；B 型 = \
标准对称做市（SOL，固定单量，库存中性）；个别账户带库存偏移（买卖比 0.91 偏买）。\
细分策略家族：贴盘 BBO jumping / 分层金字塔 / 库存偏移 / 动态价差（波动率自适应） / 跨市场 XEMM / 信号驱动 / \
事件驱动——上下挂单是共同底盘，细分在拧法。\
Lighter 测试网做市脚本（ca-brain 方案）：单文件 CLI、三层报价 50/30/20、库存中性+偏移、三道风控（价格带 $\pm 200\text{bps}$ \
/ 库存熔断 $3\times\text{size}$ / 插针过滤 75bps）、默认 dry-run；ca-dev 按总架构实现中。\
风险清单\
贴盘抢位的逆向选择（容易吃到有毒流）；库存偏移的方向性暴露；插针风险；风控参数必须与标的流动性匹配。\
对工程实践的启示\

四旋钮与做市机器人参数一一对应（价差↔hs、单量↔base、刷新↔requote、库存↔maxpos/inventory_quote_mode）；探针反推做市账户特征的方法可直接移植来分析 Polymarket CLOB 上其他做市商的挂单行为；"三层报价 + 三道风控 + 默认\

dry-run"的 Lighter 脚本结构 = 风控边界 + 库存熔断 + dry-run 部署习惯的同构。\\

— 5 —\\

结 · 三篇串联 做市的三个层面：成本、火力与旋钮\\

笔记 4: 做市的三个层面：成本、火力与旋钮\\

作者：综合整理 | 日期：2026-08-16 | 主线对照 · 贯穿一句话\\

一句话核心：做市的三层真相——策略层面看成本约束（每格价差 > 双边成本），机制层面看火力（通道存在 ≠ \

资金在），执行层面看旋钮（四个旋钮决定你能否在市场里活下来）。\\

三篇主线对照\\

策略应用层 · 网格交易（hellingercheri-gif）：网格是简化做市，区间内低买高卖，成败在成本与退出纪律。\\

机制经济学层 · Inverse Finance（darven）：机制存在 ≠ 有火力，xy=K 拍卖/空 PSM/休眠 Feds 揭示真实做市条件。\\

策略执行层 · 做市实战（gh25888）：四旋钮 + 探针反推，价差/单量/刷新/库存是唯一可控的执行变量。\\

贯穿一句话\\

做市的三个层面：策略层面看成本约束（每格价差 > 双边成本），机制层面看火力（通道存在 ≠ \

资金在），执行层面看旋钮（价差/单量/刷新/库存决定你能否在市场里活下来）

笔记 082: Day 12：链上清算与自动交易基础设施扩展

作者：feiyee2165-blip | 时间：2026-08-16 13:53 UTC | 字数：308 | 评分：46

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

Day 12：链上清算与自动交易基础设施扩展

1. Euler V2 清算系统独立部署：理解仓位转移模式（清算人接管债务+折扣抵押品，无需预付资金），搭建监控+自动执行
 2. Aave 清算扩展到多链：官方 GraphQL 查各链市场状态，利用率异常告警；事件驱动监听清算
 3. 部署第二个 Hermes 实例：VPS 初始化、SSH 安全加固（关密码登录）、跨机受限推送通道
 4. BSC meme 自动交易系统：多维度信号打分选币（链上流动性/交易量/热度/聪明钱），动态仓位，安全扫描拦截蜜罐
 5. 经验：免费 RPC 日志查询普遍限流需多源轮换；聚合交易引擎按优先级回退
- 明日计划：实盘验证自动交易，完善风控

笔记 083: 1\、修复普查解析器的持久化缺陷

作者：Wang Hua | 时间：2026-08-16 13:56 UTC | 字数：567 | 评分：49

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1\、修复普查解析器的持久化缺陷

\\ transactions.csv 改为每批增量写入，resume 以 CSV 为准，中断不再丢数据

\\ 已验证：30 笔试跑 → resume → 30 行无重复

2\、完成 4/6 冻结日解析（03-04、03-03、02-10、01-18，7,176 笔）

\\ 7,129 笔确认 J82 actual signer（99.3%），46 笔 unresolved

\\ 扫描停止，12-27/12-05 两日未解析，记录为未完成项

3\、关键发现（链上事实，E1 级）

\\ 222 笔闭环成功（成功率 3.09%），6,843 笔失败候选（95.4%）

\\ 锚点绝不是孤立事件：同一地址 4 个普查日共 222 笔闭环成功

\\ 01-18 是最高产日：单日 3,068 笔中 150+ 笔成功，含一笔 +22.43 USDC 大额

\\ 成功分布不均匀——更像事件/时段驱动，不是均匀持续

\\ USDC 闭环 67 笔毛利合计 989.70 USDC；全部费用仅 0.0357 SOL（≈3.15 USD）

笔记 084: 今天监听的PairCreated的新池和usdt/wbnb主池Swap结果是binance data

作者：Freetrip | 时间：2026-08-16 14:36 UTC | 字数：296 | 评分：45

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

今天监听的PairCreated的新池和usdt/wbnb主池Swap结果是binance datase单区块查询被拒，publicn ode 403，llamarpc ssl直接断连

匿名免费节点已把日志/事件接口全部关进付费墙

所有链上信号监控系统都是同一个三层架构：订阅层（数据源）→ 触发层（过滤/规则）→ 通知层（动作）

订阅层：拿到事件流。两种模式为\

事件驱动：节点主动推送，延迟毫秒级，需付费/注册节点\

轮询定时：自己定时拉，延迟=轮询间隔，免费可行。

触发层：对流入事件做规则过滤，规则就是信号定义。

通知层：触发后干什么——发通知、记数据库、下单。

09 风险与安全（6 篇）

笔记 085: 2026-08-15 學習打卡

作者：cht+ | 时间：2026-08-15 15:01 UTC | 字数：812 | 评分：48

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

2026-08-15 學習打卡

今天完成了什麼

完成跨市場套利 Shadow Mode 的通知稽核與故障復原強化：把候選持續性、通知呼叫、發送端回執與接收端驗收拆成不同證據層級，避免把單次合格評估或呼叫成功誤當成已送達。

建立可持久化的通知事件與回執流程；程序重新啟動時，會先恢復去重與完整追蹤狀態，再開始新的行情掃描，降低重複通知與追蹤遺失風險。

強化復原資料的嚴格驗證與失敗關閉策略，並隔離研究模式與正式狀態，避免測試或模擬資料污染正式通知紀錄。

完成去敏品質報告與多輪缺陷修正，涵蓋多級距追蹤復原、重啟冪等性、錯誤分類及非法資料型別拒絕。

證據在哪裡

今日版本紀錄顯示，通知稽核、復原流程、品質報告及其測試已整合完成，工作目錄維持乾淨。

全部 192 項自動化測試通過，程式碼品質檢查與差異格式檢查亦通過。

多輪獨立檢視曾找出復原級距錯置、研究模式狀態污染及型別驗證漏洞；修正後的最終檢視未留下高或中等嚴重度阻擋項目。

目前正式監控服務仍正常運行且未自動重啟；今日完成的新版尚未部署，因此不能把測試成果描述成正式環境已驗證。

尚未完成／下一步

在正式環境變更前，整理部署與回退檢查表，確認舊版狀態可安全遷移至新版復原格式。

部署後觀察首次重啟復原、通知去重、完整多級距追蹤及去敏品質報告，並以真實接收端證據完成通知鏈驗收。

持續評估通知已被外部服務接受、但本地尚未持久化回執時的極短故障窗口，避免把無法完全消除的外部不確定性包裝成絕對保證。

今日學習心得

套利監控的可信度不只來自價差計算，也來自每一層狀態都能被獨立證明。採用可重播事件、冪等復原與嚴格資料驗證，可以把重啟與短暫故障轉化為可控流程；但外部傳輸仍存在不可完全消除的不確定性，因此最重要的是清楚區分「系統曾嘗試」、「服務已接受」與「接收端已驗收」，並誠實保留證據邊界。

笔记 086: 当然可以。先抓住最核心的一点：

作者：hellingercheri-gif | 时间：2026-08-16 03:14 UTC | 字数：3547 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

当然可以。先抓住最核心的一点：

买 BTC 现货，是在赌“长期价格会上涨”；\

买 BTC 看涨期权，是在赌“在指定日期之前，BTC 会涨到足够高”。

所以，只判断“会不会涨”还不足以买期权。你还必须判断：

1. 什么时候涨？
2. 能涨多少？
3. 市场是否已经提前把这次上涨计入期权价格？

一、什么是看涨期权？

看涨期权 (Call Option) 赋予买方一种权利：在指定日期，以预先约定的价格获得 BTC 的上涨收益。

几个基本概念：

标的：这里是 BTC，或与 BTC 相关的期货合约

行权价：预先约定的价格

到期日：期权的有效期限

权利金：买期权时支付的价格

看涨期权：BTC 上涨时通常获利

看跌期权：BTC 下跌时通常获利

很多 BTC 期权是欧式、现金结算或与期货挂钩的，到期并不一定真的把 BTC 交给你，具体取决于交易所的合约规则。例如 CME 的不同 BTC 期权产品就存在不同结算方式。CME BTC 期权说明 (<https://www.cmegroup.com/cryptooptionsfaq>)

二、用具体数字比较

假设：

当前 BTC 价格：100,000 美元

你认为一个月后会涨

一个月到期的看涨期权：

行权价：105,000 美元

权利金：5,000 美元

合约对应 1 BTC 的价格敞口

暂时忽略手续费

方案 A：直接买 1 BTC

你需要支付 100,000 美元。

一个月后的盈亏：

$\lfloor \text{现货盈亏} = \text{到期价格} - 100,000 \rfloor$

方案 B：买看涨期权

你只支付 5,000 美元权利金。

到期盈亏：

$\lfloor \text{期权盈亏} = \max(\text{到期价格} - 105,000, 0) - 5,000 \rfloor$

因此，到期盈亏平衡点是：

$\lfloor 105,000 + 5,000 = 110,000 \text{ 美元} \rfloor$

也就是说，BTC 不只是一定要涨，还必须涨到 110,000 美元以上，你才真正盈利。看涨期权到期盈亏平衡点通常等于“行权价 + 权利金”。OIC (<https://www.optionseducation.org/strategies/all-strategies/long-call>)

| 一个月后 BTC 价格 | 买现货盈亏 | 看涨期权盈亏 |

| :----- | :----- | :----- |

| 80,000 | -20,000 | -5,000 |

| 100,000 | 0 | -5,000 |

| 108,000 | +8,000 | -2,000 |

| 110,000 | +10,000 | 0 |

| 120,000 | +20,000 | +10,000 |

| 150,000 | +50,000 | +40,000 |

这里有一个很反直觉的结果：

BTC 从 100,000 涨到了 108,000，上涨了 8%，买现货赚钱，但买期权依然亏钱。

因为这次上涨没有超过“行权价 + 权利金”。

三、两者的核心区别

| 对比项目 | 买 BTC 现货 | 买看涨期权 |

| :----- | :----- | :----- |

| 初始资金 | 较多 | 较少 |

| 是否有到期日 | 没有 | 有 |

| 最大损失 | BTC 跌到零时接近全部本金 | 通常限于已付权利金 |

| 是否有时间损耗 | 没有 | 有 |

110,000
看涨期权说明

对上涨幅度的要求	涨一点就产生账面利润	必须涨得足够多
是否受隐含波动率影响	不直接受影响	影响明显
是否真正持有 BTC	是	通常不是
收益结构	线性	非线性、带杠杆
未使用杠杆时是否爆仓	通常不会	单纯买期权通常不会，但可能归零

为什么有人仍然选择期权？

因为期权具有资本效率和有限亏损。

在上面的例子中：

买现货用了 100,000 美元

买期权只用了 5,000 美元

BTC 涨到 120,000 美元时：

现货赚 20,000 美元，投入回报约 20%

期权赚 10,000 美元，投入回报约 200%

期权的绝对利润较少，但相对于投入资金，回报率高很多。

代价是：如果判断不够准确，这 5,000 美元可能在一个月后全部归零。

四、最容易误解的地方：同等敞口和同等资金不是一回事

上面是在比较：

买 1 BTC

买对应 1 BTC 敞口的期权

这是“同等名义敞口”的比较。

但有人可能会想：

既然我有 100,000 美元，而一份期权只要 5,000 美元，那我可以买 20 份。

这样就完全不同了。

如果买20份：

总权利金仍是100,000美元

获得约20 BTC的名义敞口

如果判断错误，100,000美元可能全部归零

所以“买期权最多只损失权利金”不等于“风险小”。如果你投入了全部资金，权利金本身就是全部本金。

五、为什么 BTC 涨了，期权价格仍可能下跌？

到期前，期权价格不仅受 BTC 价格影响，还受以下因素影响。

1. 时间损耗 Theta

期权每天都在接近到期日。

如果 BTC 没有尽快上涨，期权的时间价值通常会不断减少；越接近到期，这种损耗往往越明显。OIC 时间价值说明
(<https://www.optionseducation.org/optionsoverview/leverage-risk>)

所以：

你预测“BTC 最终会涨”可能是对的

但如果上涨发生在期权到期后，这份期权依然可能归零

2. 隐含波动率 IV

隐含波动率可以粗略理解为：市场认为未来可能波动得有多剧烈。

IV 很高：期权通常比较贵

IV 下降：即使 BTC 小幅上涨，期权也可能不涨反跌

重大消息发布前，期权经常因为预期波动而变贵

消息发布后，IV 可能快速下降，即所谓“波动率坍塌”

期权价格主要由内在价值和时间价值组成，而时间、隐含波动率、行权价和标的价格都会影响权利金。OIC 期权定价说明
(<https://prd-web.optionseducation.org/optionsoverview/options-pricing>)

3. Delta 不是固定的 1

买 1 BTC，BTC 每上涨 1 美元，你的持仓价值大致上涨 1 美元。

但一份对应 1 BTC 的看涨期权，BTC 上涨 1 美元，期权可能只上涨例如 0.3、0.5 或 0.7 美元。这个敏感度叫 Delta，而且会不断变化。

因此，期权并不是“用更少的钱，完全复制一枚 BTC”。

六、什么情况下两种工具分别更符合逻辑？

直接买 BTC 现货通常更符合以下观点：

只确定长期看涨，不确定具体时间

不想承受期权到期归零

可以接受 BTC 下跌和较大的账面波动

希望长期持有或真正拥有 BTC

不想研究行权价、IV、时间损耗等因素

买看涨期权更贴近以下观点：

对上涨发生的时间有明确判断

认为涨幅会明显超过市场预期

希望事先限定最大损失

希望用较少资金获得较大的名义敞口

能接受权利金全部归零

一句很实用的总结是：

“我觉得 BTC 以后会涨”更接近现货逻辑；\

“我认为 BTC 在一个月内会大涨，而且涨幅足以覆盖昂贵的权利金”才更接近期权逻辑。

最后提醒：以上主要讨论的是“买入看涨期权”。卖出裸期权是另一回事，可能涉及保证金、强制平仓，以及非常大的甚至理论上无限的损失，不适合把它简单理解成“收取利息”。

笔记 087: 第 12 天 | 阶段三开启：捕获、实施与复盘

作者：Milli | 时间：2026-08-16 14:10 UTC | 字数：799 | 评分：72

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

第 12 天 | 阶段三开启：捕获、实施与复盘

主题：把“信号”变成“真上链”——quote → 授权 → 签名 → 广播 → /status 五步闭环

产出：experiments/executor.py（可运行，入参 token/amount/fromChain/toChain，出参 txHash + 状态日志）

一句话总结

执行链路的本质 = 把 LI.FI 报价里的 transactionRequest 字段变成一笔「由我私钥签名的、广播到链上的真实交易」，然后用 /status 盯着它从 PENDING 走到 DONE 或 FAILED。

今日核心产出

1. 五步链路（全部实测跑通）

① GET /quote → 报价 + 顶层 transactionRequest (to/data/value/gasLimit/gasPrice)

② 授权检查 → allowance(token, owner, LI.FI Diamond) < 金额？→ 构造 approve

③ 本地签名 → eth_account.sign_transaction（离线，不碰网络）

④ 广播（可选）→ eth_sendRawTransaction 到源链 RPC → txHash

⑤ /status 轮询 → PENDING → DONE / FAILED（dry-run 时会 NOT_FOUND，预期行为）

2. 实测记录

`bash

主网 dry-run（真实报价 + 真实签名，不广播）

python3 executor.py --amount 10000

输出：tool=eco | 到账

笔记 088: 01.9 钱包隔离、授权、签名与只读安全

作者：petree1 | 时间：2026-08-16 14:29 UTC | 字数：14343 | 评分：90

来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

01.9 钱包隔离、授权、签名与只读安全

1. 本节解决什么问题

扫链经常需要访问陌生代币、陌生地址和新上线平台。页面可能提示你连接钱包、签署消息、授权代币或确认交易。

本节帮助你判断：

当前研究任务是否需要钱包，以及点击下一步后究竟会暴露地址、生成签名、创建权限还是执行链上动作。

1.1 本节主问题链

我要完成什么研究任务

- 是否可以使用公开数据完成
- 是否真的需要连接钱包
- 页面请求的是连接、签名、授权还是交易
- 哪个账户正在操作
- 哪个合约将获得什么能力
- 权限有多大、持续多久、如何撤销
- 出现异常后应该隔离什么

1.2 第一遍必须建立的最低模型

第一次阅读先掌握六点：

1. 查看公开链上数据通常不需要连接钱包。
2. 连接钱包主要让网站知道你的公开地址，并允许网站向钱包提出后续请求。
3. 连接、消息签名、Token 授权和交易确认是四种不同动作。
4. 消息签名可能只用于登录，也可能授权合约以后执行某种动作，要看签名内容。
5. 断开网站连接不会自动清除链上 allowance。
6. 钱包隔离需要区分地址隔离、助记词隔离、设备隔离和资金隔离。

2. 从一个具体场景开始

假设你在研究一个刚上线的 Meme 币。

你先打开 BscScan 查看合约、交易和持有人。随后打开一个行情平台，页面提示连接钱包。连接后，页面又弹出签名请求。最后，它要求确认一笔授权交易。

这三个弹窗代表不同风险层：

连接钱包

- 网站取得当前公开地址，并可以继续发起请求

签署消息

- 钱包用当前账户对指定内容生成签名

授权交易

- 链上 Token 合约记录 Owner 对 Spender 的 allowance

如果你的目标只是观察行情和链上历史，第一步就先尝试不连接钱包完成任务。

3. 五级操作阶梯

3.1 第一级：公开只读查询

常见动作包括：

1. 在 BscScan 查看地址、交易、合约和日志。
2. 使用 RPC 读取交易、回执、代码和状态。
3. 查看公开的 Dune 查询与平台行情。
4. 把公开地址加入自己的观察列表。

这些动作不需要私钥，也不需要签名。

3.2 第二级：连接钱包

连接后，网站通常可以取得你允许公开的账户地址、网络和钱包能力，并向钱包提出签名或交易请求。

连接动作本身不会自动创建 Token allowance，也不会自动转走资产。

3.3 第三级：签署消息

钱包对一段消息或结构化数据生成签名。签名本身通常不上链，也可能被网站或其他人提交到合约中使用。

签名效果由消息内容、验证合约和使用方式决定。

3.4 第四级：创建链上权限

典型动作是：

`Token.approve(spender, amount)`

交易成功后，Token 合约会记录 Owner 对 Spender 的 allowance。Spender 后续可以在额度和合约规则允许的范围内调用 `transferFrom`。

3.5 第五级：执行链上交易

典型动作包括：

1. 转出 BNB。
2. 转出 Token。
3. 调用 Swap。
4. 添加流动性。
5. 修改授权。
6. 调用陌生合约方法。

交易会进入链上执行流程，可能改变资产、权限和其他状态，并消耗 Gas。

3.6 操作阶梯的判断原则

能用公开查询完成

→ 不连接钱包

确实需要账户识别

→ 使用隔离账户连接

出现签名或交易请求

→ 暂停并重新识别动作类型

4. 先分清六个安全对象

| 对象 | 最低正确理解 |

| :-: | :----- |

| 公开地址 | 可以公开查询的账户标识。知道地址不能直接签名。 |

| 私钥 | 控制单个账户签名能力的秘密材料。泄露后该账户失去控制安全。 |

| 助记词 | 可以派生并恢复同一钱包中的一组账户。泄露影响所有相关派生账户。 |

| 钱包密码 | 通常保护当前设备上的钱包访问。它不能替代助记词完成完整恢复。 |

| 钱包连接 | 网站取得账户地址和请求接口的权限关系。 |

| 链上授权 | Token 合约等链上状态中保存的持续权限。 |

4.1 同一助记词下的新账户能隔离什么

同一助记词派生的账户具有不同地址、余额和 allowance。某个地址给出的 Token 授权不会自动变成另一个地址的授权。

助记词一旦泄露，该助记词派生的所有账户都可能受到影响。因此，同一助记词下新建账户只能提供地址级隔离，不能提供根密钥隔离。

4.2 钱包密码能保护什么

钱包密码主要保护当前设备上的本地钱包访问。攻击者取得助记词或私钥后，可以在其他钱包中恢复账户，不需要知道原设备密码。

5. 扫链的只读优先原则

5.1 哪些任务通常无需连接钱包

1. 查看 Token 合约地址和源码。
2. 查看交易 Input、Receipt、Logs 和 Trace。
3. 查询某地址持仓和历史活动。
4. 查询池地址、储备和交易记录。
5. 观察 GMGN、DEX Screener、Arkham 和 Bubblemaps 的公开页面。
6. 运行只读 RPC 和公开 SQL 查询。
7. 建立代币筛选表和钱包标签库。

平台是否把部分页面设为登录可见属于产品规则。即使需要平台账户登录，也不代表必须连接链上钱包。

5.2 只读钱包地址

很多观察任务只需要公开地址。可以把研究对象地址复制到浏览器、RPC、Dune 或自己的监控表中，无需导入私钥。

5.3 微案例：查看聪明钱钱包

研究目标：观察地址 A 最近买过哪些代币。

安全路径：

复制公开地址 A

→ 在 BscScan 或研究平台搜索

→ 保存交易哈希和区块

→ 回到原始交易复核

整个流程不需要自己的钱包参与。

6. 钱包隔离模型

6.1 四类角色

| 钱包角色 | 用途与边界 |

| :---- | :----- |

| 观察环境 | 只使用公开地址、浏览器和 RPC，不保存可签名密钥。 |

| 实验钱包 | 用于陌生网站和测试，保持低余额，不保存长期资产。 |

| 交易钱包 | 只连接经过核查的交易平台，资金规模受风险预算限制。 |

| 长期保管钱包 | 保存长期资产，尽量不连接新平台，可使用独立硬件钱包。 |

6.2 当前课程最低配置

当前观察和纸面交易阶段建议采用：

研究浏览器配置

→ 不导入真实资产钱包

实验钱包

→ 独立助记词

→ 无长期资产

→ 只保留测试所需少量 Gas

真实资产钱包

→ 不进入当前课程陌生平台测试

6.3 隔离的四个维度

1. 地址隔离：不同地址拥有不同余额与链上授权。
2. 根密钥隔离：使用不同助记词或独立硬件钱包。
3. 浏览器隔离：研究配置不安装无关扩展，不复用高价值钱包会话。
4. 资金隔离：实验钱包只保留能承受完全损失的少量资产。

6.4 硬件钱包的边界

硬件钱包把私钥和签名动作隔离到独立设备，可以降低浏览器和电脑直接取得私钥的风险。

如果用户在设备上确认了恶意交易或危险签名，硬件钱包仍会按确认内容签署。因此，硬件设备不能替代对地址、金额、合约和权限的检查。

7. 连接钱包究竟发生了什么

7.1 网站获得的主要信息

连接后，网站通常可以看到：

1. 你允许公开的账户地址。
2. 当前网络。
3. 公开链上余额和历史。
4. 钱包支持的请求能力。

网站随后可以向钱包提出签名或交易请求。钱包仍应要求用户确认。

7.2 连接不等于 Token 授权

连接网站与 approve 写入不同位置：

网站连接

→ 钱包或浏览器保存的站点权限

Token allowance

→ Token 合约保存的链上状态

清除网站连接不会改变 Token 合约中的 allowance。

7.3 WalletConnect 风险提示的边界

WalletConnect Verify 等功能可以提示域名匹配、未知、失配或已知威胁。官方文档同时说明此类检测不能保证绝对安全。

域名显示正常仍需检查签名内容和交易参数。安全提示属于辅助线索。

8. 签名请求为什么需要逐项阅读

8.1 普通消息签名

普通消息签名常用于登录、证明地址控制权或接受服务条款。

需要检查：

1. 请求来自哪个域名。
2. 当前使用哪个账户。
3. 消息原文表达什么。
4. 是否包含长期有效的授权语句。
5. 网站是否说明签名用途。

8.2 EIP-712 结构化签名

EIP-712 让钱包能够显示结构化字段，例如：

```
domain
chainId
verifyingContract
primaryType
message
```

结构化显示有助于理解内容，但 EIP-712 本身不提供完整重放保护，也不保证业务安全。应用仍需正确使用 Domain、Nonce、Deadline 和验证逻辑。

8.3 Permit 签名

ERC-2612 permit 允许 Owner 使用 EIP-712 签名修改 Token allowance。签名可以由其他地址提交上链。

因此，下面两种授权路径效果可能相近：

路径一

Owner 发送 approve 交易
→ Token 合约写入 allowance

路径二

Owner 签署 permit
→ 任意提交者把签名交给 Token 合约
→ Token 合约验证后写入 allowance

检查 Permit 时重点读取：

1. Owner。
2. Spender。
3. Token 或 verifyingContract。
4. Value。
5. Nonce。
6. Deadline。
7. Chain ID 与 Domain。

8.4 微案例：签名登录与 Permit

签名登录可能写：

登录 example.com
账户 0xA
Nonce 123

Permit 可能写：

Owner 0xA
Spender 0xS
Value uint256.max
Token 0xT
Deadline 很晚

两者都会弹出签名窗口，但后者可能形成持续 Token 权限。

9. 授权交易需要检查什么

9.1 BEP20 approve

授权交易的核心关系是：

Owner
→ 在 Token 合约中设置
→ Spender 可以使用多少额度

顶层 tx.to 通常是 Token 合约。Spender 位于 approve 参数中。

9.2 授权额度

常见额度包括：

1. 本次实际需要的数量。
2. 自定义上限。
3. uint256 最大值形式的常见无限授权。

授权额度越大、有效时间越长，潜在暴露越大。还需要同时评估 Spender 代码、升级权限和目标 Token 余额。

9.3 撤销授权

常见撤销方式是把同一组 Owner、Token、Spender 的 allowance 设置为零。

撤销属于链上写操作：

1. 需要发出交易。
2. 需要 Gas。
3. 需要检查回执和最终 allowance。
4. 只影响指定 Token 与 Spender 组合。

9.4 撤销后的证据链

approve(spender, 0) Input

→ 成功 Receipt

→ Approval Event 为 0

→ allowance(owner, spender) 读取为 0

Event 和状态读取可以相互验证。只看到钱包界面提示“已撤销”仍应回到链上确认。

10. 交易确认前的十项检查

10.1 先识别动作类型

确认钱包请求属于：

连接

消息签名

结构化签名

Token 授权

普通转账

合约调用

10.2 再检查关键对象

1. 域名是否准确。
2. 当前网络是否为预期网络。
3. 当前签名账户是否为实验钱包。
4. 顶层 to 是谁。
5. BNB value 是否为零或预期数值。
6. 方法名和 Selector 是否匹配。
7. Spender、Recipient 或 Operator 是谁。
8. Token 和金额是否正确。
9. Deadline、Nonce 和 Domain 是否合理。
10. 钱包模拟、风险提示和页面摘要是否与原始参数一致。

模拟和风险提示可以辅助判断，不能覆盖原始参数检查。

11. 真实案例：授权归零与断开连接

11.1 案例标识

网络：BNB Smart Chain

交易哈希：0xe0caaae3848472a96dcca6bdb6eec167023e133bbaeb7c5df2921620cee6cd72

区块高度：113374111

区块哈希：0x219f4931e8f7cfccc508d2c88a725c8628aa9dc3ac28c7d19558ff6ac6e46914

复核时间：2026-08-13

数据来源：BNB Chain 公共 RPC

11.2 原始交易字段

tx.from：0x476f2b4b174890ce29ee13dc430a597fe2e997e4

tx.to：0x55d398326f99059ff775485246999027b3197955

tx.value：0

[illegible]

11.3 ABI 解码结果

```
Owner : 0x476f2b4b174890ce29ee13dc430a597fe2e997e4
Token : 0x55d398326f99059ff775485246999027b3197955
Spender : 0x07964f135f276412b3182a3b2407b8dd45000000
Amount : 0
```

```
status : 0x1
```

Event address : 0x55d398326f99059ff775485246999027b3197955
Topic0 : 0x8c5be1e5ebec7d5bd14f71427d1e84f3dd0314c0f7b2291e5b200ac8c7c3b925
Approval owner : 0x476f2b4b174890ce29ee13dc430a597fe2e997e4
Approval spender : 0x07964f135f276412b3182a3b2407b8dd45000000
Approval value : 0

11.5 状态复核

$$\text{allowance}(\text{owner}, \text{spender}) = 0$$

11.6 这个案例说明什么

1. 这是一笔链上授权归零交易，需要 Gas。
2. 它改变的是指定 Token、Owner 和 Spender 的 allowance。
3. 当前 allowance 为零与 Event 方向一致。
4. 单纯断开网站连接无法产生这笔链上状态变化。
5. 这笔交易不能证明该 Owner 对其他 Spender 或其他 Token 的授权也已清除。

12. 出现异常后的分级处理

12.1 只连接了网站

1. 断开站点连接。
2. 清除该站点的钱包权限和会话。
3. 检查是否还签过消息或交易。
4. 检查地址近期链上活动。

12.2 已簽署不明消息

1. 保存域名、账户、消息原文和时间。
2. 判断消息是否包含 Permit、订单、Operator 或其他授权语义。
3. 检查 Domain、verifyingContract、Nonce、Deadline 和资产范围。
4. 按协议机制撤销或使签名失效。
5. 无法确定签名效果时，停止继续交互并隔离相关资产。

12.3 已发送不明授权或交易

1. 保存交易哈希。
2. 检查 tx.to、Input、Receipt、Logs 和状态变化。
3. 定位 Token、Spender、Operator、额度和剩余权限。
4. 使用可信入口撤销可撤销权限。
5. 对高风险持续权限，评估把资产迁移到独立助记词钱包。

12.4 助记词或私钥可能泄露

应停止使用受影响根密钥，并在可信设备和安全环境中建立新的独立钱包。资产迁移需要同时考虑攻击者监控、Gas、授权和恶意自动转出风险。金额较大或情况复杂时，应寻求可信的专业安全协助。

13. 当前课程安全环境清单

13.1 研究环境

研究浏览器配置不保存真实资产钱包密钥。

只安装必要且来源明确的扩展。

BscScan、RPC 和公开工具优先使用无钱包模式。

收藏或手动核对常用工具的官方域名。

系统、浏览器和钱包扩展保持更新。

13.2 实验钱包

使用独立助记词。

不从真实资产钱包助记词派生。

不保存长期资产。

只保留实验所需少量 Gas 和测试资产。

地址明确标注为实验用途。

13.3 签名与交易纪律

不签署无法解释的消息。

不依赖页面按钮名称判断真实动作。

核对域名、网络、账户和合约地址。

核对 Spender、Token、金额和 Deadline。

无限授权需要明确理由。

陌生合约先在测试网或低价值实验钱包验证。

13.4 授权维护

定期检查高余额钱包的 allowance。

区分站点连接和链上授权。

不再使用的授权按风险和 Gas 成本处理。

撤销后回到链上状态复核。

保存异常授权的交易哈希和处置记录。

13.5 密钥纪律

助记词和私钥不进入 Obsidian、聊天、截图和云文档。

不把助记词输入普通网页。

备份方式与个人威胁模型匹配。

高价值资产考虑独立硬件钱包。

确认恢复材料真实可用，同时避免在线暴露。

14. 常见误判与纠正

| 误判 | 正确检查方向 |

| :----- | :----- |

| 只查看地址也必须连接钱包 | 先使用公开地址、BscScan 和 RPC。 |

| 连接钱包后网站已经能直接转走全部资产 | 连接主要公开地址并开放请求入口，资产权限取决于后续签名、授权和交易。 |

| 断开网站就撤销了 Token 权限 | 继续查询 Token allowance，链上撤销需要交易或对应协议机制。 |

| 消息签名不花 Gas，所以没有风险 | 检查签名内容是否可以被用于 Permit、订单或其他授权。 |

| EIP-712 显示结构化字段，所以一定安全 | 检查 Domain、验证合约、Nonce、Deadline 和业务效果。 |

| 硬件钱包可以自动识别恶意交易 | 硬件钱包保护私钥，用户仍需检查设备上确认的内容。 |

| 同一助记词新建账户可以隔离助记词泄露 | 它只提供地址级隔离，根密钥风险仍然共享。 |

| 授权归零后该钱包所有风险都已清除 | 只确认指定 Token、Owner 和 Spender 组合，并继续检查其他权限。 |

| 钱包显示的合约名称可以代替地址核对 | 使用网络和完整合约地址固定对象。 |

| 安全工具没有警告，所以可以签名 | 风险提示存在覆盖和识别边界，继续人工检查请求内容。 |

15. 官方来源与时效边界

15.1 必要来源

| 关键主张 | 来源、核查日期和适用边界 |

| :----- | :----- |

| ----- |
| 连接钱包主要公开账户并允许网站发起后续请求，连接本身不会创建 Token allowance | MetaMask Why Connect
(<https://support.metamask.io/start/why-am-i-being-asked-to-connect-to-a-dapp/>) 与 Disconnect Wallet
(<https://support.metamask.io/more-web3/dapps/disconnect-wallet-from-a-dapp/>)，2026-08-13 核查。钱包界面会更新。 |

| 断开网站不会撤销链上 Token allowance，撤销授权需要链上动作和 Gas | MetaMask Revoke Approvals (https://support.metamask.io/more-web3/learn/how-to-revoke-smart-contract-allowances-token-approvals/), 2026-08-13 核查。具体撤销入口和支持网络会变化。|

| EIP-712 提供结构化签名与 Domain Separation，但不自动提供完整重放保护 | EIP-712 (https://eips.ethereum.org/EIPS/eip-712), 2026-08-13 核查。实际安全取决于应用验证逻辑。|

| ERC-2612 Permit 可以使用签名修改 allowance，并使用 Nonce、Deadline 与 Domain | ERC-2612 (https://eips.ethereum.org/EIPS/eip-2612), 2026-08-13 核查。不同 Token 可能使用其他 Permit 变体。|

| WalletConnect Verify 提供域名风险状态，同时声明不能保证绝对安全 | WalletConnect Verify API (https://docs.walletconnect.network/wallet-sdk/web/verify), 2026-08-13 核查。实际提示取决于钱包实现。|

| 助记词可以控制其派生账户，硬件钱包增加独立设备签名边界 | MetaMask SRP Security (https://support.metamask.io/start/what-is-a-secret-recovery-phrase-and-how-to-keep-your-crypto-wallet-secure/) 与 Hardware Wallet Guide (https://support.metamask.io/more-web3/wallets/hardware-wallet-hub), 2026-08-13 核查。具体产品支持会变化。|

15.2 时效边界

1. 钱包界面、风险提示、支持的签名类型和高级权限会更新。
2. DApp 域名、合约地址和前端控制权可能变化，使用时需要重新核查。
3. Token 可能采用 ERC-2612 以外的 Permit 或授权机制。
4. 撤销工具和钱包安全扫描只提供辅助能力，最终回到链上状态复核。
5. 本节不保存任何私钥、助记词、API 密钥或真实签名材料。

16. 压缩总结与覆盖检查

16.1 最短安全规则

公开查询优先

- 研究、实验、交易和保管环境分离
- 连接、签名、授权和交易分别识别
- 核对账户、网络、合约、Spender、金额和期限
- 撤销后回到链上状态确认
- 根密钥泄露时迁移到独立根密钥

16.2 正文覆盖检查

| 能力目标 | 覆盖位置与当前状态 |

| :----- | :----- |

| 区分只读、连接、签名、授权和交易 | 第 2 至 3 节已建立最低模型。|

| 区分地址、私钥、助记词、密码和权限 | 第 4 节已覆盖。|

| 建立钱包隔离环境 | 第 5 至 6 节和第 13 节已覆盖。|

| 理解连接与链上授权差异 | 第 7 节和第 11 节已覆盖。|

| 识别消息签名与 Permit | 第 8 节已覆盖。|

| 检查和撤销 Token 授权 | 第 9 至 11 节已覆盖。|

| 处理异常交互 | 第 12 节已覆盖最低处置顺序。|

| 解释、定位、执行和判断能力 | 正文学习中，题目层尚未启动。|

17. 本节上下文交接卡

| 字段 | 内容 |

| :----- | :----- |

| 本节核心结论 | 公开扫链优先使用无钱包模式。连接、签名、Token 授权和交易属于不同动作。安全隔离需要同时考虑地址、根密钥、浏览器和资金。|

| 正文生成状态 | 2026 年 8 月 13 日按第一遍可读性规则生成。2026 年 8 月 14 日由学习者直接进入第二章，正文阅读阶段结束。|

| 已经掌握的动作 | 尚未通过本节互动评估。|

| 仍然容易出错的点 | 等待学习反馈。重点观察连接与授权、消息签名与 Permit、地址隔离与根密钥隔离的混淆。|

| 已保存的真实案例 | BNB Smart Chain 授权归零交易，包含完整交易哈希、区块、Input、Receipt、Approval Event 和当前 allowance。|

| 案例证据缺口 | 交易区块历史 allowance 查询返回 missing trie node。当前 allowance 只能证明观察时刻状态。|

| 必须继承的术语 | 只读查询、钱包连接、消息签名、EIP-712、Permit、Owner、Spender、allowance、助记词、根密钥隔离。|

| 不要重复展开的内容 | BEP20 approve 与 transferFrom 机制已在 01.5 建立，ABI、Receipt 与 Logs 已在 01.6 建立。|

| 留给下一节的问题 | 怎样在章节综合实验中连接网络、账户、交易、授权、证据层级与安全边界。|

| 跨章节接口 | 签名与授权风险交付第五章，隔离执行与成本控制交付第九章。|

| 教学互动状态 | 未启动。学习者已进入第二章。|

题目层状态	未启动。完成正文后只推荐下一项互动。
独立任务状态	未进入，属于选做。
案例内复习状态	尚未开始。后续案例自然复习 raw Input 与 ABI 解码、Approval Event 与 allowance 状态。
四维能力状态	解释、定位、执行和判断均待后续案例评估。
下一小节建议	01.10 已由学习者主动跳过。当前进入 02.1。

笔记 089: 1. ERC-20中规定的几个核心属性/交互方法

作者：mookixiao | 时间：2026-08-16 14:55 UTC | 字数：1320 | 评分：61
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

1. ERC-20中规定的几个核心属性/交互方法

1. Token Address（代币合约地址）：代币在链上的20字节物理地址，比如USDT在Enthereum上的地址为0xdAC17F958D2ee523a2206206994597C13D831ec7

1. 套利程序应该依赖地址识别资产，而不是依赖代币符号，防止遇到貔貅币/假币

2. Decimals（精度/小数位数）：代币可分割的最大小数位数

1. EVM底层不支持浮点数，链上用大整数（BigInt）存储代币数量，计算关系：

$$\mathrm{真实数量} = \frac{\mathrm{链上原始整数}}{\mathrm{10}^{\mathrm{decimals}}}$$

3.

Balance（余额）：查询账户（地址）在当前代币合约映射表（Mapping）中持有的链上整数数量，通过balanceOf(address)实现查询

4. Allowance（授权额度）：资产所有者（owner）允许第三方合约/地址（spender）划扣的最大代币数量，通过allowance(owner, spender)查询

5. approve(spender, amount)：由资产所有者（owner）调用，授权spender地址有权划转的代币数量为amount

6. transferFrom(from, to, amount)：由被授权方（spender）调用，在不超过Allowance的情况下将amount数量的代币从from账户划转至to账户

2. 在Ethereum的底层架构里，一切主体（合约地址、账户地址、代币地址等）均由一个20字节（160位、40个十六进制字符，加上0x前缀）的地址标识

3. 在Arbiscan上查询地址

1. 不同选项卡含义

1. Transactions：主交易，由EOA直接发起并支付Gas的顶层交易

2. Internal Transactions：内部交易，主交易在执行内部合约时，由合约代码自动触发的原生币转账

3. Token Transfers(ERC-20)：代币转移日志（代币转账属于日志事件，即Logs/Events），所有触发的ERC-20代币Transfer事件，比如USDT、USDC的进出帐，最常查阅

2. Transactions中的Amount：交易产生的原生币转账，如图1中的交易所示（没有产生原生币转账所以为0）

1. Internal Transactions中的Amount：交易产生的原生币转账，如图2中的交易所示（没有产生原生币转账所以为0）

2. Token Transfers(ERC-20)中的Amount：交易产生的代币转账，如图3中的交易所示

4. 分辨一个地址是合约还是地址：有页面图标是合约、无页面图标是地址

笔记 090: 最近完成：

作者：wangche | 时间：2026-08-16 14:57 UTC | 字数：379 | 评分：60
来源：<https://intensivecolearn.ing/programs/b43d2e97-ed88-4ca3-b12f-7ef672b01205>

最近完成：

深度复盘一笔 Morpho + Lulo + sNUSD 交易，确认其不是纯闭环套利，而是闪电流动性叠加高杠杆借款仓位。

还原资金路径，识别出 EOA 收到的 166.666013 USDC 属于借款后现金残余，不能直接当作套利利润。

跟踪该仓位的抵押、债务、预言机和清算线，发现其长期处于接近 LLTV 上限的高风险状态。

新增 Morpho/Lulo 只读影子监控模块：支持区块状态快照、事件监听、本地 SQLite 游标和历史区块复核。

增加闭环净值评估：残留债务/抵押自动否决，成本不全自动标记为证据不足，避免误报套利。

增加 CLI、示例配置和使用文档，并用真实 Ethereum RPC 验证快照与事件同步。

完成测试验证：227 passed。

全程未进行签名、授权、借款、部署或链上交易。